

计算机组成原理

- 计算机系统概述
- 数据信息的表示
- 运算方法与运算器
- 存储系统
- 指令系统
- 中央处理器
- 指令流水线
- 总线、异常与中断
- 输入与输出系统

计算机系统概述

计算机系统的组成

计算机包括**硬件**和**软件**两部分，还包括固化的软件，即**固件**
固件兼具软件和硬件的特性，如BIOS

硬件系统

按照冯诺伊曼的设计思想，硬件系统包含**运算器**、**控制器**、**存储器**、**输入设备**、**输出设备**

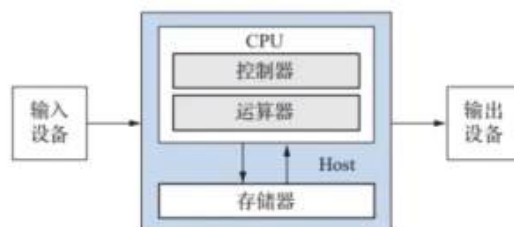


图 1.1 冯·诺依曼体系结构

其中，运算器和控制器合称为CPU，CPU和存储器称为主机，输入设备和输出设备合称为输入输出设备或外部设备

冯诺依曼机的特点如下

- 采用“**存储程序**”的工作方式
- 即将事先编制好的程序和原始数据送入主存储器后才能执行，一旦程序被启动执行，计算机会自动逐条执行指令，直至程序执行结束。
- 指令和数据以**同等地位存储在存储器**中，形式上没有区别，但计算机应能区分它们
- 指令和数据均用**二进制代码**表示
- 指令由**操作码**和**地址码**组成，操作码指出操作的类型，地址码指出操作数的地址

计算机的发展经过了**电子管、晶体管、集成电路、超大规模集成电路**的几个发展阶段

i 摩尔定律

价格不变时，集成电路可容纳的晶体管数量大约18-24个月翻一倍，性能提升一倍

存储器

存储器由许多个存储单元构成，存放程序和数据，它们分别对应的编号称为**单元地址**

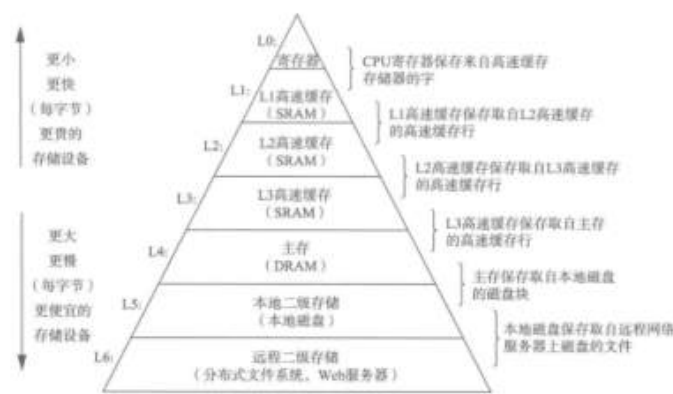


图 1-4 一个存储层次结构的示例

存储器分为主存和辅存，CPU 可以直接访问主存，辅存的内容必须调入主存后才能为 CPU 访问

运算器

对数据进行运算，由**算术逻辑单元ALU**和一系列寄存器组成

ALU完成运算，寄存器存放运算操作数，累加器存放连续运算的中间结果和最后结果

控制器

实质是**解释程序**，每次从存储器读取一条指令并分析译码产生一串操作命令再发给各功能部件

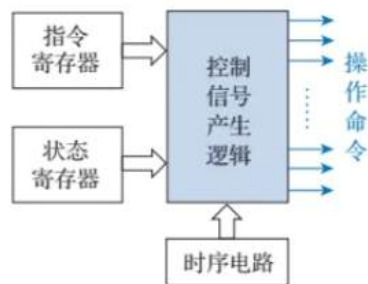


图 1.4 控制器结构图

i 流动信息

计算机有两种**流动信息**

- 由控制器发生的控制流信息，即**操作命令**
- 另一种则是数据流信息，受控制流信息的控制从一个部件流向另一个部件

控制流信息的依据是

- 存放在指令寄存器中的**机器指令**，是计算机操作的主要依据
- 存放在状态寄存器中的**状态信息**，判断顺序执行还是分支转移
- 时序电路产生的**时序信号**

输入设备

将外部信息输入计算机的外部设备，输入设备与主机之间通过接口连接

? 设置接口的原因

- 输入设备传送数据的速度远低于主机，需要用接口进行数据缓冲

- 输入设备所用信息格式与主机不同，需要进行信息格式的转换
- 接口可以向主机报告设备运行的状态，传达主机的命令

输出设备

将计算机运算结果转换为能被人或其他设备接受和识别信息的设备，同样需要接口

外存储器

既可以作为输入设备也可以作为输出设备，还能存储信息

常常作为辅存使用，如磁盘、光盘、磁带等

总线

现代计算机使用较多的是**总线互连**方案

总线是连接两个或多个设备的公共信息通路，主要由数据线、地址线、控制线组成

CPU连接计算机中各主要部件的总线称为**系统总线**，所有设备均与总线相连

由于总线是多个设备的公共连接线，因此同一时刻只能允许一个设备向总线发送信息，多个设备同时接受来自总线的信息

软件系统

一台计算机全部程序的集合统称为**软件系统**

系统软件和应用软件

- 系统软件：一组保证计算机系统高效、正确运行的基础软件，通常作为系统资源提供给用户使用

系统软件主要有操作系统（OS）、数据库管理系统（DBMS）、语言处理程序、分布式软件系统等

- 应用软件：用户为解决某个应用领域中的各类问题而编制的程序，如各种科学计算类程序

语言级别

- 1. 机器语言：也称二进制代码语言，需要编程人员记忆每条指令的二进制编码
机器语言是计算机唯一可以直接识别和执行的语言。
- 2. 汇编语言：汇编语言用英文单词或其缩写代替二进制的指令代码，更容易为人们记忆和理解
使用汇编语言编写的程序，必须经过汇编程序的翻译转换为机器语言程序后，才能在计算机的硬件系统上执行
- 3. 高级语言。高级语言（如C、C++、Java等）可以方便程序设计人员写出解决问题的处理方案和解题过程的程序

通常高级语言需要经过编译程序编译成汇编语言程序，然后经过汇编操作得到机器语言程序，或直接由高级语言程序翻译成机器语言程序。

计算机系统层次结构

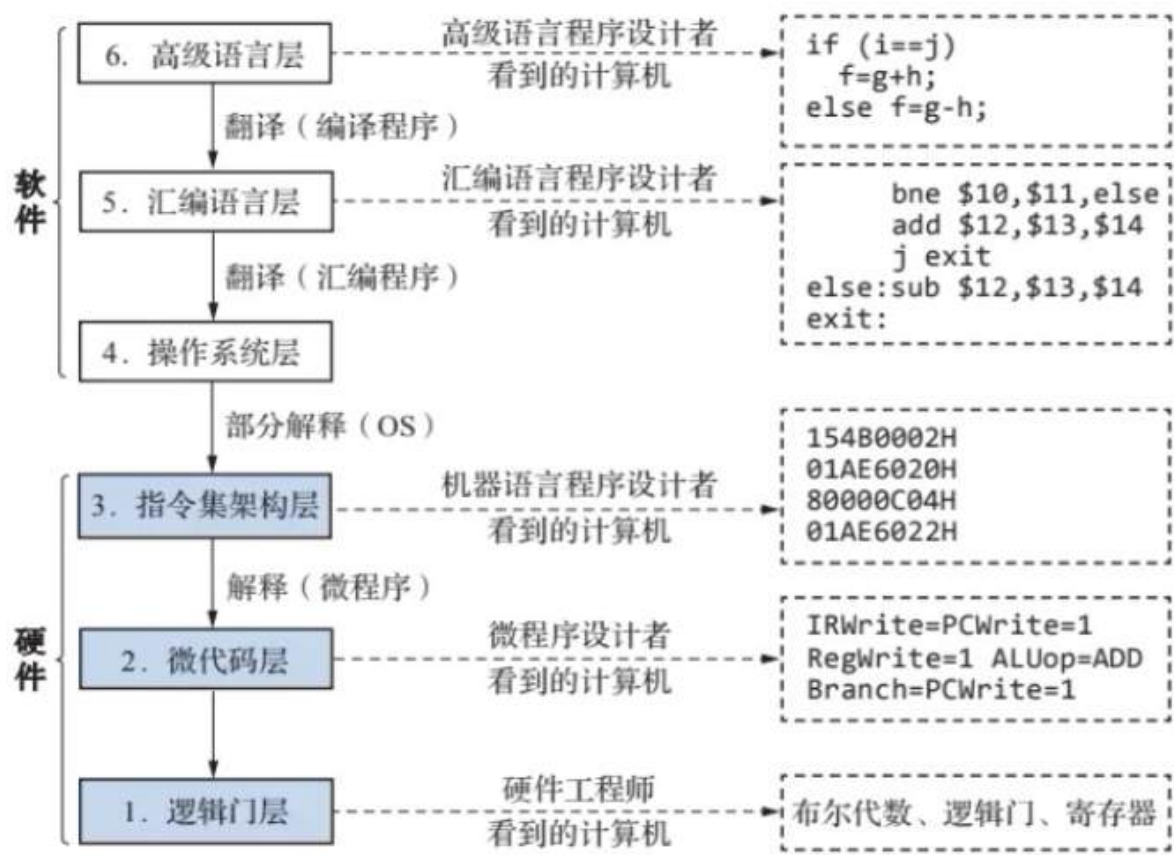


图 1.7 计算机系统的层次结构

- 逻辑门层

是计算机系统最底层的硬件系统，由逻辑电路构成

- 微代码层(可能有)

实际的机器层，用户可以使用微代码编写微程序，微程序直接由硬件直接执行

- 指令集架构层

通过机器语言编写程序实现对计算机硬件的控制，也称为传统机器层或ISA层

- 操作系统层

对计算机系统的硬件和软件资源进行统一管理和调度

- 汇编语言层

- 高级语言层

前三层为**硬件层**，后三层为**软件层**

操作系统层面向机器，而汇编语言层和高级语言层面向应用

计算机性能指标和评价

基本性能指标

- 机器字长

机器字长简称字长，字长一般指CPU一次处理的数据位数，用二进制数的长度来衡量，一般与计算机内部寄存器、运算器、数据总线的位宽相等

字长一般以字节为基本单位，现代计算机字长一般为32位或64位

字长影响计算机计算的精确度、数据表示范围和精度

- 主存容量

指主存存储最大信息量，一般用字容量×位容量的形式表示

字容量表示存储单元数，位容量表示存储单元存储的二进制位数

 增加主存容量能减少程序运行期间访问辅存的次数，利于程序执行速度的提高

表 1.1 主存容量常用单位的定义

单位	存储容量	最少地址线	单位	存储容量	最少地址线
KB (Kilobyte)	1024B	10 根	TB (Terabyte)	1024GB	40 根
MB (Megabyte)	1024KB	20 根	PB (Petabyte)	1024TB	50 根
GB (Gigabyte)	1024MB	30 根			

- **数据通路带宽**

数据通路带宽是指**数据总线**一次所能**并行传送信息的位数**，指外部数据总线的宽度
它与CPU内部的数据总线宽度（内部寄存器的大小）有可能不同

时间相关的性能指标

- **响应时间**：程序的执行时间，通常由**硬盘访问时间**、**内存访问时间**、**I/O操作时间**、**操作系统开销时间**、**CPU执行时间**构成
- **吞吐量**：系统在单位时间内处理请求的数量
- **时钟周期与主频**：时钟周期是计算机中**最基本的时间单位**，机器内部主时钟脉冲信号的宽度，一个时钟周期内CPU只完成一个基本动作
 - 时钟周期以**相邻状态单元间组合逻辑电路的最大延迟时间**为基准确定
 - 时钟周期也以指令流水线的每个流水段的**最大延迟时间**确定
 - 时钟周期是时钟频率的倒数，也称为**节拍周期**和**T周期**

主频是**机器内部主时钟的频率**，即每秒能有几个时钟周期

- **CPI与IPC**

CPI指执行每条指令所需要的**平均时钟周期数**，或一类指令、一段程序中所有指令执行所需时钟周期数的平均值

由定义可知

$$\text{CPI} = \frac{m}{\text{IC}}$$

其中，程序总指令条数用IC，程序执行所需时钟周期数为 m

也可以表示为

$$CPI = \sum_{i=1}^n (CPI_i \times P_i) = \sum_{i=1}^n (CPI_i \times \frac{IC_i}{IC})$$

其中， P_i 为每类指令的使用频率， CPI_i 表示每类指令的CPI，为每类指令的条数
IPC 指的是每个时钟周期CPU能执行的指令倒数，有

$$IPC = \frac{1}{CPI}$$

由于技术的发展，IPC的值可以大于1

• CPU时间

CPU时间一般为CPU执行时间，指CPU真正花费在该程序上的时间，它一般指用户CPU时间，指执行程序所要的时间

$$T_{cpu} = m \times T = m/f$$

其中，机器周期为T，频率为f

CPU时间的影响因素

- 时钟频率越高，程序执行速度越快，这取决于CPU的实现技术
- CPI越小，程序执行越快，这取决于计算机的实现技术和指令集结构
- 其他条件相同下，指令条数越少，执行速度越快

• MFLOPS 与 MIPS

每秒执行百万浮点运算次数，有

$$MFLOPS = \frac{IC_{flops}}{T_{cpu} \times 10^6}$$

MIPS 即**每秒百万条指令**，有

$$MIPS = \frac{f}{CPI} \times 10^{-6} = IPC \times f \times 10^{-6}$$

这也称为CPU全性能公式

🔔 通过编译后目标代码包含的指令条数多并不意味着执行更慢，如果更多的指令是CPI较小的，可能反而更快

📖 基准程序 (Benchmarks)

专门用来进行性能评价的一组程序，能够很好地反映机器在运行实际负载时的性能

可以通过在不同机器上运行相同的基准程序来比较在不同机器上的运行时间，从而评测其性能

数据信息的表示

数值数据的表示

定点数

定点数的小数点位置固定，定点整数的小数点在最低数位之后，定点小数在符号位之前

定点小数与定点整数

- 定点小数

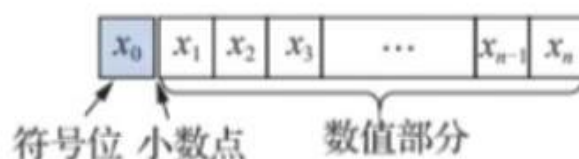


图 2.3 定点小数表示

数值部分称为尾数，而 x_1 称为最高有效位

定点小数主要用于表示浮点数的尾数

- 定点整数

定点整数如C语言中的char,short,int,long
其数据范围与机器字长、机器数表示方法有关

定点数的表示

由符号和数值一起编码表示的二进制数称为**机器数或机器码**

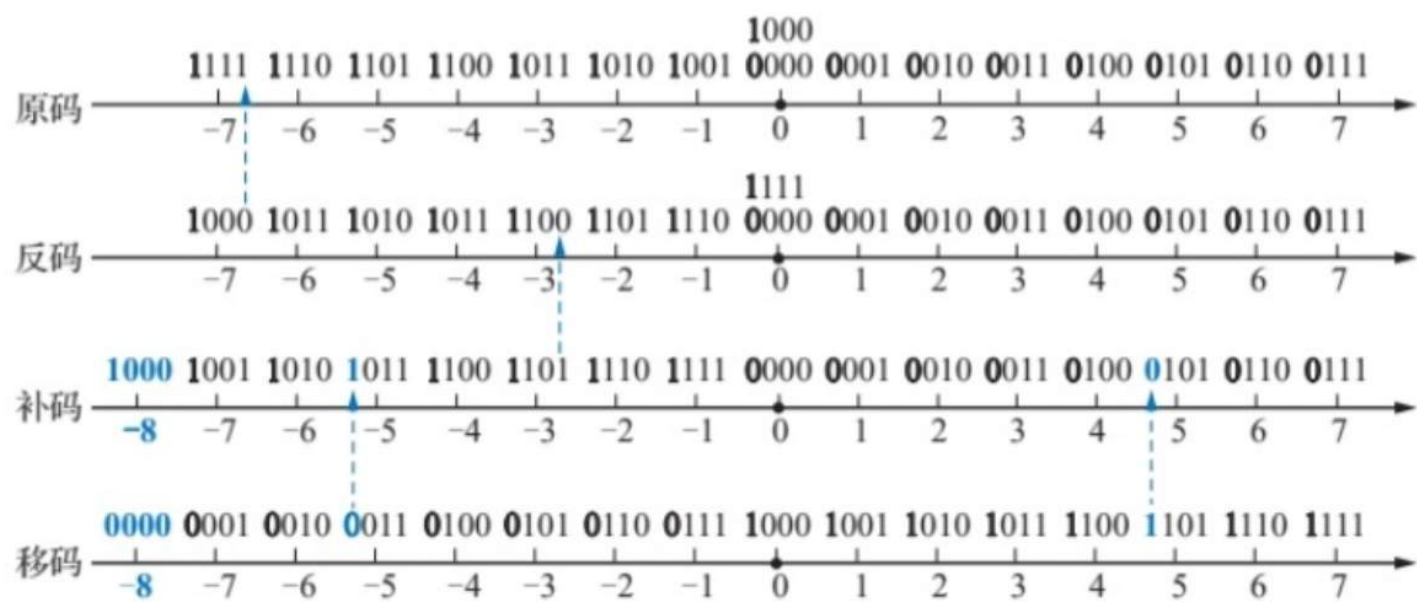


图 2.2 4 位不同机器码在数轴上的表示

- 原码

符号化的数值，正数符号位用0表示，负数用1表示

- 定点小数原码为

$$[x]_{\text{原}} = \begin{cases} x, & 0 \leq x < 1 \\ 1 + |x|, & -1 < x \leq 0 \end{cases}$$

💡 如-0.1111就会被表示为1.1111，0.1111表示为0.1111

- 定点整数略
- 0有两个编码，是符号位的差异

在字长为 $n + 1$ 的情况下，原码整数表示范围为 $-(2^n - 1) \leq x \leq 2^n - 1$ ，关于原点对称

优点在于：

- 与真值转换简便
- 易于实现乘除

缺点在于：

- 0 的表示不唯一，有正负0
- 难以实现加减运算

• 反码

原码转换到补码的中间表示形式

- 真值为正数时，反码和原码相同
- 真值为负数时，反码为真值原码数值位取反

• 补码

补码是针对模而言的，在计算机中通常指模2的补码

$$[x]_{\text{补}} = \begin{cases} 0, x, & 0 \leq x < 2^n \\ 2^{n+1} + x = 2^{n+1} - |x|, & -2^n \leq x < 0 \end{cases}$$

- 真值为正数时，补码和原码相同
- 真值为负数时，补码为真值反码加1，或是加上模数

补码的优势在于利用模使得符号位直接参与运算，将减法运算可以转化为加法运算

若字长为 $n + 1$ ，补码整数的表示范围为 $-2^n \leq x \leq 2^n - 1$

• 变形补码

用两个二进制位表示数据符号，00表示正数，11表示符号

$$[x]_{\text{变补}} = \begin{cases} 00, x, & 0 \leq x < 2^{n-1} \\ 2^{n+1} + x = 2^{n+1} - |x|, & -2^{n-1} \leq x < 0 \end{cases}$$

- 对定点小数而言，其模为4
- 对定点整数而言，其模为 2^{n+1}

变形补码易于判断是否存在溢出

- 01表示正溢出
- 10表示负溢出

• 移码

只用于**定点整数的表示**，常用于表示浮点数的阶码

移码公式一般为

$$[x]_{\text{移}} = x + bias$$

bias为偏移量，常为 2^n

- 移码的符号位0为负数，1为正数
- 移码与补码符号位相反，其他相同
- 移码中的0表示唯一，为100000...

在这几种表示方法中，

- 原码、反码的表示区间在数轴上对称，都存在两个
- 补码、移码的表示区间在数轴上不对称，0表示唯一，相对原码和反码多了一个数
- 原码、反码、补码符号位相同，正数机器码相同
- 负数的反码、补码末位相差
- 整数补码、移码符号位相反，数值位相同
- 移码保持原有大小顺序，可以直接比较大小

溢出

表 2.4 定点数表示范围

	定点小数			定点整数			
	原码	反码	补码	原码	反码	补码	移码
最大正数	0.111...11 (1-2 ⁻ⁿ)	0.111...11 (1-2 ⁻ⁿ)	0.111...11 (1-2 ⁻ⁿ)	0111...11 (2 ⁿ -1)	0111...11 (2 ⁿ -1)	0111...11 (2 ⁿ -1)	1111...11 (2 ⁿ -1)
最小正数	0.000...01 (2 ⁻ⁿ)	0.000...01 (2 ⁻ⁿ)	0.000...01 (2 ⁻ⁿ)	0000...01 (1)	0000...01 (1)	0000...01 (1)	1000...01 (1)
0	0.000...00 1.000...00	0.000...00 1.111...11	0.000...00	0000...00 1000...00	0000...00 1111...11	0000...00	1000...00
最大负数	1.000...01 (-2 ⁻ⁿ)	1.111...10 (-2 ⁻ⁿ)	1.111...11 (-2 ⁻ⁿ)	1000...01 (-1)	1111...10 (-1)	1111...11 (-1)	0111...11 (-1)
最小负数	1.111...11 (-1-2 ⁻ⁿ)	1.000...00 (-1-2 ⁻ⁿ)	1.000...00 -1	1111...11 (-2 ⁿ -1)	1000...00 (-2 ⁿ -1)	1000...00 (-2 ⁿ)	0000...00 (-2 ⁿ)

数据超出计算机所能表示的数据范围称为溢出

- 数据大于最大正数为正上溢
- 数据小于最小负数为负上溢
- 不在数轴刻度上的纯小数超出了定点小数所能表示的精度，这称为精度溢出

浮点数表示

浮点数是小数点位置不固定的数

任意一个二进制浮点数均可表示为

$$N = (-1)^S \times 2^E \times M$$

其中阶码 E 为定点整数，尾数 M 为定点小数

- 阶码位数决定数据表示的范围
- 阶码的值决定了小数点的位置
- 尾数的位数决定数据表示的精度
- 由于阶码和尾数可以采用不同机器码进行表示，其浮点数表示范围可能不同



图 2.6 浮点数数据格式

溢出

浮点数不仅存在最大正数和最小负数，由于浮点的原因还有最小正数和最大负数

因此浮点数会出现正上溢、负上溢、正下溢、负下溢、精度溢出

浮点数越往数轴左右两端，刻度越稀疏

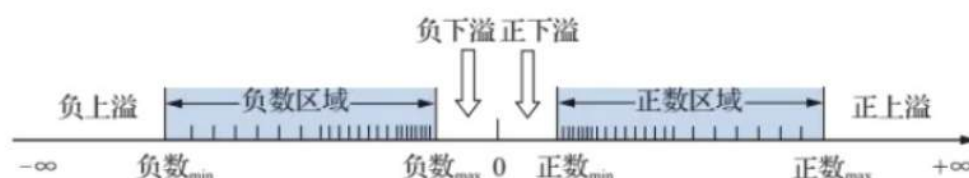


图 2.7 浮点数的表示范围

- 数据上溢时，计算机需要进行溢出处理
- 数据下溢时，直接当作 0 处理

浮点数的规格化

浮点数的规格化就是通过调整一个非规格化浮点数的尾数和阶码的大小，使非零浮点数在尾数的最高数位上保证是一个有效值

考虑同一个数 0.01111×2^{101} 和 0.11110×2^{100}

- **左规**：运算结果的尾数最高位不是有效位，令尾数左移一位、阶码减1。左规可能会进行多次。
- **右规**：运算结果的尾数的有效位在小数点前面，令尾数右移一位、阶码加1。右规只会进行一次。

两种方法都使得最高有效位为1，实际运算时会将1隐藏，这一位也称为**隐藏位**

IEEE754浮点数标准

主要包括32位单精度浮点数float和64位双精度浮点数double



图 2.8 IEEE754 32 位浮点数格式

- 单精度浮点数 float：32位，符号位S占1bit，指数E占8bit，尾数M占23bit，阶码偏移值127
- 双精度浮点数 double：64位，符号位S占1bit，指数E占11bit，尾数M占52bit，阶码偏移值1023

统一标准是为了限定阶码位数、尾码位数

以单精度浮点数为例

- 阶码E采用移码表示，偏移量为127.偏移量越大，移码最大值、最小值越小，非规格化精度越高，规格化数范围越小
- 尾数M为定点小数，小数点在M最左侧，且小数点左边有一位隐藏位，实际有效位为24位。完整形式位1.M。尾数可以看作原码表示

其具体意义为

表 2.6 IEEE754 单精度浮点数规范

符号位 S (1 位)	阶码 E (8 位)	尾数 M (23 位)	浮点数真值	说明
0/1	255	$M \neq 0$	NaN 非数	运算异常 (0/0)
0/1	255	0	$+\infty / -\infty$	正、负无穷
0/1	1 ~ 254	M	$(-1)^s \times 2^{E-127} \times 1.M$	规格化数
0/1	0	$M \neq 0$	$(-1)^s \times 2^{-126} \times 0.M$	非规格化
0/1	0	0	$+0 / -0$	两个机器 0

- 当阶码 $1 \leq E \leq 254$ 时，浮点数为规格化数

$$N = (-1)^s \times 2^{E-127} \times 1.M$$

- 当阶码 $E = 0, M \neq 0$ 时，浮点数为非规格化数

$$N = (-1)^s \times 2^{-126} \times 0.M$$

表 2.7 IEEE754 浮点数表示范围

格式	绝对值最小数	绝对值最大数
单精度规格化 $(-1)^s \times 1.M \times 2^{-127}$	$E_{\min}=1, M=0$ $f=1.0 \times 2^{-127}=2^{-126}$	$E_{\max}=254, M=1111\cdots11$ $f=2^{254-127} \times (2-2^{-23}) \approx +3.4 \times 10^{38}$
单精度非规格化 $(-1)^s \times 0.M \times 2^{-126}$	$E=0, M=0000\cdots01$ $f=2^{-23} \times 2^{-126}=2^{-149}$	$E=0, M=1111\cdots11$ $f=(1-2^{-23}) \times 2^{-126}$
双精度规格化 $(-1)^s \times 1.M \times 2^{-1023}$	$E_{\min}=1, M=0$ $f=1.0 \times 2^{-1023}=2^{-1022}$	$E_{\max}=2046, M=1111\cdots11$ $f=2^{2046-1023} \times (2-2^{-52}) \approx +1.8 \times 10^{308}$
双精度非规格化 $(-1)^s \times 0.M \times 2^{-1022}$	$E=0, M=0000\cdots01$ $f=2^{-52} \times 2^{-1022}=2^{-1074}$	$E=0, M=1111\cdots11$ $f=(1-2^{-52}) \times 2^{-1022}$

由于浮点数的舍入问题，小数和大数相加时的小数会被“吸收”，这导致浮点数的加法不满足结合律

十进制编码

十进制整数

- BCD码

用4位二进制数表示0-9这十个数，编码效率为10/16

- BID码

直接用二进制编码表示十进制整数的值，常用于十进制浮点数的尾数表示

- DPD码

利用十个二进制位表示3位十进制数，编码效率为1000/1024

DPD码可用于十进制浮点数的尾数表示

十进制浮点数

IEE754标准引入了十进制浮点数DFP编码，包括32位，64位，128位3种形式

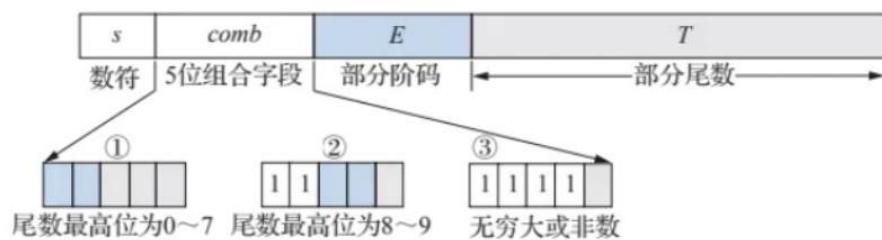


图 2.11 IEEE754-2008 中的十进制浮点数格式

十进制浮点数机器码的最高位为符号位S，comb字段为5位组合字段(包括部分阶码和部分尾数)，E为部分阶码字段，T为部分尾数字段

comb字段有三种格式

- 高两位为阶码E的最高有效位，低三位为尾数T的最高有效位(0-7)
- 高两位为11，后续两位为阶码E的最高有效位，最低位为0时表示T字段最高有效位为1000(8)，为1时表示为9
- 前四位为1，表示无穷大或非数

十进制浮点数的尾数不是定点小数，而是定点整数，小数点固定在最右侧

编程语言的数据类型

汇编语言

汇编语言中的数据类型取决于指令操作码，操作数本身没有数据类型

高级语言

以C语言为例，整型数据类型有char,short,int,long 4种，浮点数据类型有float,double 2种
4种整型数据的数据宽度为8位、16位、32位、64位，默认为有符号整数

表 2.12 整型变量的取值范围

值		char (8 位)	short (16 位)	int (32 位)	long (64 位)
无符号 最大值	机器码	0xFF	0x FFFF	0x FFFF FFFF	0x FFFF FFFF FFFF FFFF
	真值	255	65,535	4,294,967,295	18,446,744,073,709,551,615
有符号 最大值	机器码	0x7F	0x 7FFF	0x 7FFF FFFF	0x 7FFF FFFF FFFF FFFF
	真值	127	32,767	2,147,483,647	9,223,372,036,854,775,807
有符号 最小值	机器码	0x80	0x 8000	0x 8000 0000	0x 8000 0000 0000 0000
	真值	-128	-32,768	-2,147,483,648	-9,223,372,036,854,775,808
-1	机器码	0xFF	0x FFFF	0x FFFF FFFF	0x FFFF FFFF FFFF FFFF
0	机器码	0x00	0x 0000	0x 0000 0000	0x 0000 0000 0000 0000

• 整型数据转换

• 相同字长的整型数据转换

可以保持机器码不变，但值不一定相同

• 小字长转大字长

若原数据为无符号类型，进行零扩展；

否则进行符号扩展，高位部分用原数据符号位补充

• 大字长转小字长

编译器通常会直接将机器码截短处理

• 浮点数据转换

- 单精度浮点数：float (32位)，双精度浮点数：double (64位)
- 半精度浮点数：_Float16 (16位)，四精度浮点数：longdouble (128位)
- 十进制浮点数：_Decimal32 (32位)、_Decimal64 (64位)、_Decimal128 (128位)

int、float、double之间的转换关系

- float->double,int->double: 没问题
- double->float,double/float->int: 可能会溢出和精度丢失(舍入)
- int->float: 可能精度溢出(因为float数轴不均匀，int的数轴均匀)

表 2.13 数据类型转换实例

C 语言表达式	恒成立	原因
$i = (int)(float)i$	否	$int \rightarrow float$ 会发生精度溢出
$i = (int)(double)i$	是	int 是 $double$ 型的子集
$f = (float)(int)f$	否	小数会舍入
$f = (float)(double)f$	是	$float$ 型是 $double$ 型的子集
$d = (float)d$	否	$double \rightarrow float$ 会发生溢出或舍入
$f = -(-f)$	是	浮点数采用原码表示，单目运算的功能只是符号取反
$(d+f) - d = f$	否	浮点数不满足结合律，小数加大数的和可能还是大数

非数值数据的表示

字符表示

广泛采用ASCII码表示字符

表 2.14 ASCII 字符编码表

位数				$w_7 \sim 5$	000	001	010	011	100	101	110	111
w_4	w_3	w_2	w_1	行列	0	1	2	3	4	5	6	7
0	0	0	0	0	(NUL)	(DLE)	空格	0	@	P	`	p
0	0	0	1	1	(SOH)	(DC1)	!	1	A	Q	a	q
0	0	1	0	2	(STX)	(DC2)	"	2	B	R	b	r
0	0	1	1	3	(ETX)	(DC3)	#	3	C	S	c	s
0	1	0	0	4	(EOT)	(DC4)	\$	4	D	T	d	t
0	1	0	1	5	(ENQ)	(NAK)	%	5	E	U	e	u
0	1	1	0	6	(ACK)	(SYN)	&	6	F	V	f	v
0	1	1	1	7	(BEL)	(ETB)	'	7	G	W	g	w
1	0	0	0	8	(BS)	(CAN)	(	8	H	X	h	x
1	0	0	1	9	(HT)	(EM)	)	9	I	Y	i	y
1	0	1	0	A	(LF)	(SUB)	*	:	J	Z	j	z
1	0	1	1	B	(VT)	(ESC)	+	;	K	[	k	{
1	1	0	0	C	(FF)	(FS)	,	<	L	\	l	

汉字编码

由于汉字数量众多，汉字编码采用双字节编码

- GB2312

GB2312编码用2个字节表示汉字

而又为了与ASCII编码兼容并便于区分，双字节的最高位均为1，因此事实上用14位表示汉字

- **GBK**

不要求每个字节的MSB必须为1，可以表达更多汉字

- **区位码**

是国标码的另一种表示形式，有94行、94列

区位码+A0A0H=国标码

- **GB18030**

4字节的汉字编码，支持少数民族文字

- **UTF/Unicode**

为所有语言提供统一编码标准

汉字的处理包括

- **输入码**：外码，即用英文键盘输入汉字时的编码

- 流水码：国标码、区位码

- 音码：如拼音码（全拼、简拼、双拼等）

- 形码：如五笔字型码

- 音形码：如自然码、钱码

- **机内码**：是计算机内部存储、处理加工和传输汉字时所用的统一编码,如国标码等

- **输出码**：也称字形码，利用汉字字形点阵来表示字形，这些字形存放在汉字库中

大小端和对齐存储

大小端存储

在存储数据时，通常用最低有效字节 LSB 和最高有效字节 MSB 分别表示数据的低位、高位

而大端和小端则是将字节排列在内存中的不同方式，对同一个数01 23 45 67H 的存储如图所示

	0800H	0801H	0802H	0803H	
大端方式	...	01H	23H	45H	67H
	0800H	0801H	0802H	0803H	
小端方式	...	67H	45H	23H	01H

图 2.15 采用大端方式和小端方式存储数据

- **大端方式**：先存储高位字节，后存储低位字节。字中的字节顺序和原序列的**相同**
- **小端方式**：先存储低位字节，后存储高位字节。字中的字节顺序和原序列的**相反**。

对齐存储

现代计算机都是按字节编址的，对32位字长的计算机，数据按边界对齐方式存放要求其存储地址是自身大小的整数倍

如半字地址一定是2的整数倍，字地址一定是4的整数倍。这样，无论所取的数据是字节、半字还是字，均可一次访存取出。

当数据不按边界对齐方式存储时，半字长或字长的数据可能在两个存储字中，此时需要两次访存，并对高低字节的位置进行调整后才能得到所需数据，影响了系统的效率。

字节 1	字节 2	字节 3	填充
半字 1		半字 2	
半字 3		填充	
字 1			

图 2.16 按边界对齐方式存储

字节 1	字节 2	字节 3	半字 1-1
半字 1-2	半字 2		半字 3-1
半字 3-2	字 1-1		
字 1-2			

图 2.17 按边界不对齐方式存储

□

运算方法与运算器

定点运算

定点加减法

补码加减法

$$[x]_{\text{补}} + [y]_{\text{补}} = [x + y]_{\text{补}}$$

$$[x]_{\text{补}} - [y]_{\text{补}} = [x - y]_{\text{补}}$$

对定点小数和定点整数均适用

溢出及检测

溢出分为正溢和负溢

$$\begin{array}{rcl}
 (1) & [x]_{\text{补}} & 0.1011 \\
 & +[y]_{\text{补}} & 0.1100 \\
 \hline
 & [x+y]_{\text{补}} & 1.0111
 \end{array}
 \quad
 \begin{array}{rcl}
 (2) & [x]_{\text{补}} & 1.0101 \\
 & +[y]_{\text{补}} & 1.0100 \\
 \hline
 & [x+y]_{\text{补}} & \textcircled{1}0.1001
 \end{array}$$

左侧为正溢，右侧为负溢

检测溢出有下面的两种方法

- 根据操作数和运算结果符号位是否一致进行检测

因为只有两个同符号的数相加才会发生溢出(相减异符号)

设 X_f, Y_f 为操作数的符号位， S_f 为结果的符号位， V 为溢出标志位

检测的逻辑表达式为

$$V = X_f Y_f S_f + \bar{X}_f + \bar{Y}_f S_f$$

当 V 为"1"时表示有溢出，减法运算将操作数看作加法即可

- 根据运算过程中最高数据位进位与符号位进位是否一致进行检测

设运算时最高有效数据位进位为 C_d ，符号位产生进位为 C_f

溢出检测逻辑表达式为

$$V = C_f \oplus C_d$$

当V为"1"时表示有溢出

- 变形补码符号位进行检测

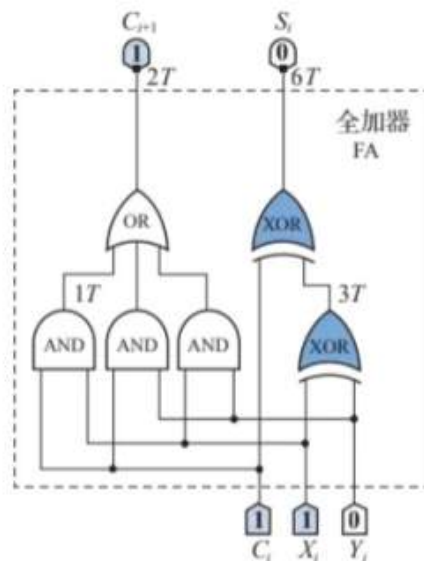
推荐手算

加减法的逻辑实现

实现加减法可以使用下面的逻辑电路

- 全加器

有三个输入和两个输出，输入端为相加数 X_i, Y_i ，低位进位输入 C_i



输出端为结果 S_i ，高位进位输出 C_{i+1} ，有

$$S_i = X_i \oplus Y_i \oplus C_i$$

$$C_{i+1} = X_i Y_i + (X_i + Y_i) C_i$$

和数的时间延迟为 $6T$ ，进位输出时间延迟为 $2T$

高位进位输出还可以表示为

$$i+1 = X_i Y_i + (X_i \oplus Y_i) C_i$$

但是这样进位输出时间延迟为 $5T$ ，因此不采用

i 半加器

半加器不考虑进位输入，但有进位输出.时间延迟为 $3T$

$$S_i = X_i \oplus Y_i$$

$$C_{i+1} = X_i Y_i$$

• 多位串行加法器

也称行波进位加法器，它将 n 个全加器的进位链串联

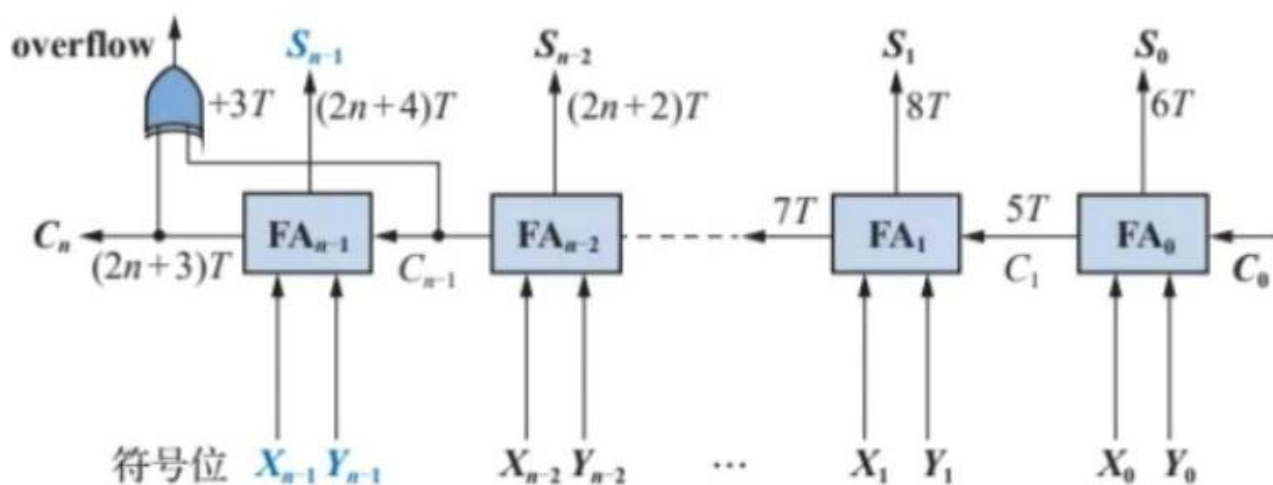


图 3.3 多位串行加法器逻辑框图

i 各个信号的时间延迟

第一次输入 X, Y 时，所有的位都会同步输入

因此在除 X_0, Y_0 的所有加法中都无需等待 X_i, Y_i 的输入并且可以略去两者通过与门的时间

所以只有第一次进位需要 $5T$ ，后面的每个进位都是 $2T$

- 溢出信号的时间延迟为 $(2n+6)T$

- S_{n-1} 的时间延迟为 $(2n+4)T$
- C_n 的时间延迟为 $(2n+3)T$

可控加减法电路

减法只需要对减数求补做加法即可，在多位串行加法器的基础上就可以得到减法

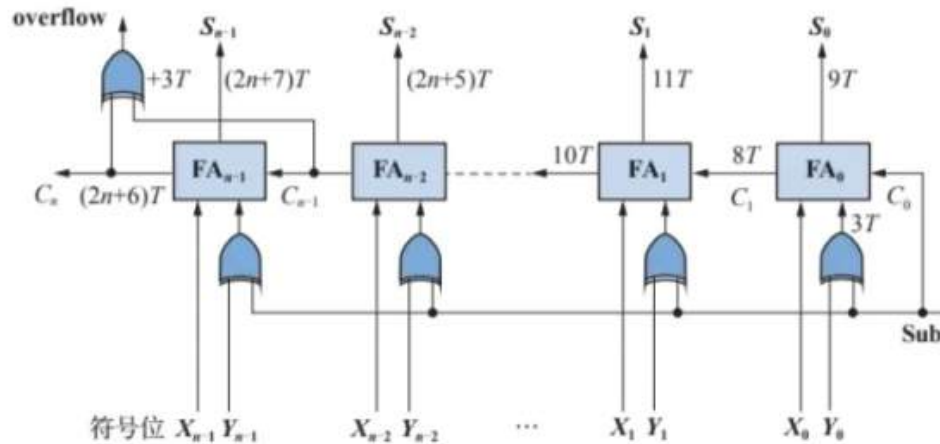


图 3.4 多位可控加减法电路逻辑框图

- 当SUB为0时，正常加法
- 当SUB为1时，SUB信号输入至减数的每一位通过异或门求反，求补中加1的操作通过给C0赋值为1实现

由于每一位多了异或门，加减法电路的所有延迟与多位串行加法器相比多了 $3T$

先行进位加法器

对全加器中

$$G_i = X_i \oplus Y_i \oplus C_i$$

$$C_{i+1} = X_i Y_i + (X_i + Y_i) C_i$$

我们设 $G_i = X_i Y_i$, $P_i = X_i \oplus Y_i$

- 由于当 $G_i = 1$ 时， $C_{i+1} = 1$ ，我们将 G_i 称为进位生成函数
- 当 $P_i = 1$ 时，才能传递进位输入信号到进位输出，因此 P_i 称为进位传递函数

那么先行进位加法器中就有

$$G_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i$$

展开进位输出的递推式可得

$$C_n = G_{n-1} + P_{n-1}G_{n-1} + P_{n-1}P_{n-1}G_{n-3} + \cdots + P_{n-1}P_{n-2} \cdots P_0 C_0$$

受到硬件条件的限制，先行进位的运算不能包括所有位，因此将数4位一组分组进行先行加法运算再串行

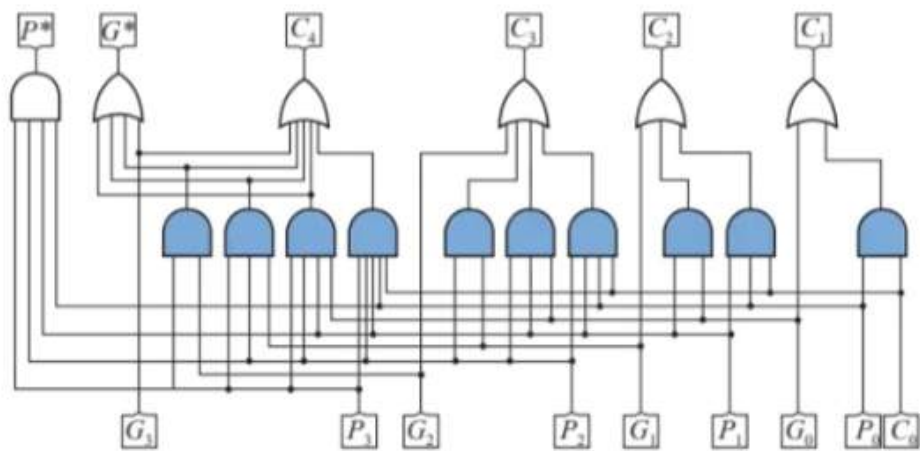


图 3.5 可级联的 4 位先行进位电路

- 并行加法器
 - 4位快速加法器

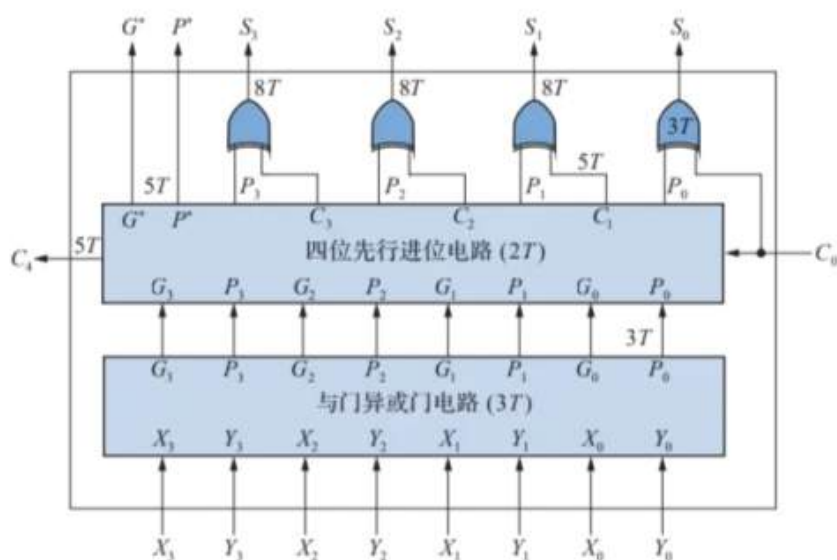


图 3.6 4 位快速加法器

关键延迟为和数信号的时间延迟 $14T$

- 16 位组内并行组间串行加法器

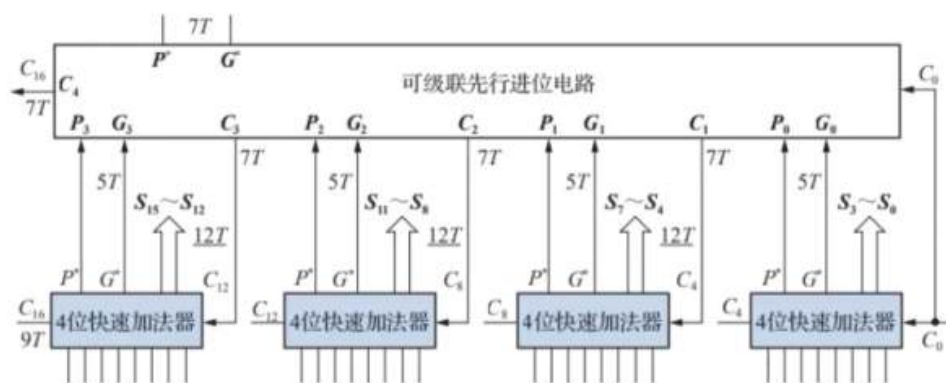


图 3.8 16 位组内并行、组间并行加法器

关键延迟为和数信号的时间延迟 $12T$

若进一步构成 64 位快速加法器就可以得到时间延迟为 $16T$ 的加法器

定点乘法

原码一位乘法

确定乘积符号只需要两数符号位异或得到，乘积数值为两者绝对值之积

设乘积为 P_i ，可得

$$P_0 = x_0 \oplus y_0$$

$$P_{i+1} = (P_i + y_{n-i}|x|)2^{-1}$$

例如

已知 $x = 0.1101$, $y = -0.1011$, 用原码一位乘法求 $x \times y$ 。

	部分积	乘数 $ y $	说明
	00.0000	1011	$P_0=1$
+	00.1101		$y_n=1, + x $
	00.1101	1011	
→	00.0110	1101	右移一位, 得 P_1
+	00.1101		$y_n=1, + x $
	0 1 .0011	1101	
→	00.1001	1110	右移一位, 得 P_2
+	00.0000		$y_n=0, +0$
	00.1001	1110	
→	00.0100	1111	右移一位, 得 P_3
+	00.1101		$y_n=1, + x $
	0 1 .0001	1111	
→	00.1000	1111	右移一位, 得 $P_4= x \times y $

其逻辑实现如下

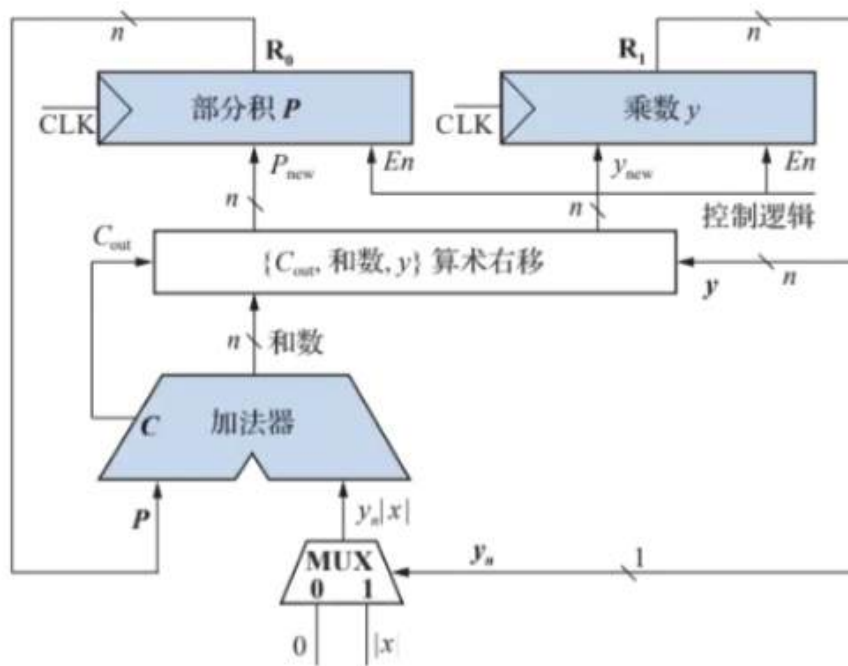


图 3.10 实现原码一位乘法的硬件逻辑

补码一位乘法

运用补码一位乘法就不需要将补码转换为原码再进行运算，即布斯算法

布斯算法对应的递归公式为

$$P_{i+1} = (P_i + (y_{n-i+2} - y_{n-i+1})[x]_{\text{补}})2^{-1}, \quad (i = 0, 1, 2, \dots, n-1, y_{n+1} = 0)$$

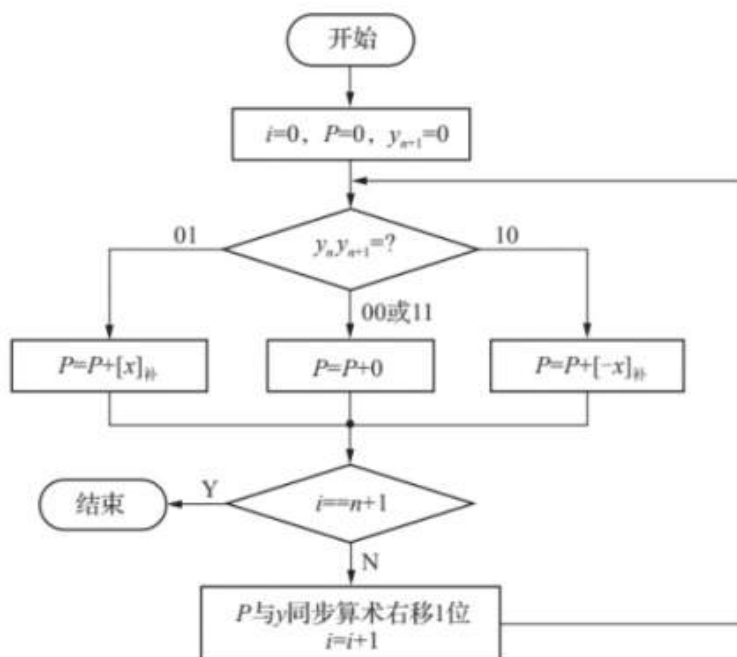


图 3.11 补码一位乘法算法流程图

- 布斯算法在乘数末尾补0，这一位记作附加位 y_{n+1}
- 乘数单符号位

例如

例 3.11 设 $[x]_{\text{补}} = 1.0111$, $[y]_{\text{补}} = 1.0011$, 求 $[x \times y]_{\text{补}}$ 。

解: $[-x]_{\text{补}} = 0.1001$

	部分积	乘数 y	说明
	00.0000	100110	$y_{n+1}=0$ $P_0=0$
+	00.1001		$y_n y_{n+1}=10$, $+[-x]_{\text{补}}$
	00.1001		
→	00.0100	110011	右移一位, 得 P_1
+	00.0000		$y_n y_{n+1}=11$, $+0$
	00.0100		
→	00.0010	011001	右移一位, 得 P_2
+	11.0111		$y_n y_{n+1}=01$, $+ [x]_{\text{补}}$
	11.1001		
→	11.1100	101100	右移一位, 得 P_3
+	00.0000		$y_n y_{n+1}=00$, $+0$
	11.1100		
→	11.1110	010110	右移一位, 得 P_4
+	00.1001		$y_n y_{n+1}=10$, $+ [-x]_{\text{补}}$
	00.0111	0101	最后一步数据不移位

所以 $[x \times y]_{\text{补}} = 0.0111\ 0101$ 。

其实现逻辑如下

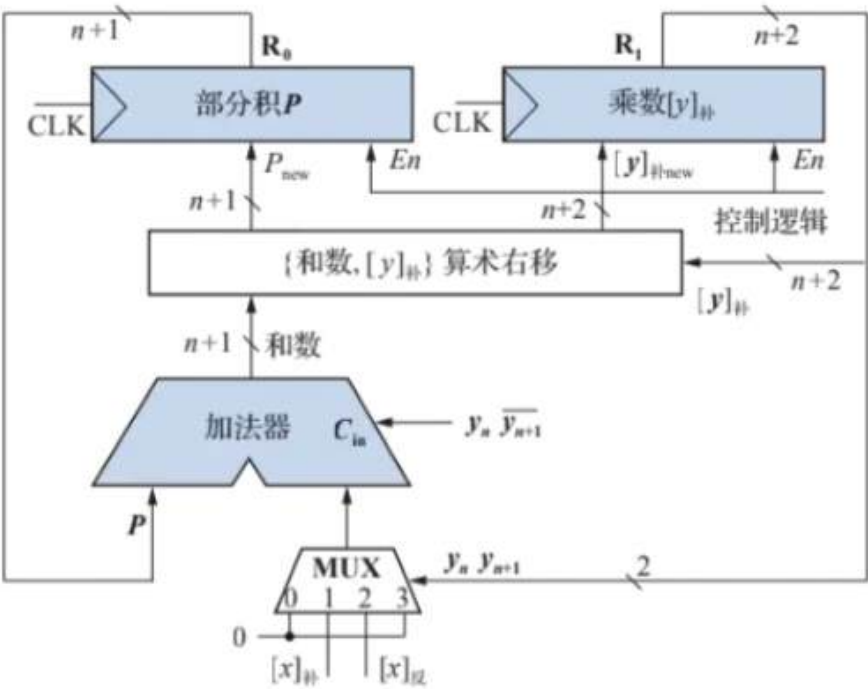


图 3.12 实现补码一位乘法的逻辑框图

阵列乘法器

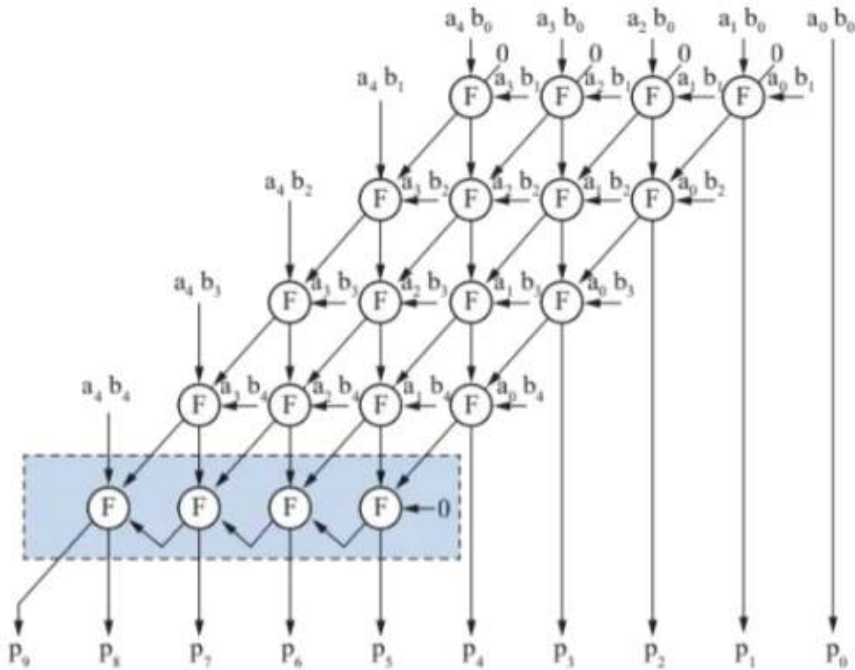


图 3.13 无符号阵列乘法器工作原理

总时间延迟为 $(8n-3)T$

补码阵列乘法器

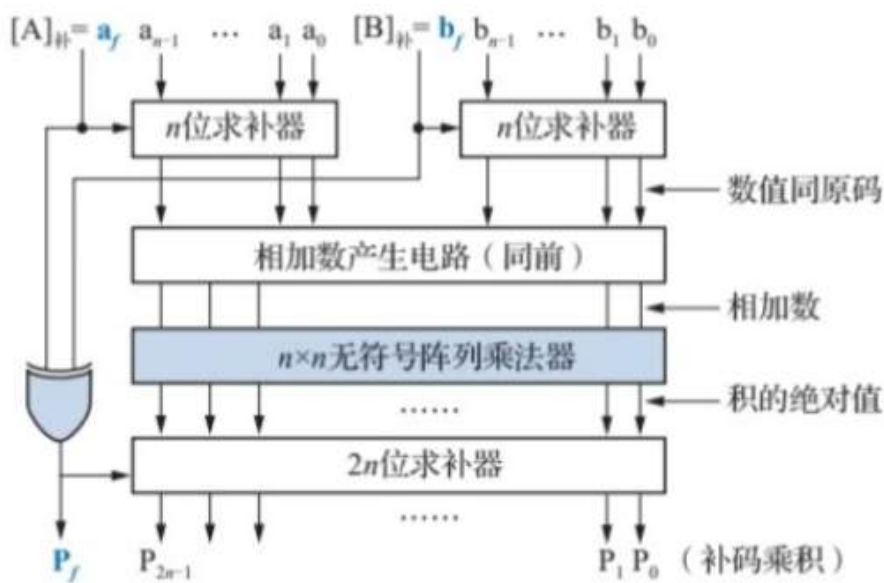


图 3.14 补码阵列乘法器的工作原理

定点除法运算

原码一位除法

设被除数为 x ，除数为 y ，商为 Q ，余数为 R

四者符号位均记为第0位，且 $|x| < |y|$

- 原码恢复余数法

运算时间不能确定，时间延迟取决于最慢除法，且控制电路也相对复杂

例 3.12 $[x]_{原}=0.1001$, $[y]_{原}=0.1011$, 求 $[x]_{原} \div [y]_{原}$ 。

解: $[y]_{补}=0.1011$, $[-y]_{补}=1.0101$ 。

	余数 R	商 Q	说明
	00.1001	0.0000	初始余数 $R_0=x$
$+[-y]_{补}$	11.0101		减 $ y $ 比较
	11.1110	0.0000	$R_1<0$, 商 q_1 上 0, $q=\bar{r}_0$
$+ y $	00.1011		加 $ y $ 恢复余数
	00.1001		
←	01.0010	0.000	R 、 Q 同步左移一位
$+[-y]_{补}$	11.0101		减 $ y $ 比较
	00.0111	0.0001	$R_2>0$, 商 q_2 上 1
←	00.1110	0.001	左移一位
$+[-y]_{补}$	11.0101		减 $ y $ 比较
	00.0011	0.0011	$R_3>0$, 商 q_3 上 1
←	00.0110	0.011	左移一位
$+[-y]_{补}$	11.0101		减 $ y $ 比较
	11.1011	0.0110	$R_4<0$, 商 q_4 上 0
$+ y $	00.1011		加 $ y $ 恢复余数
	00.0110		
←	00.1100	0.110	左移一位
$+[-y]_{补}$	11.0101		减 $ y $ 比较
	00.0001	0.1101	$R_5>0$, 商 q_5 上 1

故 $[x]_{原} \div [y]_{原}=[Q]_{原}=0.1101$, 余数 $=[R]_{原}=0.0001 \times 2^{-4}$ 。

- 原码不恢复余数法

例 3.13 $[x]_{\text{原}}=0.1001$, $[y]_{\text{原}}=0.1011$, 用不恢复余数法求 $[x]_{\text{原}} \div [y]_{\text{原}}$ 。

解: $[y]_{\text{补}}=0.1011$, $[-y]_{\text{补}}=1.0101$ 。

	C 余数 R	商 Q	说明
	00.1001	0.0000	初始余数 $R_0=x$
$+[-y]_{\text{补}}$	11.0101		减 $ y $ 比较
	0 11.1110	0.000 0	$R_1 < 0$, 商上 0 , $q = C = \bar{r}_0$
$\leftarrow$	11.1100	0.000	R 、 Q 同步左移一位
$+ y $	00.1011		加 $ y $ 比较
	1 00.0111	0.000 1	$R_2 > 0$, 商上 1
$\leftarrow$	00.1110	0.00 1	左移一位
$+[-y]_{\text{补}}$	11.0101		减 $ y $ 比较
	1 00.0011	0.00 11	$R_3 > 0$, 商上 1
$\leftarrow$	00.0110	0.0 11	左移一位
$+[-y]_{\text{补}}$	11.0101		减 $ y $ 比较
	0 11.1011	0.0 110	$R_4 < 0$, 商上 0
$\leftarrow$	11.0110	0.110	左移一位
$+ y $	00.1011		加 $ y $ 比较
	1 00.0001	0.1101	$R_5 > 0$, 商上 1

故 $[x]_{\text{原}} \div [y]_{\text{原}} = [Q]_{\text{原}} = 0.1101$, 余数 $= [R]_{\text{原}} = 0.0001 \times 2^{-4}$ 。

逻辑实现如下

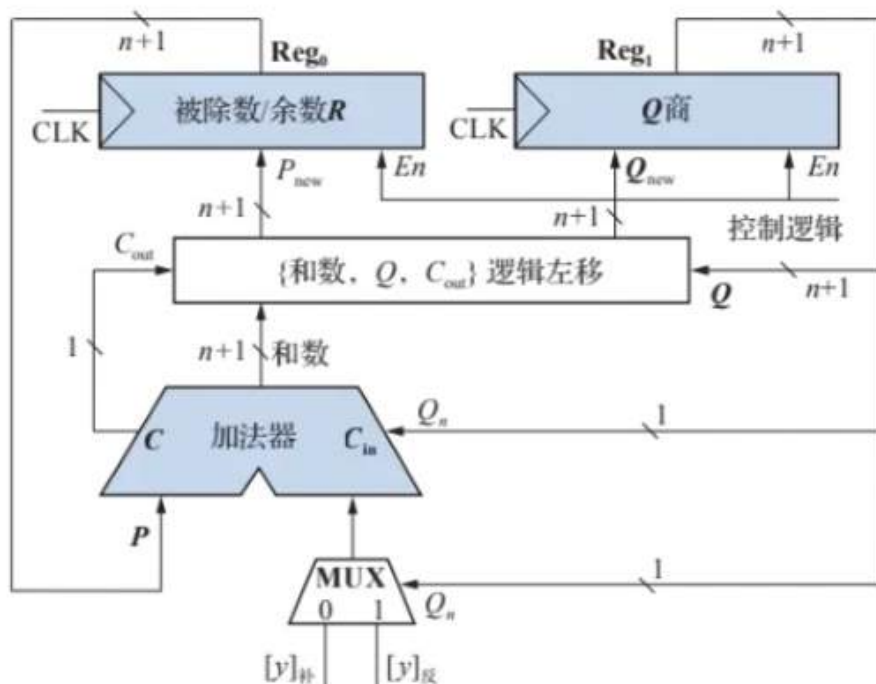
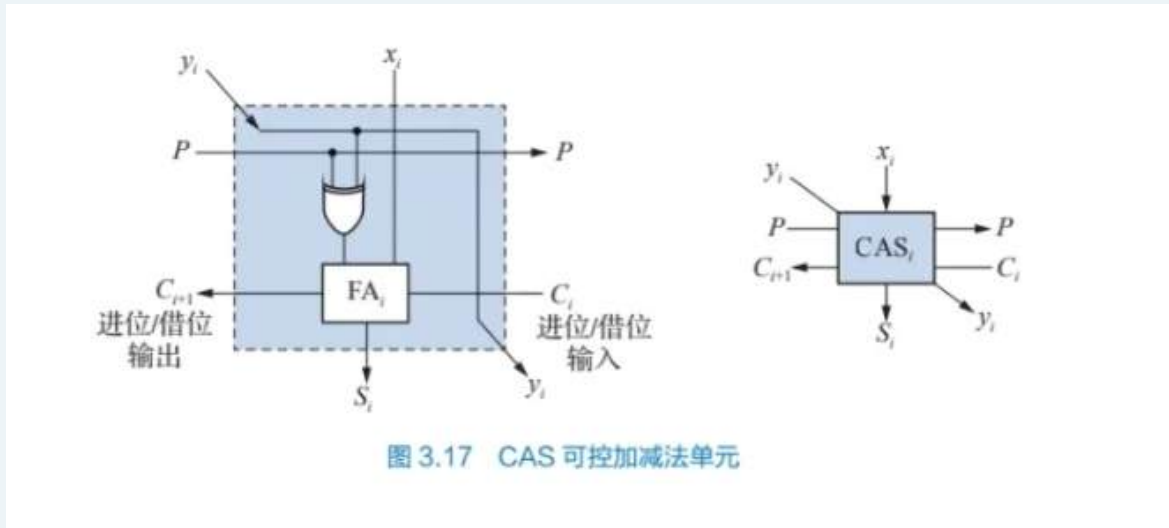


图 3.16 原码不恢复余数法的硬件逻辑框图

i CAS结构

CAS是可控加减法单元，分为4个输入端和4个输出端



- $P=1$ 时CAS执行加法
- $P=0$ 时执行减法
- 斜向输入和输出的 y_i 为除数
- 垂直输入的是被除数，垂直输出的是商，CAS单元输入和输出的关系可表示为

$$S_i = x_i \oplus (P \oplus y_i) \oplus C_i$$

$$C_{i+1} = (x_i + C_i)(P \oplus y_i) + x_i C_i$$

在CAS结构的基础上，我们可以设计阵列除法器的电路结构

图 3.18 所示为字长为 3 位的阵列除法器的基本结构，其中被除数 $x=0.x_1x_2x_3x_4x_5x_6$ （双字长），除数 $y=0.y_1y_2y_3$ ，商 $Q=0.q_1q_2q_3$ ，余数 $R=0.00r_3r_4r_5r_6$ 。

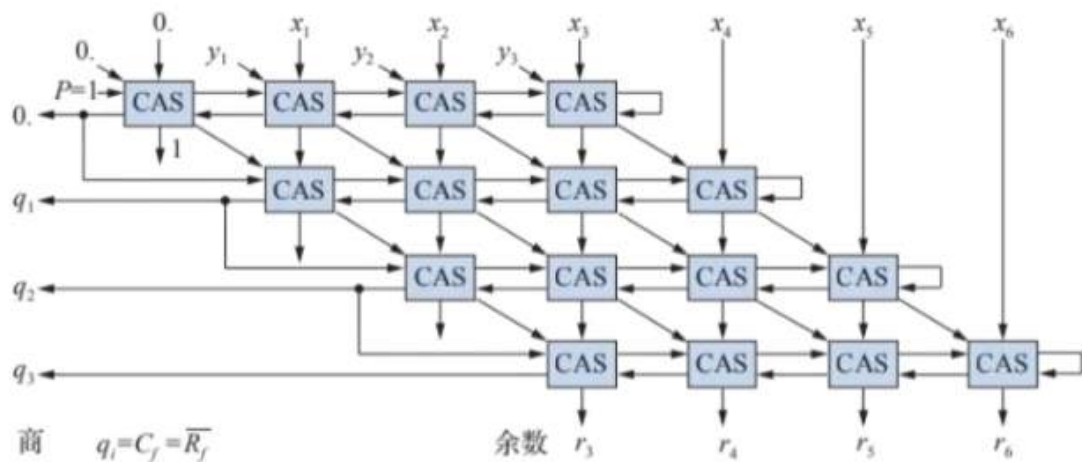


图 3.18 阵列除法器的基本结构

阵列除法器时间延迟为 $2n^2 + 11n + 9$ ，这也就是为什么高级语言中变量除常量常会被优化为移位操作

浮点运算

浮点数用规格化数进行运算

浮点加减法运算

1. 对阶

当两个浮点数阶数相同时，尾数部分直接运算即可得到浮点形式运算结果

对于两个阶码不相等的浮点数，首先要使得两个浮点数的阶码相等

原则是**小阶码向大阶码看齐**

- 1) 求两者阶数之差
- 2) 若阶差为正，则浮点数 y 尾数右移 $m - n$ 位；否则 x 右移 $n - m$ 位

尾数右移时通常要保留最低位的移出位以保证精度，称为**保留附加位**

在IEEE754中有三个保留附加位，从左到右为保护位、舍入位、粘位

舍入位右侧数据非0则粘位为1，否则为0

2. 尾数运算

对阶完成后按照定点数的补码加减运算**执行尾数加减操作**，但要记得先将隐藏位还原
阶码小的那个浮点数要用右移后的尾数运算

3. 结果规格化

可让尾数符号位扩展为双符号位

- 符号位为01或10，要向右规格化，尾数右移一位，结果的阶码值加1
- 运算结果为11.1.../00.0...时向左规格化，尾数连同符号位一起左移直至尾数部分为11.0.../00.1...

向左规格化阶码做减法，左移多少位减多少

4. 舍入

向右规格化时，尾数的末几位因超出计算机字长而被丢弃，需要舍入

- 末位恒置1法：只要移位使得有一位1丢失，运算结果最低位就恒为1
- 0舍1入法：丢失位数最高位为1时，尾数末位加1
- 就近舍入：舍入为最近可表示的数，如果数据正好处于两个可表示的中间则向偶数舍入
- 取左/右侧最近可表示的数

舍入后还需要再次规格化处理

5. 溢出判断

阶码符号位为01或10时，说明有溢出

浮点数的乘除

- 浮点数的乘法流程包括阶码相加、尾数相乘、规格化舍入
- 浮点数的除法流程包括尾数调整、阶码求差、尾数相除、规格化舍入

运算器

定点运算器

定点运算器包含算术逻辑运算单元(ALU)，通用寄存器，输入数据选择电路和输出数据控制电路

算术逻辑运算单元

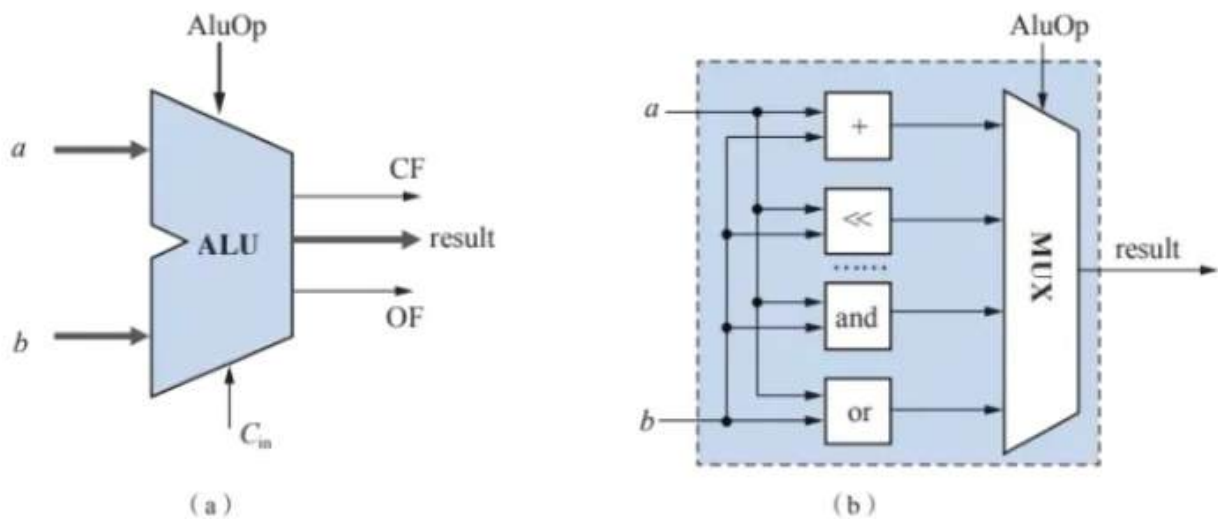


图 3.19 ALU 的封装及内部原理

- 输入：两个 n 位的操作数和进位输入 C_{in} ， $AluOp$ 为运算功能选择操作码，可用于选择运算功能

此处的 $AluOp$ 并非机器指令的操作码，而是编码电路得到的编码

- 输出：运算结果和若干状态标志位，状态标志位会存储在寄存器中

ALU作为**最基本的运算单元**，时延必须要短，因此通常会把乘除从ALU中剥离出来

通用寄存器

寄存器作用有

- 暂时存放运算结果，减少指令访问主存的次数
- 作为状态寄存器
- 作为变址寄存器、堆栈指示器

输入、输出数据选择控制

对送入ALU的数据和输出ALU的数据进行选择和控制，一般还有移位功能

可以将输出数据输送到ALU输入端，通用寄存器的通路，送往总线的控制电路

内部总线

内部总线是连接各个ALU、通用寄存器组、缓冲寄存器等部件的信息通道

运算器的基本结构

单总线结构运算器

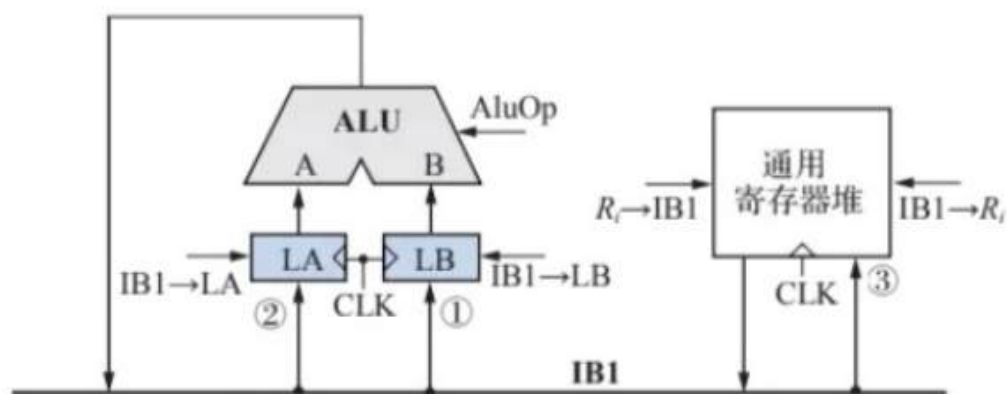


图 3.20 单总线结构运算器

一条总线上同时只能有一个数据传输，因此其运算流程为

1. 第一个时钟周期，将操作数A送入LA
2. 第二个时钟周期，将操作数B送入LB
3. 第三个时钟周期，给出运算选择控制信号AluOp和通用寄存器写回信号(将输出送回寄存器)

可见单总线结构运算器运算速度慢

双总线结构运算器

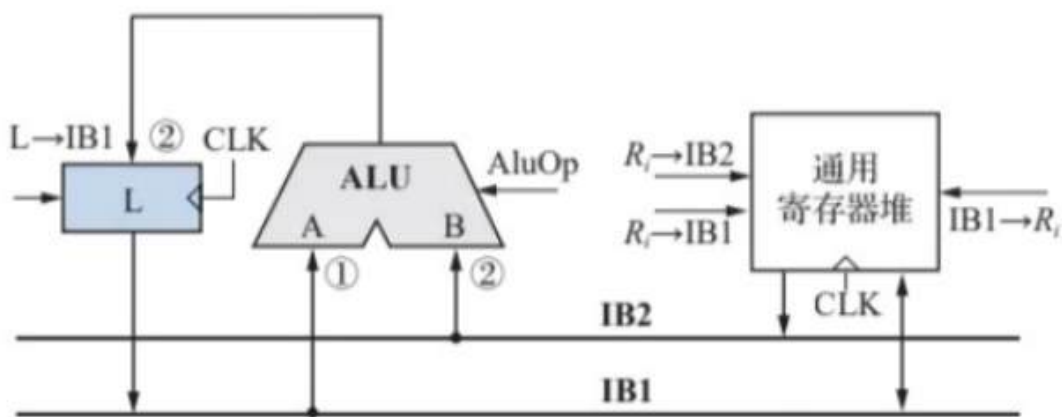


图 3.21 双总线结构运算器

1. 第一个时钟周期，取出操作数，分别走两条总线同时传入ALU，并输入AluOp得到结果存入L中
2. 第二个时钟周期，将寄存器L的数据写回到寄存器中

三总线结构运算器

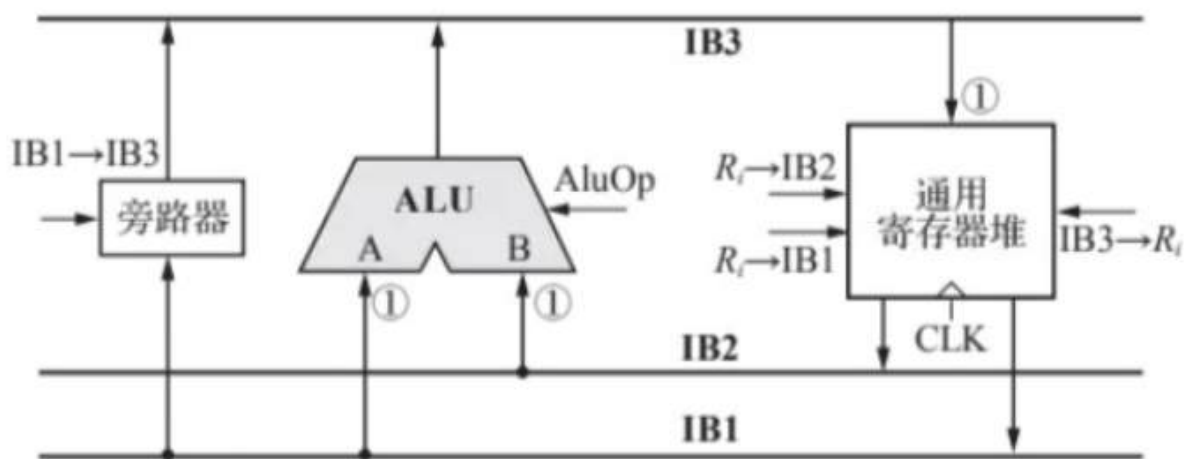


图 3.22 三总线结构运算器

一个时钟周期完成所有动作，但通用寄存器堆需要提供两个读端口，一个写端口

浮点运算器流水线

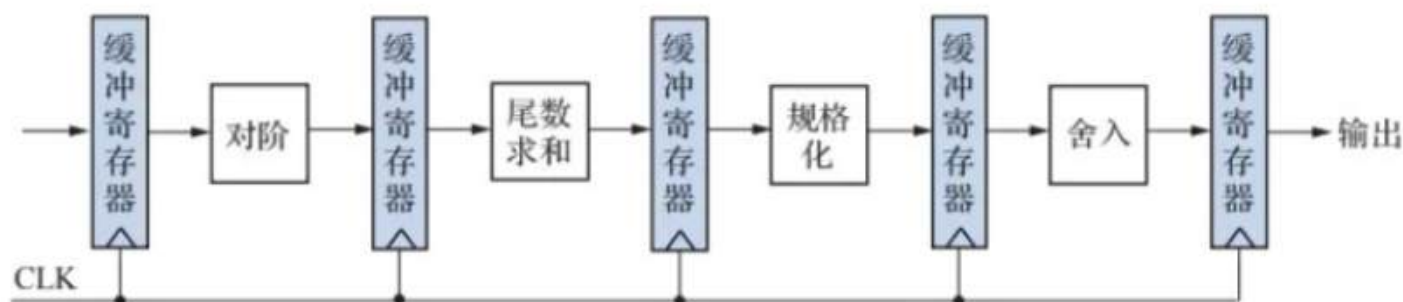


图 3.26 浮点加减法流水线框图

其中5个缓冲寄存器用于暂存各功能段的运算结果

每来一个时钟，所有寄存器同步锁存前端运算部件的运算结果，其输出为后续运算提供操作数

浮点流水线充满时，每隔一个时钟周期就可以完成一个浮点运算

存储系统

存储器是计算机中存放指令和数据的主要场所

存储器概述

分类

- 按存储介质
 - **磁存储器**：以磁性材料作为存储介质，如磁盘或磁带，存取慢但单位容量成本最低
 - **半导体存储器**：用半导体器件制作。分为双极型存储器(TTL/ECL)和金属氧化物半导体存储器(MOS)。体积小，存储速度快，但单位容量成本相对高
 - **光存储器**：利用介质光学特性读出数据。便于携带，成本低廉,如光盘
- 按存取方式
 - **随机存储器(RAM)**：按照地址随机读写数据存储单元，存取访问时间与存储单元位置无关。

包括早期的磁芯存储器和半导体存储器

- **顺序存储器(SAM)**: 存储单元中内容只能按地址顺序访问, 访问时间与存储位置有关, 如磁带
- **直接存储器(DAM)**: 不需要顺序搜索直接在存储器中读取信息, 兼有随机存储器和顺序存储器的访问特性, 如磁盘
- **按信息的可改写性**
 - **读写存储器**: 能读能写
 - **只读存储器(ROM)**: 只读不能写, 如半导体只读存储器和部分光盘
- **按信息的可保存性**
 - **易失性存储器**: 断电后保存信息会丢失, 如半导体RAM
 - **非易失性存储器**: 断电后保存信息不丢失, 如半导体ROM、闪存、磁盘、光盘
- **按功能和存取速度**
 - **寄存器存储器**: 由多个寄存器组成, 字长与计算机字长相同, 速度与CPU匹配, 容量小
 - **高速缓冲存储器**: 又称高速缓存cache, 隐藏在寄存器和主存之间的一个高速小容量存储器

常用于存放CPU即将或经常要用的指令或数据, 一般由静态RAM构成。用于缓冲CPU和慢速主存之间的性能差异, 加快存储系统访问
 - **主存储器**: CPU中除寄存器外唯一能直接访问的存储器, 用于存放指令和数据。

CPU通过主存地址直接、随机读写主存, 主存一般由半导体存储器构成。主存还包括BIOS、硬件端口等
 - **外存储器**: 也称为辅存。外存容量大, 存取速度低。

技术指标

- **存储容量**: 存储器可以存储的二进制信息总量
 - **位表示法**: 以存储器中存储单元总数与存储字位数乘积表示
 - **字节表示法**: 以存储器中单元总数表示
- **单位成本**: 即总成本除以总容量
- **存取速度**
 - **存取时间**: 存储器的访问时间, 指一次读写完成所经历的时间

- **存取周期**：连续两次访问之间最短的时间间隔，存取周期略大于存取时间
- **存储器带宽**：单位时间内存储器所能传输的信息量

! 存取时间不等于存取周期

通常存取周期大于存取时间，因为对任何一种存储器，在读/写操作之后，总要有一段恢复内部状态的复原时间

对于破坏性读出的存储器，存取周期往往比存取时间大得多，因为存储器中的信息读出后需要马上进行再生。

存储系统的层次结构

对于存储系统，有三个互相制约的因素：容量、速度、成本

为此在计算机系统中通常使用多级存储器结构

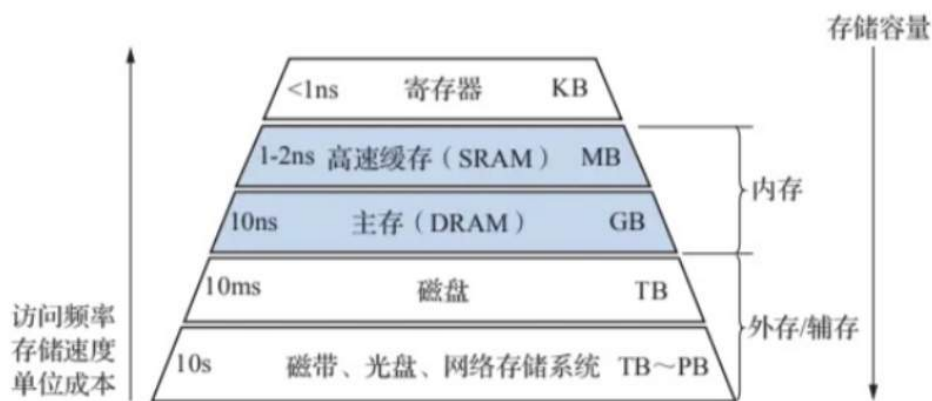


图 4.1 存储系统的分级结构

- 越向上离CPU越近，访问速度越快，单位容量成本越高
- 越向下存储容量越大

存储器层次结构的主要思想是上一层的存储器作为低一层存储器的高速缓存

由于程序访问的局限，上层存储器可以为下层存储器做缓冲，将最常使用的数据调度到上层，有效提高系统访问速度

主存

SRAM 与 DRAM

SRAM

存储体以**静态MOS(场效应管)存储元**为基本单元组成的存储器称为**静态MOS存储器(SRAM)**

存储元是存储器中的**最小存储单位**，存储1位信息

6 管 SRAM 存储单元如图所示

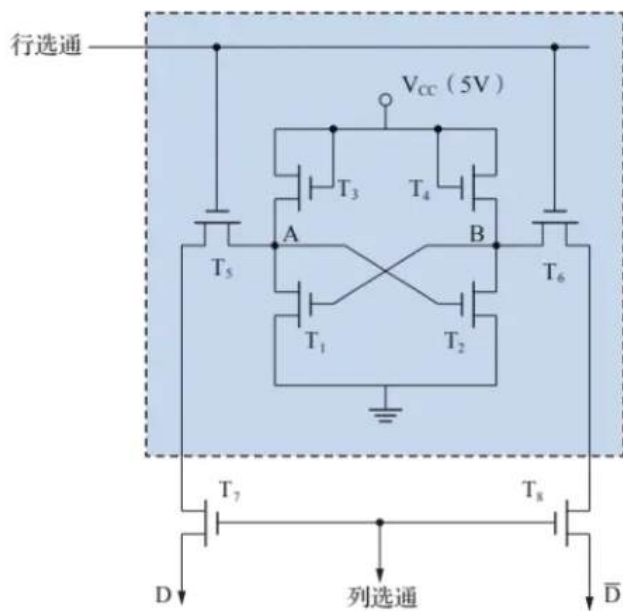


图 4.6 6 管 SRAM 存储单元

- T1，T2为工作管。静态存储单元由T1和T2组成的双稳态触发器存储1位二进制信息
- T3，T4为负载管
- T5，T6，T7，T8为门控管，相当于开关。T7和T8为各个存储单元共用

存储单元的存储靠电路状态表示

- A点高电平，B点低电平，表示存储数据1，T1导通，T2截止
- A点低电平，B点高电平，表示存储数据0，T2导通，T1截止

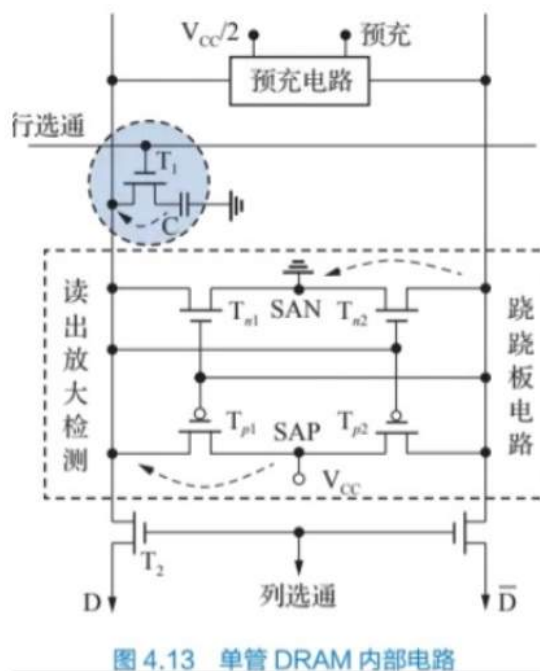
SRAM的**存取速度快**，但**集成度低**，**功耗较大**，**价格昂贵**，一般用于高速缓冲存储器

DRAM

存储体以**动态MOS存储元**为基本单元组成的存储器称为**动态MOS存储器**，也称为**动态随机存取存储器**

- 静态MOS存储器（SRAM）存储1位二进制数需要6个MOS管，其**存储密度较低**
 - 动态MOS存储器（DRAM）则采用数量较少的MOS管（如4个MOS管、3个MOS管、1个MOS管）和存储电容来存储1位二进制数
- 其**存储密度较高**，且去掉了MOS管中的功耗管

单管动态MOS存储单元如图所示，仅用一个MOS管和一个电容构成存储单元



用电容的有无电荷表示数据，有则为1，无则位0

该存储单元的特点有

- 单管DRAM的**读写速度比SRAM慢**，因为电容的**充放电过程较慢**
 - 单管DRAM读出时，会引起**电容电荷的减少**，从而**破坏原有数据**
- 为避免数据丢失，在读出后需要将数据重新写入，该过程称为数据恢复（再生）
- 各个操作的工作原理为
- **读**：行选通和列选通信号使T1与T2导通
 - 如果电容C上有电荷（表示存放的是“1”），则使输出D为高电平（表示读出“1”）
 - 如果电容C上无电荷（表示存放的是“0”），则使输出D为低电平（表示读出“0”）
 - **写**：行选通和列选通信号使T1与T2导通

- 如果是写“1”，则输入D上的高电平将对电容C充电（表示写入“1”）
- 如果是写“0”，则输入D上的低电平将对电容C放电（表示写入“0”）

• 刷新

因为电容上的电荷会逐渐泄露，数据只能保存较短的时间

为避免数据丢失，必须定期采用类似读操作的方式对存储单元补充电荷，这个过程称为刷新

两次完整刷新之间的时间间隔被称为**刷新周期**，可能为2/4/8ms，信息存储到数据丢失之前的时间称为**最大刷新周期**

DRAM的刷新按行进行，因此**减少刷新周期可以减少存储矩阵的行数**

🔔 读操作也有**刷新功能**，但刷新只需要给出行地址，读操作要给出**行地址和列地址**

! 内存争用

DRAM刷新由内存控制器完成，此时CPU不能访问存储器，称为内存争用（也称死区）
为此出现了不同的刷新方式

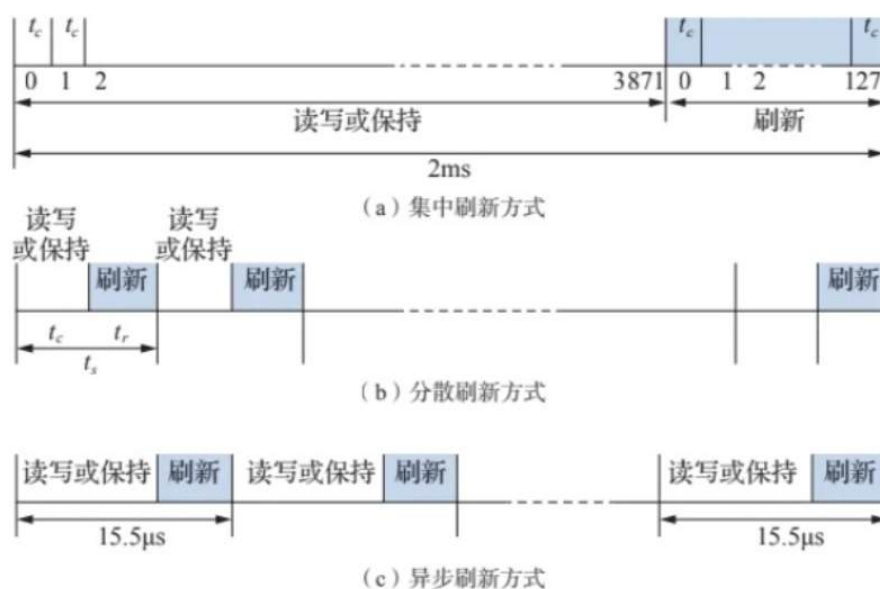


图 4.14 刷新时间分配

设DRAM为128行，128列，读写周期为0.5微秒，刷新间隔为2ms

• 集中刷新

读写操作期间不受到刷新操作的影响，存储较快，但死区很长

- **分散刷新**

将每一行的刷新分散到每一个周期中，这种刷新不存在死区(在存储周期中完成刷新)，但存储周期增加了

而且，每128微秒，所有行都会被刷新一次，则在一个刷新周期内所有行都要被刷新15次，而这是不必要的

- **异步刷新**

在2ms间刷新完128行，也就是在15.5微秒刷新一次即可。那么只需在15.5微秒的最后一个0.5微秒进行刷新即可

是集中刷新和分散刷新的结合，常用的刷新方式。

既没有增加存储周期，也没有过长时间的死区

DRAM的发展历史如下

1. **传统DRAM**

先送行地址，再送列地址选择存储单元，工作方式低下

2. **SDRAM**

同步DRAM，自带时钟控制信号，与系统总线频率同步。第1个列地址就绪后，每个时钟周期均可得到一个数据

前面的DRAM均为异步DRAM，本身没有时钟信号，时序控制、地址控制、数据访问均由内存控制器以异步方式控制实现

3. **DDR SDRAM**

双倍数据速率同步DRAM，采用2路预取技术，在时钟周期的上升沿和下降沿分别进行数据传输

即每个时钟周期可以传输2个数据，其速度是SDRAM的2倍

MOS的结构

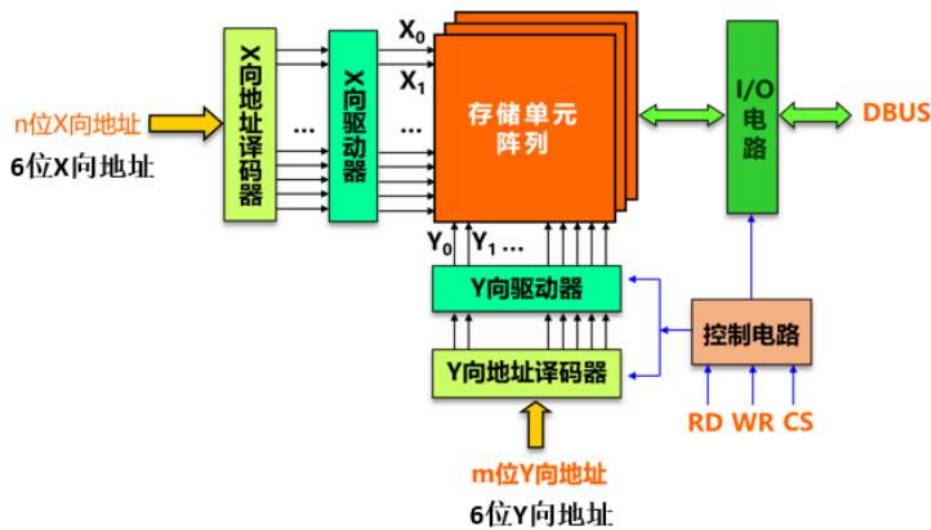
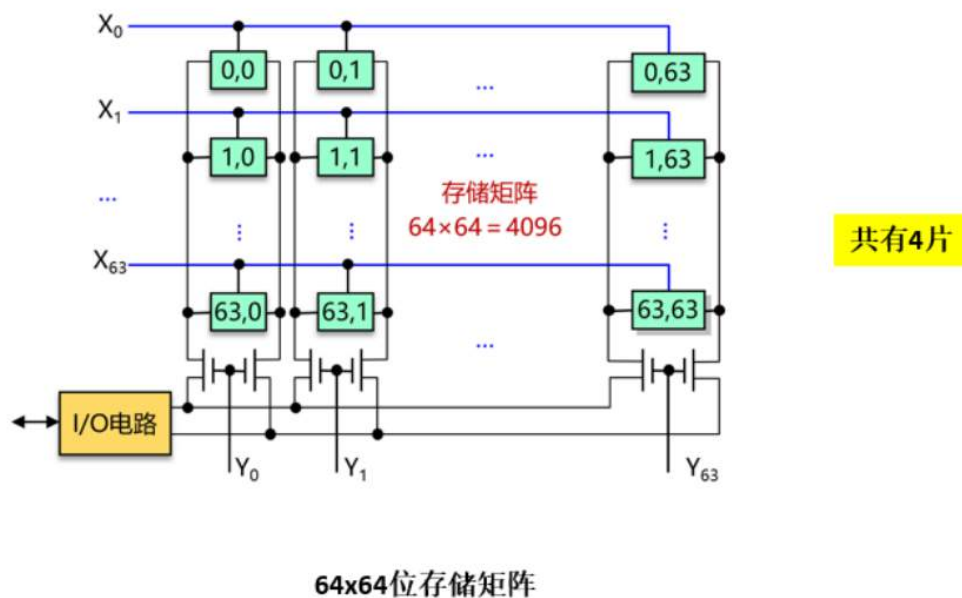


图4.10 4096x4位的静态MOS存储器结构框图

- **存储体(存储单元序列)**：存储单元的集合，由行选择线 (X) 和列选择线 (Y) 选择所访问的单元



- **地址译码器**：包括X向地址译码器和Y向地址译码器，两者均为6-64译码器，6个输入64个输出

- **驱动器**：包括X向驱动器和Y向驱动器，驱动器输入和输出逻辑上相等

当行译码输出信号时，这一行上所有位存储体的T5, T6门控管都要被驱动，4个存储阵列并发，会有更多的门控管需要被驱动，负载过大

因此需要驱动器

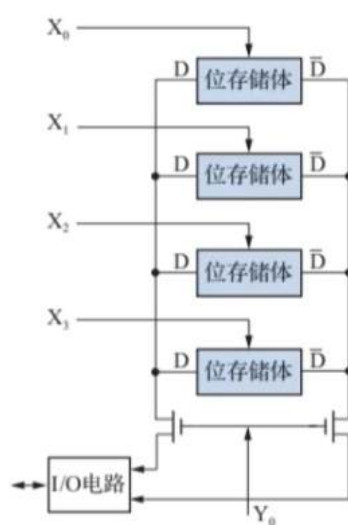
- **I/O电路**：用来控制数据的输入和输出，在存储单元和数据总线间

- 片选和读写控制电路

- CS: 片选信号，只有CS信号有效的芯片才能进行读写操作
- WR: 写信号
- RD: 读信号

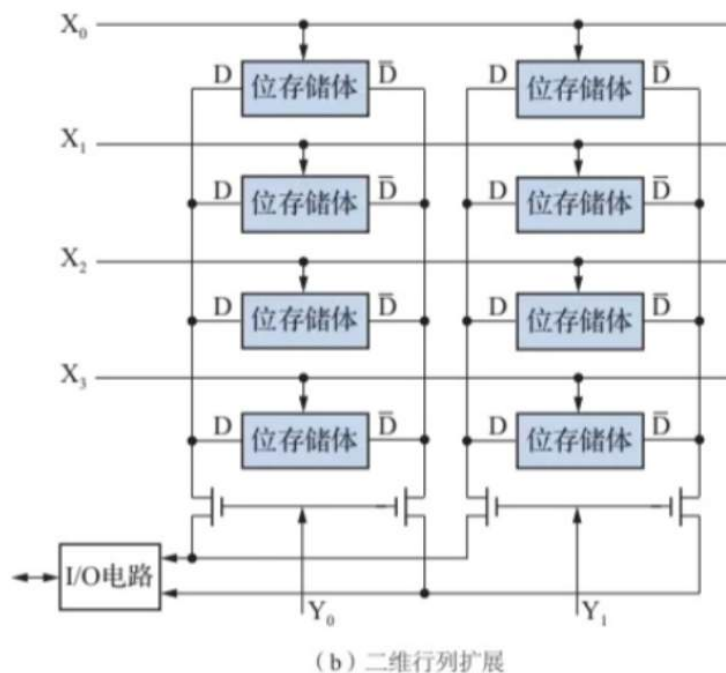
存储单元扩展

将6管 SRAM 存储单元的位线并联到一起，并各自连接不同的行选通地址，就可以得到一个4位存储单元



(a) 一维行扩展

这也就是行扩展，图a所示的存储体存放4位二进制数
也可以进行列扩展



图b所示的存储体存放8位二进制数

为了同时访问多位数据，只需要将多个一位存储体的地址线并联即可，这就是字长扩展
同一时刻最多有一根行选通/列选通信号为高电平，保证只选中一行存储单元并输出到位线
一维扩展和二维扩展也各自对应了两种译码结构

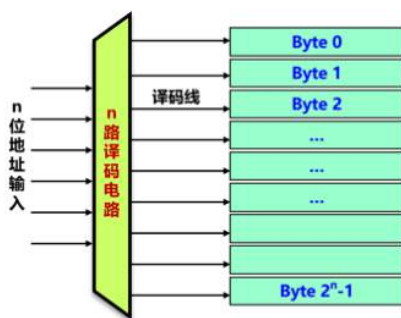


图4.9a 单译码结构

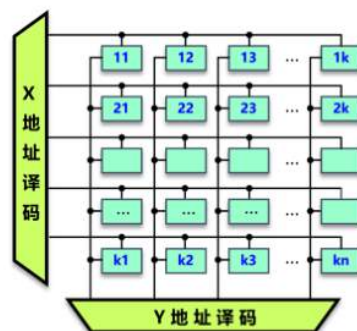


图4.9b 双译码结构

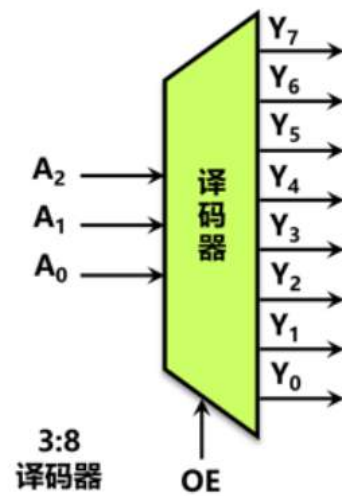
- 单译码结构：如左图所示， n 位地址译码后得到 2^n 个译码线，共有 2^n 个存储单元
- 双译码结构：如右图所示，X地址译码后得到 k 行，Y地址译码后得到 n 列，共有 kn 个存储单元。这是普遍采用的结构，也就是**地址引脚复用技术**

i 译码器

- 输入：地址的各位，OE(控制是否允许输出)

输出：选中的地址线

输入				输出							
A2	A1	A0	OE	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
0	0	0	1	0	0	0	0	0	0	0	1
0	0	1	1	0	0	0	0	0	0	1	0
0	1	0	1	0	0	0	0	0	1	0	0
0	1	1	1	0	0	0	0	1	0	0	0
1	0	0	1	0	0	0	1	0	0	0	0
1	0	1	1	0	0	1	0	0	0	0	0
1	1	0	1	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0
x	x	x	0	x	x	x	x	x	x	x	x



3-8译码器的输入输出关系

只读存储器ROM

ROM 常用于存放一些不需要修改的程序，如微程序、系统软件（ROM-BIOS）等

其优点在于结构简单、存储密度高，且具有非易失性

- **掩模式只读存储器(MROM)**: MROM内容在厂家生产时就按要求做好，被固定的信息不能再改变，灵活性差，但可靠性高
- **可编程只读存储器(PROM)**: 用户可写一次的ROM，但一旦写入内容无法改变
- **可擦除可编程只读器(EPROM)**: 写入信息可长期保存，可以擦除信息重写，但编程次数有限，写入时间长
- **电可擦除可编程只读器(EEPROM)**: 写入方式与EPROM相同，但擦除时可以精准地删除某一存储单元
- **闪存(Flash)**: 闪存是一种快速擦写、非易失性存储器，可以在线进行擦除和重写，其逻辑结构与EEPROM相似。常见的U盘为这种存储单元
- **固态硬盘 (SSD)**: 基于闪存实现。由控制单元和存储单元组成，保留了 Flash 存储器长期保存信息、快速擦除与重写的特性。

主存的基本结构

主存是指令直接操作的存储器，用主存地址进行随机访问

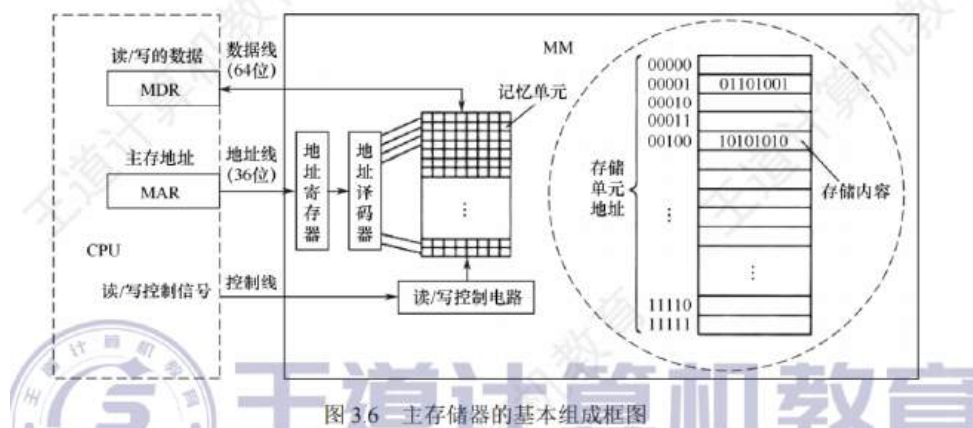


图 3.6 主存储器的基本组成框图

主存由**存储体**和**外围电路**组成，外围电路包括**地址译码器**、**数据寄存器**、**读写控制电路**

- **地址译码器**接收来自CPU的 n 位地址信号，经译码、驱动后形成 2^n 根**地址译码信号**
每根地址译码信号连接一个**存储单元**，对于给定的地址，只有一个**存储单元**会被选中
- **数据寄存器**暂存CPU的 m 位数据或存储体中的 m 位数据
- **读写控制电路**接收CPU的读写控制信号后产生存储器内部的控制信号
- 在这里，CPU 内部的地址寄存器位数与地址线位数相同，CPU 内部的数据寄存器位数与数据线位数相同

主存中数据的存放

存储字长与数据字长

- **存储字长**：主存的一个**存储单元**所存储的二进制位数
- **数据字长(简称字长)**：计算机一次能处理的二进制数的位数

两者不一定相同

地址访问模式

地址访问模式可以分为**字节地址**、**半字地址**、**字地址**

X86ASM

MOV AH,[8];按字节AH=12H

MOV AX,[8];按半字AH=3412H

```
MOV EAX,[8];按字AH=78563412H
```

字节地址逻辑右移一位即可得到半字地址，右移两位可得到字地址

可见指令中用字节地址代表半字地址和字地址

💡 在上例中，实际上的半字地址是4，字地址是2

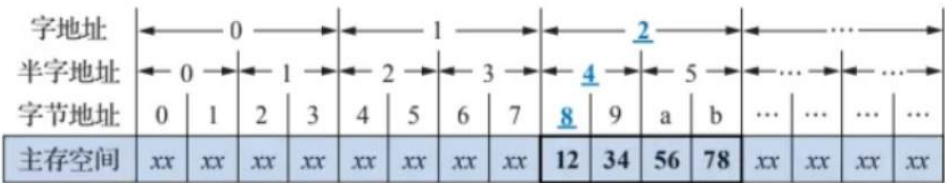


图 4.3 不同的主存地址访问模式

并行存储系统

双端口存储器

双端口存储器是指同一个存储器具有两组相互独立的端口

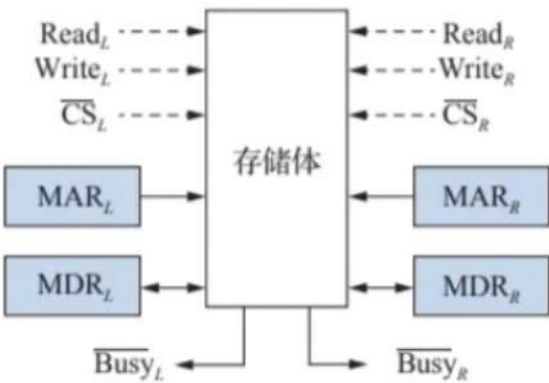


图 4.27 双端口存储器结构示意图

每个端口均有各自独立的数据端口、地址端口、读写控制端口、片选端口，每个端口均可独立读写操作

并行读写

当左右两个端口地址不同时，两个端口使用各自的地址线、数据线和控制线

此时可以同时存储体的两个不同的存储单元进行访问（读写）

冲突处理

当左右两个端口地址相同时就会发生冲突

此时先访问存储体的端口优先访问，后访问的端口/Busy置为低电平(不能访问存储体)

只有在前一个端口访问结束后，后访问端口的/Busy置为高电平(可以访问)

多模块存储器

主存速度慢于CPU，因此需要通过一些方法提升主存的速度

存储体的并行工作就提高了存储器的访问速度

单体多字存储器

多个存储模块**共享地址总线**，实现同时访问多个存储模块

💡 存储器中只有一个存储体，每个单元存储 m 个字，总线宽度也为 m 个字。
一次并行读出 m 个字，地址必须顺序排列并处于同一存储单元。

但指令和数据在主存内必须是**连续存放**的，一旦遇到**转移指令**，或操作数**不能连续存放**，这种方法的效果就不明显

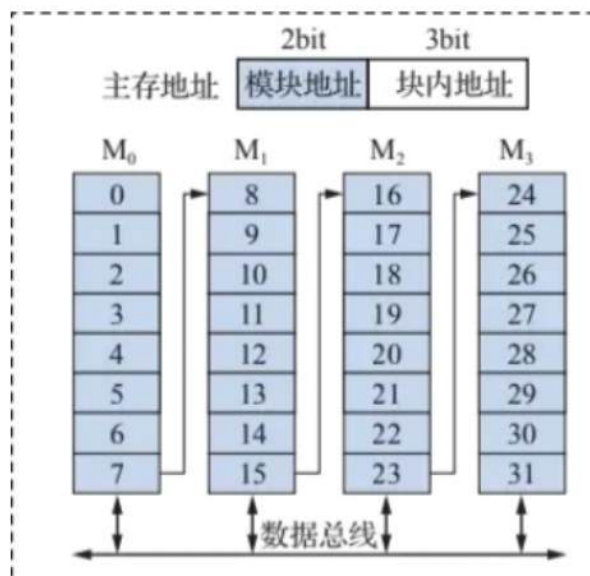
多通道内存技术

采用**单体多字技术**，常见的有双通道、三通道、四通道技术

高位多体交叉

多体并行存储器由多体模块组成。每个模块都有相同的容量和存取速度，各模块都有独立的读/写控制电路、地址寄存器和数据寄存器

它们既能并行工作，又能交叉工作。



计算机的加装内存就是这种形式

和字扩展完全相同，高位地址译码产生片选信号选择存储模块，低位地址译码选择一个存储模块内的存储单元

不同存储模块对应不同的地址区间，按顺序为模块分配地址。因此高位多体交叉也称为顺序编址模式

- 相邻的地址在同一存储体内
- 不同存储体中的地址不相邻
- 无法实现多个存储体并行工作

由于程序的指令和数据往往分布在同一个存储体中，从而导致一个存储体被频繁访问，其它存储体基本处于空闲状态

低位多体交叉



和字扩展基本相同，但低位地址用于选择存储体，高位地址送至存储体内

低位多体交叉也称为交叉编址

- 相邻地址往往在不同的存储体内
- 同一存储体中地址不相邻
- 可以实现存储体的并行工作

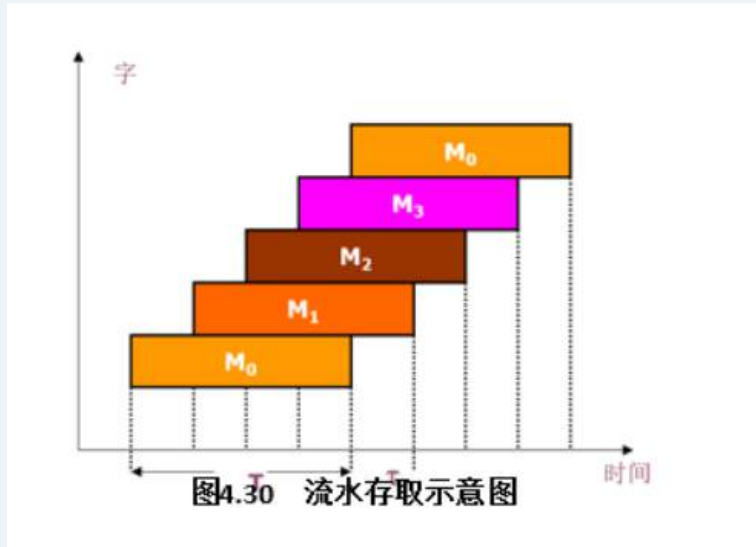
因为程序指令数据等在连续的地址中，而连续地址在不同存储体中

i 交叉存储的流水存取

设存储模块存储周期为 T ，总线传输周期及相应的处理时间延迟总和为 τ ，交叉模块数为 m ，要实现流水方式存取必须满足 $T = m\tau$

- 每个存储单体的存储周期仍然为 T
- 各个存储体错开 τ 时间分时启动读写操作，每个存储周期可以访问内存 m 次

因此连续读取 n 个字所需时间为 $T + (n - 1)\tau$



可见交叉存储的访问快于顺序存储

主存的组织及与CPU的连接

连接原理

- 主存储器通过**数据总线**、**地址总线**和**控制总线**与CPU连接
- 数据总线的位数与工作频率的乘积正比于**数据传输速率**
- 地址总线的位数决定了可寻址的**最大内存空间**

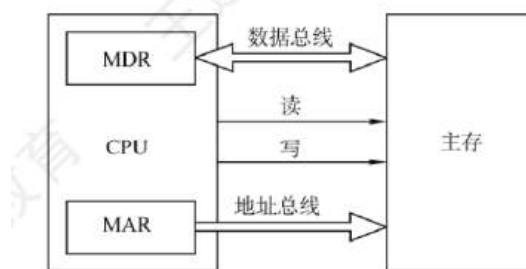


图 3.10 主存储器与 CPU 的连接

存储器的扩展

单个芯片的容量是有限的，因此通过存储器芯片扩展技术，将多个芯片集成在一个内存条上，然后由多个内存条及主板上的ROM芯片组成计算机所需的主存空间，再通过总线与CPU相连。

位扩展

又称**字长扩展**或**数据总线扩展**。当存储芯片的数据总线位宽小于CPU数据总线位宽时需要**位扩展**

下面是一个需要位扩展的例子

假设存储器数据位宽为 N ，存储芯片数据位宽为 k ，就需要 N/k 个芯片进行存储扩展

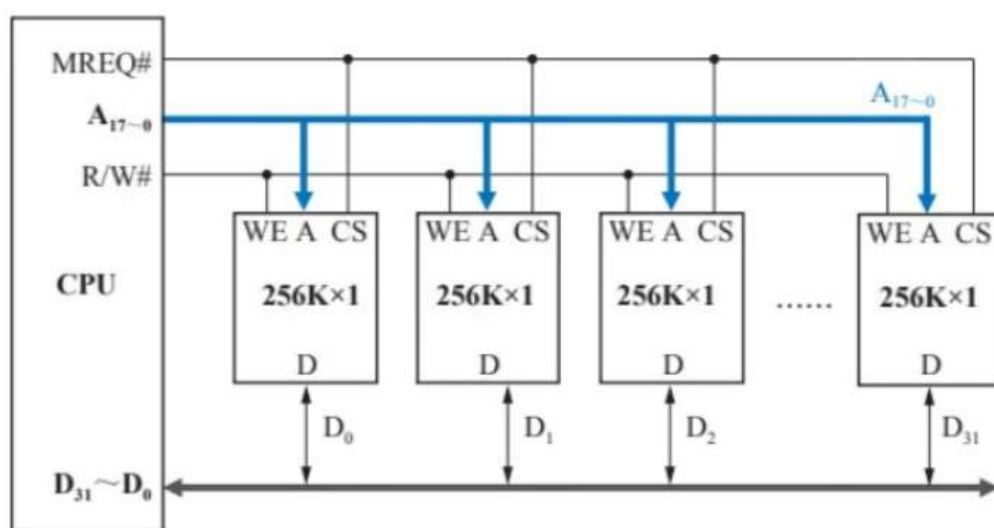


图 4.24 存储器位扩展

图中的芯片为256K=18位，CPU需要32位，所以要32个芯片组成32位的存储器与CPU连接

- 地址线：18根 ($2^{18}=256K$)，同时连接到每个SRAM芯片的地址线A（18根）
- 数据线：32根，分别连接到每个SRAM芯片的数据线D（1根）
- 控制线：MREQ（相当于/MREQ）连接到每个SRAM芯片的CS端（片选信号）
- R/W（相当于-R/W）连接到每个SRAM芯片的WE端（读写控制信号）

字扩展

又称**容量扩展**或**地址总线扩展**，当存储芯片的存储容量不能满足存储器对存储容量的要求时需要进行字扩展

假设存储器容量为 M ，存储芯片容量为 l ，若 $M > l$ ，需要 M/l 个芯片进行存储扩展

如用256Kx8位的SRAM存储芯片组成2Mx8位的存储器并与CPU连接，需要 $2M/256K=8$ 个芯片

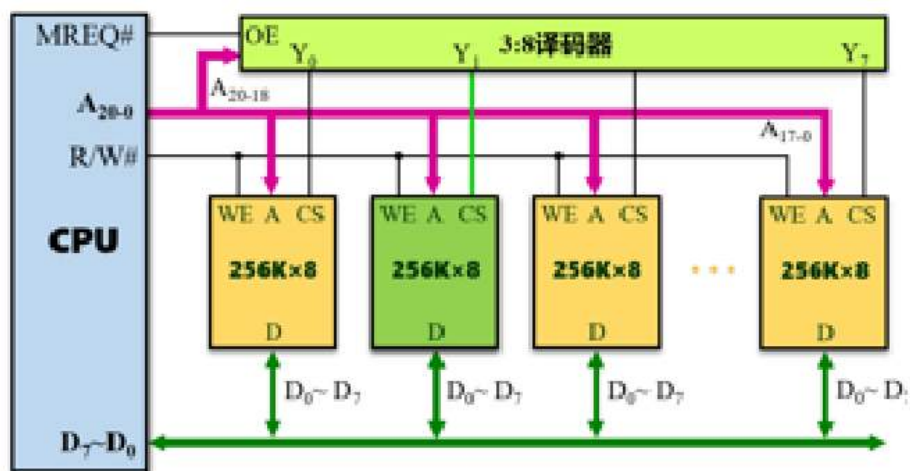


图4.25 存储器的字扩展

- 地址线：21根，其高位部分（A20~A18）连接3-8译码器的输入，低位部分（A17~A0）同时连接到每个SRAM芯片的地址线A（18根）
- 数据线：8根，同时连接到每个SRAM芯片的数据线D（8根）
- 控制线：MREQ连接到3-8译码器的OE端；R/W连接到每个存储器芯片的WE端

输入				输出							
A2	A1	A0	OE	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
0	0	0	1	0	0	0	0	0	0	0	1
0	0	1	1	0	0	0	0	0	0	1	0
0	1	0	1	0	0	0	0	0	1	0	0
0	1	1	1	0	0	0	0	1	0	0	0
1	0	0	1	0	0	0	1	0	0	0	0
1	0	1	1	0	0	1	0	0	0	0	0
1	1	0	1	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0
x	x	x	0	x	x	x	x	x	x	x	x

3-8译码器的输入输出关系

根据译码器选中一个芯片，地址线表示在这个芯片中选取某一位

- 地址总线= $A_{20} \sim A_0$ (21根)
- 地址范围= $0\ 0000\ 0000\ 0000\ 0000 \sim 1\ 1111\ 1111\ 1111\ 1111 = 000000H \sim 1FFFFFFH$
- 第1个SRAM芯片的地址范围= $000000H \sim 03FFFFH$; 容量为256K
- 第2个SRAM芯片的地址范围= $040000H \sim 07FFFFH$; 容量为256K
- 第3个SRAM芯片的地址范围= $080000H \sim 0BFFFFH$; 容量为256K
- 第4个SRAM芯片的地址范围= $0C0000H \sim 0FFFFFFH$; 容量为256K
- 第5个SRAM芯片的地址范围= $100000H \sim 13FFFFH$; 容量为256K
- 第6个SRAM芯片的地址范围= $140000H \sim 17FFFFH$; 容量为256K
- 第7个SRAM芯片的地址范围= $180000H \sim 1BFFFFH$; 容量为256K
- 第8个SRAM芯片的地址范围= $1C0000H \sim 1FFFFFFH$; 容量为256K

字位同时扩展

当存储芯片的**数据位宽**和**存储容量**均不能满足存储器的数据位和存储总容量要求时，可以用字位同时扩展方式组织存储器

下图是一个字位同时扩展的例子

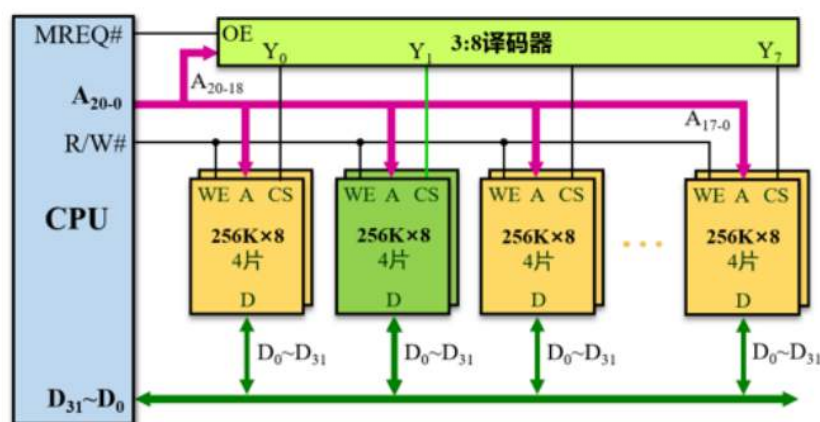


图4.26 存储器的字位扩展

现在利用32个256Kx8位的SRAM芯片，构成1个2Mx32位的存储器，可行的方案是

- 地址线：21根 ($2^{21}=2M$)，其高位部分 ($A_{20} \sim A_{18}$) 连接到3-8译码器的输入，低位部分 ($A_{17} \sim A_0$) 同时连接到每个SRAM芯片的地址线A (18根)
- 数据线：32根，分为4组 ($D_{31} \sim D_{24}$; $D_{23} \sim D_{16}$; $D_{15} \sim D_8$; $D_7 \sim D_0$)，分别连接到各个SRAM芯片的数据线D (8根)
- 控制线：MREQ连接到3-8译码器的OE端；R/W连接到每个SRAM芯片的WE端



高速缓冲存储器

Cache 工作原理

为了提高CPU访问主存的速度，通常在CPU与主存之间增加一个**隐藏的小容量快速SRAM**，称为cache（高速缓冲存储器）

将主存中经常访问或即将访问的数据的副本放在cache中，就可以使得大部分数据访问在快速的SRAM中进行

程序局部性

采用上面的方法是因为CPU执行的程序具有较强的**程序局部性**

程序局部性是在一段时间内，**整个程序的执行仅限于程序中的某一部分（时间）**

而执行程序所需的指令和数据也仅局限于某个存储区域内（空间）

- **时间局部性**：当程序访问一个存储位置时，该位置在未来可能会被多次访问
如程序的循环结构和调用过程就很好地体现了时间局部性
- **空间局部性**：一旦程序访问了某个存储单元，则其附近的存储单元也即将被访问；
如计算机中的指令代码、数组、结构体元素，通常在主存中顺序存放，对其数据访问就具有很强的空间局部性

基本概念

- **命中**：如果CPU要访问的数据在cache中，则称为**命中**

将命中时的数据访问时间称为命中访问时间，通常为**cache访问时间 t_C**

假设为命中cache的次数，为从主存中访问信息的次数，命中率 **h** 为 $h = N_c / (N_c + N_m)$ ，缺失率为 $1 - h$

命中率与程序局部性、cache容量、数据块（Block）大小、cache的组织方式有关

- **数据缺失**：如果CPU要访问的数据不在cache中，则称为**数据缺失**

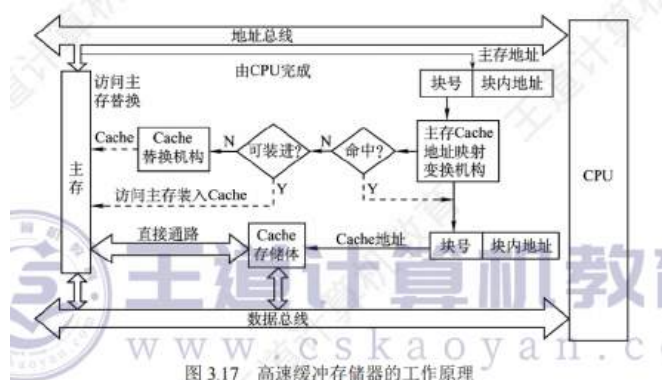
此时要将缺失数据从主存调入cache中才能访问数据

将数据缺失时的访问时间称为**缺失补偿**，通常为主存访问时间

- **数据块**：通常将cache和主存分为若干个固定大小的数据块
 - 主存地址=主存块地址+块内偏移
 - cache地址=cache块地址+块内偏移
 - 主存和cache的块内偏移是一样大的，即主存块和cache块的大小是相同的
- **平均访问时间**： $t_a = ht_c + (1 - h)t_m$
- **访问效率**： $e = t_c/t_a = \frac{1}{h+(1-h)r}$ ，其中 $r = t_m/t_c$

Cache的读写

为了使 Cache 与主存更好地交换信息，Cache 和主存都被划分为大小相等的块，Cache 块也称 Cache 行，每块由若干字节组成，块的长度称为**块长**



关键技术

- **数据查找**：快速查找数据是否在cache中
- **地址映射**：主存中的数据块如何放置到cache中
- **替换策略**：如何对cache中的某个数据块进行替换
- **写入策略**：采用何种写入策略

读

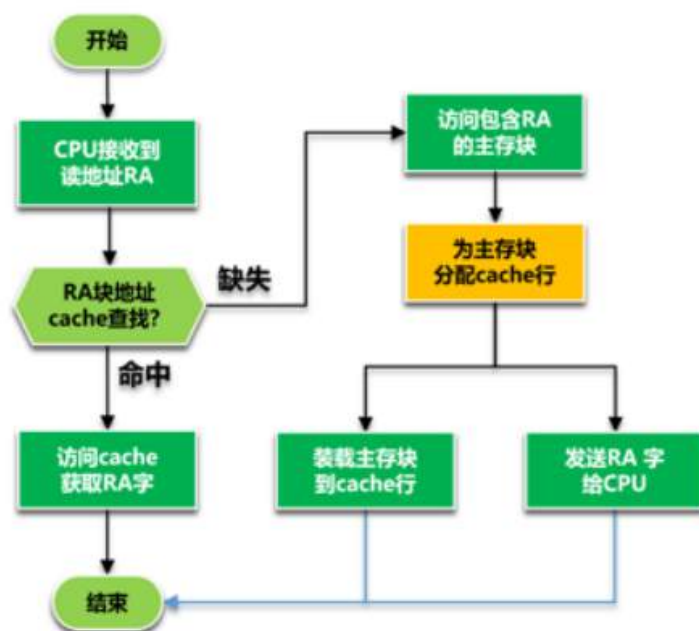


图4.33 cache读操作的基本流程

1. CPU 给出主存地址 **RA** (包括块地址和块内偏移), 以 RA 的块地址为关键字在cache中**查找**, 找到则直接访问 Cache 得到RA处的数据
2. 找不到标识**数据缺失**, 访问主存, 将RA处主存块调入 Cache 中, 并将数据返回至CPU
3. 如果 Cache 已经满了, 要进行替换, 更新查找表

写

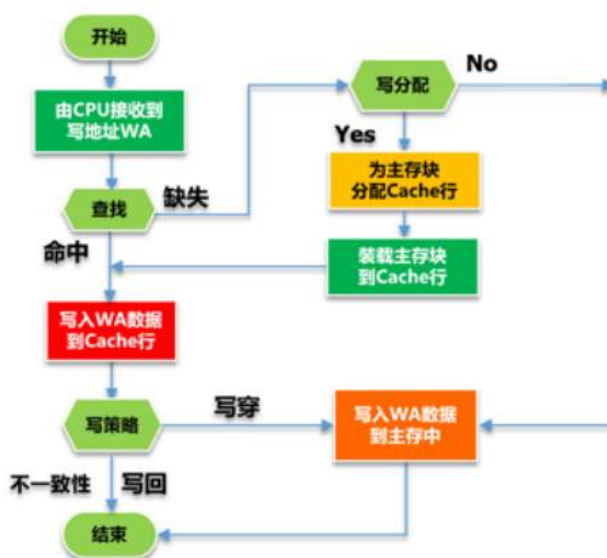


图4.34 cache写操作的基本流程

1. CPU 给出主存地址 WA(包括块地址和块内偏移), 以 WA 块地址为关键字在cache中进行查找
2. 找到则将 WA 处数据写入cache中
 - 若采用写回策略, 则数据不需要同时写入主存
 - 若采用写直达策略, 要同时写入主存
找不到则标识写缺失
 - 若采用写分配法, 先将数据块调入cache中, 再进行数据写入cache的操作
 - 否则直接将数据写入主存中
3. 若cache已满, 则再进行替换, 更新查找表

脏数据

新写入cache中的数据与主存中的原始数据不一致时, 新写入cache中的这部分数据称为**脏数据**

cache是主存一部分内容的副本, 因此cache的内容改变时, 主存相应内容也要改变, 为此产生了不同的写入策略

• 写回法

- 内容: 当写入cache时, 只修改cache的内容, 而不立即修改主存的内容。只有当该cache行被替换出去时, 才将脏数据写回主存
- 实现: 实现写回法, cache的每一位配置一个修改位(也称为脏位)

若该行被修改, 则脏位为1, 否则为0.

当该行被替换出去时, 如果脏位=1, 则需要将该行的内容写入主存

- 优点: 写回法可以使cache在大部分时间(没有发生替换的时间)的写入速度等于访问cache的速度

只有在发生替换时, cache的写入速度才变为访问主存的速度(此时要执行写入主存的操作)

• 写穿法

- 内容: 也称为直写法, 当写入cache时, 同时对cache的行和主存对应的数据块进行修改
- 分析

cache的每行不需要设置一个修改位(脏位), 并且发生替换时, 直接将该cache行丢弃,

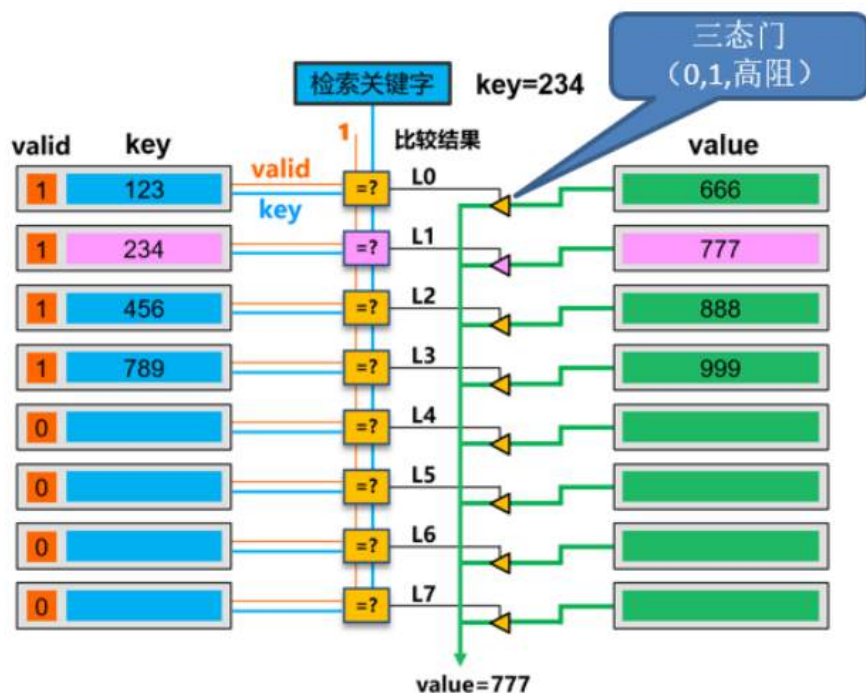
不需要写回主存。

较好维护了cache与主存的一致性，但cache的写入时间将变为访问主存的时间，从而大大降低了写cache的速度

相联存储器(CAM)

相联存储器对应的是cache的数据查找

普通的存储器是按地址进行访问，相联存储器按**内容进行访问**



相联存储器用于存放**cache的查找表**，其内部存储的基本数据是键值对，通过检索关键字得到值

其中，每个键值对还存放一个有效位valid，当valid为1时说明键值对有效

地址映射

即 Cache 关键技术中的**地址映射**

全相联映射

主存的某一块可以映射到 Cache 的任意块中

全相联映射

主存 (块)	cache (块)
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	0
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	1
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	2
.....	...
.....	...
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	30
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	31

假设cache块大小为4W，共8行，主存按字访问，主存地址长度为9位，被划分为7位主存块地址和2位块内偏移offset字段

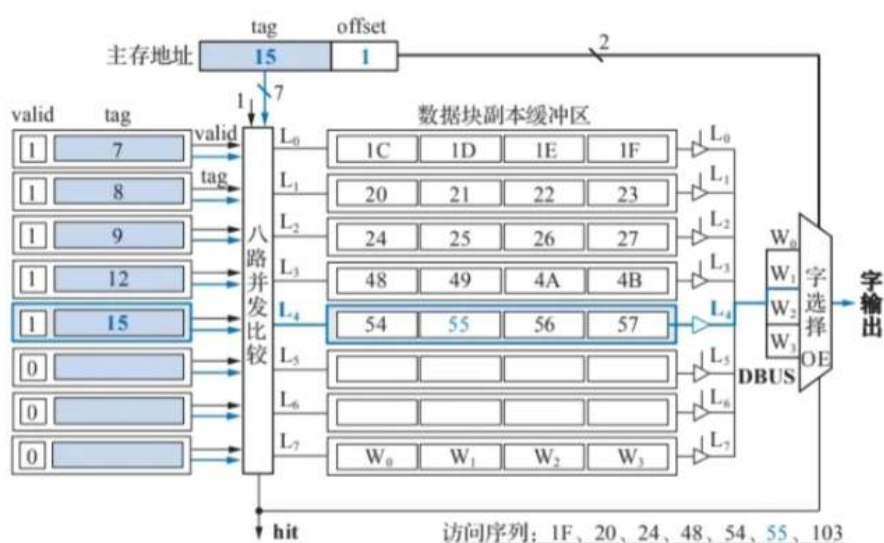


图 4.37 全相联映射的硬件逻辑实现框图

将主存地址中的主存块地址（tag）与n个cache行的主存块地址进行比较（n路并发比较，即1个与n个进行比较）

比较相等且有效位=1的那一行cache为命中

💡 在下图中，第五次已经将15块主存存入cache中，因此第六次就命中了

表 4.4 全相联映射访问过程

访问序列	1	2	3	4	5	6	7
十六进制	1F	20	24	48	54	55	103
二进制	00001111 11	00010000 00	00010010 00	00100100 00	00101010 00	00101010 01	10000000 11
(tag,offset)	(2,3)	(8,0)	(9,0)	(12,0)	(15,0)	(15,1)	(40,3)
cache 行	0	1	2	3	4	4	5
访问情况	载入	载入	载入	载入	载入	命中	载入

全相联映射方式的特点：

- 主存数据块可以映射到cache的任意一行（块），因此cache的利用率高
- 只要cache中还有空行就不会引起冲突，因此cache的冲突率低
- 查找时需要并发比较查找表中所有项，每一个cache行对应一个比较电路，硬件成本较高，只适合小容量cache使用
- cache满时，载入新数据块需要利用替换算法进行替换，替换策略和算法较为复杂

直接相联映射方式

主存的某一块，只能映射到cache的固定一块

直接相联映射

主存 (块)	cache (块)
0、32、64、96、128、160、192、224、.....、960、992	0
1、33、65、97、129、161、193、225、.....、961、993	1
2、34、66、98、130、162、194、226、.....、962、994	2
.....	...
.....	...
30、62、94、126、158、190、222、254、.....、990、1022	30
31、63、95、127、159、191、223、255、.....、991、1023	31

💡 例如主存的第0块，只能映射到cache的第0块；主存的第33块，只能映射到cache的第1块

有地址映射公式

$$i = j \bmod n$$

- i: cache块
- j: 主存块
- n: cache块数/行数

主存地址为区地址+行索引+块内偏移

根据主存地址中的行索引（index）找到cache的对应行，将主存地址中的区地址（tag）与该cache行的区地址（tag）进行比较

比较相等且有效位=1则为命中

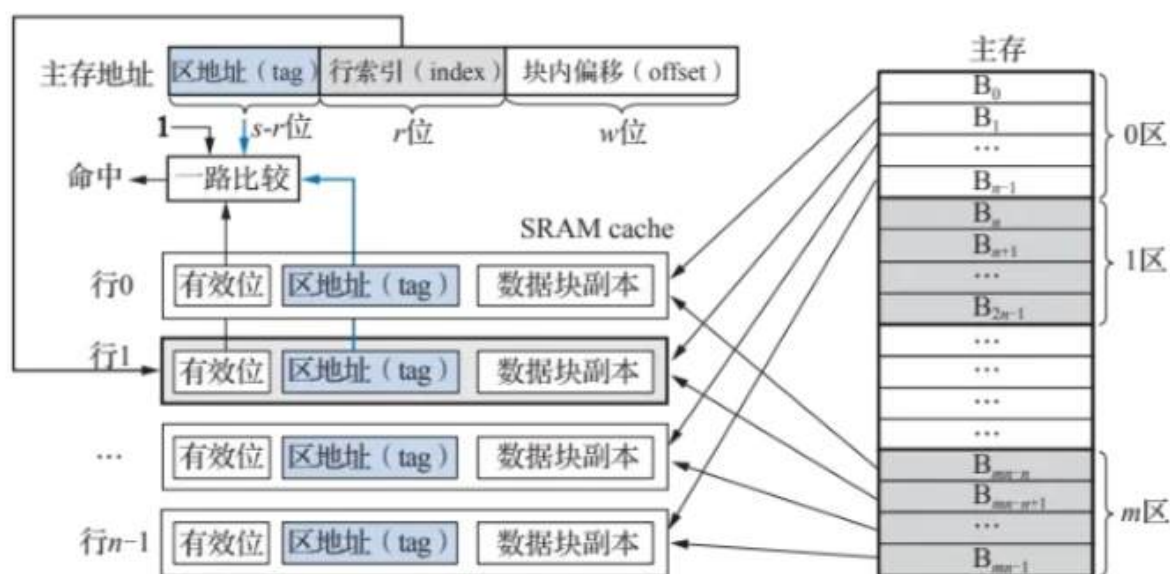


图 4.38 直接相联映射逻辑示意图

假设cache块大小为4W，共8行，主存按字访问，主存地址长度为9位，被划分为4位tag，3位index，1位offset

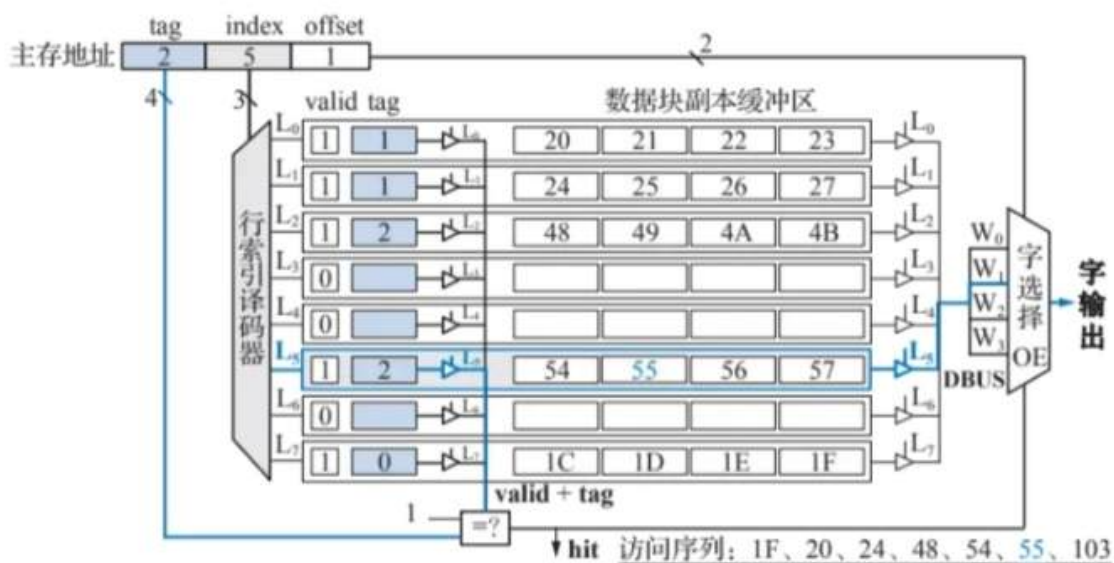


图 4.39 直接相联映射的硬件逻辑实现框图

如图所示，首先将主存地址转化为二进制并进行分割，根据中间三位得到行索引

- 若数据不在cache中，将数据所在字都存入cache中
- 否则先根据行索引查找行，再比较区号tag是否相同

在最后一次中，将数据存入第0行，此时数据有效，因此需要进行替换

直接相联映射方式的特点：

- cache的利用率低、命中率低。由于主存数据块只能映射到cache的特定行（块）
- cache的冲突率高。因为行索引（index）相同的所有主存块都将映射到cache的同一行，即使cache未满，也会发生数据冲突
- 只需要一个比较器（1路比较），硬件成本低，适合于大容量cache使用。

因为查找时只需要根据主存地址的行索引访问对应的cache行，并比较主存地址的区地址与该cache行的区地址

- 无须使用复杂的替换算法，直接替换冲突的数据块即可

表 4.5 直接相联映射访问过程

访问序列	1	2	3	4	5	6	7
十六进制	1F	20	24	48	54	55	103
二进制	0000 111 11	0001 000 00	0001 001 00	0010 010 00	0010 101 00	0010 101 01	1000 000 11
(tag,index,offset)	(0,7,3)	(1,0,0)	(1,1,0)	(2,2,0)	(2,5,0)	(2,5,1)	(8,0,3)
cache 行	7	0	1	2	5	5	0
访问情况	载入	载入	载入	载入	载入	命中	替换

组相联映射方式

2路组相联映射

主存 (块)	cache	
	组	块
0、16、32、48、64、80、96、.....、992、1008	0	0、1
1、17、33、49、65、81、97、.....、993、1009	1	2、3
2、18、34、50、66、82、98、.....、994、1010	2	4、5
.....	...	...
.....	...	...
14、30、46、62、78、94、110、.....、1006、1022	14	28、29
15、31、47、63、79、95、111、.....、1007、1023	15	30、31

将cache块分为若干组，主存的某一块，只能映射到cache的固定组，在该组中可以映射到其中的任意一块

每个组有k行，这称为k路组相联

首先采用直接相联映射方式映射到cache的某一组，然后按照全相联映射方式映射到该组内的任意一行

💡 例如主存的第0块，只能映射到cache的第0组（可以映射到第0组的第0块或第1块）

主存的第33块，只能映射到cache的第1组（可以映射到第1组的第2块或第3块）

有地址映射公式

$$i = j \bmod Q$$

- i: cache组号
- j: 主存块
- Q: cache组数

根据主存地址中的组索引（index）找到cache的对应组，

将主存地址中的标记（tag）与该cache组的所有行的标记（tag）进行比较，比较相等且有效位=1则为命中

映射方式总结



- **直接相联映射:** 查找容易（主存的块对应cache的唯一块），淘汰简单，但命中率较低
- **全相联映射:** 查找时并发比较的硬件成本最高，淘汰算法复杂，但命中率较高
- **组相联映射:** 兼具直接相联映射和全相联映射的优势，既能提高命中率，又能降低查找电路的硬件成本

替换算法

替换算法也称为淘汰算法。如果cache中已经装满数据，当新的数据块要载入时，必须从cache中替换（淘汰）掉一个数据块

如果淘汰掉的数据块中存在脏数据（Dirty Data），还需要将淘汰的数据同步到主存中

cache除了直接相联映射方式外，均需要替换算法

先进先出FIFO算法

- FIFO算法按照数据块进入cache的先后决定替换的顺序
- 在需要进行替换时，选择最先被载入cache的行（数据块）进行替换
- FIFO算法需要记录每个数据块载入cache的时间戳或时间计数，以方便替换时比较先后顺序
- FIFO算法系统开销较小，但没有考虑程序访问的局部性，命中率低

LFU算法

- LFU算法将访问次数最少的cache行替换。
- cache的每行设置一个计数器，每访问该行一次，计数器加1

当需要替换时，对所有可淘汰行的计数值进行比较，将计数值最小的行淘汰掉

LFU算法不能严格反映近期访问情况，而是历史访问情况

LRU算法

- LRU算法将近期内最久未被访问过的cache行淘汰掉
- cache的每行设置一个计数器，当访问某cache行时，该cache行的计数器清0，其它cache行的计数器加1

因此计数器是用于统计未被访问的次数

- 需要替换时，比较所有可淘汰行的计数值，将计数值最大的行淘汰掉

LRU算法保护了刚载入cache的新数据，使cache具有较高的命中率，但需要快速比较多个cache行的计数器值，硬件实现比较复杂

随机替换算法

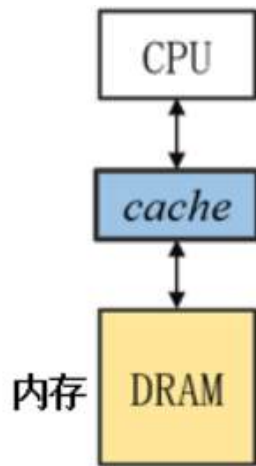
随机替换在需要进行替换时，从所有可淘汰行中，随机选取一行进行替换

随机替换算法实现最容易，替换速度最快，命中率较低

模拟研究表明，随机替换算法的性能只是比LFU和LRU稍差一些

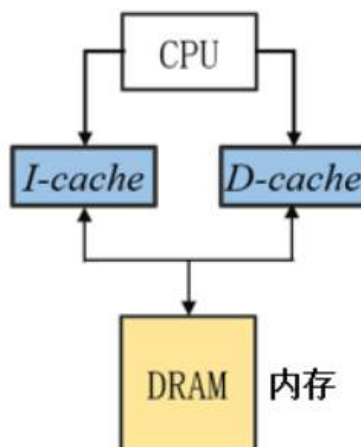
cache设计

- 统一cache：既存放指令又存放数据的cache

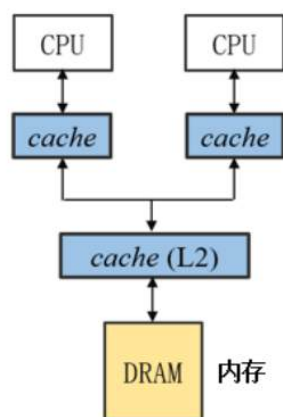


- **分离cache**：将指令cache和数据cache分开设计。

实现CPU在存取数据时（访问数据cache），同时完成取指令的功能（访问指令cache），从而提高CPU访问cache的速度



- **多级cache**：将cache和处理器集成在同一个芯片内，这个cache称为第一级cache
由于第一级cache容量小，还可以在主存和cache间加设第二级、第三级等cache
这种设计减少了对片外总线的访问，加快了存取的速度，提高了系统的性能



- **软件cache**：在软件设计时可以参考cache设计的理念

虚拟存储器

虚拟存储器简介

- 虚拟存储器处于“主存-辅存”存储层次，是为了解决存储系统“存储容量大”的问题
- 虚拟存储器使得程序可以用虚拟地址访问内存(虚拟地址空间更大)
- 虚拟存储器采用按需加载的方式加载程序代码和数据，将辅存中经常访问的程序和数据的副本调度到主存中
- 如果主存已满，则会选择最不经常访问的程序和数据进行淘汰，这一点和cache是类似的
- 虚拟存储器在替换数据时，对于静态代码和数据可以直接丢弃；动态数据丢弃时要同时保存到辅存中

i 虚拟存储器的三个特性

- **多次性**：指一个作业中的程序和数据允许被多次调入内存运行，即只需当前要运行的那部分程序和数据装入内存即可开始运行。
- **对换性**：在进程运行期间允许将那些暂不使用的代码和数据从内存调至外存的交换区，待以后需要时再将它们从外存调至内存。
- **虚拟性**：是指能从逻辑上扩充内存容量

虚拟存储器的地址映射与变换

虚拟存储器中有三种地址空间

- 虚拟地址空间：即虚拟空间、虚地址空间
- 主存地址空间：即物理地址空间、实地址空间
- 辅存地址空间：磁盘的地址空间

虚拟存储器的地址映射是将虚拟地址空间映射到主存地址空间，地址变换是运行程序时实时将虚地址转换为实地址或磁盘地址

虚拟地址到物理地址转换的过程依靠存储器管理单元(MMU)实现

虚拟存储器有

- 页式虚拟存储器
- 段式虚拟存储器
- 页段式虚拟存储器

下面以页式虚拟存储器为例介绍虚拟存储器的机制

页式虚拟存储器

页式虚拟存储器是以页为逻辑结构划分信息传送单位的虚拟存储器

虚拟空间和主存空间均被划分为固定大小的页，常见的页大小为4KB，也有更大容量的页

虚拟地址划分

- **虚拟地址 (VA) = 虚拟页号 (VPN) + 虚拟页偏移 (VPO)**
- **物理地址 (PA) = 物理页号 (PPN) + 物理页偏移 (PPO)**

其中，虚拟页号也称为**虚页号**；物理页号也称为**页框号**、**实页号**

虚拟页偏移和物理页偏移的大小是相等的，也就是**页式虚拟存储器**页的大小



图4.46 页式虚拟存储器的地址划分

页表

页表是一张保存**虚拟页号**和**物理页号**对应关系的查找表，是一个由若干个表项组成的数组，常驻于主存中

页式虚拟存储器中虚拟地址到物理地址之间的转换是基于页表进行的，通过页表将虚拟页号转换为物理页号

页表以**虚页号**为索引，存储包括**有效位**和**物理页号（PPN）**、**修改位**、**使用位**、**权限位**等信息
页表的每一行称为**页表项**，主存和磁盘都分为固定大小的页面，用于存放虚拟页面

i 有效位

有些页在执行程序的时候可能不在内存里，但是这些页肯定要在页表中存有记录，用有效位标记记录是否有效

- 有效位=0，标记为null的页表项，其对应的虚拟页可能是未分配的页
- 有效位=0，没有标记为null的页表项，虚拟页在磁盘中
- 有效位=1，虚拟页在主存中

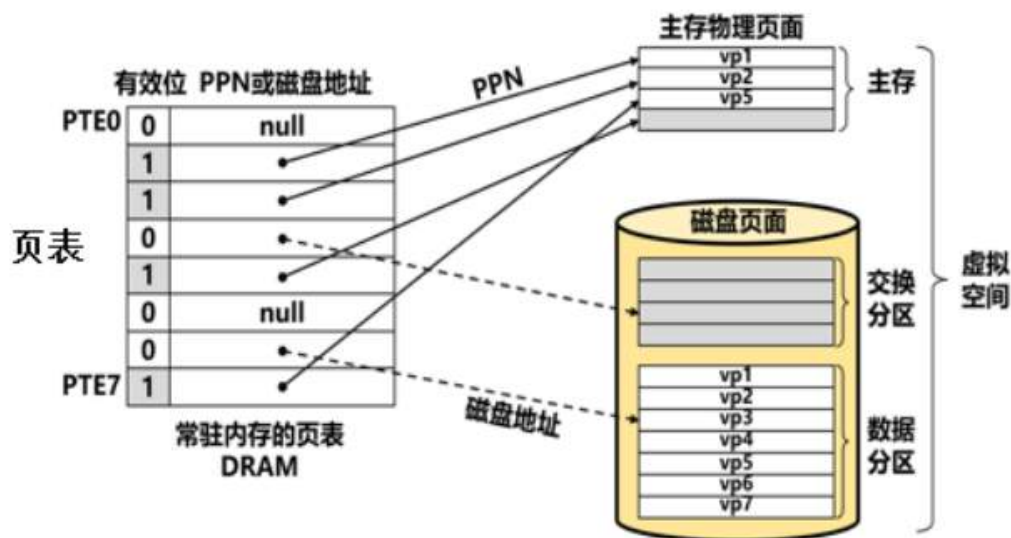


图4.47 页式逻辑结构示意图

磁盘中的交换分区用于存放主存页面替换出来的动态修改数据，磁盘中的数据分区用于存放用户程序和数据

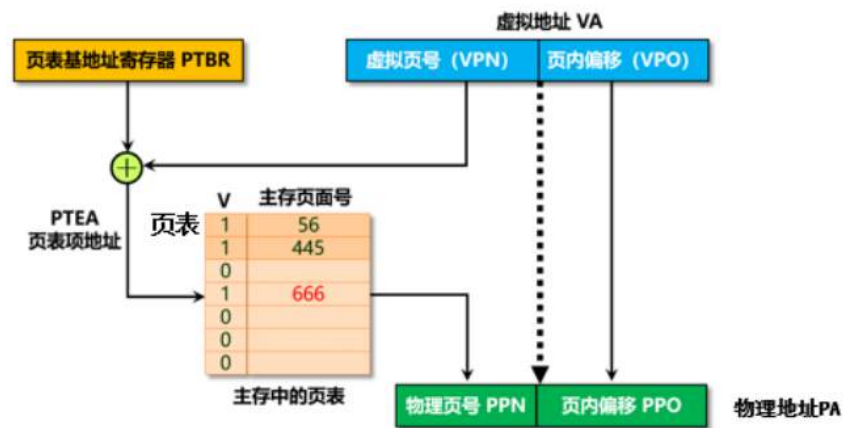
页表在主存中的首地址记录在页表基址寄存器，在进行进程切换时只需切换首地址即可切换页表

基于页表的虚拟地址与物理地址的转换过程

将虚拟地址中的虚拟页号 (VPN) 与页表基址寄存器的内容 (PTBR) 相加，结果作为页表项的地址 (PTEA)，得到页表的某一行

- 若该页表项有效位 $V=1$ ，则当前页数据在主存中，通过物理页号与页内偏移相加可以得到物理地址
- 若有效位 $V=0$ ，表示缺页。需要磁盘将对应也载入主存中

虚拟存储器访问流程



• 普通虚拟存储器的访问

• 页面命中时

1. CPU将虚拟地址VA传给MMU
2. MMU根据VA中的虚拟页号VPN、页表基址寄存器PTBR的内容，计算得到页表项地址PTEA。根据页表项地址PTEA访问页表
3. 页表向MMU返回页表项PTE
4. 若页表项PTE中的有效位=1，则根据页表项PTE中的物理页号PPN和虚拟地址中的页内偏移VPO，构成物理地址PA
5. 由物理地址PA，从cache/主存中访问得到数据，并返回给CPU

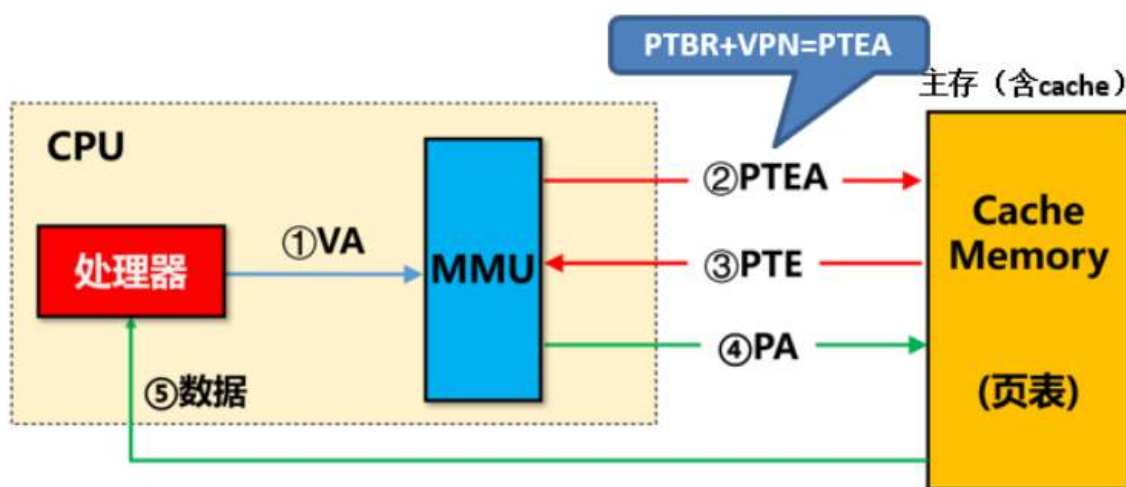


图4.49 (a) 页式虚拟存储器的访问过程 (页面命中)

• 页面缺失时

1. CPU将虚拟地址VA传给MMU

2. MMU根据VA中的虚拟页号VPN、页表基址寄存器PTBR的内容，计算得到页表项地址PTEA。根据页表项地址PTEA访问页表
3. 页表向MMU返回页表项PTE
4. 页表项PTE中的有效位=0，表示所访问的页不在主存中，MMU触发缺页异常，调用操作系统内核中的缺页异常处理程序
5. 若主存页满，则需要根据替换算法确定替换那一页；如果换出页的修改位=1，则将该页换出到磁盘（调出页），否则直接丢弃
6. 缺页处理程序直接从磁盘调入新的页（调入页），并更新存储器中的页表项PTE
7. 缺页处理程序返回原来进程，接续页面命中时步骤处理

• 结合cache的虚拟存储器访问流程

先前的cache直接放在主存中，在这个流程中，cache和主存分开。部分页表块放在cache中与正常访问流程不同的是，在MMU根据PTEA访问页表时

- 如果位于cache中的页表有该PTEA，则可直接向MMU返回页表项PTE，表示PTEA命中
- 否则照常从主存页表中获取页表项PTE

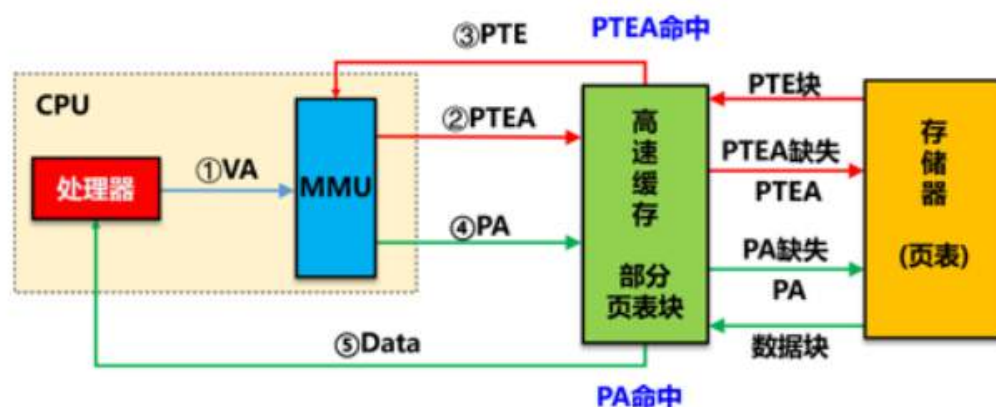


图4.50 结合cache的页式虚拟存储器的访问过程

• TLB加速虚拟存储器地址转换

TLB即转换旁路缓冲区，用于缓冲经常访问的页表项。通常它被称为快表，而主存中的页表称为慢表

本质上TLB是一个容量较小的cache，采用全相联或组相联方式和随机替换算法

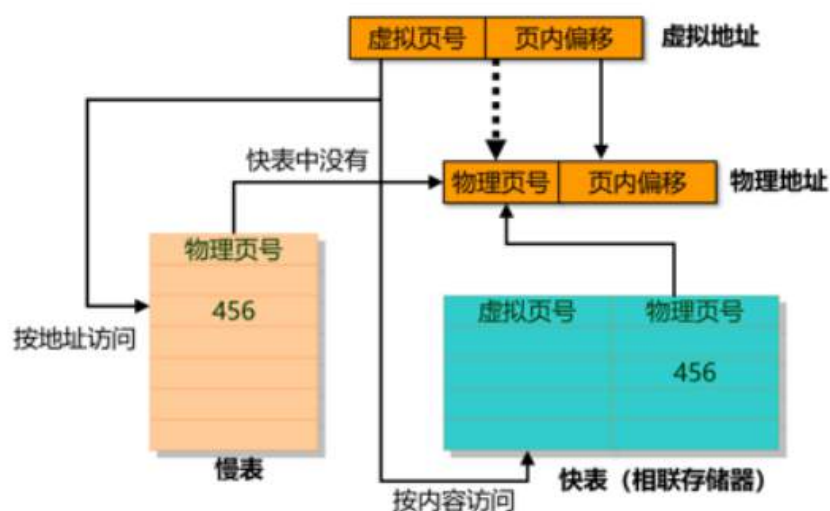


图4.52 基于TLB的虚拟地址与物理地址的转换逻辑

- 快表的查找按内容访问，即虚拟页号与物理页号以类似哈希表的方式对应
- 慢表的查找按地址访问，类似数组的访问

转换地址时同时查快表和慢表，如果快表命中，慢表的查找直接终止

• 命中时访问流程

1. CPU（处理器）给出一个虚拟地址VA，并送到MMU
2. MMU根据虚拟地址VA中的虚拟页号VPN，查询TLB
3. 如果TLB访问命中，则TLB向MMU返回与虚拟页号VPN对应的物理页号PPN
4. MMU根据物理页号PPN和虚拟地址中的页内偏移VPO，构成物理地址PA
5. 由物理地址PA，从cache/主存中访问得到数据，并返回给CPU

• 不命中时访问流程

1. CPU（处理器）给出一个虚拟地址VA，并送到MMU
2. MMU根据虚拟地址VA中的虚拟页号VPN，查询TLB
3. TLB访问不命中（缺失），则TLB向MMU返回TLB访问缺失信息
4. MMU根据虚拟地址VA中的虚拟页号VPN和页表基址寄存器PTBR的内容，计算得到页表项地址PTEA
5. 再根据页表项地址PTEA访问页表
 - 1) 页表向MMU返回页表项PTE

2) 若页表项PTE中的有效位=1，则需要更新TLB

同时根据页表项PTE中的物理页号PPN和虚拟地址中的页内偏移VPO，构成物理地址PA

6. 由物理地址PA，从cache/主存中访问得到数据，并返回给CPU

CPU访存过程

CPU在不同模式下以不同方式访问主存

- **实地址模式**：只能用物理地址访问主存，即实模式
- **虚地址模式**：只能用虚拟地址访问主存，即保护模式

访问流程如图所示

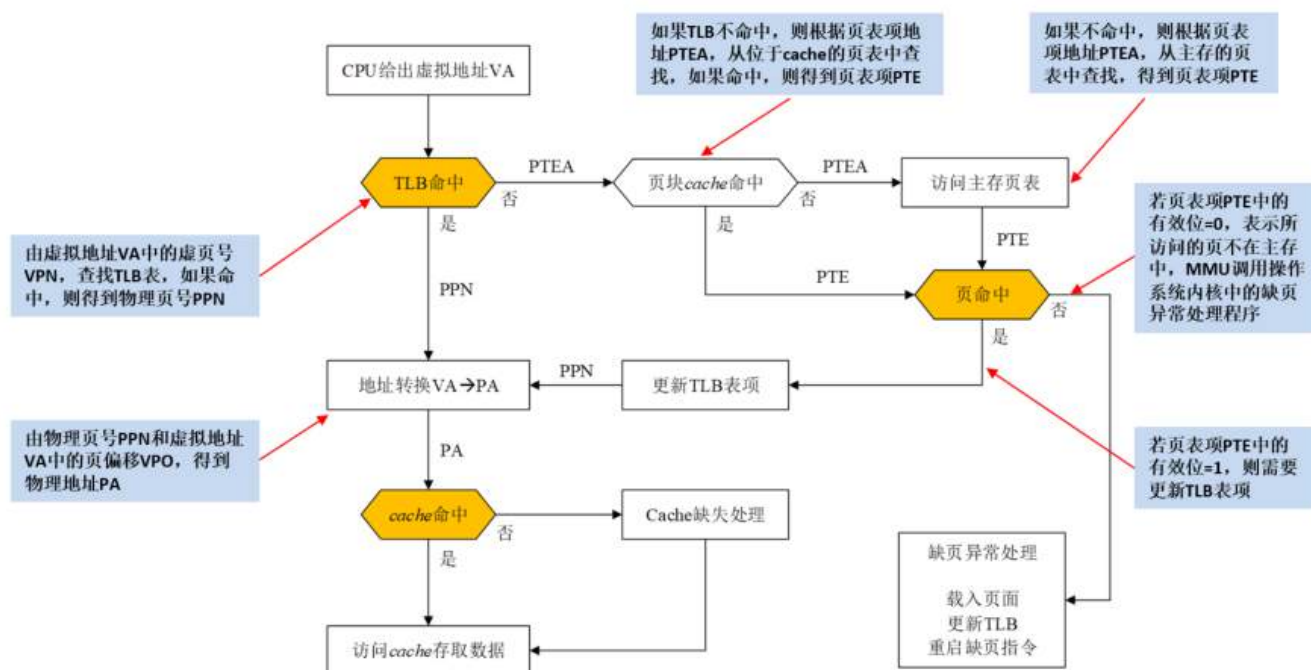


图4.55 虚存访问操作流程

三者命中组合情况如下

表4.11 TLB命中、页命中、cache命中组合

序号	TLB	页	cache	可能性	说明
1	命中	命中	命中	可能	TLB命中则页一定命中，页载入主存和数据块载入cache并不同步。所以cache有可能命中，也有可能缺失。三者均命中时访存速度最快
2	命中	命中	缺失		
3	缺失	命中	命中		
4	缺失	命中	缺失	可能	TLB缺失后还可以访问慢速页表（慢表），页载入主存和数据块载入cache并不同步，所以cache有可能命中，也有可能缺失
5	缺失	缺失	缺失	可能	这是最糟糕的情况，虚存系统初始化时常见
6	命中	缺失	缺失	不可能	页缺失说明页不在主存中，TLB中一定没有对应页表项，TLB不可能命中
7	命中	缺失	命中		
8	缺失	缺失	命中	不可能	页缺失说明数据也不在主存中，所以cache不可能命中

指令系统

计算机中的指令包括高级语言指令、汇编语言指令、机器指令、微指令

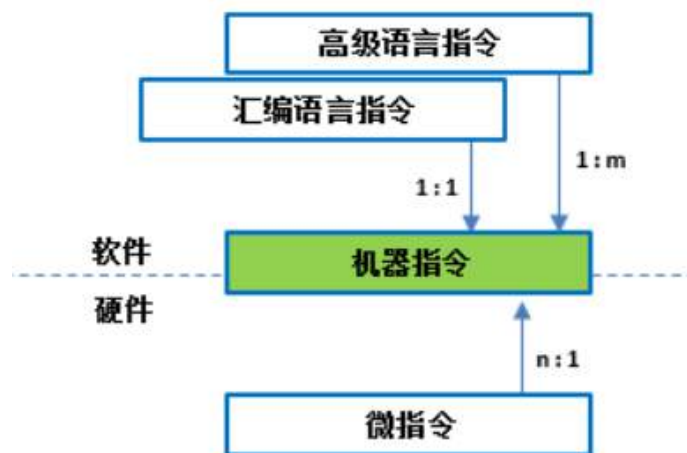


图5.1 不同级别指令之间的关系

本节内容主要介绍机器指令，**机器指令**（简称**指令**）是指示计算机执行某种操作的命令。一台计算机的所有指令的集合构成该机的指令系统，也称**指令集**。

指令系统概述

指令系统是**指令集体系结构**（ISA）中最核心的部分，ISA 规定的内容主要包括

- **指令格式**：指令寻址方式，操作类型，以及每种操作对应的操作数的相应规定。

- **操作数的类型**：操作数寻址方式，以及是按大端方式还是按小端方式存放
- **程序可访问的寄存器编号、个数和位数**，存储空间的大小和编址方式
- **指令执行过程的控制方式**：包括程序计数器、条件码定义等。

指令格式

指令包括**操作码 OP** 和**地址码 A**



图5.2 指令的一般格式

- **操作码 OP** 用于解决进行何种操作的问题
- **地址码 A** 用于解决处理什么操作数的问题
- 通过何种方式获取操作数通常由寻址方式决定，寻址方式可能在地址码中或操作码中给出

指令字长度

指令机器码的长度，也称为**指令字长**，它应当为字节的整倍数

根据指令字长与机器字长的关系，分为：

- **半字长指令**：指令字长为机器字长的一半，CPU访问一次主存可以读取两条指令
- **单字长指令**：指令字长等于机器字长，CPU访问一次主存只能读取一条指令
- **多字长指令**：指令字长为机器字长的多倍，CPU需要多个存储周期才能读取一条指令

通常最常用的指令是**短指令格式**

i 定长指令系统与边长指令系统

- **定长指令系统**
 - 指令长度固定，RISC计算机中多采用定长指令系统
 - 优点：结构简单，有利于CPU取指令、译码和指令顺序寻址，方便硬件执行
 - 缺点：平均指令长度较长，冗余状态较多，不容易扩展

- **变长指令系统**

- 指令长度可变，CISC计算机中多采用变长指令系统，如Intel x86序列计算机
- 优点：结构灵活，冗余状态较少，平均指令长度较短，可扩展性好
- 缺点：取指令和译码不方便，下一条指令的地址必须在指令译码完成后才能确定，大大增加了硬件控制系统的设计难度

指令地址码

指令中**地址码** A 的作用随指令类型和寻址方式的不同而不同

根据一条指令中所含操作数地址的数量，可将指令分为：

- **三地址指令**

ASM

```
add rd,rs,rt
```

需要较大的指令长度，如果是寄存器地址可以缩短指令长度

- **二地址指令**

ASM

```
add al,bl
```

有三种类型：

- **RR(寄存器-寄存器)**：MOVAL,BL
- **RS(寄存器-存储器)**：MOV[2000H],AL
- **SS(存储器-存储器)**：MOV[2000H],[3000H]

RR型指令执行速度最快，SS最慢

- 单地址指令

ASM

```
inc al
```

- 零地址指令

只有操作码 OP，不需要操作数或操作数隐式地取出

ASM

```
daa
```



指令操作码

操作码表示具体进行何种操作

- **定长操作码:** 操作码OP长度固定，且操作码在指令中的位置也是固定的

💡 假设指令系统有 m 条指令，则操作码的位数 n 应该满足： $n \geq \log_2 m$ ；如 $m=128$ ， $\log_2 m = 7$ ， $n \geq 7$

- **变长操作码:** 操作码OP长度可变，且操作码在指令中的位置也是不固定的

可以采用扩展操作码技术实现变长操作码，即操作码的长度随地址码数目的增加而减少

寻址方式

寻址方式是寻找指令或操作数有效地址的方法，分为指令寻址方式和操作数寻址方式

指令寻址方式

• 顺序寻址方式

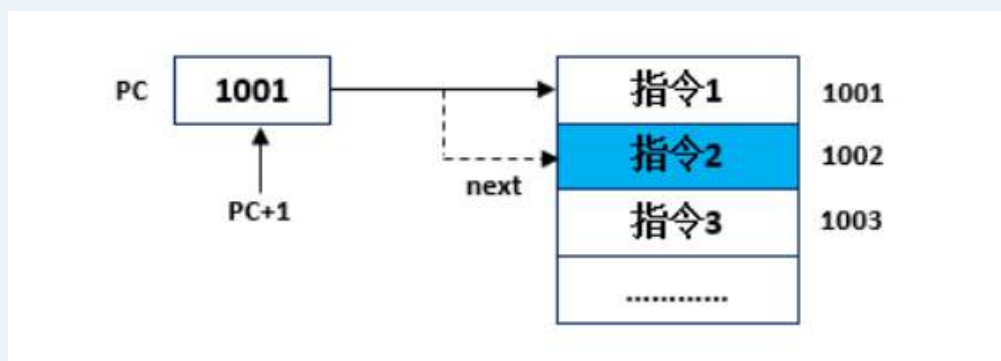
大部分程序中的机器指令序列在主存中是按顺序存放的，根据前一条指令的有效地址来推断后续指令的有效地址

将下一条指令的地址存放在程序计数器PC中

i 指令预取

假设主存按照字节进行编址

- 如果是单字节指令，则PC应自动+1
- 如果是双字节指令，则PC应自动+2



• 跳跃寻址方式

程序出现分支或转移时需要跳跃寻址

跳跃寻址方式时，下一条指令的地址由指令本身及指令需要测试的条件决定

💡 如jmp1003使得下一条执行的指令在1003处

操作数寻址方式

操作数寻址方式就是形成**操作数有效地址**的方法

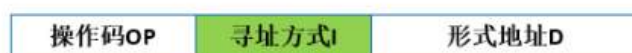


图5.5 含寻址方式字段的单地址指令格式

寻址方式字段标识了寻址方式

- **隐含寻址**

如一些零地址指令，隐含了操作数的地址

- **立即寻址**

ASM

```
MOV EAX,2000H;2000H
```

机器指令为



图5.6a 立即寻址

采用立即寻址方式，形式地址即操作数为2000H

指令存储在主存，但是执行时不需要访问主存，因为数据不在主存

- **直接寻址**

ASM

```
MOV EAX,[2000H];2000H采用直接寻址方式
```

机器指令为



图5.6b 直接寻址

形式地址为2000H

直接寻址方式的特点是地址直观，但指令执行阶段需要访问一次主存，且寻址范围受限于地址字段D的宽度

- **寄存器寻址**

ASM

```
MOV EAX,ECX
```

机器指令为



图5.6c 寄存器寻址

形式地址为寄存器编号

- 间接寻址

ASM

```
MOV EAX,@2000H
```

机器指令为

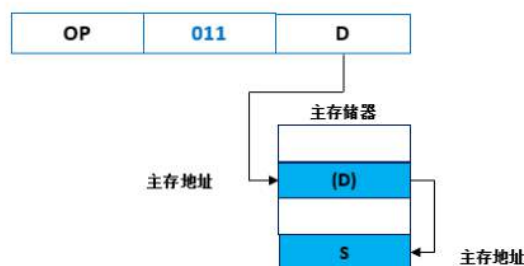


图5.7 间接寻址

形式地址为2000H，有效地址为2000H单元中的内容，即操作数为[[2008H]]

间接寻址方式指令执行阶段需要访问至少两次主存，如果多次间址，则多次访存

- 寄存器间接寻址

ASM

```
MOV EAX,[ECX]
```

机器指令为

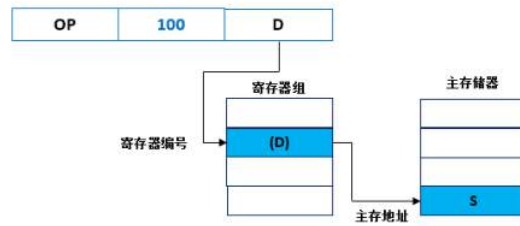


图5.8 寄存器间接寻址

形式地址为寄存器编号，有效地址为寄存器内容

寄存器间接寻址方式指令执行阶段只需要访问一次主存

- 相对寻址

ASM

```
MOV EAX,32[PC]
```

机器指令为

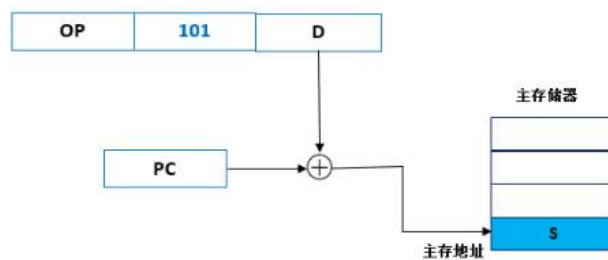


图5.9 相对寻址

此处有效指令为32，有效地址为PC+32，常用于跳转指令

- 变址寻址

ASM

```
MOV EAX,32[ESI]
```

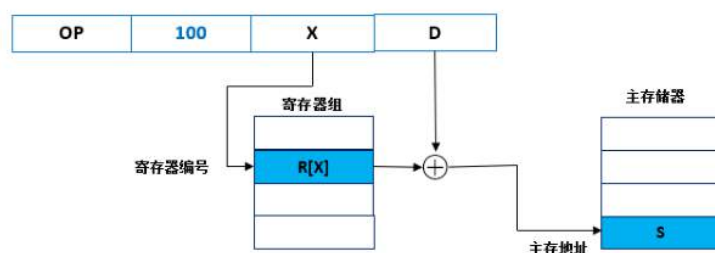


图5.10a 变址寻址

形式地址为32。此处的X为寄存器编号

- 基址寻址

ASM

```
MOV EAX,32[EBX]
```

机器指令为

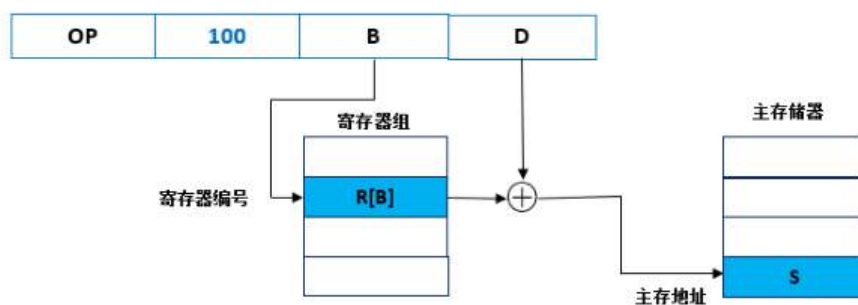


图5.10b 基址寻址

基址寻址方式的指令格式和变址寻址方式相同，寻址过程也与变址寻址方式相同，此时地址码字段增加一个基址寄存器B编号字段

🔔 基址寻址是面向系统的，主要用于程序的重定位,如页表基址寄存器PTBR
相对寻址、变址寻址、基址寻址等3种寻址方式非常类似，也称为偏移寻址

- 堆栈寻址

堆栈以先进后出的方式存储数据，通常有存储器堆栈和寄存器堆栈两种

- 存储器堆栈

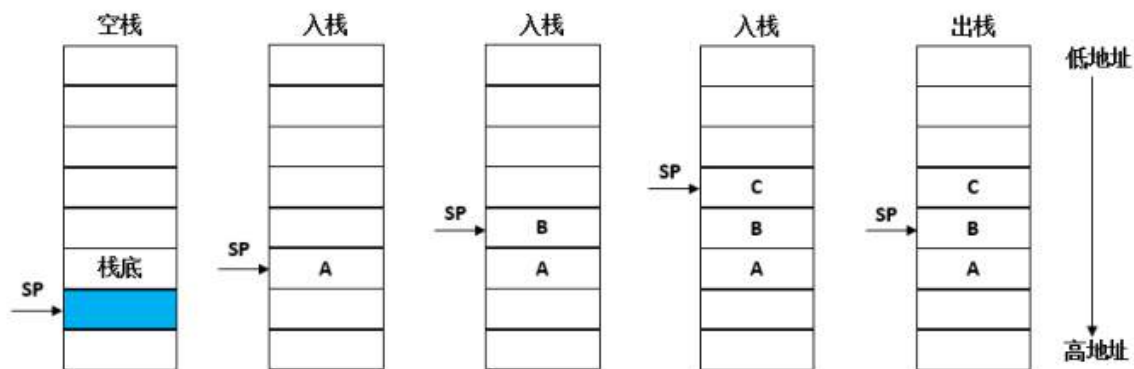


图5.11 存储器堆栈进出栈的操作过程

- 入栈: $SP = SP - 1$, $M[SP] = R$
- 出栈: $M[SP] = R$, $SP = SP + 1$
- 若进出栈数据为16位, 要-2; 32位为-4

• 寄存器堆栈

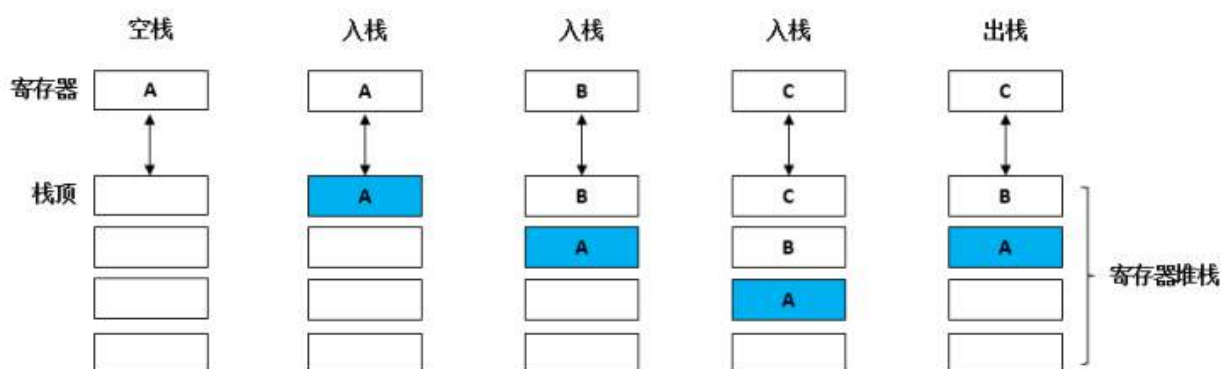


图5.12 寄存器堆栈进出栈的操作过程

不需要SP

• 两者比较

- 寄存器堆栈栈顶固定不动, 存储器堆栈栈顶随着堆栈操作而移动
- 进行堆栈操作时, 寄存器堆栈中的数据移动, 而存储器堆栈中的数据不动
- 寄存器堆栈速度快, 但容量有限; 存储器堆栈速度慢, 但容量很大
- 寄存器堆栈必须采用专用堆栈指令进行控制, 存储器堆栈则不一定

• 混合寻址

- **变址+间接寻址方式**

先进行变址寻址，再进行间接寻址

有效地址 $EA = (R[X] + D)$ ， $R[X]$ 为变址寄存器X的内容，D为形式地址

- **间接+变址寻址方式**

先进行间接寻址，再进行变址寻址

有效地址 $EA = R[X] + (D)$ ，其中 $R[X]$ 为变址寄存器X的内容，D为形式地址

- **相对+间接寻址方式**

先进行相对寻址，再进行间接寻址

有效地址 $EA = (PC + D)$ ，其中PC为程序计数器，D为形式地址

指令类型

- **算术逻辑运算指令**

包括定点、浮点数的加、减、乘、除等算术运算指令，与、或、非、异或等逻辑运算指令，有些计算机还专门设置了十进制运算指令

- **移位操作指令**

- 算术移位指令是针对带符号数的移位操作，包括算术左移指令和算术右移指令
- 逻辑移位指令是针对无符号数的移位操作，包括逻辑左移指令和逻辑右移指令



- 循环移位指令



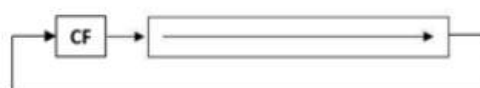
循环左移



带进位 (CF) 循环左移



循环右移



带进位 (CF) 循环右移

• 数据传送指令

完成两个部件之间的数据传送

• 通用数据传送指令(MOV)

支持寄存器与寄存器之间、寄存器与存储器之间、存储器与存储器之间的数据传送

• 存储器读写指令(LOAD/STORE)

访问存储器，此时MOV指令只能用于寄存器之间的数据传送

• 交换指令(SWAP)

双向数据传送

• 串传送指令(MOVS)

与REP配合将数据块从一个区域传送到另一个区域

• 堆栈操作指令

• 压栈指令(PUSH)

• 出栈指令(POP)

• 字符串处理指令

属于非数值处理指令

包括字符传送、字符串比较、字符串查找、字符串抽取、字符串转换等指令

但不是所有的计算机都支持字符串处理指令

• 程序控制指令

• 转移指令

包括无条件转移指令(JMP)和条件转移指令

通常在同一个程序内转移，可以跨越不同CS代码段

- 循环控制指令
- 子程序调用和返回指令

包括CALL指令和RET指令

CALL指令会在不同程序之间转移，其下一条指令的地址会被记为断点，断点将被压入堆栈。

而RET指令将会从堆栈取出断点

- **输入输出指令**
 - 输入指令(IN)：IN AL, DX将以寄存器DX为端口地址的输入设备的内容送到寄存器AL中
 - 输出指令(OUT)：OUT DX, AL将寄存器AL的内容送到以寄存器DX为端口地址的输出设备中
 - 当外部设备与主存采用统一编址时，无需专用的输入输出指令，可以使用访存指令实现输入和输出指令的功能

- **其他指令**

包括停机、等待、空操作、开中断、关中断、自陷、置条件码、特权指令等

其中，特权指令一般只能用于操作系统或其他系统软件，而不直接提供给用户使用

CISC和RISC

复杂指令系统计算机(CISC)

拥有庞大复杂的指令系统，有如下特点

- 指令系统复杂庞大，指令数目多达200-300条
- 寻址方式多
- 指令格式多
- 指令字长不固定
- 对访存指令不加限制
- 各种指令使用频率相差大

- 各种指令执行时间相差大
- 大多数采用微程序控制器

精简指令系统计算机(RISC)

考虑到80%的程序仅用了20%的指令，RISC体系结构对CISC进行了简化以提高处理器性能
它有如下特点

- 优先选择使用频率最高的一些简单指令或有用但不复杂的指令，避免使用复杂指令
- 大多数指令在一个时钟周期内执行完成
- 采用LOAD/STORE结构，只允许LOAD/STORE指令访问主存，其余指令只能对寄存器操作数进行处理
- 采用简单的指令格式和寻址方式，指令长度固定
- 有固定的指令格式。指令长度、格式固定，可简化指令的译码逻辑，有利于提高流水线的执行效率

为了便于编译的优化，常采用三地址指令格式

- 面向寄存器的结构。为减少访问主存，CPU内设大量的通用寄存器
- 采用硬布线控制逻辑。由于指令系统的精简，控制部件可由组合逻辑实现，不用或少用微程序控制
- 注重编译的优化，力求有效地支持高级语言程序

两者对比如表所示

对比项目	CISC	RISC
指令系统	复杂，庞大	简单，精简
指令数目	一般大于200条	一般小于100条
指令字长	不固定	定长
可访存指令	不加限制	只有LOAD/STORE指令
各种指令执行时间	相差较大	绝大多数在一个周期内完成
各种指令使用频度	相差很大	都比较常用
通用寄存器数量	较少	多

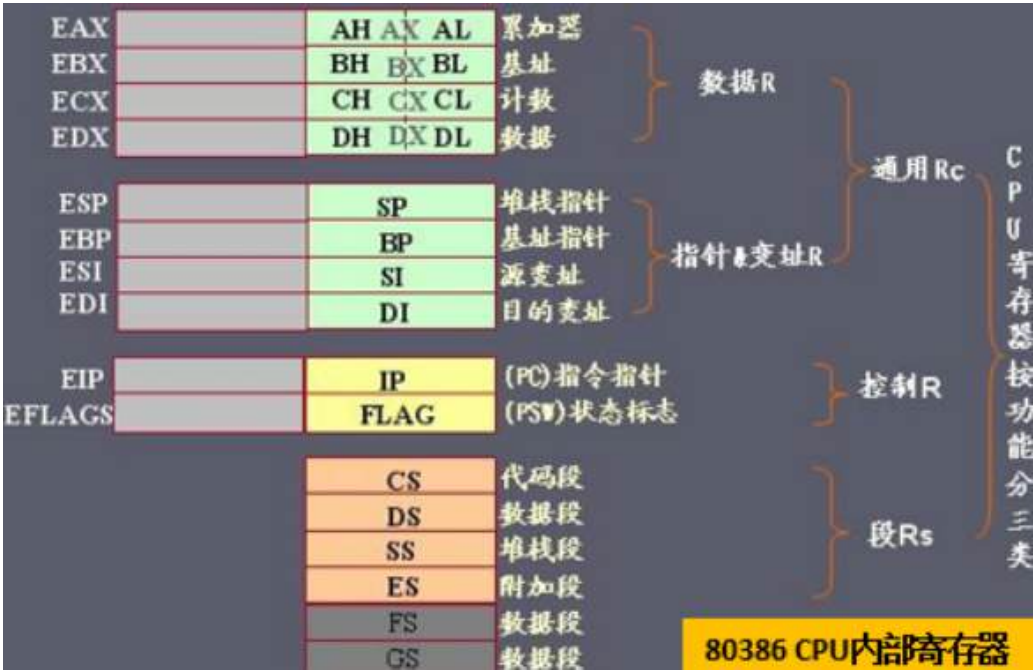
对比项目	CISC	RISC
目标代码	难以用优化编译生成高效的目标代码程序	采用优化的编译程序，生成代码较
控制方式	绝大多数为微程序控制	绝大多数为组合逻辑控制
指令流水线	可以通过一定方式实现	必须实现

指令系统实例

Intel x86指令系统

是由16位的8086/8088指令系统发展而来的32位CISC指令系统

80386CPU采用32位的Intelx86指令系统，有8个32位通用寄存器、6个16位的段寄存器，以及32位的状态寄存器和程序计数器



x86系统采用变长指令格式，包括前缀、操作码、ModR/M，SIB，偏移量Disp，立即数Imm

指令格式

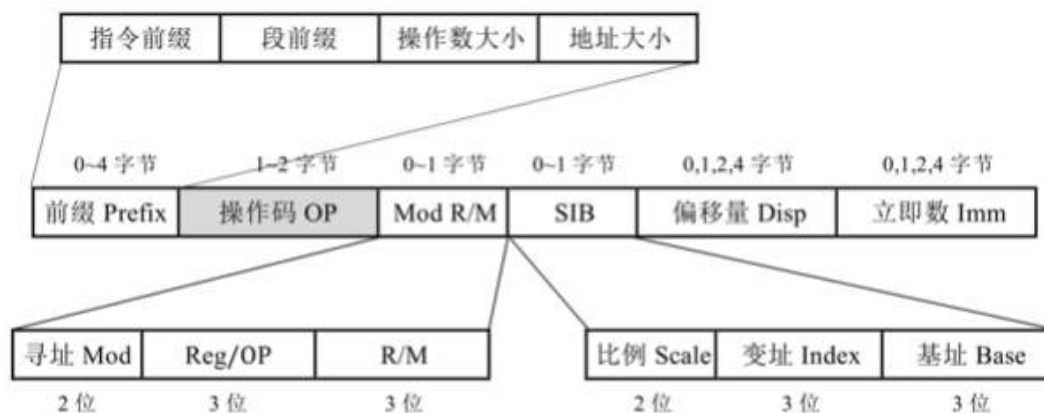


图5.15 Intel x86 指令格式

• 前缀

• 指令前缀：包括锁定前缀和重复前缀两种

- 锁定前缀 (Lock) 用于多处理器环境中共享存储器的排他性访问，值为F0H
- 重复前缀 (REPNE/REPBZ, REP/REPE/REPZ) 用于进行字符串循环处理操作
 - REPNE/REPZ值为F2H，当ECX≠0且两数不等时重复执行
 - REPE/REPZ值为F3H，当ECX≠0且两数相等时重复执行
 - REP值为F3H，当ECX≠0时重复

• 段前缀：指定使用哪个段寄存器取代默认的段寄存器，共有CS,DS,SS,ES,FS,GS六个段寄存器，对应的值分别为2EH,3EH,36H,26H,64H,65H

• 操作数大小：在32位和16位操作数之间切换，字段值为66H

• 地址大小：在32位地址和16位地址空间之间切换，字段值为67H

它决定了指令格式中偏移量的大小和在有效地址计算中生成的偏移量的大小

• 操作码OP

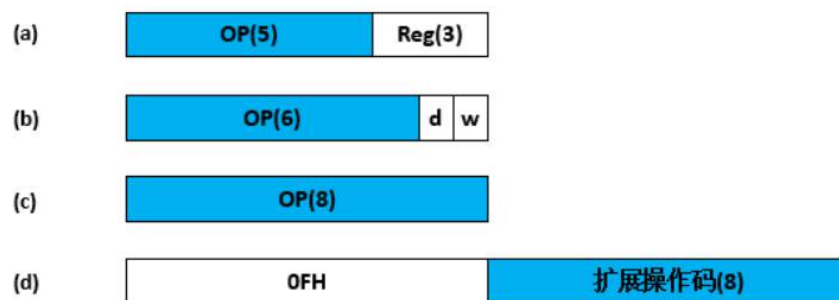


图5.17 Intel x86 指令操作码字段格式

- a: 代表单地址指令，OP5位，REG3位以表示8个寄存器
 - b: 代表双地址指令，OP6位
 - d为1时表示ModR/M字段中寄存器操作数为目的寄存器，为0时表示为源寄存器
 - w为1时表示为16位或32位操作数，为0时表示8位操作数
 - c: 可以代表零地址、单地址、双地址指令，OP8位
 - d: OP16位，高8位为0FH，低8位为扩展操作码
 - Mod R/M
- ModR/M字段用于描述操作数及其寻址方式
- 在Intelx86指令集中，双操作数最多只能有一个存储器操作数
- 寻址Mod: 2位，与R/M字段一起构成8种寻址方式
 - Reg/OP: 3位，表示寄存器操作数的编号，若为单地址指令，则为操作码的扩展字段OP
 - R/M: 3位，表示另一个操作数，可能为寄存器或存储器

表5.4 Mod R/M字段对应的寻址方式

Mod	R/M值	操作数类型	寻址方式	有效地址EA/操作数S
00		存储器	寄存器间接寻址	EA=R[R/M] S在内存
00	100	存储器	基址+比例变址寻址	EA=SIB
00	101	存储器	直接寻址（偏移量）	EA=Disp32
01		存储器	寄存器相对寻址	EA=R[R/M]+Disp32
01	100	存储器	基址+比例变址+偏移量寻址	EA=SIB+Disp8
10		存储器	寄存器相对寻址	EA=R[R/M]+Disp32
10	100	存储器	基址+比例变址+偏移量变址	EA=SIB+Disp32
11		寄存器	寄存器寻址	S=R[R/M] S在寄存器

• SIB

SIB字段与ModR/M字段组合以指定寻址方式

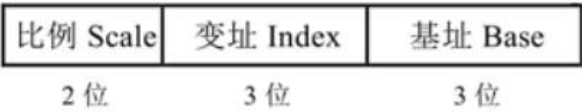


图5.19 SIB字段

- 比例Scale：2位，指定比例变址中的比例因子，Scale为0~3之间，表示比例因子为1、2、4、8
- 变址Index：指定变址寄存器的编号
- 基址Base：指定基址寄存器的编号

SIB字段对应有效地址为

$$EA = R[Base] + 2^{Scale} \times R[Index]$$

• 偏移量Disp

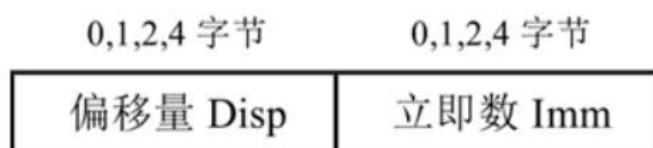
可以为一字节、两字节、四字节或没有字节

偏移量字段是否存在与ModR/M字段中的寻址方式有关

- 立即数Imm字段

立即数Imm字段提供指令所需的8位（1字节）、16位（2字节）、32位（4字节）的立即数操作数，也可以没有（0字节）

立即数字段是否存在与指令操作码有关



指令译码

1. 前缀分析：首先判断有无前缀，有则需要根据不同的前缀值进行指令功能控制
2. 操作码译码：判断是单字节还是双字节操作码
3. 寻址方式译码：根据ModR/M字段，确定指令中的操作数和寻址方式，并配合后续的SIB字节、Disp（偏移量）字段、Imm（立即数）字段，计算操作数地址

寻址方式

x86处理器采用段页式存储管理机制，使用逻辑地址、线性地址、物理地址3类地址

- 逻辑地址：为段式虚拟地址，包括段寄存器和段内偏移地址，即指令中的有效地址EA
- 线性地址：页式虚拟地址
- 物理地址：实地址

当操作数为存储单元时，需要进行地址转换

1. 通过分段方式将逻辑地址转换为线性地址
 2. 通过分页方式将线性地址转换为物理地址
- 实模式：物理地址（20位）=段寄存器（16位）左移4位+段内偏移地址
 - 保护模式：线性地址（32位）=段基址（32位）+段内偏移地址；再将线性地址通过存储器管理部件（MMU）转换为物理地址（32位）

MIPS指令系统

MIPS指令系统是一种非常优雅、简洁、高效的RISC体系结构，包括MIPS32，MIPS64

寄存器

表5.7 MIPS的通用寄存器

寄存器#	助记符	释义
\$0	\$zero	固定值为0 硬件置位
\$1	\$at	汇编器保留，临时变量
\$2~\$3	\$v0~\$v1	函数调用返回值
\$4~\$7	\$a0~\$a3	4个函数调用参数
\$8~\$15	\$t0~\$t7	暂存寄存器，被调用者按需保存
\$16~\$23	\$s0~\$s7	save寄存器，调用者按需保存
\$24~\$25	\$t8~\$t9	暂存寄存器，同上
\$26~\$27	\$k0~\$k1	操作系统保留，中断异常处理
\$28	\$gp	全局指针 (Global Pointer)
\$29	\$sp	堆栈指针 (Stack Pointer)
\$30	\$fp	帧指针 (Frame Pointer)
\$31	\$ra	函数返回地址 (Return Address)

- 32个32位通用寄存器：31，并有对应的助记符。其中\$fp用于过程调用，\$ra用于子程序返回地址
- 32个32位单精度浮点寄存器：f0~f31，可配对为16个64位的浮点寄存器
- 2个特殊的寄存器：*hi*、*lo*，用于保存乘（64位的乘积）、除（商和余数）运算结果

指令格式

指令均为32位定长指令，包括寄存器R型，立即数I型，无条件转移J型

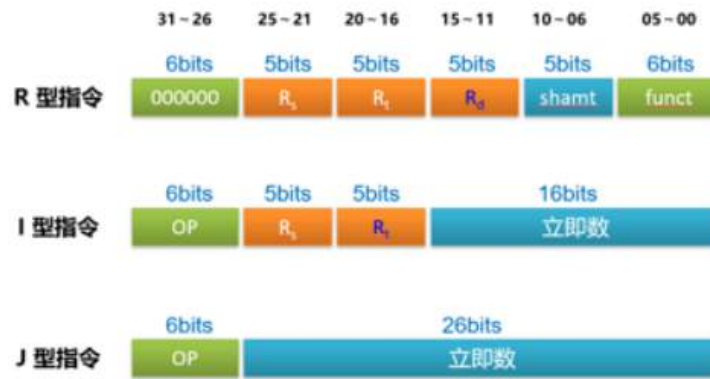


图5.24 MIPS的指令格式

- 操作数OP为6位，R型指令OP为000000，用扩展操作码的方式，指令功能由6位funct字段决定
 - R_s , R_t , R_d 为寄存器操作数字段，均为5位，可以访问32个通用寄存器
 - shamt字段为5位的移位变量，用于移位指令
 - 立即数可能为16位(I型)或26位(J型)
 - 指令字中没有独立的寻址方式字段，指令的寻址方式由操作码决定
- 其指令包括
- R型指令

R型指令的操作数只能是寄存器，运算结果也只能存入寄存器

- 若为双目运算， r_s 、 r_t 分别是第一和第二源操作数， r_d 为目的操作数
- 若为移位运算，则表示对 r_t 的内容进行移位，移位位数由shamt指定

表5.8 常用R型指令及其funct编码

funct	指令助记符	指令功能描述	备注
00	<u>sll rd,rt,shamt</u>	$R[rd]=R[rt]<<shamt$	逻辑左移, 注意rs字段未使用
02	<u>srl rd,rt,shamt</u>	$R[rd]=R[rt]>>shamt$	逻辑右移, 注意rs字段未使用
03	<u>sra rd,rt,shamt</u>	$R[rd]=R[rt]>>shamt$	算术右移, 注意rs字段未使用
04	<u>sliv rd,rt,rs</u>	$R[rd]=R[rt]<<R[rs]$	可变左移
08	<u>jr rs</u>	$PC=R[rs]$	$R[rs]$ 值应是4的倍数, 字对齐
09	<u>jalr rs</u>	$R[31]=PC+8$ $PC=R[rs]$	子程序调用
12	<u>syscall</u>	系统调用	无操作数
16	<u>mfhi rd</u>	$R[rd]=HI$	取HI寄存器的值, mflw取LO
17	<u>mfhi rs</u>	$HI=R[rs]$	存HI寄存器的值, mflw存LO
24	<u>mult rs,rt</u>	$\{HI,LO\}=R[rs]*R[rt]$	有符号乘, 64位结果送入HI、LO寄存器
32	<u>add rd,rs,rt</u>	$R[rd]=R[rs]+R[rt]$	溢出时发生异常, 且不修改R[rd]
34	<u>sub rd,rs,rt</u>	$R[rd]=R[rs]-R[rt]$	溢出时发生异常, 且不修改R[rd]
36	<u>and rd,rs,rt</u>	$R[rd]=R[rs]\&R[rt]$	逻辑与
37	<u>or rd,rs,rt</u>	$R[rd]=R[rs] R[rt]$	逻辑或
42	<u>slt rd,rs,rt</u>	$R[rd]=(R[rs]<R[rt])?1:0$	小于置位指令, 有符号比较

• I型指令

包含立即数的指令

- 若是双目运算, 则将源操作数rs和立即数imm分别作为第一和第二操作数, 结果送rt
- 若是Load/Store指令, 则将rs的值和立即数imm的值相加, 生成存储单元地址
- 若是条件分支指令, 则对rs和rt的值进行比较, 并根据比较结果决定是否转移, 转移的目标地址采用相对寻址方式

表5.9 常用I型指令及其操作码

OP	指令助记符	指令功能描述	备注
04	<u>beq rs,rt,imm</u>	If($R[rs]==R[rt]$) $PC=PC+4+imm<<2$	条件分支(相等跳转) imm左移2bit, 末位00, 确保地址为4的倍数, 因为字长是32bit=4Byte
05	<u>bne rs,rt,imm</u>	If($R[rs]!=R[rt]$) $PC=PC+4+imm<<2$	条件分支(不等跳转)
08	<u>addi rt,rs,imm</u>	$R[rt]=R[rs]+imm$	立即数加, 溢出发生异常
10	<u>slti rt,rs,imm</u>	$R[rt]=(R[rs]<imm)?1:0$	小于置位指令, 有符号比较
12	<u>andi rt,rs,imm</u>	$R[rt]=R[rs]\&imm$	立即数逻辑与指令
15	<u>lui rt,imm</u>	$R[rt]=imm<<16$	加载立即数指令, 长度对齐, 左移16bit, 正好是32bit
35	<u>lw rt,imm(rs)</u>	$R[rt]=M[R[rs]+imm]$	取数指令, 类似指令还有lb、lh、lbh等
43	<u>sw rt,imm(rs)</u>	$M[R[rs]+imm]=R[rt]$	存数指令, 类似指令还有sb、sh

• J型指令

主要为无条件转移指令

指令中给出的为26位直接地址, J型指令通过伪直接寻址方式进行寻址



PC+4的高4位(4bit)和address经左移2位后（28bit）的值拼接得到目标地址

如PC+4=25000000H， address=1234567H， PC+4的高4位=2H， address左移2位=0100100011010001010110011100=0100100011010001010110011100=48D159CH， 拼接后的目标地址=248D159CH

表5.10 常用J型指令及其操作码

OP	指令助记符	指令功能描述	备注
02	j address	PC <- {(PC+4) _{31:28} , address, 00}	无条件分支
03	jal address	R[31] <- PC+8 (无延迟槽+4) PC <- {(PC+4) _{31:28} , address, 00}	子程序调用指令

中央处理器

中央处理器概述

中央处理器的功能

中央处理器由**运算器**和**控制器**组成，是计算机的核心

CPU的主要功能是**执行程序**，周而复始地进行取指令和执行指令的工作

CPU应该具有的功能有

- **程序控制**：控制程序中的指令执行顺序，CPU必须能够正确地确定下条指令的地址
- **操作控制**：产生指令执行过程中需要的操作控制信号
- **时序控制**：对每个操作控制信号进行定时，以便按规定的时间顺序执行各操作，控制各功能部件。
- **数据加工**：对数据进行算术、逻辑运算，或将数据在各部件之间传送。
- **中断处理**：CPU应能及时响应内部异常和外部中断请求。

中央处理器的组成

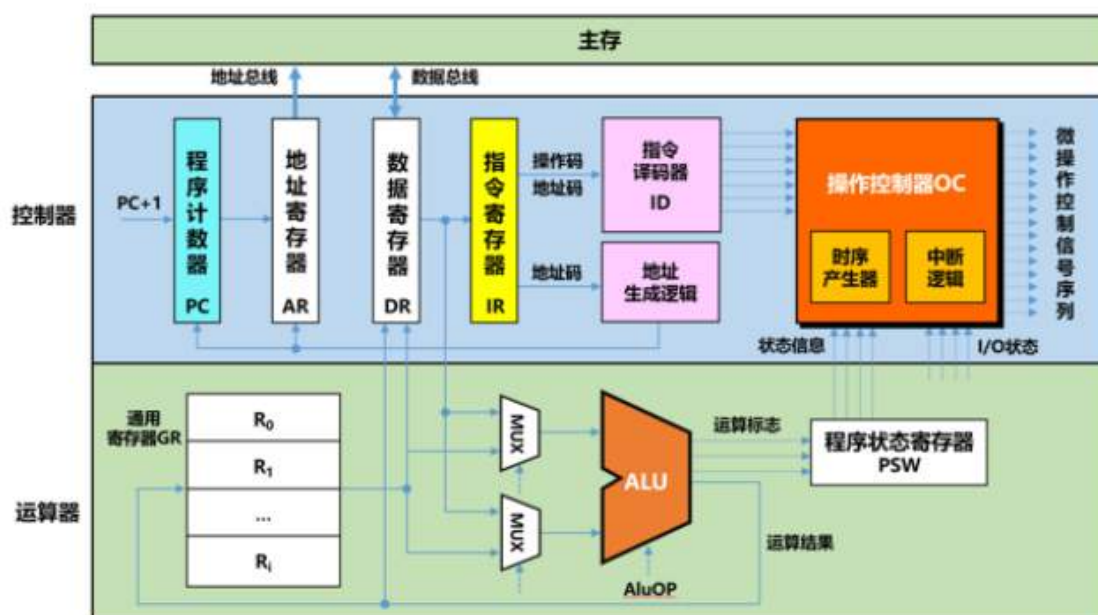


图6.1 CPU的基本组成

- **运算器**是执行部件，由ALU和各种寄存器组成，负责数据加工。
- **控制器**负责取指令、计算下一条指令的地址、译码、生成指令对应的操作控制信号序列、控制指令执行的步骤和数据流动的方向

寄存器

- **程序计数器(PC):** Intelx86称为IP，用于保存将要执行的指令的**字节地址**。
- **存储器地址寄存器(AR/MAR):** 保存CPU访问主存的单元地址
- **存储器数据寄存器(DR/MDR):** 用于存放从主存中读出的数据或准备写入主存的数据
- **指令寄存器(IR):** 用于保存当前正在执行的指令
 - 指令中的操作码、地址码送指令译码器
 - 指令中的地址码送地址生成逻辑

- **通用寄存器组(GR):** 也称寄存器堆

通用寄存器可以作为ALU的累加器、变址寄存器、基址寄存器、地址指针、数据缓冲器用来存放操作数、中间结果以及各种地址信息

- **程序状态字寄存器(PSW/PSR):** Intelx86称为EFLAGS，用于保存由算术运算指令、逻辑运算

指令、测试指令等建立的各种条件标志

常见的标志

- C（进位标志）
- V（溢出标志）
- S（结果为负数标志）
- Z（结果为零标志）
- 其他中断和系统工作的状态信息

操作控制器

操作控制器接收指令译码器送来的**指令译码信号**

指令译码信号与时序信号、条件及状态信息进行组合，形成各种具有严格时间先后顺序的操作控制信号，也就是微操作控制信号序列

CPU执行指令的过程就是CPU控制信息流的过程，操作控制器产生的微操作控制信号序列就是控制流

时序产生器

指令执行过程中的所有操作都必须按照一定的次序执行，因此需要引入**时序**

也即对完成指令而执行的微操作控制信号进行时间调制，严格规定各信号的产生时间和持续时间

根据时序调制方法的不同，操作控制器分为**硬布线控制器(硬时序)**和**微程序控制器(软时序)**

指令执行

基本概念

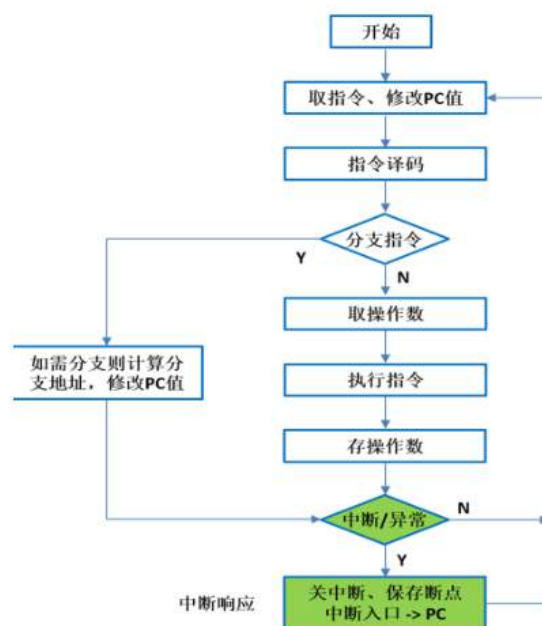


图6.2 指令执行的一般流程

指令执行一般流程如图所示

指令周期即一条指令从取出到执行完成所需要的时间，分为取指周期、译码/取操作数周期、执行周期、写回周期和一些其他周期

• 取指周期

取指周期中，CPU以程序计数器PC的内容为地址从主存取出指令，并计算后续指令的地址作为PC的新值

- 顺序执行指令时，PC的新值为PC加当前指令的字节长度
- 分支执行指令时，要根据寻址方式、分支条件、分支目标地址等内容，计算PC的新值

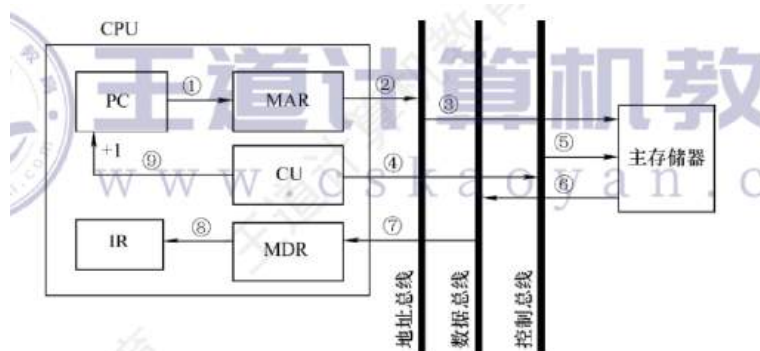


图 5.4 取指周期的数据流

• 译码/取操作数周期

译码/取操作数周期中，对指令寄存器 IR 中的指令字进行译码，识别指令类型
根据指令地址码生成操作数的有效地址，然后访问相应的寄存器或主存单元

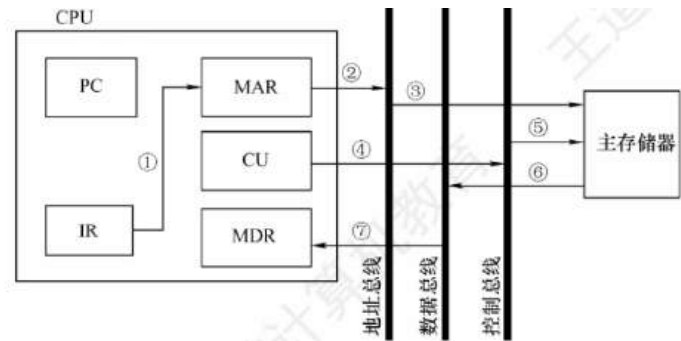


图 5.5 一次间址周期的数据流

💡 当寻址方式为**间接寻址**时，则需要加入访存周期才能得到操作数的地址，这个访存周期又称为**间址周期**

在指令 `MOV EAX,@2008H` 中，访问地址为 `2008H` 主存单元的访存周期称为**间址周期**

假设 `2008H` 处存的是 `A0A0F000H`，访问 `@2008H(A0A0F000H)` 的访存周期

• 执行周期

控制器向ALU及数据通路中的其他相关部件发出操作控制命令处理操作数，并将处理的状态信息记录在PSW中。

当程序出现分支时，执行周期还要计算分支的目标地址。

• 中断周期：当指令执行完毕后，如果出现异常或外部中断请求，CPU将进入中断响应周期（中断周期）

CPU将利用硬件逻辑保存断点，然后将中断处理程序入口地址送入程序计数器PC，实现当前程序与中断处理程序的切换。

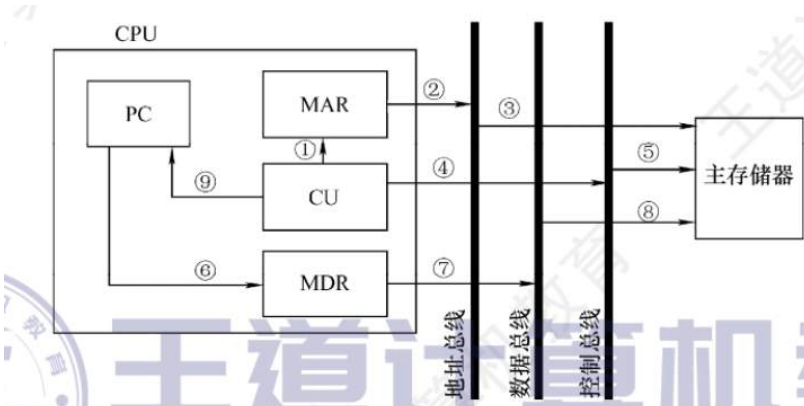


图 5.6 中断周期的数据流

还有一些其它周期，如

- **写回周期**：将运算结果写回到目的寄存器或存储器中
- **总线周期**：用于完成总线操作及总线控制权的转移
- **I/O周期**：用于完成输入输出操作

可以将指令周期划分为若干个机器周期(CPU周期)，每个机器周期又可包含若干个时钟周期

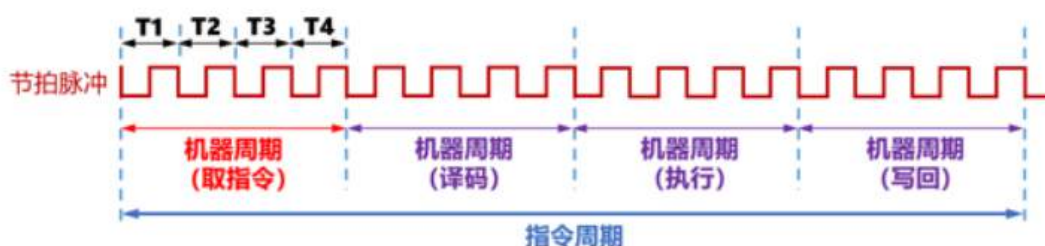


图6.3 指令周期、机器周期与时钟周期的关系

✎ 从主存取出一个存储字所需的最短时间定义为机器周期

寄存器传送语言

寄存器传送语言 (RTL) 用来表示指令执行过程中的操作

每条指令的执行过程都可以分解为一组操作序列，进而可分解为一组微操作序列

✎ 操作是功能部件级的动作，微操作是指令序列中最基本的、不可再分割的动作

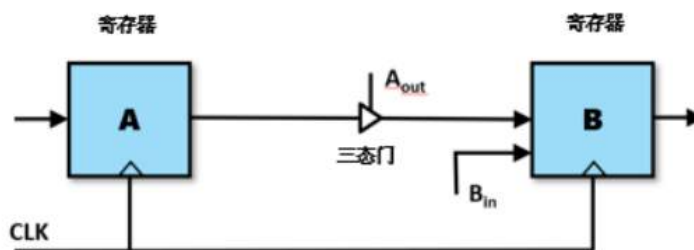


图6.4 指令执行过程的操作和微操作

💡 在此图中，微操作Aout表示控制三态门将寄存器A的输出数据送入寄存器B的输入端口
微操作Bin用于打开寄存器B的写使能控制信号，通过时钟信号的配合最终将数据锁存到寄存器B中

RTL的描述规则如下：

- **M[addr]**

表示主存单元或内容；addr为主存单元的地址

作为源操作数使用时表示主存内容，作为目的操作数使用时表示主存单元

- **R[i]**

表示通用寄存器组中的第i号寄存器或内容；i为寄存器编号

作为源操作数使用时表示寄存器内容，作为目的操作数使用时表示寄存器

- **A->B/B<-A**：表示数据传送，其中A为源操作数，B为目的操作数

- **X_{y:z}**：表示寄存器X的第y位到第z位的数据字段

- **SignExt(X)**：将X带符号扩展到32位

- {X,Y}：表示将X、Y的比特位连接在一起

 **M[PC]**表示程序计数器PC所指向的主存单元的内容，而**M[R[i]]**表示以第i号寄存器内容作为地址的主存单元（寄存器间接寻址）

指令执行方案

- **单周期处理器**

单周期处理器对所有指令都选用**相同的执行时间**来完成。此时每条指令都在一个时钟周期内完成（CPI=1），指令之间串行执行

因此，指令周期取决于**执行时间最长的指令的执行时间**

对于那些本来可以在更短时间内完成的指令，仍要在这个较长的周期内完成，会降低整个系统的运行速度

- **多周期处理器**

多周期处理器对不同类型的指令选用不同的执行步骤。指令需要几个周期就为其分配几个周期。

因此可选用不同个数的时钟周期来完成不同指令的执行过程（CPI>1），不再要求所有指令占用相同的执行时间

- **流水线处理器**

流水线处理器采用**指令之间并行执行**的方案，其追求的目标是力争在每个时钟周期完成一条指令的执行过程。

这种方案通过在每个时钟周期启动一条指令，尽量让多条指令同时运行，但各自处在不同的执行步骤中。

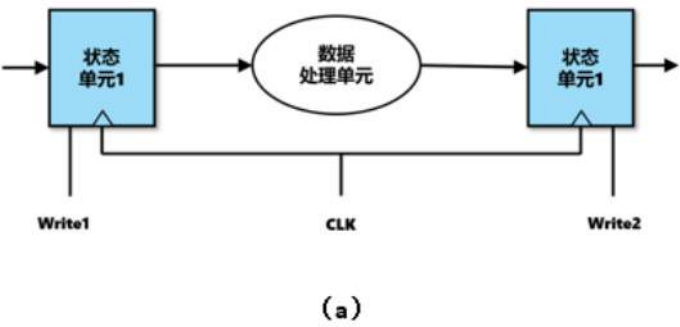
数据通路及指令操作流程

数据通路的组成

数据通路是数据在各功能部件之间传送的路径，可用总线或专用通路两种方式来构建
构成数据通路的功能部件根据性质可分为**数据处理单元**和**状态存储单元**



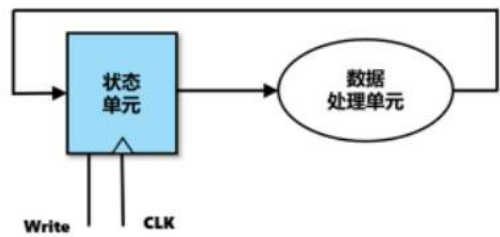
- **数据处理单元**：由组合逻辑电路构成，其输出只与当前的输入有关，负责对数据进行加工和处理
如ALU、符号扩展单元、译码器等
- **状态存储单元**：也称状态单元，是指带有存储功能的单元，如存储器和寄存器等
状态单元一般包含输入端、写使能控制端、时钟输入端CLK、输出端
在数据通路模型中，一个**数据处理单元**连接**两个状态单元**



各状态单元只有在写使能信号有效，且时钟触发到来时，才能将输入端的数据写入，还要遵

循较为严格的时序约定

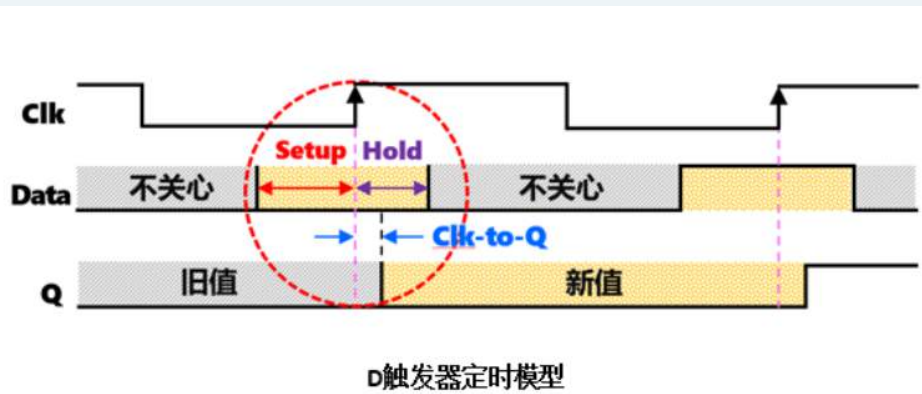
数据通路模型也可以简化为只有一个数据处理单元和一个状态单元



数据通路最小时钟周期

存储器、寄存器等状态单元的定时模型与D触发器相同

i D触发器定时模型



- 建立时间(set_{up}): 时钟Clk触发前 (上跳沿), 输入数据Data须稳定一段时间
- 保持时间(hold): 时钟Clk触发后 (上跳沿), 输入数据Data还须稳定一段时间
- 触发器延迟(Clk_{to}Q): 从时钟Clk触发 (上跳沿), 到输出Q稳定的时间

在这个模型中, 设

- 数据处理单元对状态单元1的输出数据处理加工需要一个关键路径延迟 $T_{\max}$ (数据处理单元所有输出信号稳定的最大延迟时间)
- 数据通路最小时钟周期为 t

则必须满足

$$t > T_{clk_to_q} + T_{max} + T_{setup}$$

否则，将导致数据处理单元（组合逻辑）的输出来不及送到寄存器B

! 数据通路保持时间违例

设数据处理单元（组合逻辑）的最短路径延迟为 T_{min} ，即数据处理单元所有输出信号稳定的最小延迟时间

则必须满足

$$t_{hold} < T_{clk_to_q} + T_{min}$$

否则在状态单元2还没有写入完成旧值时，又需要写入新值，这也就是保持时间违例

数据通路实例

单总线结构的数据通路

单总线的计算机数据通路中，CPU中的运算器、控制器、寄存器堆通过一条内总线(CPU内总线)连接

而连接CPU、存储器和输入输出设备的总线称为**系统总线（外总线）**

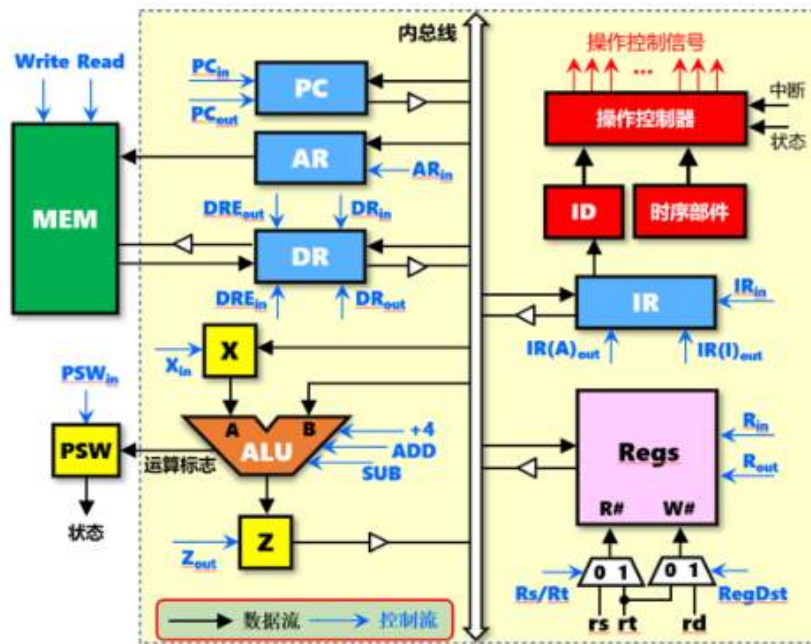


图6.8 单总线结构的计算机框图

功能模块

- PC：程序计数器。输入来自内总线，输出送到内总线。
- AR：存储器地址寄存器。输入来自内总线，输出送到存储器MEM。
- DR：存储器数据寄存器。输入来自内总线或存储器，输出送到内总线或存储器。
- X：暂存器；输入来自内总线，输出送到ALU。
- Z：暂存器；输入来自ALU，输出送到内总线。
- ALU：算术逻辑单元；输入A来自暂存器X，输入B来自内总线，输出送到暂存器Z。
- PSW：程序状态字寄存器；输入来自ALU，输出送到操作控制器。
- IR：指令寄存器；输入来自内总线，输出送到内总线和指令译码器ID。
- ID：指令译码器；输入来自IR，输出送到操作控制器。
- Regs：寄存器堆；输入来自内总线，输出送到内总线；R为读出的寄存器编号，W为写入的寄存器编号
- MEM：存储器；地址由AR提供，数据来自或送到DR。

单总线结构的计算机，总线上连接很多功能部件

为了防止总线上出现数据冲突，某一时刻只允许一个功能部件向总线发送数据

因此，每个功能部件的输出端都采用三态门

• 控制信号和典型MIPS32指令介绍

表6.1 控制信号及其功能

序号	控制信号	功能说明
1	PC_{in}	控制PC接收来自内总线的的数据，需配合时钟控制
2	PC_{out}	控制PC向内总线输出数据
3	AR_{in}	控制AR接收来自内总线的的数据，需配合时钟控制
4	DR_{in}	控制DR接收来自内总线的的数据，需配合时钟控制
5	DR_{out}	控制DR向内总线输出数据
6	DRE_{in}	控制DR接收从主存读出的数据，需配合时钟控制
7	DRE_{out}	控制DR向主存输出数据，以便最后将该数据写入主存
8	X_{in}	控制暂存器X接收来自内总线的的数据，需配合时钟控制
9	+4	将ALU的A端口的数据加4输出
10	ADD	控制ALU执行加法，实现A端口和B端口的两个数相加
11	SUB	控制ALU执行加法，实现A端口和B端口的两个数相减
12	PSW_{in}	控制程序状态字寄存器PSW接收ALU的运算状态，需配合时钟控制
13	Z_{out}	控制暂存器Z向内总线输出数据，注意暂存器Z没有输入控制，每个时钟都会锁存新值
14	IR_{in}	控制IR接收来自内总线的的数据，需配合时钟控制
15	$IR(A)_{out}$	控制IR中的分支目标地址输出到内总线，指令字中的立即数要转换成目标地址，需要相应逻辑
16	$IR(I)_{out}$	控制IR中的立即数输出到内总线，指令字中的立即数符号扩展为32位才能输出
17	Write	存储器写命令，需配合时钟控制
18	Read	存储器读命令
19	R_{in}	控制寄存器堆接收来自内总线的的数据，写入W#端口对应的寄存器中，需配合时钟控制
20	R_{out}	控制寄存器堆输出指定编号R#寄存器的数据，该寄存器堆为单端口输出
21	Rs/Rt	控制多路选择器选择送入R#的寄存器编号，为0时送入指令字中的rs字段，为1时送入rt
22	$RegDst$	控制多路选择器选择送入W#的寄存器编号，为0时送入指令字中的rt字段，为1时送入rd

根据指令周期的概念，任何指令的第一个机器周期均为取指周期，在取值周期内需要

1. $M[PC] \rightarrow IR$ ：以程序计数器PC值为地址从存储器中取出指令，并送入指令寄存器IR中保存
2. $PC + \text{指令长度} \rightarrow PC$ ：计算顺序指令的地址，修改PC的值，因为MIPS32是32位定长指令，因此这里是： $PC + 4 \rightarrow PC$
3. 指令译码：以确定指令在执行阶段将要进行何种操作

而后则是执行周期，根据不同的指令执行不同操作

下面是不同指令的执行流程

• lw指令

lw指令（取数指令）属于I型指令，汇编代码为


```
lw rt,imm(rs)
```

其RTL功能描述为 $R[rt] \leftarrow M[R[rs] + \text{SignExt}(imm)]$

lw指令执行需要3个机器周期

1. 第一个机器周期

取指周期，使用两条数据通道

- PC→AR→MEM→DR→IR;以PC为地址访存，取指令，并送入IR
- PC→X→ALU→Z→PC;修改PC的值，为取下一条指令做准备

2. 第二个机器周期

计算周期，用于计算访存地址，使用数据通道为 $R[rs] \rightarrow X \rightarrow ALU; IR(I) \rightarrow ALU \rightarrow Z$

3. 第三个机器周期

执行周期，使用数据通道 $Z \rightarrow AR \rightarrow MEM \rightarrow DR \rightarrow R[rt]$

表6.3 lw指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	PC → AR; PC → X	将程序计数器PC内容送AR，同时送入暂存器X	$PC_{out} = AR_{in} = X_{in} = 1$
	T_2	$X + 4 \rightarrow Z$	将PC值加4并送入暂存器Z	$+4 = 1$
	T_3	$Z \rightarrow PC; M[AR] \rightarrow DR$	将暂存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out} = PC_{in} = 1$ $Read = DRE_{in} = 1$
	T_4	DR → IR	将DR内容送入IR，完成取指令	$DR_{out} = IR_{in} = 1$
计算周期 M_{cal}	T_1	$R[rs] \rightarrow X$	将rs寄存器内容送入暂存器X，准备计算访存地址	$R_{out} = X_{in} = 1$
	T_2	$IR(I) + X \rightarrow Z$	将IR中的立即数符号扩展为32位并送入ALU做加法运算	$IR(I)_{out} = ADD = 1$
执行周期 M_{ex}	T_1	$Z \rightarrow AR$	将Z中暂存的访存地址送入AR	$Z_{out} = AR_{in} = 1$
	T_2	$M[AR] \rightarrow DR$	读AR内容对应主存单元的值并送入DR	$Read = DRE_{in} = 1$
	T_3	DR → R[rt]	将DR内容送入寄存器rt	$DR_{out} = R_{in} = 1$

💡 实际上，取指周期中M[AR]→DR，可以放在节拍T3（上面），也可以放在节拍T2（下面），两者功能等价

计算周期中，暂存器X可以是暂存rs寄存器的值（上面），也可以是暂存符号扩展后的立即数的值（下面）；两者结果一样

• sw指令

sw指令（存数指令）也属于I型指令，汇编代码为

```
sw rt,imm(rs)
```

其RTL功能描述为 $R[rt] \rightarrow M[R[rs] + \text{SignExt}(imm)]$

sw指令执行需要3个机器周期

1. 第一个机器周期

取指周期，使用两条数据通道

- PC→AR→MEM→DR→IR;以PC为地址访存，取指令，并送入IR
- PC→X→ALU→Z→PC;修改PC的值，为取下一条指令做准备

2. 第二个机器周期

计算周期，用于计算访存地址，使用数据通道为R[rs]→X→ALU;IR(I)→ALU→Z

3. 第三个机器周期

执行周期，使用数据通道Z→AR;R[rt]→MEM→DR

表6.4 sw指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T ₁	PC → AR; PC → X	将程序计数器PC内容送AR，同时送入暂存器X	$\overline{PC}_{out} = \overline{AR}_{in} = X_{in} = 1$
	T ₂	X+4 → Z	将PC值加4并送入暂存器Z	+4=1
	T ₃	Z → PC; M[AR] → DR	将暂存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out} = \overline{PC}_{in} = 1$ $\overline{Read} = \overline{DRE}_{in} = 1$
	T ₄	DR → IR	将DR内容送入IR，完成取指令	$\overline{DR}_{out} = \overline{IR}_{in} = 1$
计算周期 M_{cal}	T ₁	R[rs] → X	将rs寄存器内容送入暂存器X，准备计算访存地址	$R_{out} = X_{in} = 1$
	T ₂	IR(I) + X → Z	将IR中的立即数符号扩展为32位并送入ALU做加法运算	$\overline{IR(I)}_{out} = \overline{ADD} = 1$
执行周期 M_{ex}	T ₁	Z → AR	将Z中暂存的访存地址送入AR	$Z_{out} = \overline{AR}_{in} = 1$
	T ₂	R[rt] → DR	将rt寄存器内容送入DR	$R_{out} = \overline{Rs/Rt} = \overline{DR}_{in} = 1$
	T ₃	DR → M[AR]	将DR内容写入AR内容所指的主存单元	$\overline{DRE}_{out} = \overline{Write} = 1$

• beq指令

beq指令（条件分支指令）也属于I型指令，汇编代码为

```
beq rs,rt,imm
```

RTL功能描述为If(R[rs]==R[rt])PC<-PC+4+SignExt(imm)<<2

beq指令需要三个机器周期

1. 第一个机器周期

取指周期，使用两条数据通道

- PC->AR->MEM->DR->IR;以PC为地址访存，取指令，并送入IR
- PC->X->ALU->Z->PC;修改PC的值，为取下一条指令做准备

2. 第二个机器周期

计算周期，用于比较两个寄存器的值，并产生用于条件分支的标志位。

R[rs]->X->ALU;R[rt]->ALU->PSW

3. 第三个机器周期

执行周期，负责计算分支目标地址，并根据计算周期生成的标志位，决定是否分支跳转。

PC->X->ALU;IR(A)->ALU->Z;if(PSW.equal)Z->PC

表6.5 beq指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T ₁	PC->AR; PC->X	将程序计数器PC内容送AR，同时送入寄存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T ₂	X+4->Z	将PC值加4并送入寄存器Z	+4=1
	T ₃	Z->PC; M[AR]->DR	将寄存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T ₄	DR->IR	将DR内容送入IR，完成取指令	$DR_{out}=IR_{in}=1$
计算周期 M_{cal}	T ₁	R[rs]->X	将rs寄存器内容送入寄存器X，准备进行比较	$R_{out}=X_{in}=1$
	T ₂	X-R[rt]->PSW	将rt寄存器内容送入ALU做减法，可产生结果为零的标志位	$R_{out}=Rs/Rt=SUB-PSW_{in}=1$
执行周期 M_{ex}	T ₁	PC->X	将PC内容送入寄存器X	$PC_{out}=X_{in}=1$
	T ₂	IR(A)+X->Z	将IR中的立即数符号扩展为32位后，左移两位送入ALU做加法，计算出分支目标地址	$IR(A)_{out}=ADD=1$
	T ₃	if(PSW.equal) Z->PC	如果equal标志位1，将分支目标地址送入PC	$Z_{out}=1$ $PC_{in}=PSW.equal$

• addi指令

addi指令（立即数加法指令）也属于I型指令，汇编代码为

```
ASM

addi rt,rs,imm
```

addi只需要2个机器周期

1. 第一个机器周期

取指周期，使用两条数据通道

- PC→AR→MEM→DR→IR;以PC为地址访存，取指令，并送入IR
- PC→X→ALU→Z→PC;修改PC的值，为取下一条指令做准备

2. 第二个机器周期

执行周期，将寄存器rs和立即数相加，结果送入寄存器rt中。R[rs]→X;IR(I)→ALU→Z→R[rt]

• add指令

add指令（寄存器加法指令）属于R型指令，汇编代码为

ASM

```
add rd,rt,rs
```

RTL功能描述为 $R[rd] \leftarrow R[rs] + R[rt]$ ，add只需要2个机器周期

1. 第一个机器周期

取指周期，使用两条数据通道

- PC→AR→MEM→DR→IR;以PC为地址访存，取指令，并送入IR
- PC→X→ALU→Z→PC;修改PC的值，为取下一条指令做准备

2. 第二个机器周期

执行周期，将寄存器rs和寄存器rt相加，结果送入寄存器rd中，使用数据通道

R[rs]→X→ALU;R[rt]→ALU→Z→R[rd]



对单总线结构的性能分析

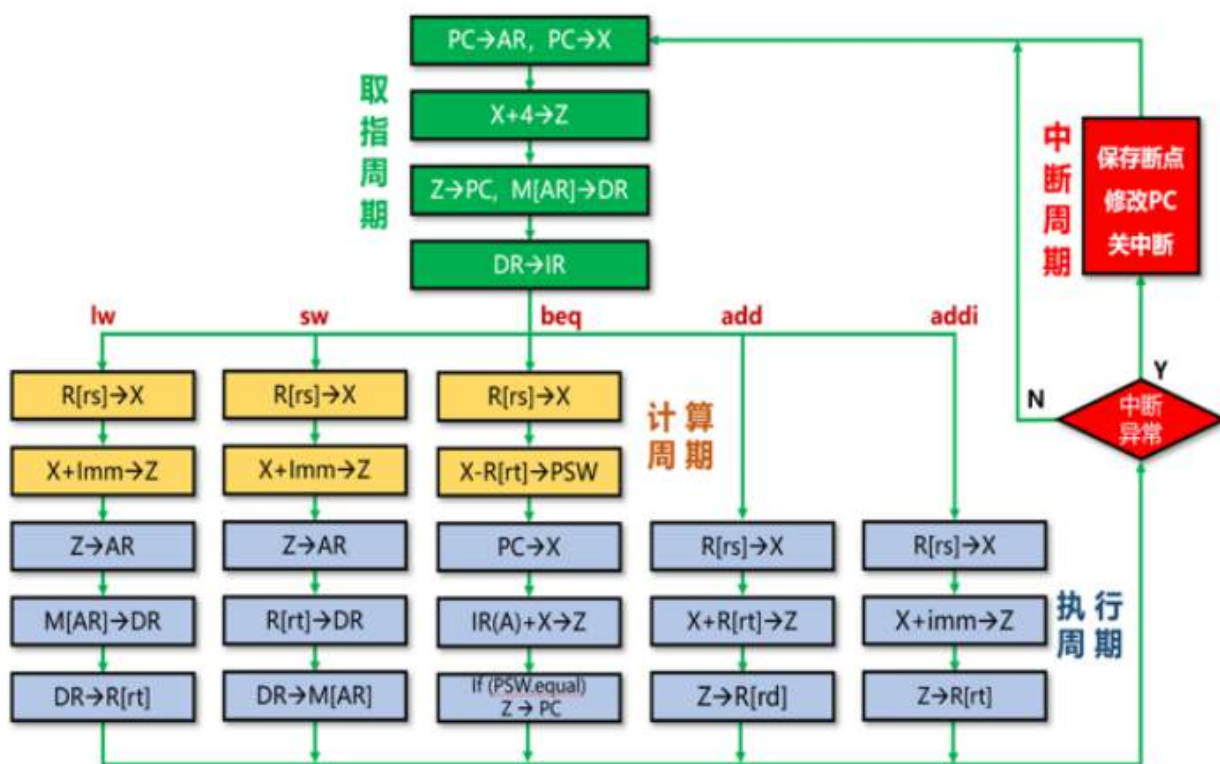


图6.14 单总线结构数据通路指令方框图

图中的每个方框代表一个时钟周期(节拍)，方框中的内容为该时钟周期内的操作

指令执行完毕后，首先判断是否有中断/异常/DMA请求，有则进入中断周期，否则为下一条指令的取指周期

单总线结构计算机的最小时钟周期（最大时钟频率），取决于不同机器周期、不同时钟节拍中最慢的关键路径时间延迟

通常访存通路和运算通路时延最长，因此最小时钟周期应为

$$t = T_{clk_to_q} + \max\{T_{alu}, T_{mem}\} + T_{setup}$$

专用通路结构的数据通路

采用**专用通路**的CPU中，各寄存器之间或寄存器与ALU之间，均基于专用的数据传输通路连接，而非基于总线方式共享连接

这使得各通路中的数据可并行传输，控制起来比总线结构的CPU要简单



图6.16 数据通路中的组合逻辑部件

- **程序计数器PC**

采用32位寄存器实现；顺序执行时： $PC=PC+4$

- **指令存储器**

采用ROM实现；输入为32位地址，输出为32位MIPS指令字

- **数据存储器**

采用RAM实现（分离模式）；输入为32位地址、32位数据，输出为32位数据，WE为读写控制信号，CLK为时钟信号

- **寄存器堆**

32个32位寄存器，有2个读出口、1个写入口

输入为2个读出口的寄存器编号（均为5位）、1个写入口的寄存器编号（5位）、写入数据

输出为2个读出口数据，WE为读写控制信号，CLK为时钟信号

- **多路选择器**

- **符号扩展单元**

将数据按符号位扩展

- **移位运算器、加法器、ALU**

单周期处理器典型数据通路

单周期处理器的所有指令在一个时钟周期内完成，因此其时钟周期取决于执行最慢的指令

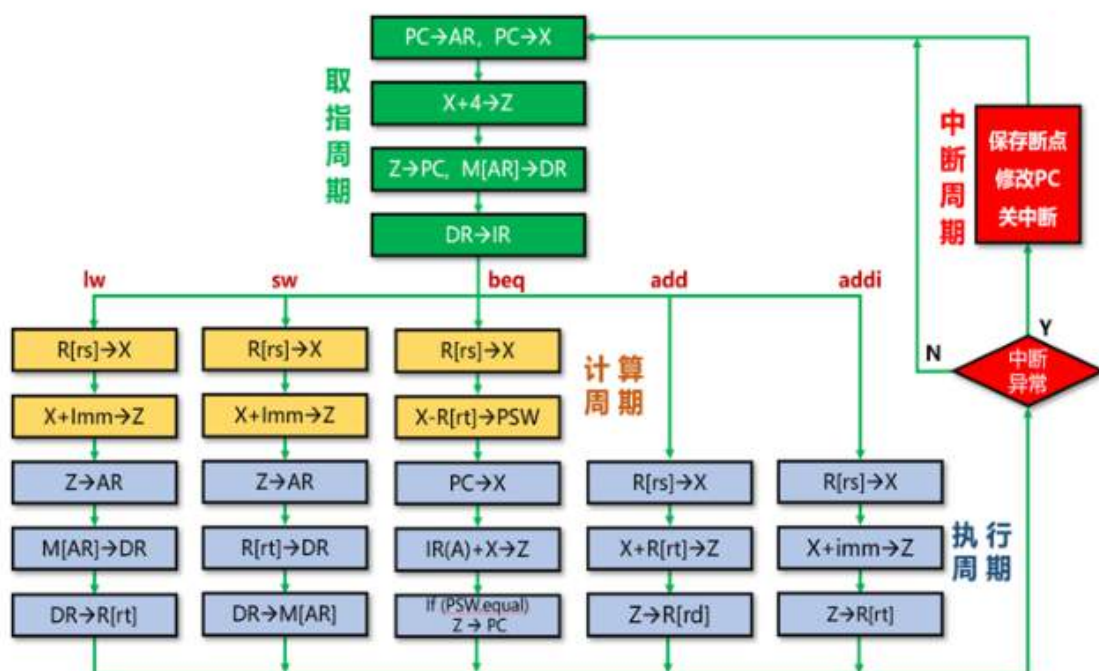


图6.14 单总线结构数据通路指令方框图

- 指令执行过程中数据通路的任何资源都不能被重复使用，都为专用数据通路。因为要在一个时钟周期内完成指令的取出和执行操作
- 需要被多次使用的资源（如加法器，图6.25中有2个）都需要设置多个
- 指令和数据分别存放在两个存储器中（指令存储器、数据存储器），因为取指令和取操作数都需要访问存储器

其中数据通路包括

- 取指令数据通路

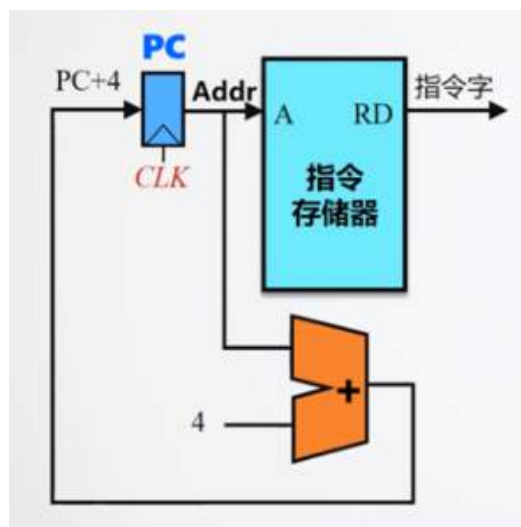


图6.17 取指令数据通路

包括程序计数器PC、加法器、指令存储器

- 加法器用于完成PC+4（因为MIPS32每条指令的长度都是32位），即计算下一条指令的地址
- 程序计数器PC的写入受时钟信号CLK的控制（上跳沿）
- **R型运算类指令数据通路**

包括程序计数器PC、指令存储器、寄存器堆（寄存器组）和ALU等功能部件

加法指令过程

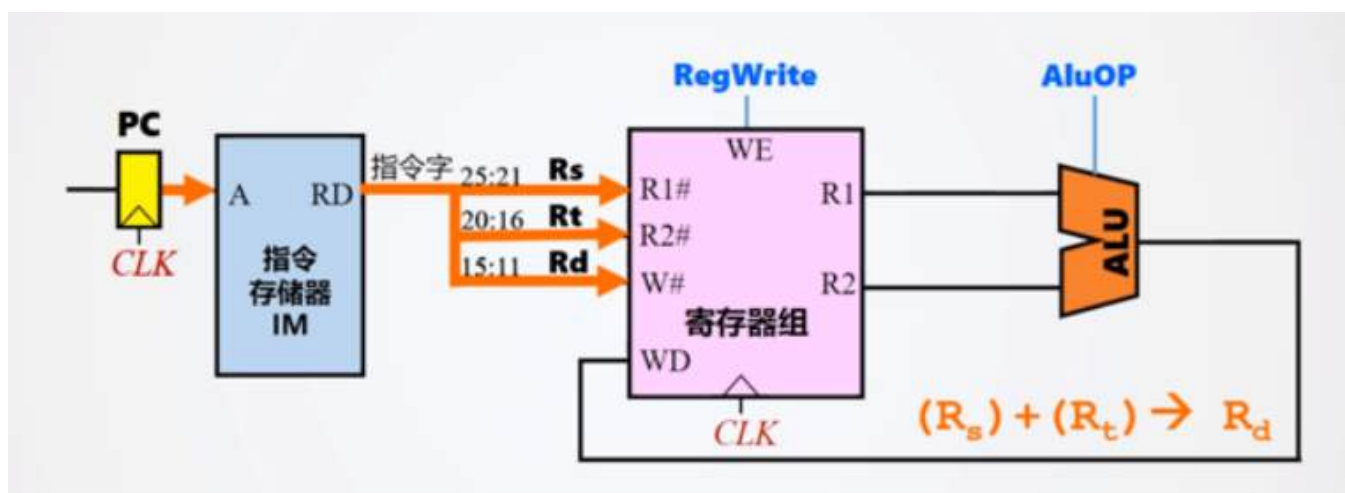


图6.18 R型运算类指令的数据通路

1. 将指令字中的源寄存器字段rs、rt分别送寄存器堆的R1和R2，将目的寄存器字段rd送寄存器堆的W#

2. 从寄存器堆读出的R1和R2送到ALU；指令字中的funct字段决定ALU进行加法运算（发出AluOP控制信号，加法运算）
3. 运算结果送寄存器堆的输入数据口WD，从而完成 $R[rs]+R[rt] \rightarrow R[rd]$ 的功能。

• I型运算类指令数据通路

包括程序计数器PC、指令存储器、寄存器堆（寄存器组）、符号扩展器、ALU、数据存储器等功能部件

• lw指令过程

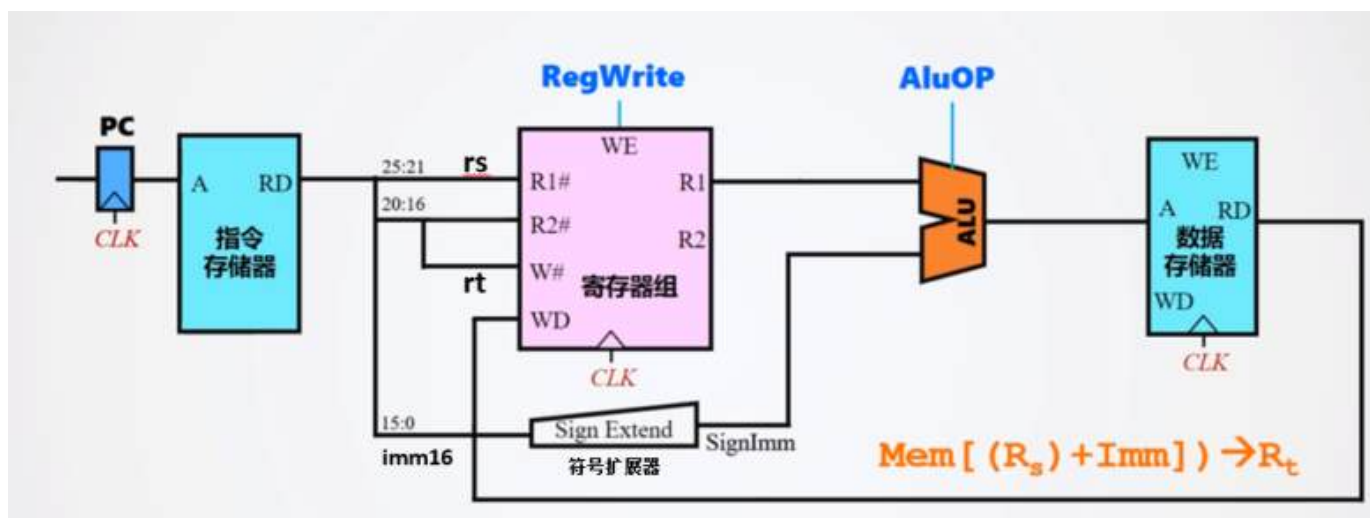


图6.19 lw指令的数据通路

1. 源寄存器字段rs送寄存器堆的R1，目的寄存器字段rt送寄存器堆的W，立即数字段imm16送符号扩展器扩展为32位
2. 从寄存器堆读出的R1，与符号扩展器的输出，一起送到ALU，相加后形成数据存储器的地址
3. 从数据存储器中读取数据，送到寄存器堆的WD口，从而完成 $M[R[rs]+SignExt(imm16)] \rightarrow R[rt]$ 的功能

• sw指令过程

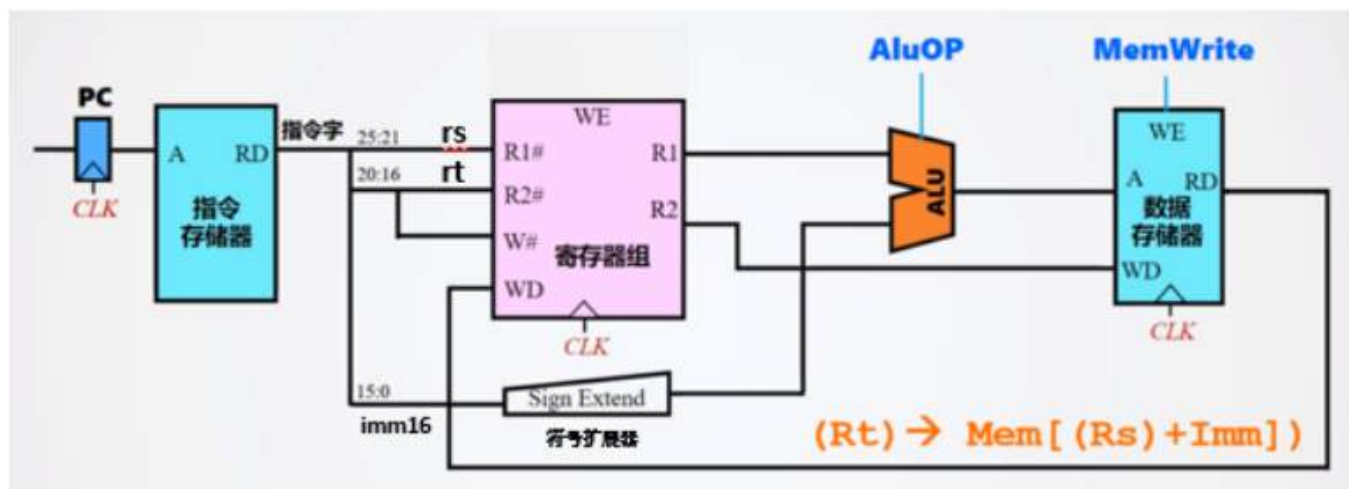


图6.20 sw指令的数据通路

1. 将指令字中的源寄存器字段rs、rt分别送寄存器堆的R1、R2，立即数字段imm16送符号扩展器（16位扩展为32位）
2. 从寄存器堆读出的R1与符号扩展器的输出一起送到ALU，相加后（发出AluOP控制信号，加法运算）形成数据存储器地址
3. 从寄存器堆中读出R2，送数据存储器的WD，从而完成 $R[rt] \rightarrow M[R[rs] + \text{SignExt}(\text{imm16})]$ 的功能

• R型与I型指令混合数据通路

增加3个多路选择器

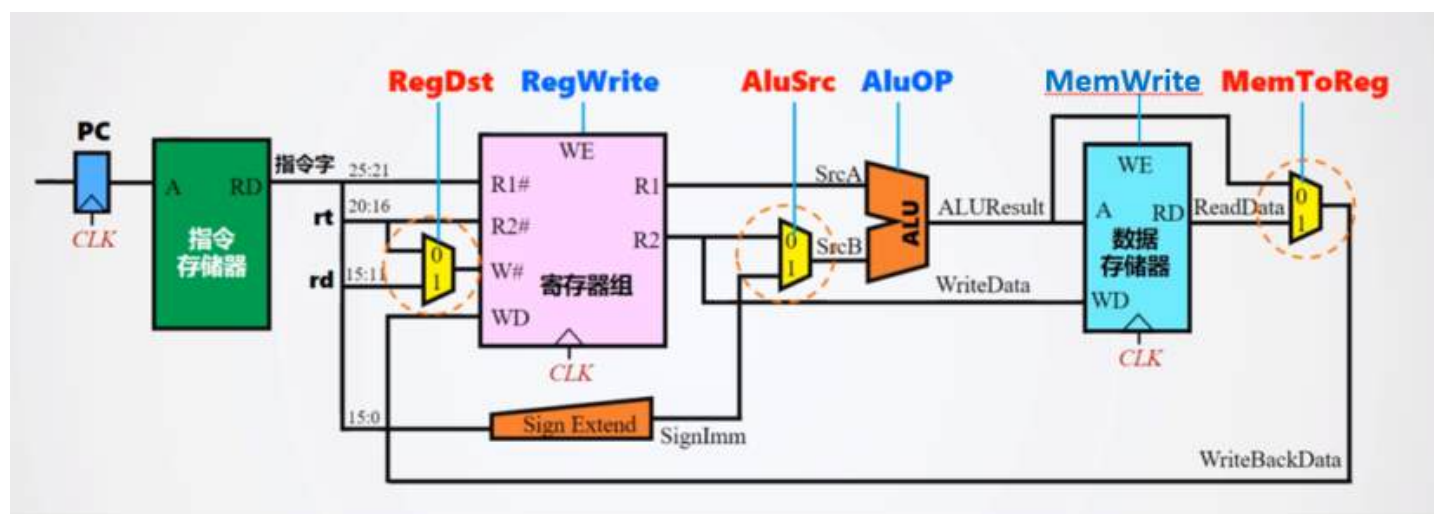


图6.21 支持R型运算类指令和访存指令的混合数据通路

1. 第一个多路选择器

- RegDst=0, W# =rt, 用于lw指令
- RegDst=1, W# =rd, 用于R型运算类指令。

2. 第二个多路选择器

- AluSrc=0, SrcB=R2, 用于R型运算类指令
- AluSrc=1, SrcB=符号扩展器的输出SignImm, 用于lw指令或sw指令。

3. 第三个多路选择器

- MemToReg=0, 寄存器堆的WD=ALUResult, 用于R型运算类指令
- MemToReg=1, 寄存器堆的WD=数据存储器的输出ReadData, 用于lw指令

• 条件分支指令数据通路

也属于I型指令，以beq指令为例

数据通路包括程序计数器PC、指令存储器、寄存器堆（寄存器组）、符号扩展器、移位运算器、ALU、数据存储器、2个加法器、4个多路选择器、与门等功能部件

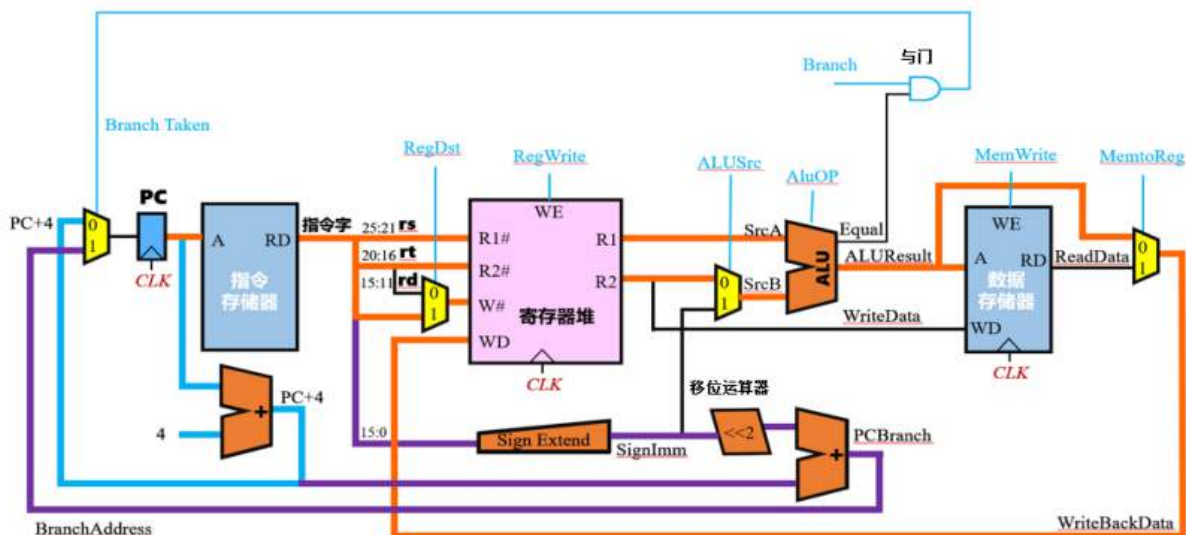


图6.22 beq指令的数据通路

- 将立即数字段imm16送符号扩展器，扩展为32位，然后通过移位运算器左移2位，经过加法器后，得到条件成立后的分支目标地址
- 将指令字中的寄存器字段rs、rt送寄存器堆的R1、R2，寄存器堆的输出R1、R2送ALU，进行比较运算

- 如果 $rs=rt$ ，则 $Equal=1$ ；Branch为beq的译码信号，故 $Branch=1$ ，与门的输出 $BranchTaken=1$

因此，将 $PC+4+SignExt(imm)<<2$ 送程序计数器PC；

- 如果 $rs \neq rt$ ，则 $Equal=0$ ， $BranchTaken=0$ ，将 $PC+4$ 送程序计数器PC
- 完成 $if(R[rs]==R[rt])PC+4+SignExt(imm16)<<2 \rightarrow PC$ 的功能
- 无条件分支指令数据通路

无条件分支指令属于J型指令，以j指令为例

数据通路包括程序计数器PC、指令存储器、寄存器堆（寄存器组）、符号扩展器、2个移位运算器、ALU、数据存储器、2个加法器、5个多路选择器、与门等功能部件

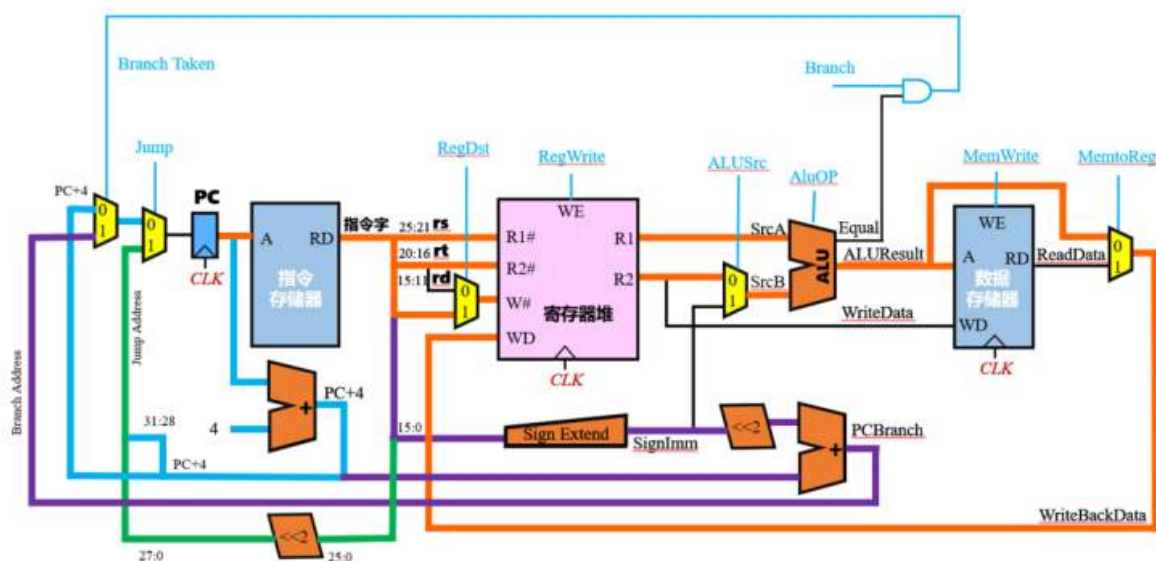


图6.24 j指令的数据通路

- 将指令字中的26位地址address字段通过移位运算器左移2位，得到28位地址
- 再与 $PC+4$ 的高4位（31:28位）拼接成32位跳转地址，送多路选择器
- j指令经过译码，使 $Jump=1$ ，多路选择器将JumpAddress送PC
- 完成 $\{(PC+4)31:28, address<<2\} \rightarrow PC$ 的功能。

这些指令数据通路合并后，增加必要的控制线路就可以得到支持MIPS指令系统子集的完整数据通路

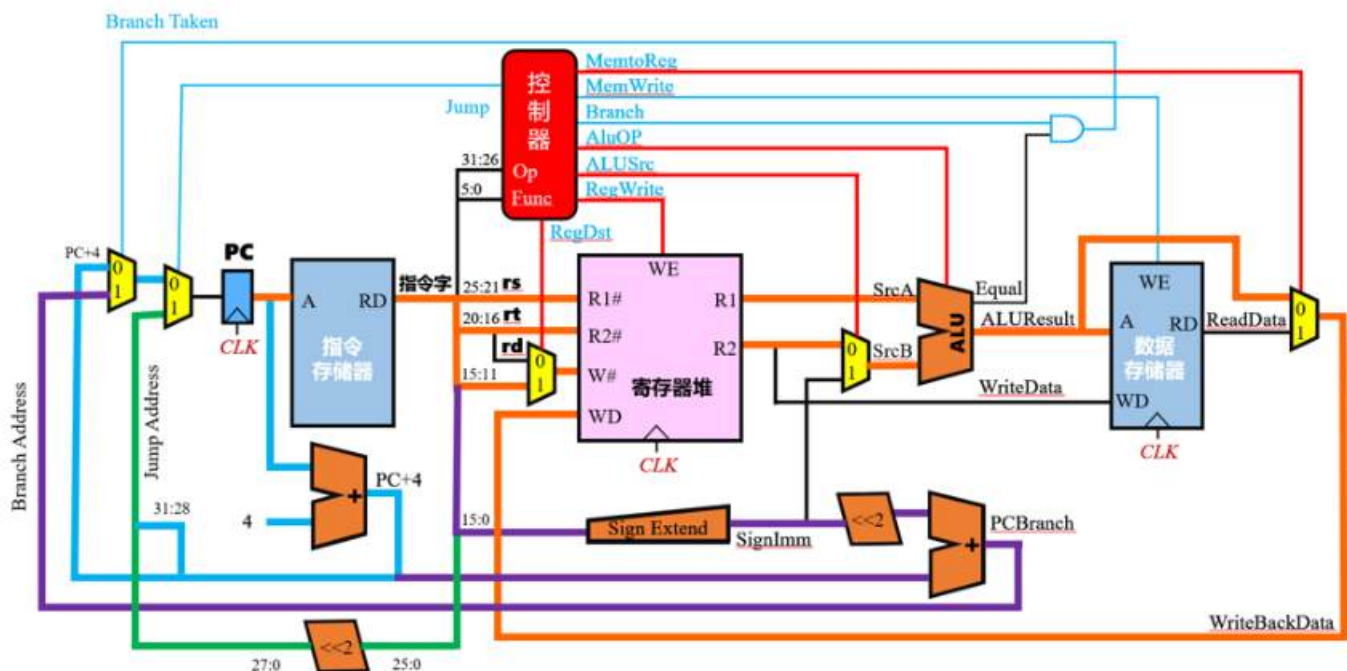


图6.25 单周期MIPS处理器的数据通路

- 程序计数器PC、寄存器堆、数据存储器受统一时钟信号CLK控制，上跳沿有效。
- 指令存储器为ROM，不用设置指令寄存器IR。
- ALU的操作控制信号AluOP为4位，可以有16种运算，而AluOP的值由指令字的OP字段和funct字段经译码产生。
- 单周期MIPS处理器的操作控制器是组合逻辑电路；控制器的输入为指令的OP字段和funct字段，控制器的输出为8个控制信息

表6.9 单周期MIPS处理器的控制信号及功能

控制信号	信号分类	功能说明
Jump	指令译码信号	j、jal等无条件分支指令时生成
Branch	指令译码信号	分支指令译码信号，beq、bne指令需要产生类似的译码信号
RegDst	多路选择控制	写入目的寄存器的选择，为1时写入rd寄存器，为0时写入rt寄存器
AluSrc	多路选择控制	控制ALU的第二输入，为0时输入R2（rt寄存器），为1时输入扩展后的立即数
MenToReg	多路选择控制	为1时将从数据存储器读出的数据写回寄存器堆，为0时将ALU的运算结果写回寄存器堆
RegWrite	功能部件控制	控制寄存器堆写操作，为1时数据需要写入指定的寄存器，受时钟驱动
AluOp	功能部件控制	控制ALU进行不同的运算，具体取指和位宽（如AluOP为4位）与ALU的设计有关
MemWrite	功能部件控制	控制数据存储器的写操作，为1时进行写操作，为0时读操作，受时钟驱动



单周期处理器的性能分析

单周期处理器的时钟周期不能太短，效率较低，因为时钟周期由执行速度最慢的指令决定，其CPI=1

通常，lw指令的执行时间最长，关键延迟为：

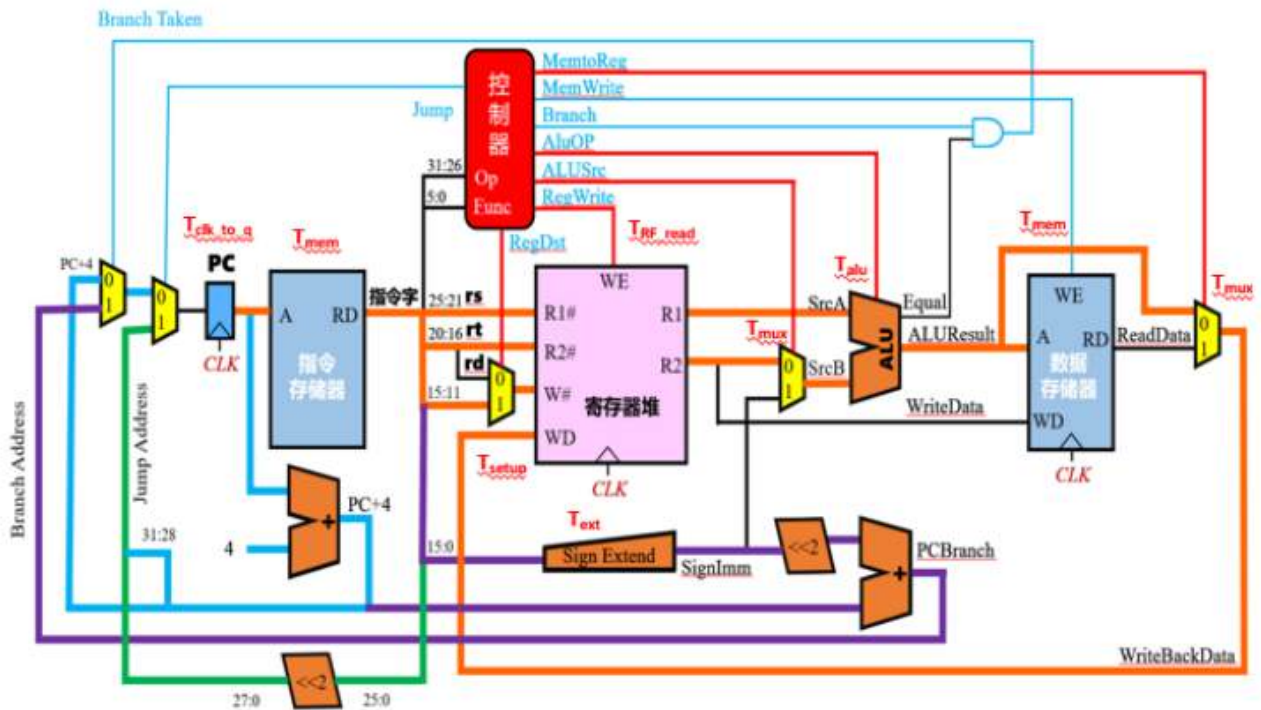


图6.26 lw指令数据通路的关键延迟

- 程序计数器PC的触发器延迟: $T_{clk_to_q}$
- 指令存储器的访存延迟: T_{mem}
- 寄存器堆的读延迟: T_{RF_read}
- 符号扩展单元的延迟: T_{ext}
- SrcB端的多路选择器的延迟: T_{mux}
- 运算器的延迟: T_{alu}
- 数据存储器的访存延迟: T_{mem}
- 数据存储器输出端的多路选择器的延迟: T_{mux}
- 写入寄存器堆的寄存器建立时间: T_{setup}

最小时钟周期为：

$$T_{\min_clk} = T_{clk_to_q} + 2T_{mem} + T_{RF_read} + T_{alu} + 2T_{mux} + T_{setup}$$

多周期处理器典型数据通路

多周期处理器执行一条指令需要多个指令周期

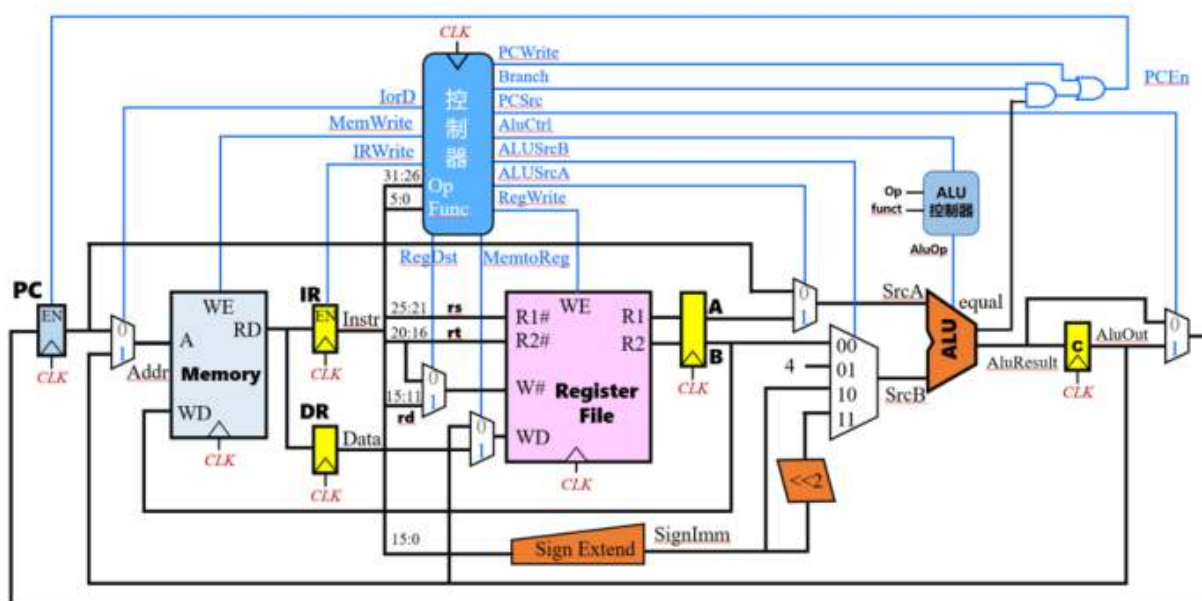


图6.27 多周期MIPS处理器的数据通路

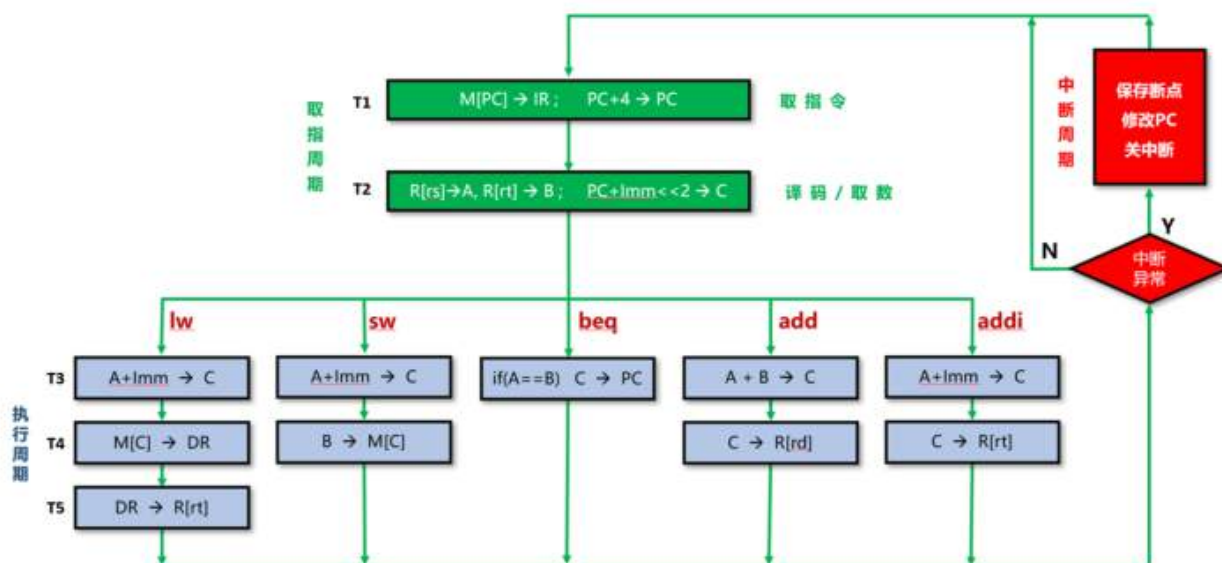


图6.35 多周期MIPS处理器数据通路的指令图



多周期处理器与单周期处理器的不同

- 不再区分指令存储器和数据存储器
- 部分功能单元可在一条指令执行的不同时钟周期中多次使用，从而不需要额外设置加法器；单周期处理器需要额外设置加法器
- 主要功能部件输出端都增加了一些附加寄存器，方便暂存当前时钟周期加工处理的数据给后续时钟周期使用
 - 指令寄存器IR：用于暂存从存储器中读出的指令，需要设置专门的写使能控制信号Irwrite。
 - 数据寄存器DR：用于暂存从存储器中读出的数据，不需要设置写使能控制信号。
 - 寄存器A、寄存器B：用于暂存从寄存器堆读出的数据，不需要设置写使能控制信号。
 - 寄存器C：用于暂存ALU的运算结果，不需要设置写使能控制信号。
- 程序计数器PC增加了使能端PCEn，由写操作控制信号PCwrite通过一定的逻辑进行控制
- 增加和扩展了部分多路选择器
 - 存储器前的多路选择器用于区分是指令地址还是数据地址。
 - 寄存器堆前的多路选择器（W）用于确定寄存器堆的写入编号W是rt还是rd。
 - 寄存器堆前的多路选择器（WD）用于确定寄存器堆的写入数据WD是来自ALU还是来自存储器。
 - ALU前的多路选择器（SrcA）用于确定ALU的输入A是来自PC还是来自寄存器A。
 - ALU前的多路选择器（SrcB，4选1）用于确定ALU的输入B是
 - 来自寄存器B（ $R[rs]+R[rt]\rightarrow R[rd]$ ）
 - 来自常数4（ $PC+4\rightarrow PC$ ）
 - 来自符号扩展输出（ $M[R[rs]+\text{SignExt}(\text{imm16})]\rightarrow R[rt]$ ）
 - 来自符号扩展输出左移2位（ $PC+4+\text{SignExt}(\text{imm16})\ll 2\rightarrow PC$ ）。
 - ALU后的多路选择器用于确认程序计数器PC的输入是来自“PC+4”还是来自分支目标地址
- 增加了ALU控制器，其输入为AluCtrl、Op、funct，输出为AluOP

- 当AluCtrl为0时，ALU做加法运算
- AluCtrl为1时，ALU的运算功能由Op和funct决定

多周期MIPS处理器的控制信号及功能如下表所示

表6.10 多周期MIPS处理器的控制信号及功能

控制信号	信号分类	功能说明
Branch	指令译码信号	分支指令译码信号， <u>beq</u> 、 <u>bne</u> 指令需要产生类似的译码信号
RegDst	多路选择控制	写入目的寄存器的选择，为1时写入rd寄存器，为0时写入rt寄存器
AluSrcA	多路选择控制	控制ALU的第一输入，为0时输入PC值，为1时输入寄存器A的值
AluSrcB	多路选择控制	控制ALU的第二输入，值为03时分别输入B值、常量4、立即数、偏移地址
AluCtrl	多路选择控制	ALU控制器选择控制信号，为0时，ALU做加法运算，1为1时，ALU的运算功能由Op和funct决定
MemToReg	多路选择控制	为0时写入暂存在C寄存器中的ALU运算结果，为1时写入DR寄存器的值
PCSrc	多路选择控制	控制程序计数器PC的输入，为0时输入PC+4，为1时输入分支目标地址
IorD	多路选择控制	控制存储器地址输入，为0时输入指令地址，为1时输入寄存器C中的数据地址
RegWrite	功能部件控制	控制寄存器堆写操作，为1时数据需要写入指定的寄存器，受时钟驱动
MemWrite	功能部件控制	控制数据存储器的写操作，为1时进行写操作，为0时为读操作，受时钟驱动
PCWrite	功能部件控制	程序计数器PC的使能信号，为1时数据写入程序计数器PC，受时钟驱动
IRWrite	功能部件控制	指令寄存器IR的使能信号，为1时数据写入指令寄存器IR，受时钟驱动

多周期处理器指令执行过程如下

1. 确定时钟周期

将指令的执行分解为若干步骤，每个时钟周期执行一个步骤，因此执行一条指令需要若干个时钟周期

通常在指令执行过程中存储器访问、寄存器堆访问、ALU操作等，都是相对比较费时的操作
因此时钟周期至少要大于这3个操作中耗时最长的操作所需的时间

2. 并行操作及控制

基于专用通路结构的多周期处理器为指令的执行提供了良好的并行性

- 指令译码（在控制器中完成）和从寄存器堆中取数据暂存到寄存器A、B的操作，可以同时进行
- 利用ALU提前计算分支目标地址并暂存到寄存器C中，即使指令最后未实现分支跳转，这样做对指令的执行也没有影响

如果指令实现分支跳转（条件成立），提前计算分支目标地址将有利于缩短分支指令的执行时间

部分指令对应数据通路如下

• 取指令阶段数据通路

多周期MIPS处理器指令周期的第一个时钟周期T1是取指令阶段，共包括两条并发的数据通路

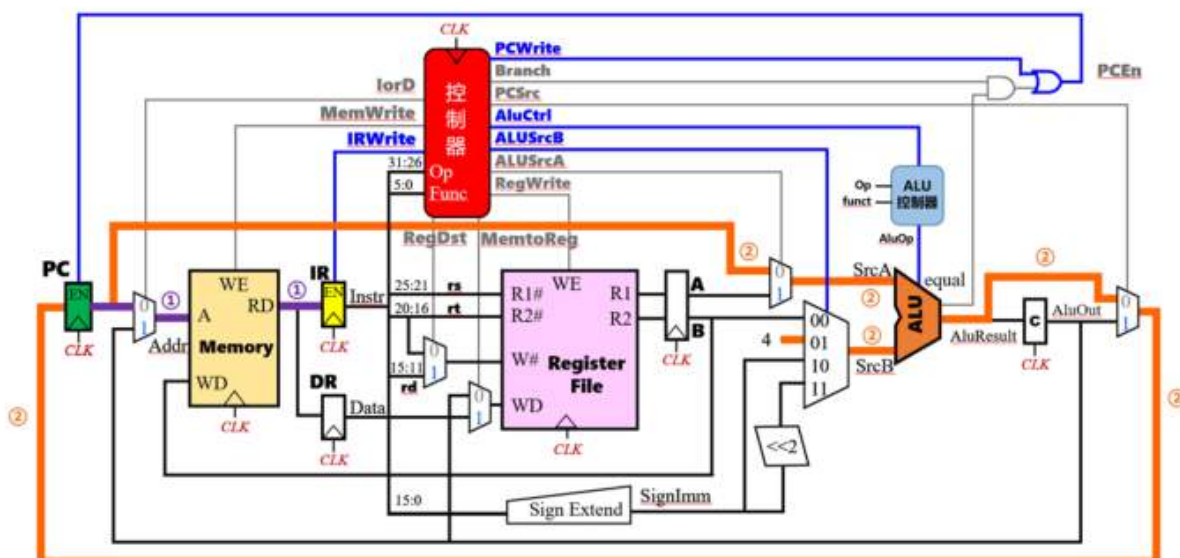


图6.28 多周期MIPS处理器取指令阶段的数据通路

这两条数据通路没有冲突，可以在一个时钟周期内同时完成

1. 第一个数据通路

实现 $M[PC] \rightarrow IR$ ，依靠图中的蓝色通路实现。用PC访存指令存储器后，将指令字送入IR中锁存

控制信号有

- $lorD=0$ (PC->存储器)
- $MemWrite=0$ (存储器读)
- $IRWrite=1$ (存储器->IR)

2. 第二个数据通路

实现 $PC+4 \rightarrow PC$ ，依靠图中橙色通路实现。PC自增，修改为下一条顺序指令的地址

控制信号有：

- $AluSrcA=0$ (PC->ALU的A端)

- AluSrcB=1 (常数4->ALU的B端)
- AluCtrl=0 (ALU做加法运算, 即PC+4)
- PCSrc=0 (ALU运算结果 (即PC+4) ->PC输入口)
- PCWrite=1 (PCEn=1, PC+4->PC)

• 译码/取数阶段数据通路

多周期MIPS处理器指令周期的第二个时钟周期T2是译码/取数阶段, 共包括三条并发的数据通路

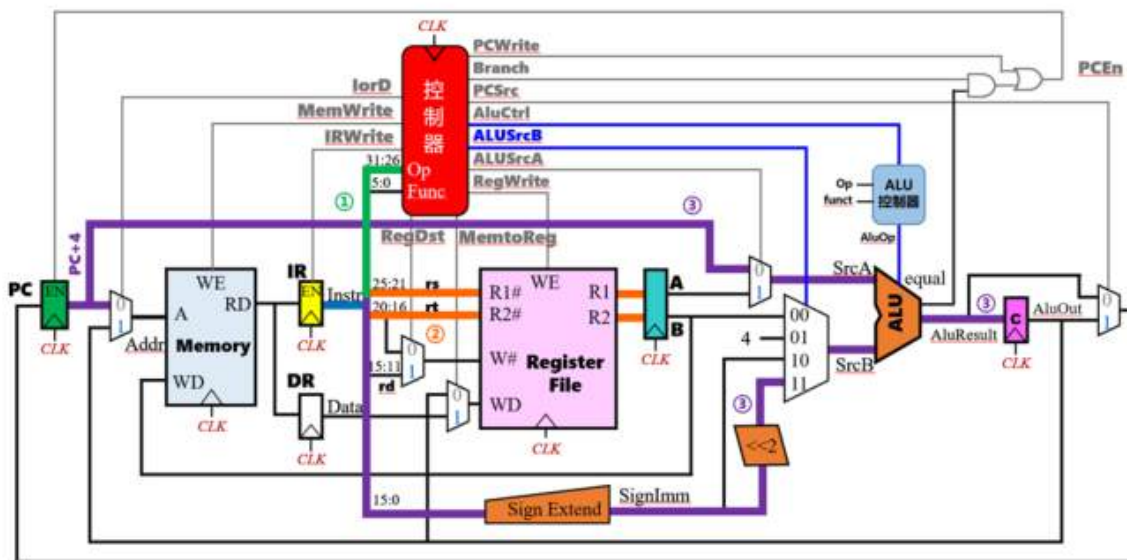


图6.29 多周期MIPS处理器译码/取数阶段的数据通路

这三条数据通路没有冲突, 可以在一个时钟周期内并发

1. 第一个数据通路

实现指令译码, 依靠图中绿色通路实现。控制器根据IR中的Op和funct字段进行译码, 生成指令译码信号, 无需控制信号

2. 第二个数据通路

取操作数, 依靠图中橙色通路实现。将IR中的rs、rt字段送寄存器堆的R1、R2
寄存器堆的输出送A、B寄存器中暂存, 即R[rs]->A;R[rt]->B

3. 第三个数据通路

提前计算分支目标地址, 并将结果暂存在C寄存器中, 依靠图中紫色通路实现, 实现
 $PC+4+SignExt(imm16) \ll 2 \rightarrow C$

控制信号有：

- $\text{AluSrcA}=0$ ($\text{PC}+4 \rightarrow \text{ALU}$ 的A端) -
- $\text{AluSrcB}=3$ ($\text{SignExt}(\text{imm16}) \ll 2 \rightarrow \text{ALU}$ 的B端)
- $\text{AluCtrl}=0$ (ALU做加法运算, 即 $\text{PC}+4+\text{SignExt}(\text{imm16}) \ll 2 \rightarrow \text{C}$)

- R型算术逻辑运算指令执行阶段的数据通路

以加法指令addrd,rs,rt为例

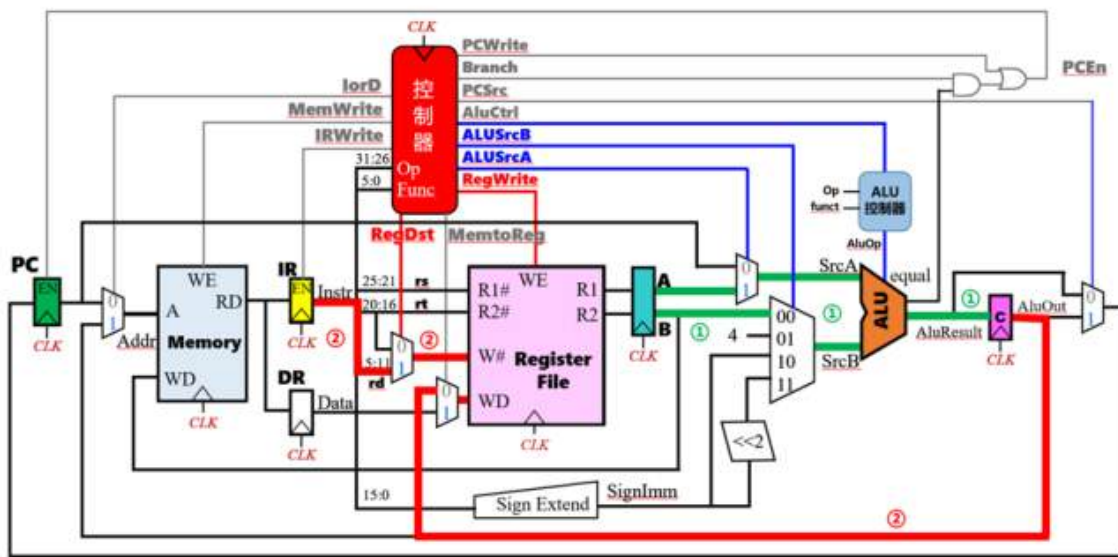


图6.30 R型算术逻辑运算指令执行阶段的数据通路

R型算术逻辑运算指令执行阶段需要2个时钟周期(T3,T4, 接着前面的两个时钟周期), 每个时钟周期各有一条数据通路且为先后执行

- **第一个数据通路:**

数据运算，结果送C寄存器，依靠图中的绿色通路实现AopB->C

控制信号有：

- AluSrcA=1 (寄存器A->ALU的A端)
- AluSrcB=0 (寄存器B->ALU的B端)
- AluCtrl=1 (运算结果AopB->C)

- 第二个数据通路

结果写回，将运算结果从寄存器C，写回到寄存器堆的rd寄存器中，依靠图中红色通路实现C->R[rd]

控制信号有：

- RegDst=1 (rd->寄存器堆的W# 端)
- MemToReg=0 (寄存器C->寄存器堆的WD端)
- RegWrite=1 (寄存器堆写，即：C->R[rd])

• I型算术逻辑运算指令执行阶段的数据通路

I型指令除lw,sw外，还有addirt,rs,imm指令，功能为 $R[rs] + \text{SignExt}(\text{imm}) \rightarrow R[rt]$

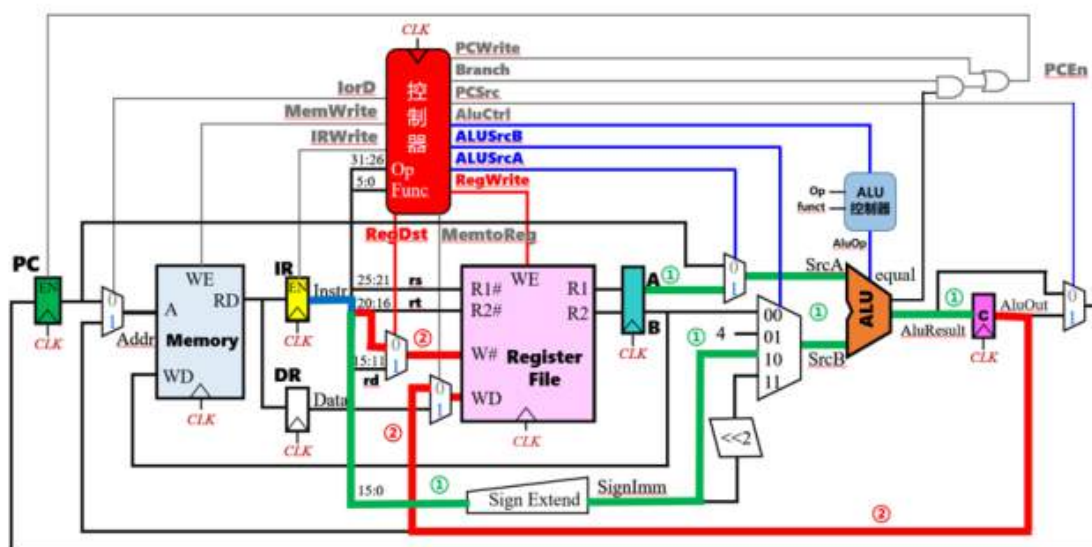


图6.31 I型算术逻辑运算指令执行阶段的数据通路

I型算术逻辑运算指令执行阶段也需要2个时钟周期 (T3、T4)，每个时钟周期各对应一条数据通路

这两条数据通路也是先后执行的

1. 第一个数据通路

进行数据运算，A和立即数进行运算，结果送C寄存器，依靠图中绿色通路实现 $Aop\text{SignExt}(\text{imm})$

控制信号有：

- AluSrcA=1 (寄存器A->ALU的A端)
- AluSrcB=2 ($\text{SignExt}(\text{imm})$ ->ALU的B端)
- AluCtrl=1 (ALU控制器根据IR的Op和funct产生AluOp信号，决定ALU做什么运算，运算结果 $Aop\text{SignExt}(\text{imm}) \rightarrow C$)

2. 第二个数据通路

结果写回，将运算结果从寄存器C，写回到寄存器堆的rd寄存器中，依靠图中红色通路C->R[rt]

控制信号有：

- RegDst=0 (rt->寄存器堆的W# 端)
- MemToReg=0 (寄存器C->寄存器堆的WD端)
- RegWrite=1 (寄存器堆写，即：C->R[rt])

lw指令执行阶段的数据通路

lw指令属于l型指令：lwrt,imm，功能描述Mem[R[rs]+SignExt(imm)]->R[rt]

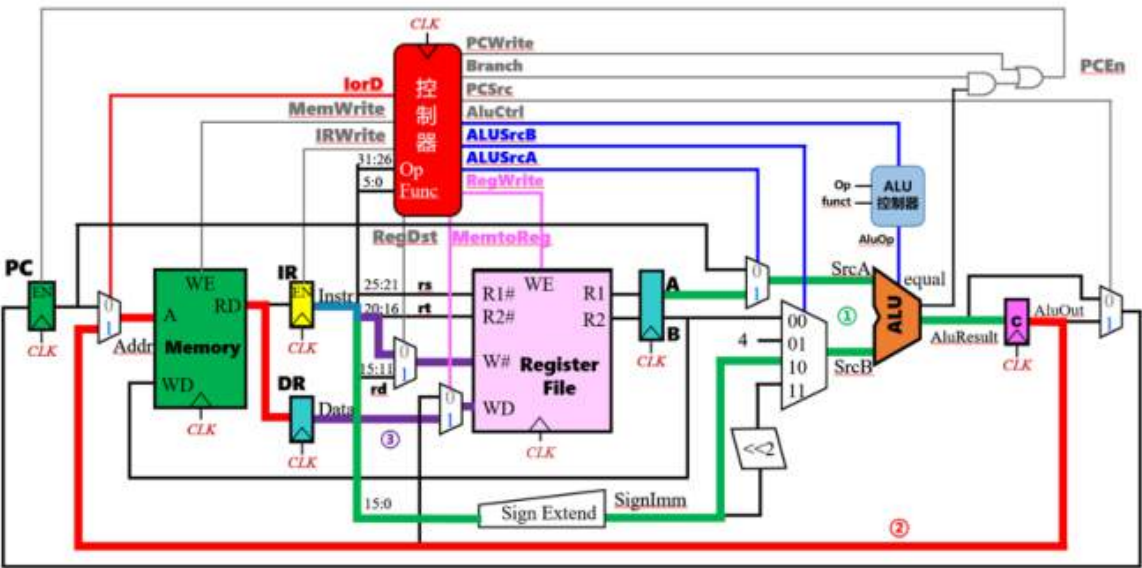


图6.32 lw指令执行阶段的数据通路

lw指令执行阶段需要3个时钟周期（T3、T4、T5），每个时钟周期各对应一条数据通路
这三条数据通路也不是并发执行，而是先后执行的

1. 第一个数据通路：

计算访存地址，A和立即数相加，结果送C寄存器，依靠绿色通路实现A+SignExt(imm)->C

控制信号有：

- AluSrcA=1 (寄存器A->ALU的A端)
- AluSrcB=2 (SignExt(imm)->ALU的B端)

- $AluCtrl=0$ (ALU进行加法运算, 运算结果 $A+SignExt(imm) \rightarrow C$)

2. 第二个数据通路:

访存, 以C寄存器的内容作为访存地址, 取出的数送DR, 依靠红色通路实现 $M[C] \rightarrow DR$
控制信号有:

- $lorD=1$ (C寄存器 $\rightarrow$ 存储器的地址端A)
- $MemWrite=0$ (存储器读, $M[C] \rightarrow DR$)

3. 第三个数据通路:

结果写回, 将DR中的数据写入寄存器堆的rd寄存器中, 依靠紫色通路实现 $DR \rightarrow R[rt]$
控制信号有:

- $RegDst=0$ (rt $\rightarrow$ 寄存器堆的W# 端)
- $MemToReg=1$ (DR $\rightarrow$ 寄存器堆的WD端)
- $RegWrite=1$ (寄存器堆写, 即: $DR \rightarrow R[rt]$)

• sw指令执行阶段的数据通路

sw也属于I型指令swrt,imm(rs), 功能描述为 $R[rt] \rightarrow Mem[R[rs]+SignExt(imm)]$

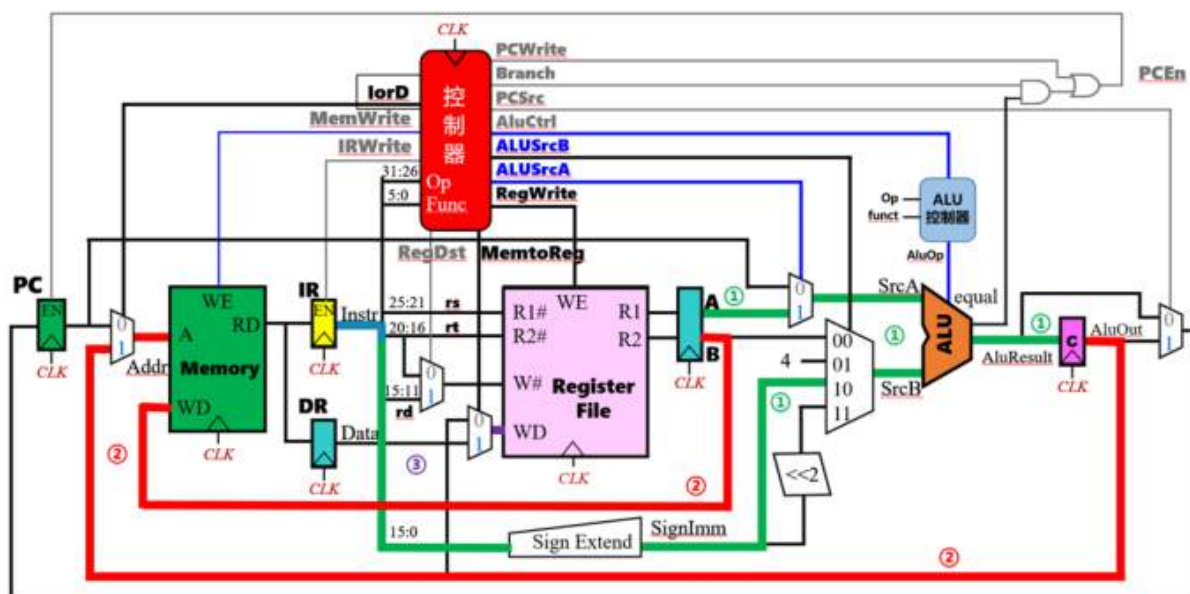


图6.33 sw指令执行阶段的数据通路

sw指令执行阶段需要2个时钟周期 (T3、T4), 每个时钟周期各对应一条数据通路
这两条数据通路也是先后执行的

1. 第一个数据通路:

计算访存地址，A和立即数相加，结果送C寄存器，依靠绿色通路实现 $A + \text{SignExt}(\text{imm}) \rightarrow C$

控制信号有:

- $\text{AluSrcA}=1$ (寄存器A $\rightarrow$ ALU的A端)
- $\text{AluSrcB}=2$ ($\text{SignExt}(\text{imm}) \rightarrow$ ALU的B端)
- $\text{AluCtrl}=0$ (ALU进行加法运算，运算结果 $A + \text{SignExt}(\text{imm}) \rightarrow C$)

2. 第二个数据通路:

存储器写，将B寄存器的内容（即 $R[\text{rt}]$ ）写入以C寄存器内容为地址的存储器中，依靠红色通路实现 $B \rightarrow M[C]$

控制信号有:

- $\text{lorD}=1$ (C寄存器 $\rightarrow$ 存储器的地址端A)
- $\text{MemWrite}=1$ (存储器写， $B \rightarrow M[C]$)

• 条件分支指令执行阶段的数据通路

条件分支指令也属于I型指令，例如beq指令为beqrs,rt,imm。RTL功能描述是:

$\text{if}(R[\text{rs}] == R[\text{rt}]) \text{PC} + 4 + \text{SignExt}(\text{imm}) \ll 2 \rightarrow \text{PC}$

只需要1个时钟周期，对应一个数据通路

数据通路为

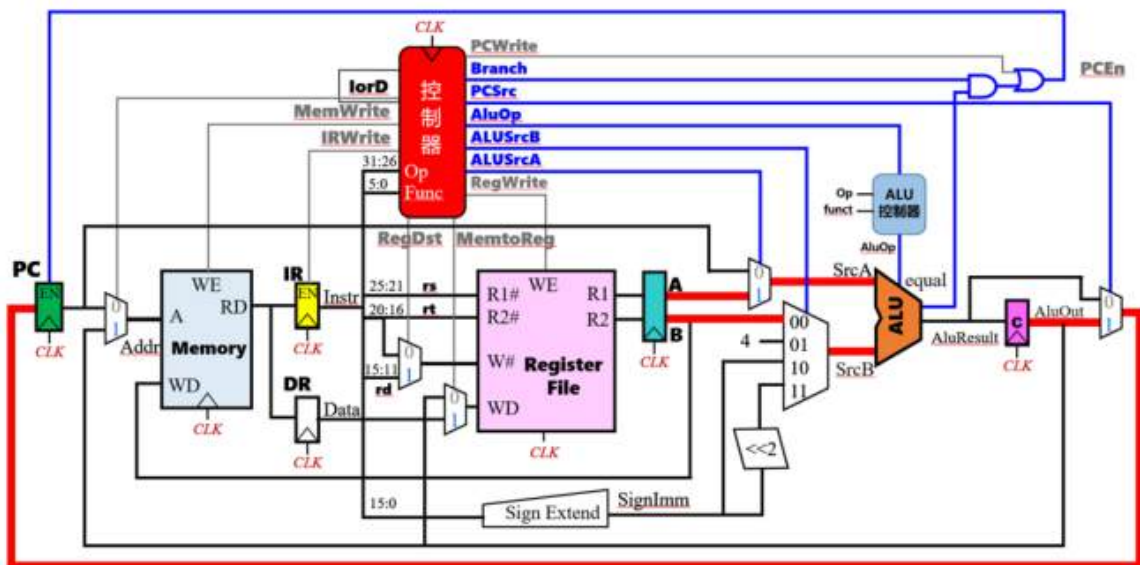


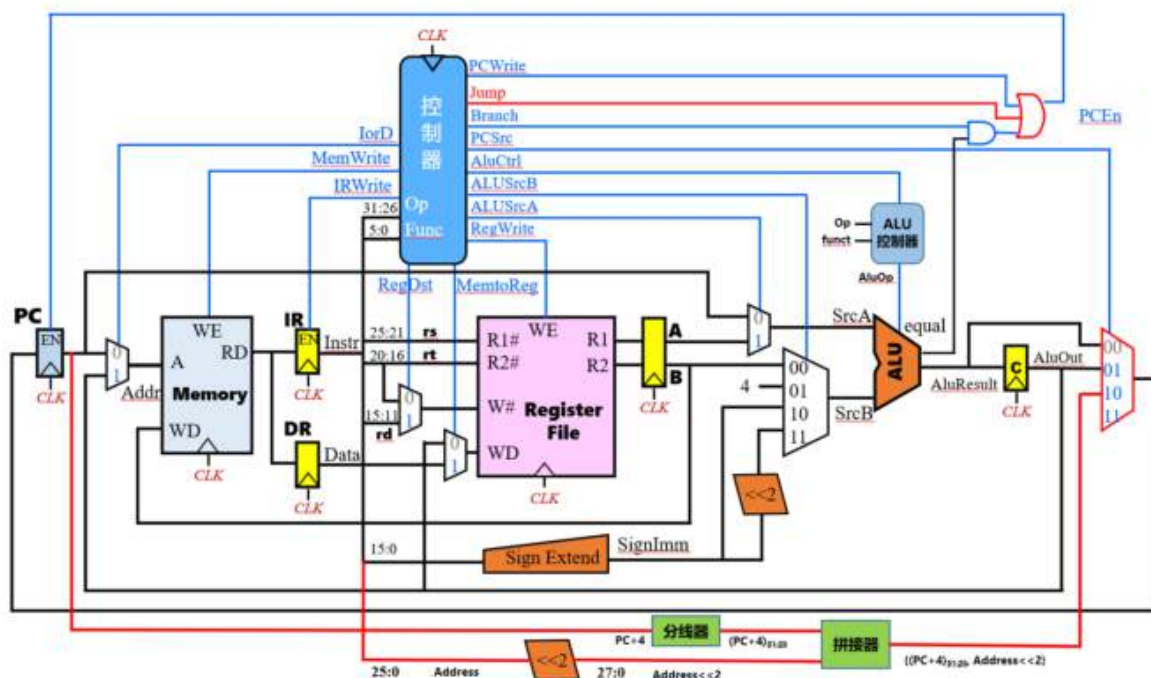
图6.34 beq指令执行阶段的数据通路

比较寄存器A和寄存器B的值，如果相等，则将寄存器C的内容 ($PC+4+\text{SignExt}(\text{imm})\ll 2$) 送入PC，依靠图中红色通路实现

控制信号为：

- AluSrcA=1 (寄存器A->ALU的A端)
- AluSrcB=0 (寄存器B->ALU的B端)
- PCSrc=1 (C寄存器->PC输入端)
- Branch=1 (如果A=B, 则equal=1; 与门输出=1, 或门输出=1, PCEn=1; C寄存器->PC, 即: $PC+4+\text{SignExt}(\text{imm})\ll 2 \rightarrow PC$)
- 支持j指令的数据通路

前面所示的数据通路不支持j指令，下面给出了一个支持j指令的数据通路



MIPS指令多周期数据通路及控制信号如表所示

表6.11 MIPS指令多周期数据通路及控制信号

指令类型	时钟周期	操作	控制信号（非零值）
取指令	T1	M[PC] -> IR; PC+4 -> PC	IRWrite=1; AluSrcB=1; PCWrite=1
	T2	R[rs] -> A, R[rt] -> B; PC+Imm<<2 -> C	AluSrcB=3
R型运算	T3	A + B -> C	AluSrcA=1; AluCtrl=1
	T4	C -> R[rd]	RegDst=1; RegWrite=1
I型运算	T3	A+Imm -> C	AluSrcA=1; AluSrcB=2; AluCtrl=1
	T4	C -> R[rt]	RegWrite=1
lw指令	T3	A+Imm -> C	AluSrcA=1; AluSrcB=2
	T4	M[C] -> DR	IorD=1
	T5	DR -> R[rt]	MemToReg=1; RegWrite=1
sw指令	T3	A+Imm -> C	AluSrcA=1; AluSrcB=2
	T4	B -> M[C]	IorD=1; MemWrite=1
beq指令	T3	if(A==B) C -> PC	AluSrcA=1; PCSrc=1; Branch=1



多周期处理器的性能分析

指令执行的每个时钟周期只能包含存储器访问、寄存器堆访问及ALU操作等3个慢速操作中的一个

前面所提到的所有数据通路有两条关键路径(用时最长)

- 取指令时的PC+4->PC数据通路
- lw指令执行时的M[C]->DR数据通路

多周期MIPS处理器的最小时钟周期为 $T_{min_clk} = T_{clk_to_q} + T_{mux} + \max(T_{alu} + T_{mux}, T_{mem}) + T_{setup}$

- $T_{clk_to_q}$ 为程序计数器的触发器延迟
- T_{mux} 为多路选择器的延迟
- T_{alu} 为运算器的延迟
- T_{mem} 为存储器的访存延迟
- T_{setup} 为写入寄存器堆的寄存器建立时间

时序与控制

在单周期处理器中，由于所有指令都在一个时钟周期内完成，控制信号同时产生，不需要考虑时序

在单总线结构处理器和多周期处理器中，要求控制信号有严格的时间先后顺序

传统计算机中有三级时序结构：指令周期、机器周期、时钟周期（节拍脉冲）

- 状态周期：表示当前指令周期处于哪个机器周期
- 节拍电位：表示当前机器周期处于哪个节拍
- 节拍脉冲：一个节拍电位内设置一个或多个节拍脉冲，例如1个节拍电位内设置2个节拍脉冲

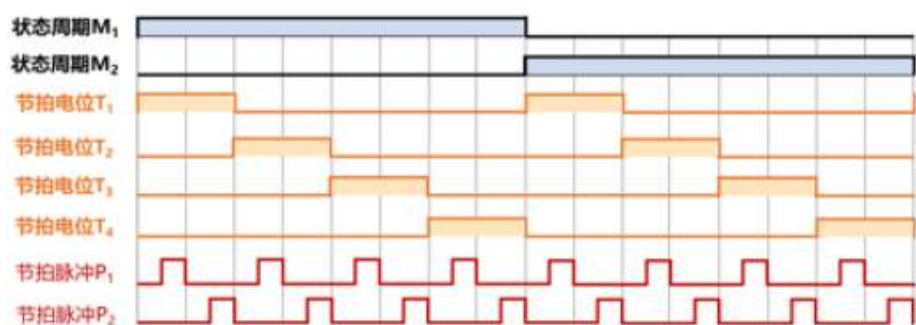


图6.36 三级时序系统示意图

现代计算机不再设置节拍脉冲，一个时钟周期就是一个节拍，称为现代时序系统

控制方式

同步控制方式

又称为固定时序方式，选取各部件中最长的操作时间作为统一的时间间隔标准

• 定长指令周期

每条指令包含机器周期数固定，如1个指令周期做取指周期、计算周期、执行周期，就等于3个机器周期

每个机器周期包含时钟周期数固定

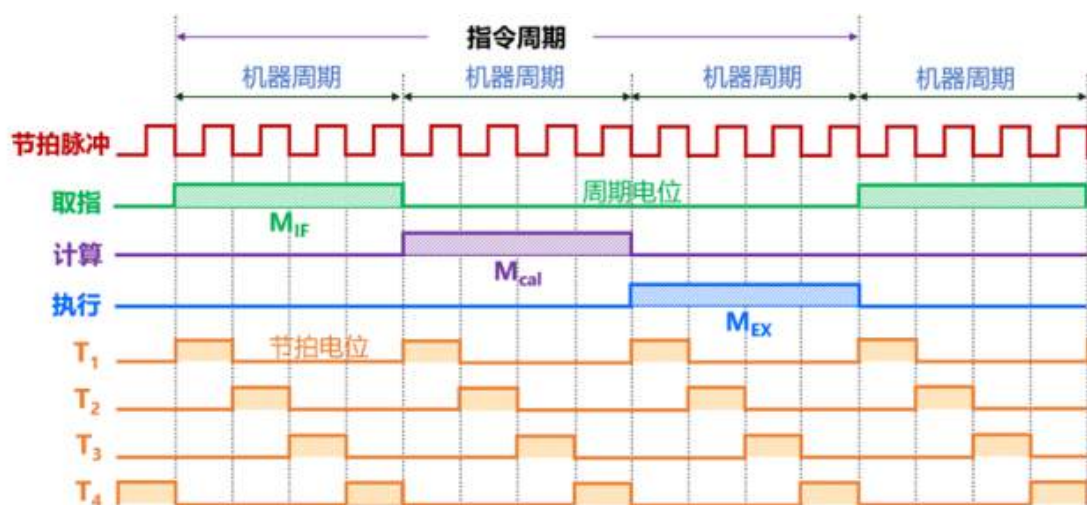


图6.37 定长指令周期的三级时序

• 变长指令周期

- 不同的指令包含的机器周期数不同
- 不同的机器周期包含的时钟周期数也不同

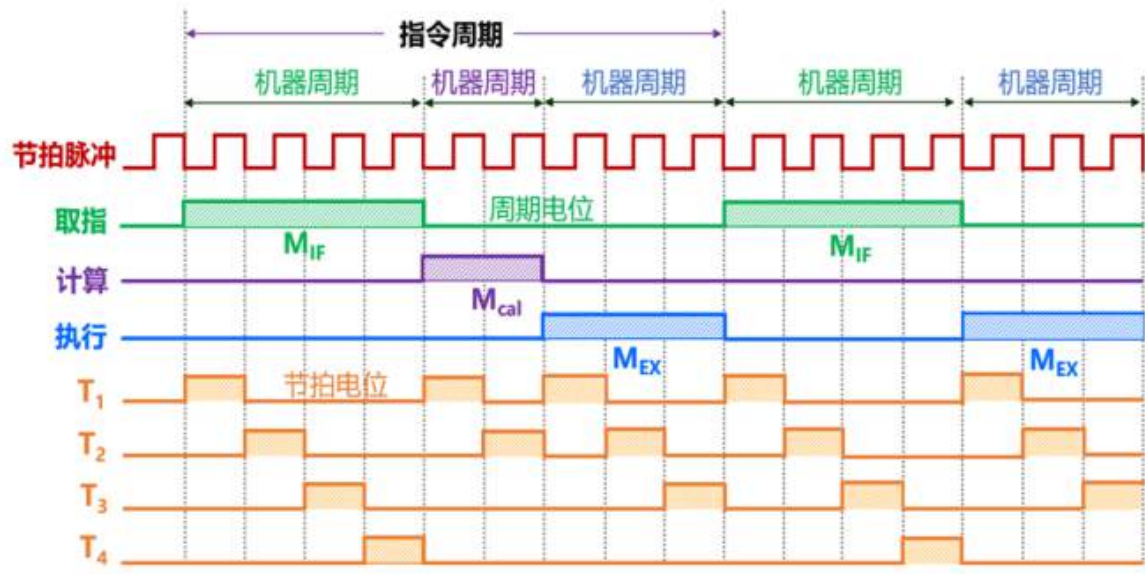


图6.38 变长指令周期的三级时序

异步控制方式

没有统一时钟信号，没有固定的机器周期和节拍，各功能部件及操作的时序采用**应答机制**实现

i

应答机制

控制部件发出操作控制信号给功能部件后，必须等到功能部件发出应答信号，才能开始下一步的操作

联合控制方式

同步控制和异步控制相结合的方式。

- 对大多数操作控制序列采用机器周期、节拍电位进行同步控制
- 对少数时间难以确定的操作可以采用异步控制方式。

时序发生器

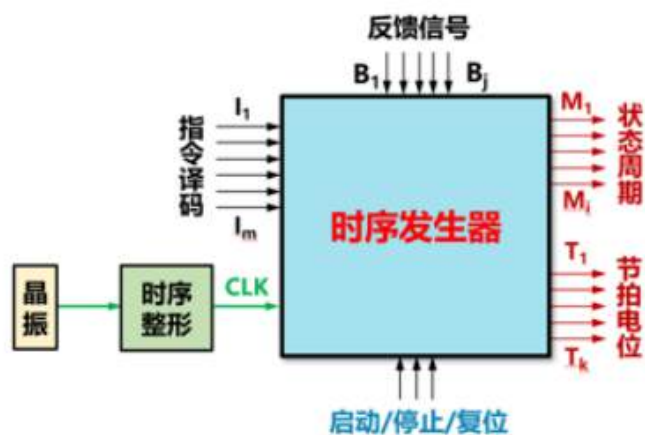


图6.39 时序发生器工作原理

时序发生器用于产生定长指令周期和变长指令周期中的状态周期与节拍电位

其输入有：

- 时钟信号CLK
- 指令译码信号
- 反馈信号
- 启动/停止/复位信号

其输出有：

- 状态周期
- 节拍电位

定长指令周期三级时序状态机

其状态周期和节拍电位只和时钟信号CLK有关，与指令译码、反馈信号无关

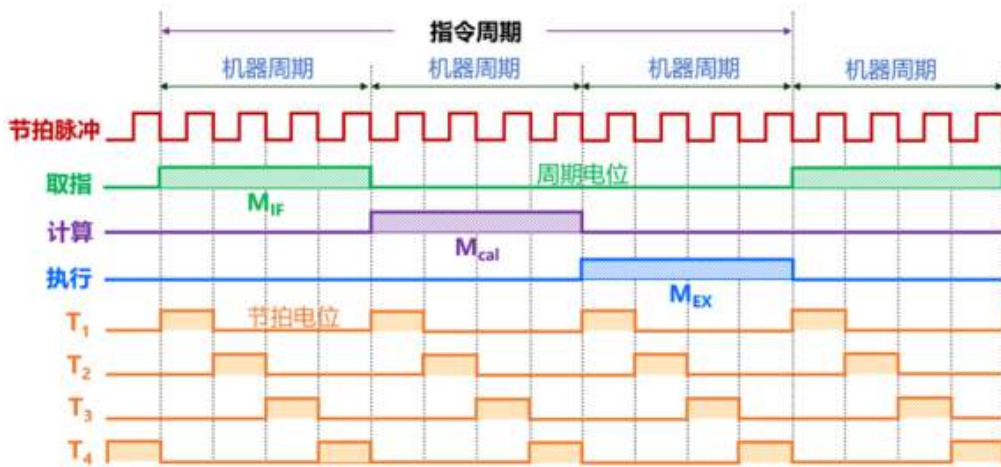


图6.37 定长指令周期的三级时序

其时序发生器可以用有限状态机实现，每条指令均有这样的12个状态



图6.40 定长指令周期三级时序状态机

- 取指周期：S0~S3
- 计算周期：S4~S7
- 执行周期：S8~S11

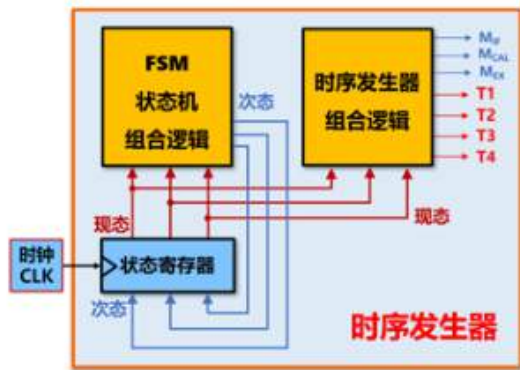


图6.42 时序发生器内部实现逻辑（定长指令周期）

变长指令周期三级时序状态机

其状态周期和节拍电位和时钟信号CLK、指令译码、反馈信号均相关

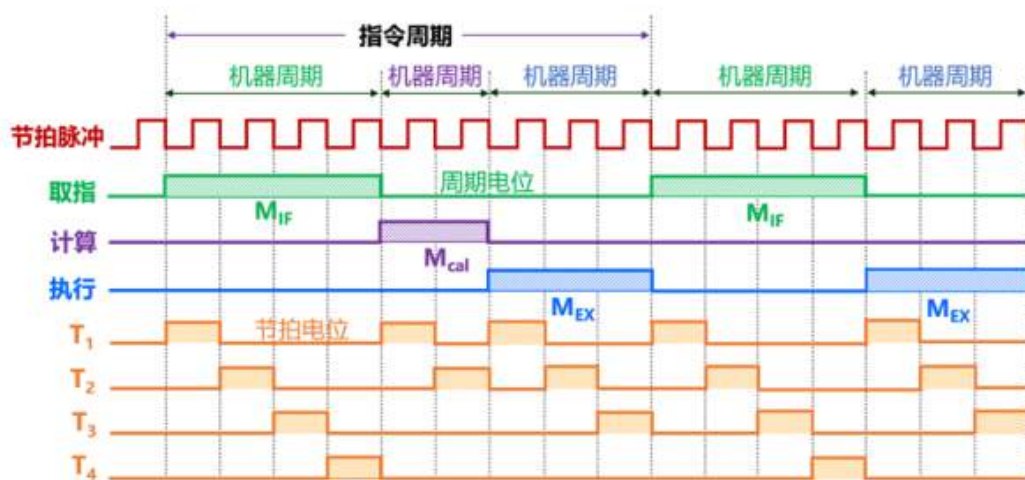


图6.38 变长指令周期的三级时序

其时序发生器也可以采用有限状态机描述，共有9个状态

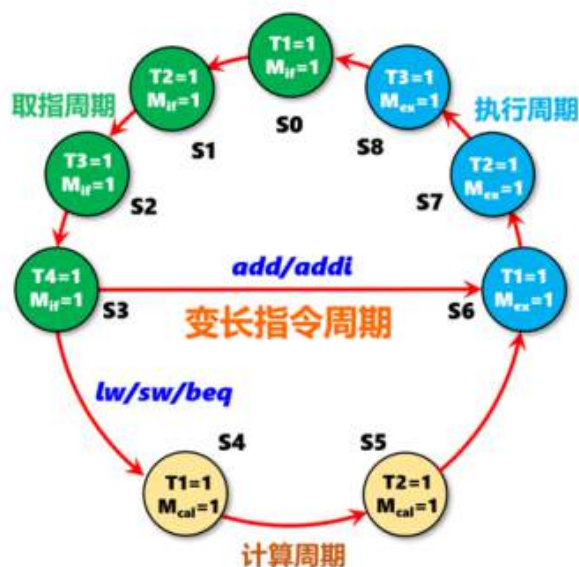


图6.41 变长指令周期三级时序状态机

- 取指周期：S0~S3
- 计算周期：S4~S5
- 执行周期：S6~S8

下表反映了时序发生器中状态的转换

表6.12 时序发生器状态转换表（变长指令周期）

现态	输入/次态			周期、节拍电位输出
	任意指令	add/addi	lw/sw/beq	
S ₀ (0000)	S ₁ (0001)			$M_{if}=T_1=1$
S ₁ (0001)	S ₂ (0010)			$M_{if}=T_2=1$
S ₂ (0010)	S ₃ (0011)			$M_{if}=T_3=1$
S ₃ (0011)		S ₆ (0110)	S ₄ (0100)	$M_{if}=T_4=1$
S ₄ (0100)	S ₅ (0101)			$M_{cal}=T_1=1$
S ₅ (0101)	S ₆ (0110)			$M_{cal}=T_2=1$
S ₆ (0110)	S ₇ (0111)			$M_{ex}=T_1=1$
S ₇ (0111)	S ₈ (1000)			$M_{ex}=T_2=1$
S ₈ (1000)	S ₀ (0000)			$M_{ex}=T_3=1$

其中，现态代表现在的状态，输入代表目前执行的是什么指令，次态代表下一个状态



lw、sw、beq指令有9个状态，add、addi指令只有7个状态（S0-S3-S6-S8）

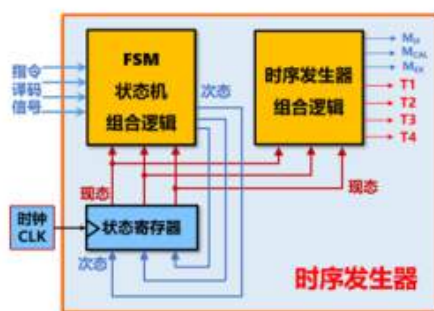


图6.42 时序发生器内部实现逻辑（变长指令周期）

硬布线控制器

硬布线控制器也称为组合逻辑控制器，其控制器由组合逻辑电路实现，控制器结构复杂，但是速度较快

三级时序硬布线控制器

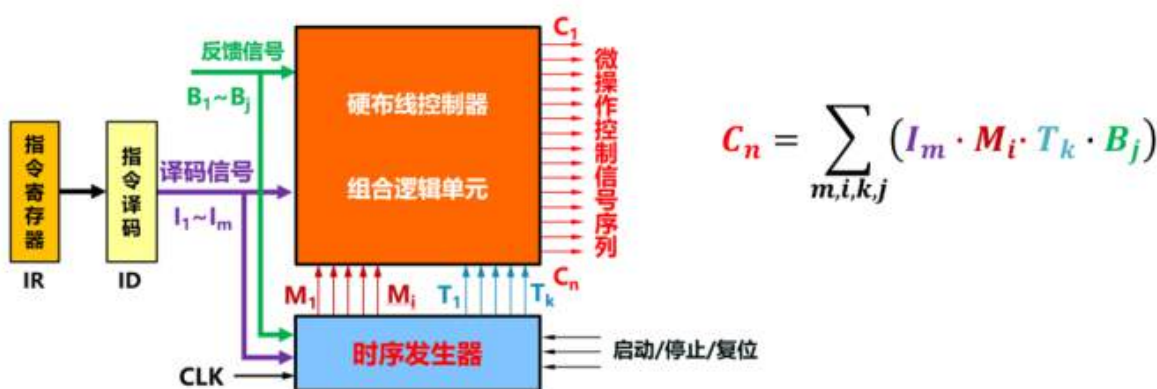


图6.43 传统三级时序硬布线控制器模型

硬布线控制器组合逻辑单元的输入为：

- 指令译码器的输出
- 反馈信号
- 时序发生器的输出

输出为微操作控制信号序列，即先前单总线结构计算机、单周期处理器、多周期处理器所需的控制信号

三级时序硬布线控制器设计

硬布线控制器的一般设计流程为：

- 1. 分析指令执行的数据通路，画出指令周期流程图，列出各节拍的控制信号。
- 2. 设计时序发生器，根据机器周期、节拍划分构建状态图
- 3. 找出同一微操作控制信号产生条件。
- 4. 写出各微操作控制信号的逻辑表达式。
- 5. 利用组合逻辑电路（逻辑门、PLA(可编程逻辑阵列)、ROM）实现

单总线结构处理器硬布线控制器

单总线结构计算机共有22个控制信号

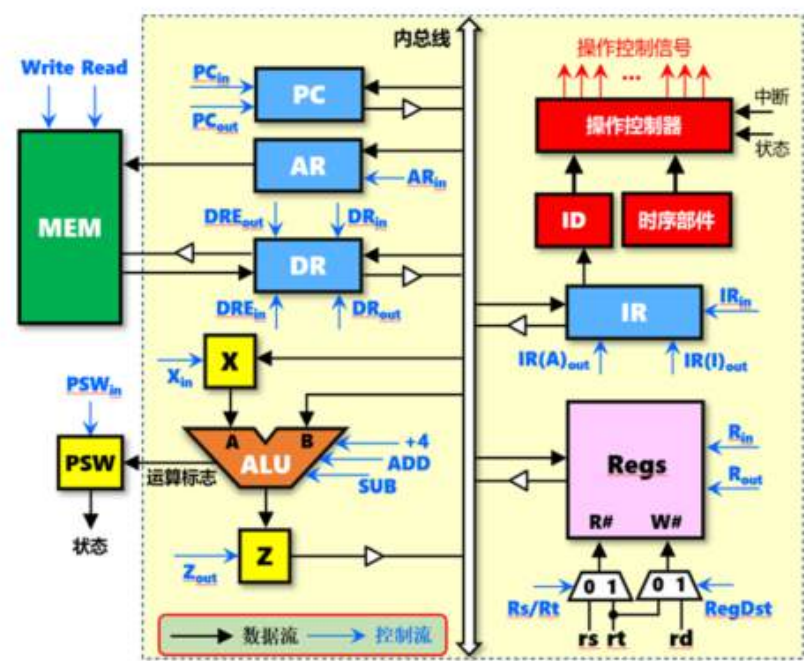


图6.8 单总线结构的计算机框图

以 X_{in} , Z_{out} , PC_{in} 控制信号实现为例

表6.1 控制信号及其功能

序号	控制信号	功能说明
1	X_{in}	控制暂存器X接收来自内总线的的数据，需配合时钟控制
2	Z_{out}	控制暂存器Z向内总线输出数据，注意暂存器Z没有输入控制，每个时钟都会锁存新值
3	PC_{in}	控制PC接收来自内总线的的数据，需配合时钟控制

- Xin
 - 所有指令的取指周期和计算周期的T1节拍都要发出Xin信号,为 $(Mif+Mcal) \cdot T1$
 - beq,addi,add指令执行周期的T1节拍要发出Xin信号, 为 $Mex \cdot T1 \cdot (beq+addi+add)$
 - 因此Xin逻辑表达式为 $Xin=Mif \cdot T1+Mcal \cdot T1+Mex \cdot T1 \cdot (beq+addi+add)$
- Zout
 - 所有指令的取指周期的T3节拍 (Z->PC) 都要发出Zout信号: $Mif \cdot T3$
 - lw、sw指令的执行周期的T1节拍 (Z->AR) 都要发出Zout信号: $Mex \cdot T1 \cdot (lw+sw)$
 - Beq (Z->PC)、addi (Z->R[rt])、add (Z->R[rd]) 指令的执行周期的T3节拍都要发出Zout信号: $Mex \cdot T3 \cdot (beq+addi+add)$
 - 因此, 控制信号Zout的逻辑表达式为:

$$Zout=Mif \cdot T3+Mex \cdot T1 \cdot (lw+sw)+Mex \cdot T3 \cdot (beq+addi+add)$$
- PCin
 - 所有指令的取指周期的T3节拍都要发出PCin信号: $Mif \cdot T3$
 - 当PSW的equal为1时, beq指令的执行周期的T3节拍 (分支目标地址送入PC) 要发出PCin信号: $Mex \cdot T3 \cdot beq \cdot equal$
 - 因此, 控制信号PCin的逻辑表达式为: $PCin=Mif \cdot T3+Mex \cdot T3 \cdot beq \cdot equal$

• 单周期MIPS处理器硬布线控制器

单周期MIPS处理器有8个控制信号, 由于单周期处理器每条指令都在1个时钟周期内执行完成, 没有状态周期和节拍电位

因此单周期处理器的控制信号只与指令译码信号有关

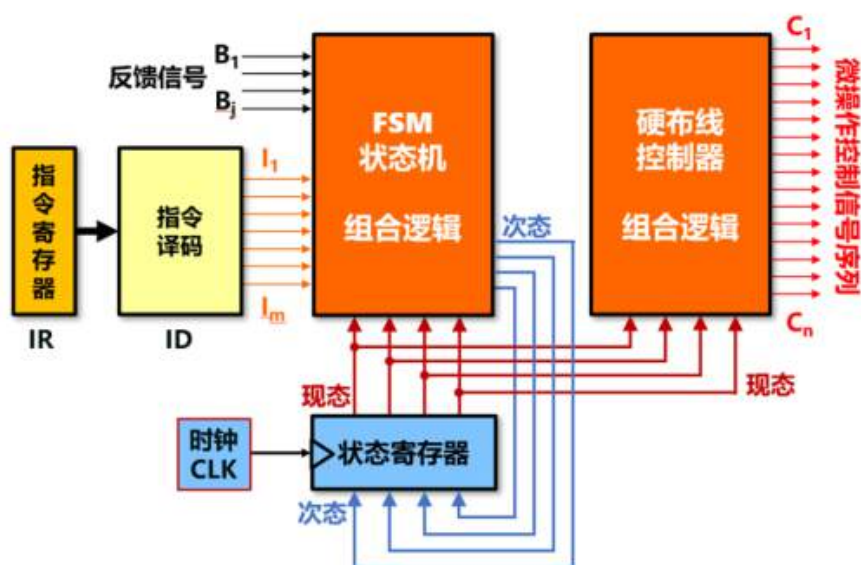
其对应的控制信号逻辑表达式为

表6.13 单周期MIPS控制器逻辑表达式

序号	控制信号逻辑表达式	功能说明
1	$\text{Jump} = j + jal$	j、jal等无条件分支指令时生成
2	$\text{Branch} = beq + bne$	分支指令译码信号，beq、bne指令需要产生类似的译码信号
3	$\text{RegDst} = add + sub + and$	写入目的寄存器的选择，为1时写入rd寄存器，为0时写入rt寄存器
4	$\text{AluSrc} = addi + andi$	控制ALU的第二输入，为0时输入R2（rt寄存器），为1时输入扩展后的立即数
5	$\text{MemToReg} = lw$	为1时将从数据存储器读出的数据写回寄存器堆，为0时将ALU的运算结果写回寄存器堆
6	$\text{RegWrite} = add + sub + and + addi + andi + lw$	控制寄存器堆写操作，为1时数据需要写入指定的寄存器，受时钟驱动
7	$\text{AluOp} = f(\text{add} + \text{sub} + \text{and} + \text{andi})$	控制ALU进行不同的运算，具体取指和位宽（如AluOP为4位）与ALU的设计有关
8	$\text{MemWrite} = sw$	控制数据存储器的写操作，为1时进行写操作，为0时读操作，受时钟驱动

现代时序硬布线控制器

现代计算机已经不再使用多级时序系统，指令执行过程中的定时信号就是基本时钟，一个时钟周期就是一个节拍



$$C_n = f(\text{现态})$$

图6.44 现代时序硬布线控制器模型

- FSM状态机输入为现态、指令译码信号、反馈信号，输出为次态
- 硬布线控制器的输入为现态，输出为微操作控制信号序列

现代时序硬布线控制器设计

- 1. 分析数据通路，画出指令周期流程图，确定各节拍（时钟周期）的控制信号。
 - 2. 绘制指令执行状态转换图。
 - 3. 构建状态转换表。
 - 4. 实现有限状态机FSM组合逻辑。
 - 5. 实现硬布线控制器组合逻辑电路
- 单总线结构处理器硬布线控制器

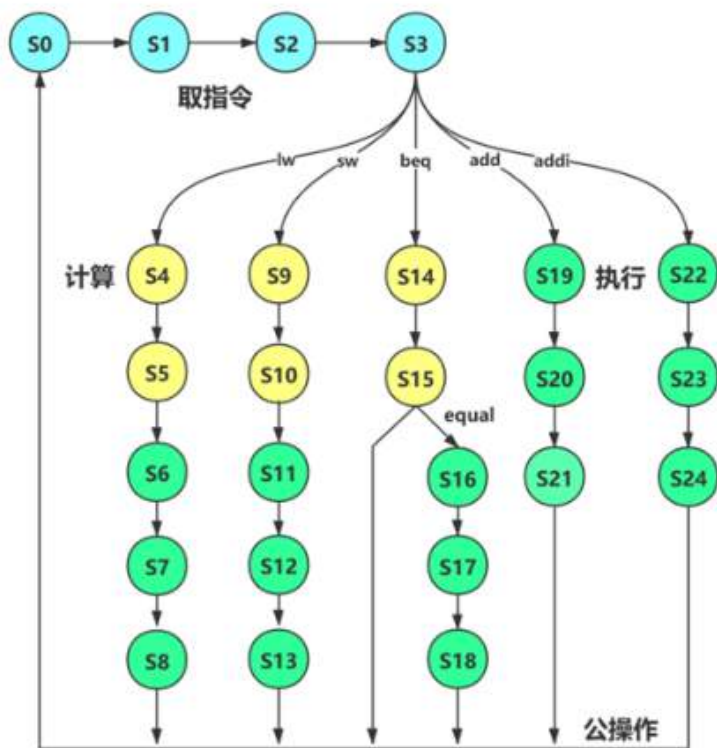


图6.45 指令执行状态转换图（单总线结构）

其状态转换表为

表6.14 现代时序指令状态转换表（单总线结构）

现态	输入/次态								操作控制信号输出
	xxx	lw	sw	beq	add	addi	equal	有中断	
S ₀	S ₁								PC _{out} =AR _{in} =X _{in} =1
S ₁	S ₂								+4=1
S ₂	S ₃								*****
S ₃		S ₄	S ₉	S ₁₄	S ₁₉	S ₂₂			*****
S ₄	S ₅								*****
S ₅	S ₆								*****
S ₆	S ₇								*****
S ₇	S ₈								*****
S ₈	S ₀							S ₂₆	*****
S ₉	S ₁₀								*****
S ₁₀	S ₁₁								*****
S ₁₁	S ₁₂								*****
S ₁₂	S ₁₃								*****
S ₁₃	S ₀							S ₂₆	*****
S ₁₄	S ₁₅								*****
S ₁₅							S ₁₆		*****
S ₁₅	S ₀							S ₂₆	*****
S ₁₆	S ₁₇								*****
S ₁₇	S ₁₈								*****
S ₁₈	S ₀							S ₂₆	*****
S ₁₉	S ₂₀								*****
S ₂₀	S ₂₁								*****
S ₂₁	S ₀							S ₂₆	*****
S ₂₂	S ₂₃								*****
S ₂₃	S ₂₄								*****
S ₂₄	S ₀							S ₂₆	Z _{out} =R _{in} =1



表中增加了1个状态S₂₆（中断响应周期状态）；在每条指令的执行周期结束时，如果有中断请求，则进入中断响应周期

• 多周期MIPS处理器硬布线控制器

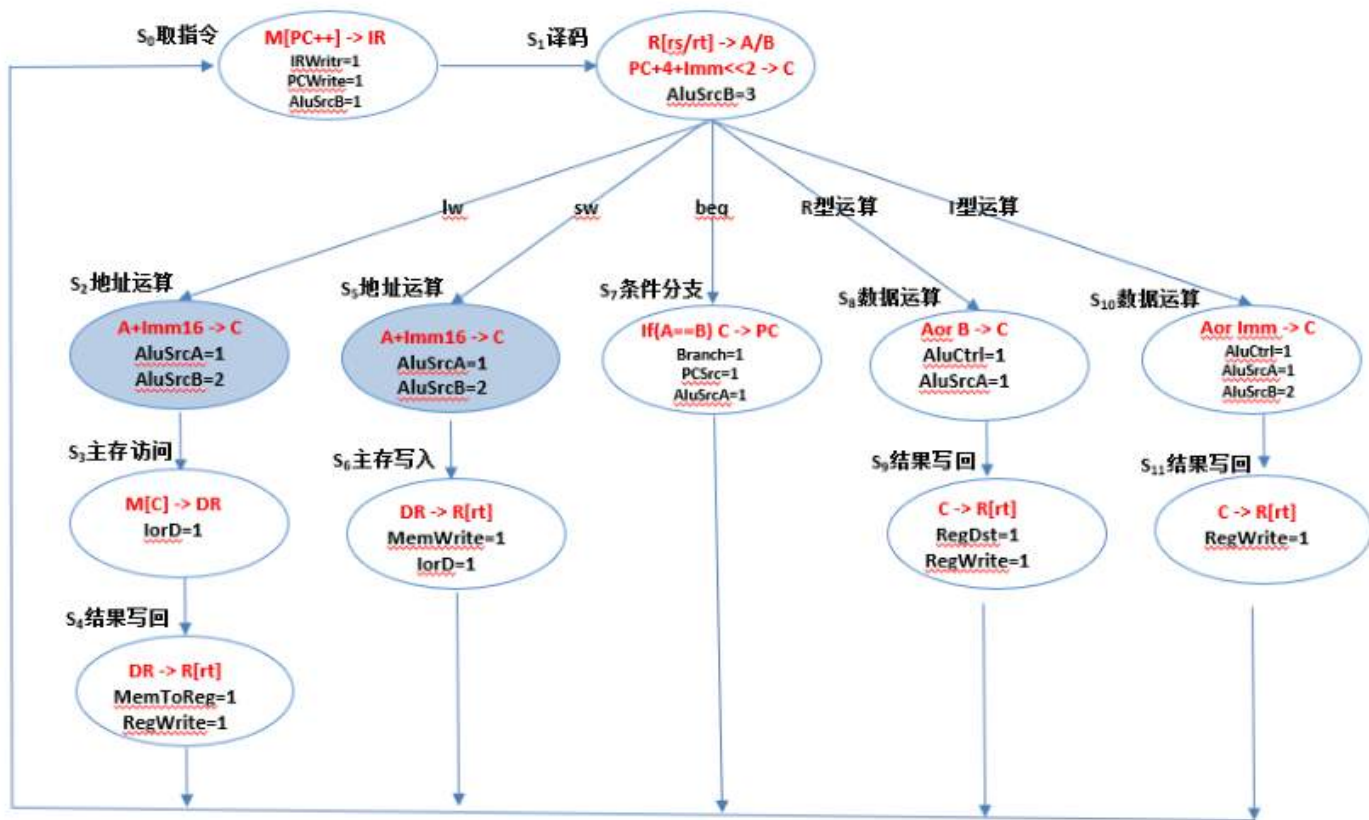


图6.46 多周期MIPS指令执行状态转换图

其状态转换表为

表6.15 多周期MIPS指令状态转换表

现态	输入/次态						操作控制信号输出
	xxx	lw	sw	beq	add	addi	
s ₀	s ₁						IRWrite=PCWrite=1 AluSrcB=1
s ₁		s ₂	s ₅	s ₇	s ₈	s ₁₀	AluSrcB=3
s ₂	s ₃						
s ₃	s ₄						
s ₄	s ₀						
s ₅	s ₆						
s ₆	s ₀						
s ₇	s ₀						
s ₈	s ₉						
s ₉	s ₀						
s ₁₀	s ₁₁						
s ₁₁	s ₀						RegWrite=1

微程序控制器

基本概念

微程序控制是仿照程序设计的基本方法，将实现指令功能所需要的控制信号编码成微指令，存放在控制存储器(控存，用ROM实现)

一条指令对应一段微程序（若干条微指令），执行指令的过程就是执行微程序的过程。



相比硬布线控制器中的硬件时序，微程序控制器是一种软件时序。微程序控制器的设计采用了存储技术和程序设计技术

微程序控制器使复杂的控制逻辑得到简化

微命令与微操作

控制器发出的控制信号称为**微命令**，是控制序列的最小单位

执行部件收到微命令后所进行的操作称为**微操作**

- **相容性微操作**：能在一个时钟周期内同时执行的微操作
- **互斥性微操作**：不能在同一个时钟周期内并行执行的微操作

互斥性的微操作不能出现在同一条微指令中，只有相容性的微操作才能出现在同一条微指令中



在单总线结构计算机中，**互斥性微操作**有

- 单总线结构计算机中的存储器读写控制信号（Read、Write）
- 所有向内总线输出的控制信号（PCout、DRout、Zout、Rout、IR(A)out、IR(I)out）
- 运算器控制信号（ADD、+4、SUB）
- DR寄存器的两个输入控制信号（DRin、DREin）
相容性微操作有
- 内总线向寄存器锁存数据的使能信号（PCin、ARin、DRin、Xin、Rin、IRin）
- DR寄存器的两个输出控制信号（DRout、DREout）

微指令与微程序

在一个时钟周期内，一组实现一定操作功能的相容性微操作称为微指令

单总线结构计算机的微指令格式如下，包含**操作控制字段**和**顺序控制字段**

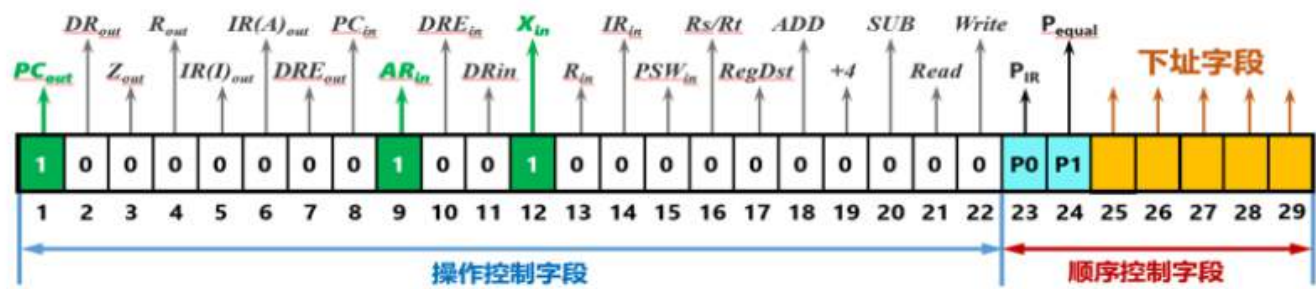


图6.47 微指令格式示例（单总线结构）

- 操作控制字段对应每一个控制信号（微命令、微操作）
- 顺序控制字段包括判断测试字段（P0、P1）和下地址字段（下址字段）
 - 测试位P0（PIR）用于取指周期结束，判断微程序进入到哪一条指令的计算周期或执行周期
 - 测试位P1（Pequal）用于beq指令判断是否相等

通常将取指周期的微指令编制成一段公共的微程序，称为**取指微程序**

微指令周期

将取出并执行一条微指令所需的时间定义为**微指令周期**，简称**微周期**

通常1个微指令周期对应1个机器周期；1个微指令周期包含4个节拍，T1节拍用于取微指令，T2~T4节拍用于执行微指令

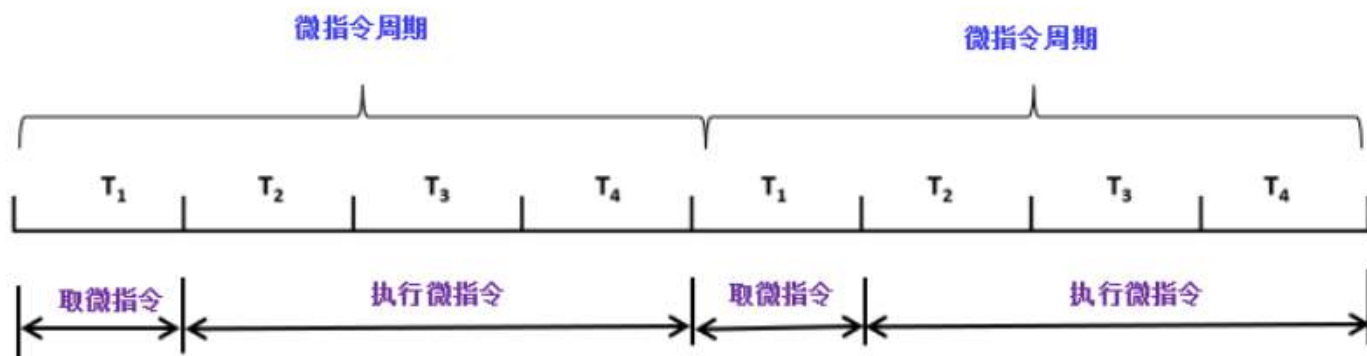


图6.48 微指令周期

微程序控制器组成原理

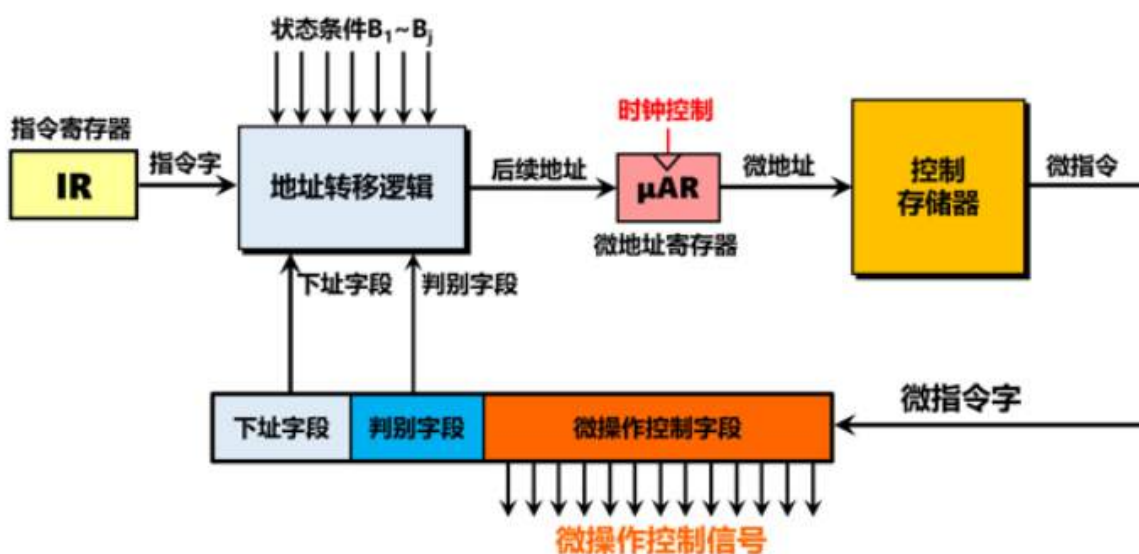


图6.49 微程序控制器组成框图

微程序控制器的部件

主要包括控制存储器（控存）、微地址寄存器（ μAR ）、地址转移逻辑等

- 控制存储器

控制存储器用于存放全部指令的所有微程序，控制存储器通常采用ROM

其长度等于微指令的长度；其容量等于所有指令的微程序包含的微指令数量，取决于指令系统

• 微地址寄存器

为控制存储器提供微地址

初始化时（上电，按Reset键） $\mu AR=0$ 。因此控制存储器0号单元的微指令应该是取指微程序的第1条微指令。

微地址寄存器的输入为地址转移逻辑的输出（后续地址）， μAR 受时钟信号CLK的控制

• 地址转移逻辑

地址转移逻辑用于产生后续地址（下一条微指令的地址）。

地址转移逻辑的输入为指令译码信号（指令字）、状态条件、微指令的判断测试字段、微指令的下址字段。

后续微地址的形成

• 下址字段法

又称为断定法

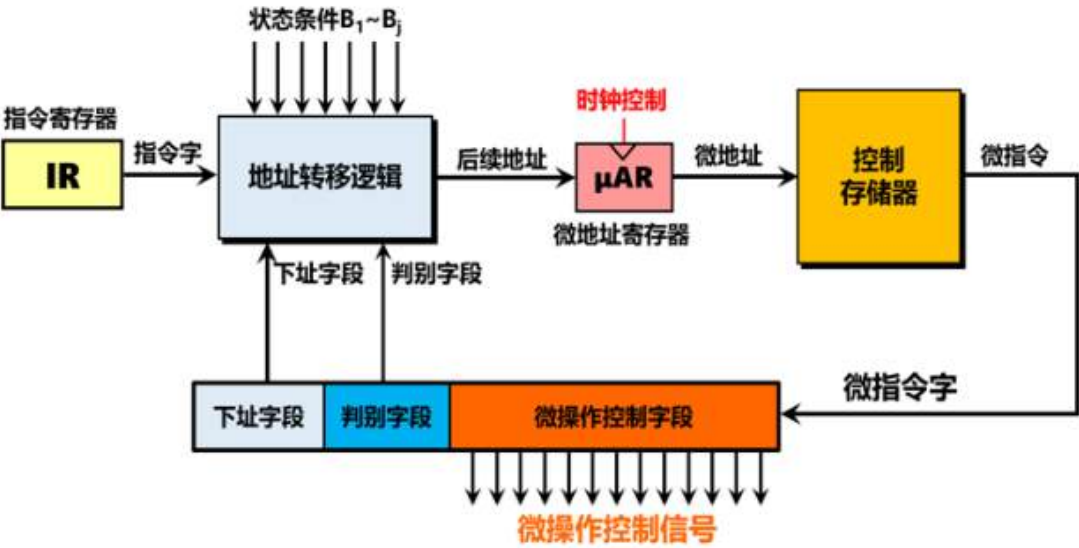


图6.49 微程序控制器组成框图（下址字段法）

• 计数器法

其思路与程序计数器PC相同，后续微指令地址由微指令寄存器+1得到

微指令不再包括下址字段

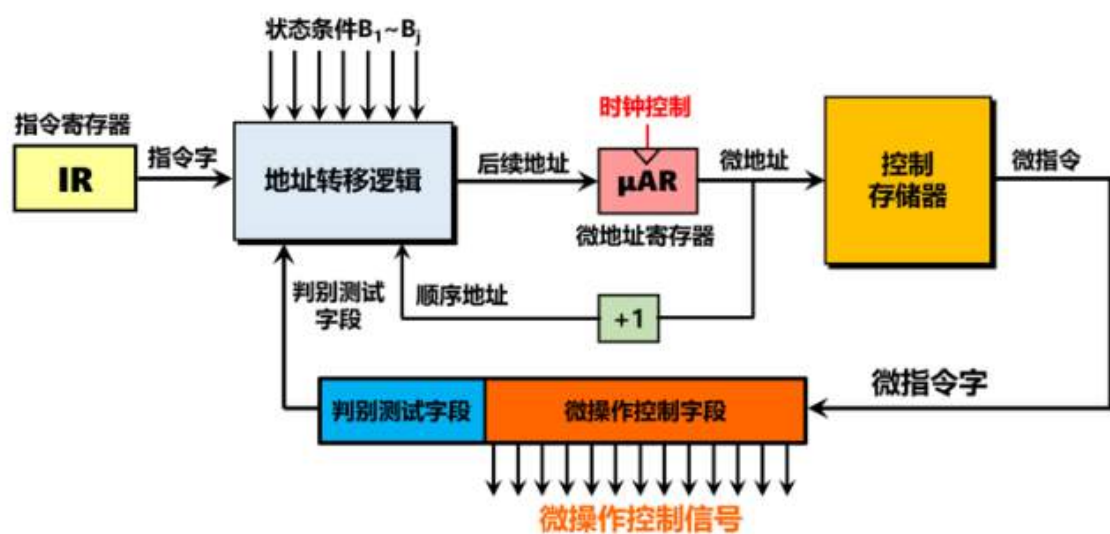


图6.50 微程序控制器组成框图（计数器法）

地址转移逻辑

- 下址字段法地址转移逻辑

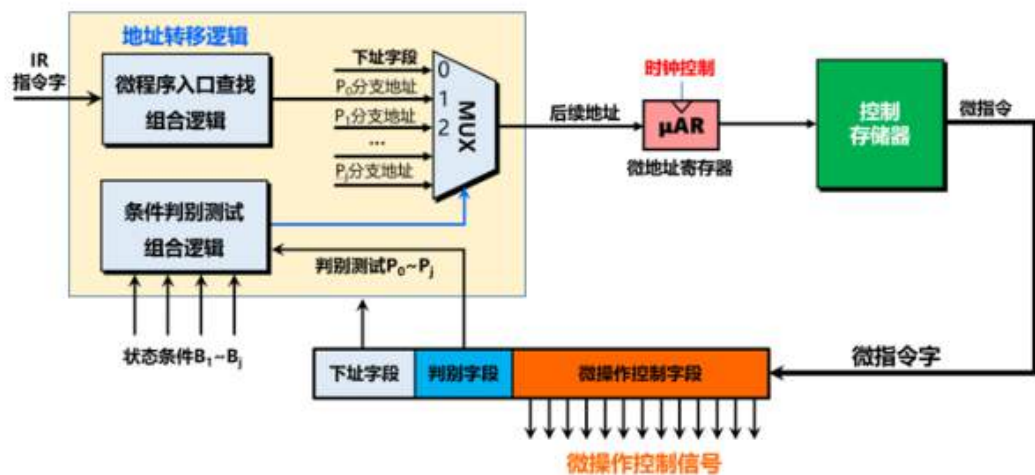


图6.51 地址转移逻辑（下址字段法）

地址转移逻辑根据指令字（指令译码信号）、状态反馈条件、判别测试字段、下址字段，给出后续微地址

- 微程序入口查找 $P_0=1$

将IR的指令字（指令译码信号）直接转换成微程序入口地址

- 条件判别测试

用于比较判别测试字段的 $P_1 \sim P_j$ 与状态反馈条件中的 $B_1 \sim B_j$ 是否相符

如果相符，则转到相应的微程序分支地址处

💡 例如beq指令微程序的 P_1 (Pequal) 判断测试位，当PSW中的equal=1时，表示条件成立

- 判别测试字段中的 P_0 (PIR) 用于指令译码测试， $P_0=1$ 时，微程序的后续地址由微程序入口查找给定。

即每条指令的计算周期或执行周期对应的微指令入口地址

- 计数器法地址转移逻辑

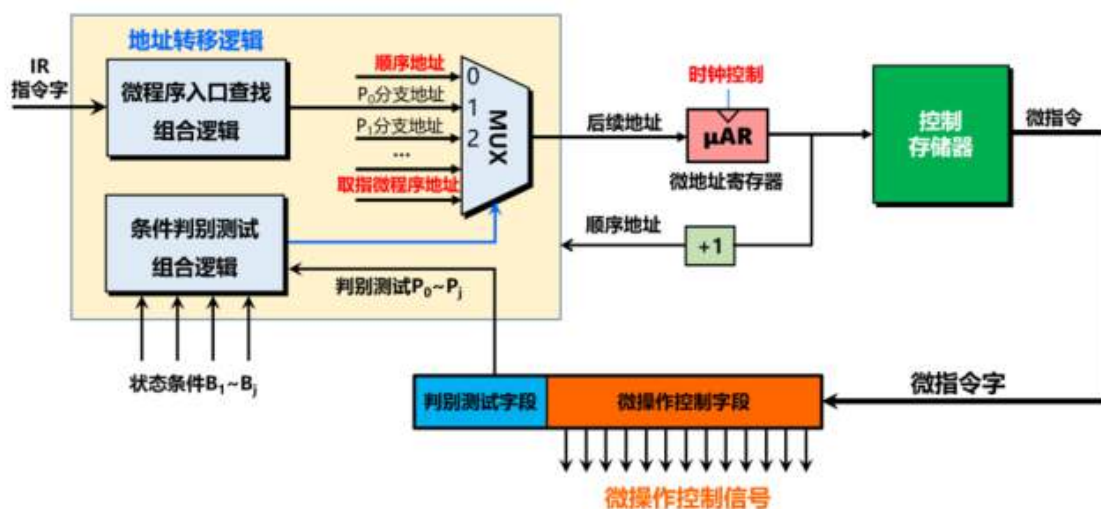


图6.52 地址转移逻辑（计数器法）

- 计数器法微指令中没有下址字段，用顺序地址代替下址字段
- 在判别测试字段增加一个结束微指令判断测试位Pend，以判断当前微指令是不是该指令的最后一条微指令。

如果是则转到取指微程序的第1条微指令地址

- 采用ROM实现的地址转移逻辑

当指令系统规模较大、判别测试条件较多时，微程序入口查找和条件判别测试将变得较为复杂，也不便于扩展

此时可以采用ROM实现微程序入口查找和条件判别测试，下面以计数器法为例

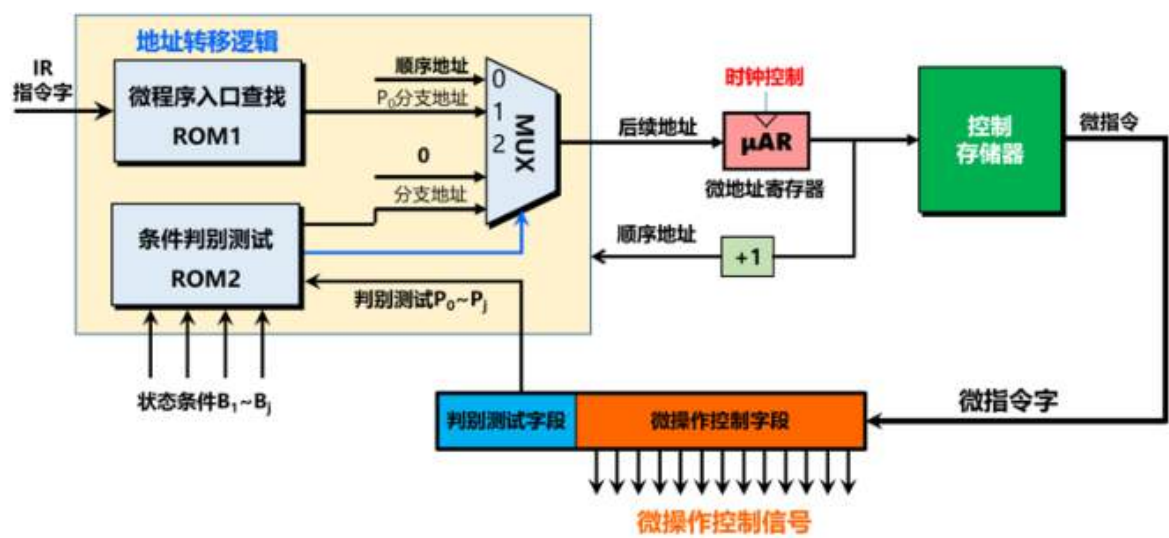


图6.53 采用ROM实现的地址转移逻辑（计数器法）

其中，多路选择器MUX的输入有4个

- 顺序地址：由微地址寄存器+1得到，用下址字段时，则为下址字段
- P0分支地址：以指令字（指令译码信号）为地址，从ROM1中得到，为每条指令的计算周期或执行周期对应的微指令入口地址
- 0：微程序的起始地址，即：取指微程序的入口，上电或按Reset键后，μAR=0
- 分支地址：为P1、P2、...、Pend判别测试位对应的分支地址

 每条指令对应的微程序入口地址均存放在ROM1，根据IR指令译码结果进行寻址，从而定位对应的入口

即对应输出P0分支地址

条件分支地址存放在ROM2，由条件判别测试逻辑形成地址去寻址定位

微程序控制器设计

以单总线结构计算机为例，用微程序方法设置其控制器

指令周期和数据通路设计已经在先前提过，接下来介绍微指令和微程序的设计

微指令设计

由于单总线结构计算机有22个控制信号，微指令的操作控制字段需要22位

- 采用下址字段法形成后续微地址，判别测试字段有P0(Pir)、P1(Pequal)2位。下址字段5位，可以支持32条微指令

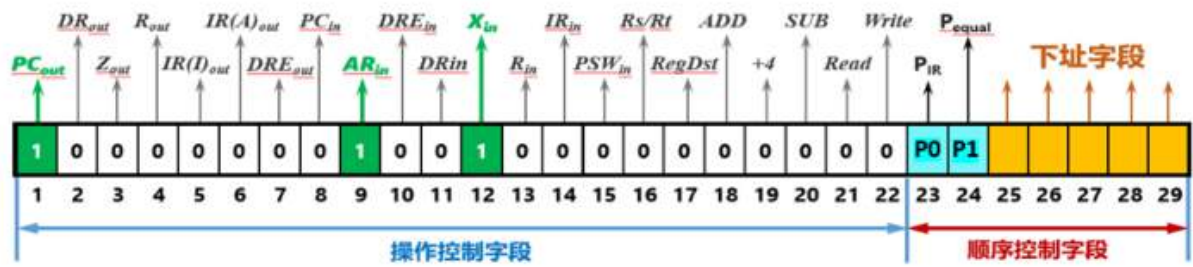


图6.54 微指令格式（下址字段法）

- 采用计数器法形成后续微地址，判别测试字段有P0(Pir)、P1(Pequal)、P2(Pend)3位，无下址字段

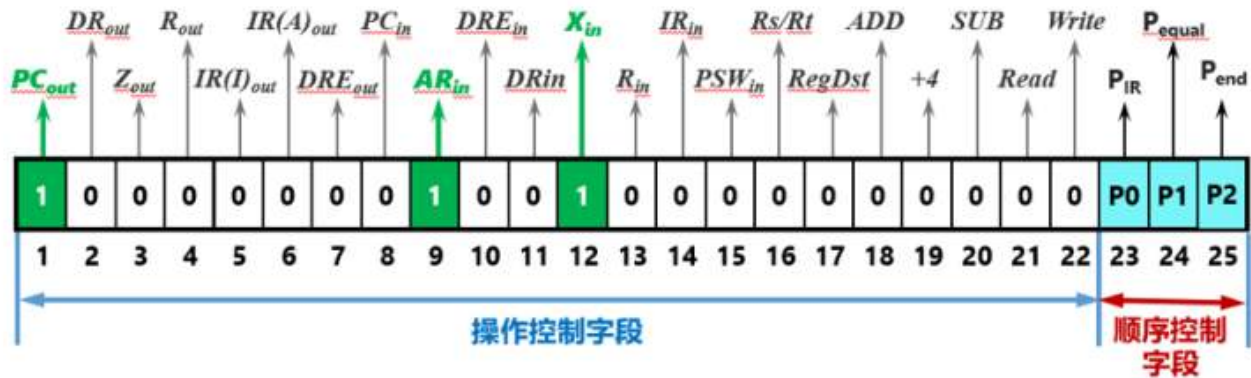


图6.55 微指令格式（计数器法）

微程序设计

采用下址字段法设计

- 取指微程序

取指周期需要4条微指令，分别与节拍T1,T2,T3,T4对应

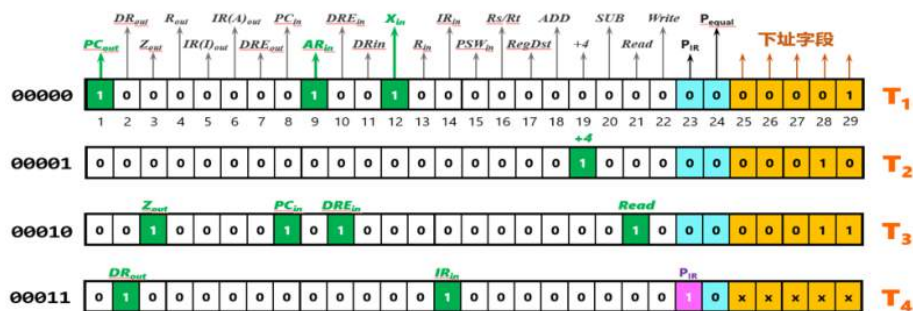


图6.56 取指微程序（下址字段法）

表6.3 lw指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	控制信号
取指周期 M_{if}	T_1	PC $\rightarrow$ AR; PC $\rightarrow$ X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	$X+4 \rightarrow Z$	$+4=1$
	T_3	Z $\rightarrow$ PC; M[AR] $\rightarrow$ DR	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	DR $\rightarrow$ IR	$DR_{out}=IR_{in}=1$

此处T4的判断测试位Pir=1，表示要根据指令字（指令译码信号）来确定微指令的后续地址，下址字段不确定

• lw指令微程序

1. 第1条微指令（微地址=00100）对应计算周期的节拍T1，下址字段=00101。
2. 第2条微指令（微地址=00101）对应计算周期的节拍T2，下址字段=00110。
3. 第3条微指令（微地址=00110）对应执行周期的节拍T1，下址字段=00111。
4. 第4条微指令（微地址=00111）对应执行周期的节拍T2，下址字段=01000。
5. 第5条微指令（微地址=01000）对应执行周期的节拍T3，下址字段=00000，表示进入指令取值周期

接下去执行取指微程序的第1条微指令

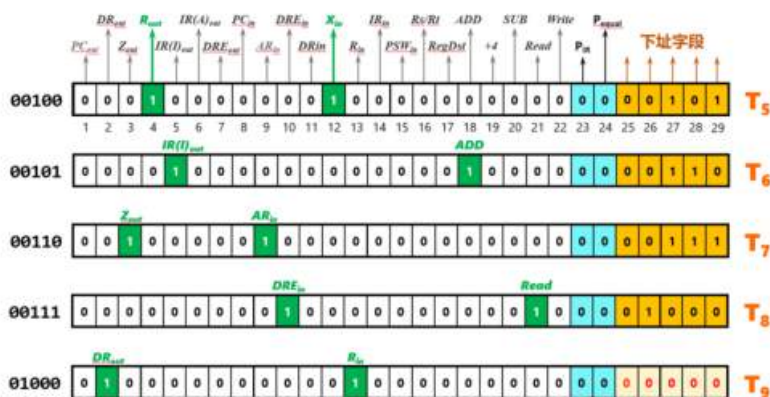


图6.57 lw指令微程序（下址字段法）

表6.3 lw指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	控制信号
计算周期 M_{cal}	T_1	R[rs] $\rightarrow$ X	$R_{out}=X_{in}=1$
	T_2	IR(I) + X $\rightarrow$ Z	$IR(I)_{out}=ADD=1$
执行周期 M_{ex}	T_1	Z $\rightarrow$ AR	$Z_{out}=AR_{in}=1$
	T_2	M[AR] $\rightarrow$ DR	$Read=DRE_{in}=1$
	T_3	DR $\rightarrow$ R[rt]	$DR_{out}=R_{in}=1$

• sw指令微程序

1. 第1条微指令（微地址=01001）对应计算周期的节拍T1，下址字段=01010。

- 第2条微指令（微地址=01010）对应计算周期的节拍T2，下址字段=01011。
- 第3条微指令（微地址=01011）对应执行周期的节拍T1，下址字段=01100
- 第4条微指令（微地址=01100）对应执行周期的节拍T2，下址字段=01101。
- 第5条微指令（微地址=01101）对应执行周期的节拍T3，下址字段=00000，表示接下去执行取指微程序的第1条微指令

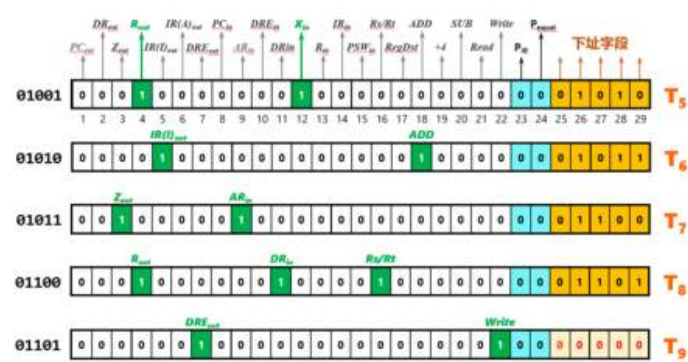


图6.58 sw指令微程序（下址字段法）

表6.4 sw指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	控制信号
计算周期 M_{cal}	T_1	$R[rs] \rightarrow X$	$R_{out}=X_{in}=1$
	T_2	$IR(I) + X \rightarrow Z$	$IR(I)_{out}=ADD=1$
执行周期 M_{ex}	T_1	$Z \rightarrow AR$	$Z_{out}=AR_{in}=1$
	T_2	$R[rt] \rightarrow DR$	$R_{out}=Rs/Rt=DR_{in}=1$
	T_3	$DR \rightarrow M[AR]$	$DRE_{out}=Write=1$

• beq指令微程序

- 第1条微指令（微地址=01110）对应计算周期的节拍T1，下址字段=01111。
- 第2条微指令（微地址=01111）对应计算周期的节拍T2，下址字段=10000。
- 第3条微指令（微地址=10000）对应执行周期的节拍T1，下址字段=10001。
- 第4条微指令（微地址=10001）对应执行周期的节拍T2，下址字段=10010。
- 第5条微指令（微地址=10010）对应执行周期的节拍T3，下址字段=00000，表示接下去执行取指微程序的第1条微指令

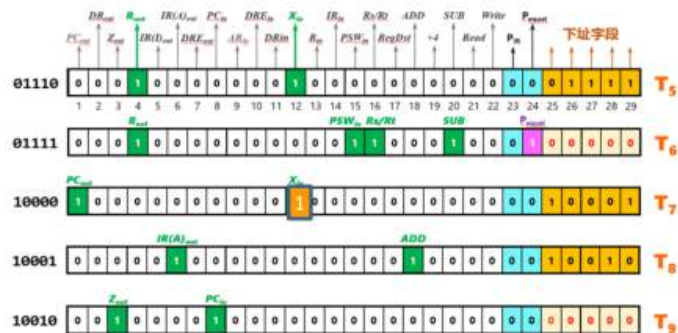


图6.59 beq指令微程序（下址字段法）

表6.5 beq指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	控制信号
计算周期 M_{cal}	T_1	$R[rs] \rightarrow X$	$R_{out}=X_{in}=1$
	T_2	$X - R[rt] \rightarrow PSW$	$R_{out}=Rs/Rt=SUB=PSW_{in}=1$
执行周期 M_{ex}	T_1	$PC \rightarrow X$	$PC_{out}=X_{in}=1$
	T_2	$IR(A) + X \rightarrow Z$	$IR(A)_{out}=ADD=1$
	T_3	$if(PSW_{equal}) Z \rightarrow PC$	$Z_{out}=1$ $PC_{in}=PSW_{equal}$

• add指令微程序

- 1. 第1条微指令（微地址=10011）对应执行周期的节拍T1，下址字段=10100。
- 2. 第2条微指令（微地址=10100）对应执行周期的节拍T2，下址字段=10101。
- 3. 第3条微指令（微地址=10101）对应执行周期的节拍T3，下址字段=00000，表示接下去执行取指微程序的第1条微指令

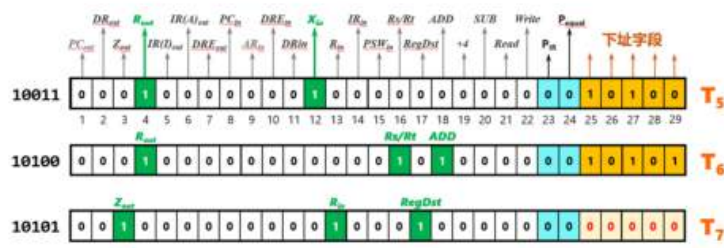


表6.7 add指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	控制信号
执行周期 M _{add}	T ₁	R[rs] -> X	R _{out} =X _{in} =1
	T ₂	R[rt] + X -> Z	R _{out} =R _s /R _t =ADD=1
	T ₃	Z -> R[rd]	Z _{out} =RegDst=R _{in} =1

图6.60a add指令微程序（下址字段法）

• addi指令微程序

- 1. 第1条微指令（微地址=10110）对应执行周期的节拍T1，下址字段=10111。
- 2. 第2条微指令（微地址=10111）对应执行周期的节拍T2，下址字段=11000。
- 3. 第3条微指令（微地址=11000）对应执行周期的节拍T3，下址字段=00000，表示接下去执行取指微程序的第1条微指令

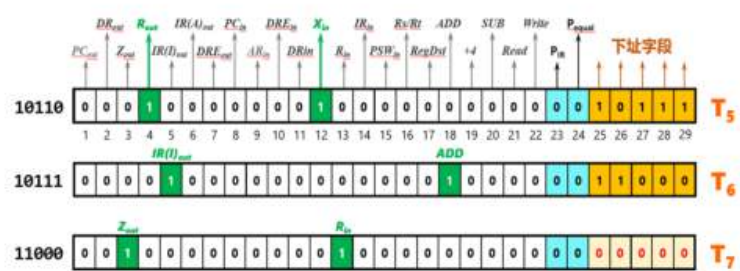


表6.6 addi指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	控制信号
执行周期 M _{addi}	T ₁	R[rs] -> X	R _{out} =X _{in} =1
	T ₂	IR(I) + X -> Z	IR(I) _{out} =ADD=1
	T ₃	Z -> R[rt]	Z _{out} =R _{in} =1

图6.60b addi指令微程序（下址字段法）

因此，采用下址字段法的单总线结构的计算机5条机器指令共需要25条微指令

每条微指令的长度=22位（操作控制字段）+2位（判别测试字段）+5位（下址字段）=29位，控制存储器的容量为：25x29位

状态	微地址	操作控制字段																顺序控制字段							
0	00000	1						1			1									0	0	0		0	1
1	00001																			0	0	0	1	0	
2	00010			1				1		1							1			0	0	0	1	1	
3	00011	1															1			*	*	*	*	*	
4	00100				1							1								0	0	1	0	1	Lw
5	00101					1														0	0	1	1	0	
6	00110			1						1										0	0	1	1	1	
7	00111										1								1	0	1	0	0	0	
8	01000	1																		0	0	0	0	0	SW
9	01001				1									1						0	1	0	1	0	
10	01010					1														0	1	0	1	1	
11	01011			1							1									0	1	1	0	0	
12	01100				1							1								0	1	1	0	1	beq
13	01101						1												1	0	0	0	0	0	
14	01110				1								1							0	1	1	1	1	
15	01111					1										1	1			0	0	0	0	0	
16	10000	1											1							1	0	0	0	1	add
17	10001							1												1	0	0	1	0	
18	10010				1						1									0	0	0	0	0	
19	10011					1								1						1	0	1	0	0	
20	10100						1									1				1	0	1	0	1	addi
21	10101				1								1							0	0	0	0	0	
22	10110					1								1						1	0	1	1	1	
23	10111						1													1	1	0	0	0	
24	11000			1																0	0	0	0	0	

图6.61 单总线结构计算机的微程序（下址字段法）

如果用计数器法设计，则不需要下址字段，但还要一个判别测试位，此时每条微指令长度为25位，控制存储器的容量为：25x25位

状态	微地址	操作控制字段																顺序控制字段		
																		P ₀	P ₁	P _{end}
0	00000	1																		
1	00001																			
2	00010			1																
3	00011			1																
4	00100																			
5	00101																			
6	00110																			
7	00111																			
8	01000																			
9	01001																			
10	01010																			
11	01011																			
12	01100																			
13	01101																			
14	01110																			
15	01111																			
16	10000																			
17	10001																			
18	10010																			
19	10011																			
20	10100																			
21	10101																			
22	10110																			
23	10111																			
24	11000																			

图6.61b 单总线结构计算机的微程序（计数器法）

微程序控制器设计流程

1. 分析指令执行的数据通路，列出每条指令的执行操作流程和每一步需要的控制信号。
2. 对指令的操作流程进行细化，将每条指令的每个微操作分配到具体的机器周期的各个时间节拍信号上
3. 以时钟周期为单位构建指令执行状态图。
4. 设计微指令格式、微命令编码方式。
5. 根据指令执行状态图编制每条指令的微程序，按照状态机组织微程序并存放到控制存储器中。
6. 根据微程序组织方式构建微程序控制器中的地址转移逻辑，微地址寄存器AR、控制存储器之间的通路，实现微程序控制器



微程序控制器与硬布线控制器的比较

- 微程序控制器因为要频繁访问控制存储器，其性能比硬布线控制器要差

- 微程序控制器的设计规整、实现容易

如果控制存储器采用可写的ROM（如EEPROM、FlashROM），可方便修改微程序、扩展指令的功能

- 微程序控制器适合CISC计算机，如x86、IBMS/360、DECVAX等；

硬布线控制器适合RISC计算机，如MIPS、ARM、RISC-V

微指令及其编码方法

微命令编码方法

微命令的编码方法就是微指令中的操作控制字段的表示方法

- 直接表示法



图6.62a 直接表示法

微指令操作控制字段的每1位对应1个微命令，为1时发出该控制信号，否则为不发出该控制信号

直接表示法较为简单，并行能力强，操作快，但控制字段很长

先前的微程序设计均为直接表示法

- 编码表示法

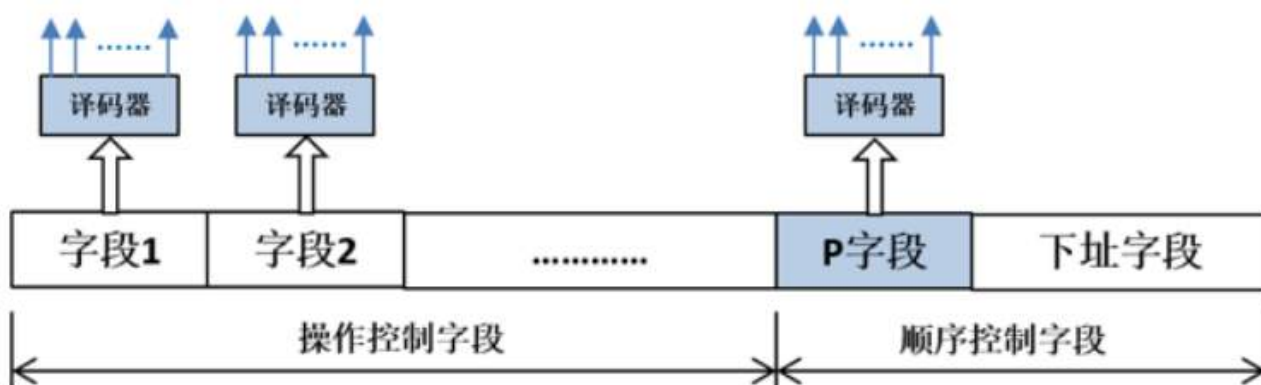


图6.62b 编码表示法

编码表示法也称为字段译码法，该方法将互斥性的微操作分成若干组，1个组对应1个字段
 各组内部的控制信号均是互斥的，各字段通过译码器生成控制信号
 编码表示法有效缩短微指令字长，但译码器会降低微指令执行速度

- 混合表示法

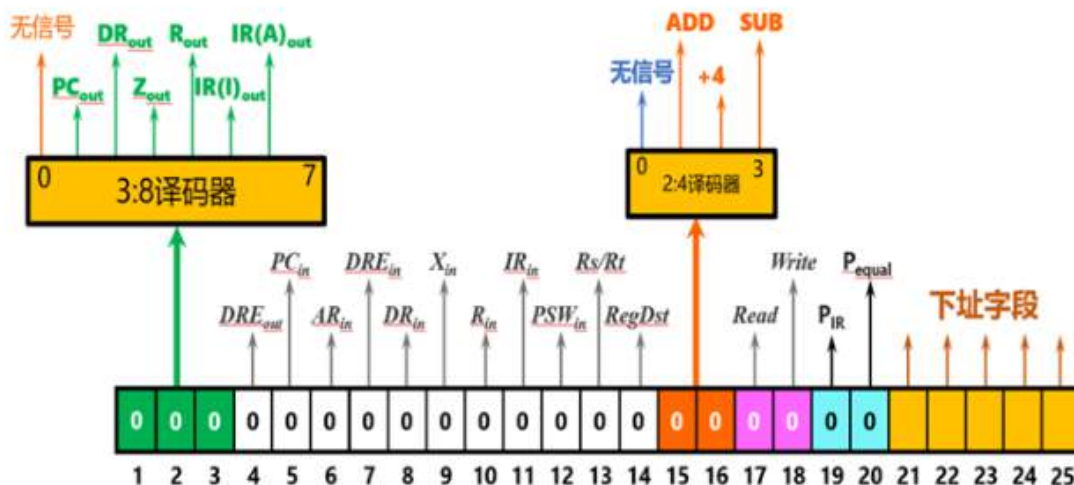


图6.62c 混合表示法

将直接表示法和编码表示法混合使用，以便在微指令字长、并行性及执行速度和灵活性等方面进行折中

• 水平型微指令

在一个微指令周期内，能同时给出多个微命令的微指令，称为水平型微指令

前面的直接表示法、编码表示法、混合表示法都属于水平型微指令，分别称为全水平型、编码水平型和混合水平型微指令

• 垂直型微指令

垂直型微指令类似于机器指令的编码方法，一条垂直型微指令包含微操作码字段和地址码字段

如果采用扩展指令格式设计，表6.16的垂直型微指令最多只需要13位的指令字即可实现，因此微程序容量为169位

表6.16 垂直型微指令格式与功能

类型	垂直型微指令举例	指令格式	功能说明
寄存器传送	MOV AR,PC	双操作数	PC -> AR
运算控制	ADD 4	单操作数	X+4 -> Z
运算控制	ADD DATA	单操作数	X + DATA -> Z
运算控制	SUB DATA	单操作数	X - DATA -> Z
主存传送	LOAD	零操作数	M[AR] -> DR
主存传送	STORE	零操作数	DR -> M[AR]
条件转移	BRANCH P ₀	单操作数	如果P ₀ =1，则根据指令译码信号转到相应指令的计算周期或执行周期的微程序入口
条件转移	BRANCH P _{end}	单操作数	如果P _{end} =1，则转到取指微程序的入口

垂直型微指令的每条微指令每个时钟周期只控制1~2个微操作，其微指令结构简单规整、字长短、易于编制微程序

但编制的微程序较长，不能充分利用数据通路固有的并行性，几乎没有并行能力，故执行效率低

💡 水平型微指令是面向数据通路的描述，垂直型微指令是面向操作算法的描述，现在垂直型微指令基本被淘汰

多处理器基本概念

SISD、SIMD、MISD、MIMD

基于指令流的数量和数据流的数量，将计算机体系结构分为 SISD、SIMD、MISD 和 MIMD 四类

常规的单处理器属于SISD，而常规的多处理器属于MIMD

- **SISD (单指令流单数据流):** 传统的**串行计算机结构**

这种计算机通常仅包含一个处理器和一个存储器，处理器在一段时间内仅执行一条指令，按指令流规定的顺序串行执行指令流中的若干指令

为了提高速度，有些SISD计算机采用流水线的方式，因此，SISD处理器有时会设置多个功能部件，并且采用多模块交叉方式组织存储器

- **SIMD (单指令流多数据流):** 指一个指令流同时对多个数据流进行处理，一般称为**数据级并行技术**。这种结构的计算机通常由一个指令控制部件、多个处理单元组成。

每个处理单元虽然执行的都是同一条指令，但是每个单元都有自己的地址寄存器，这样每个单元就都有不同的数据地址

因此，不同处理单元执行的同一条指令所处理的数据是不同的

一个**顺序应用程序**被编译后，既可能按SISD组织并运行于串行硬件上，又可能按SIMD 组织并运行于并行硬件上

- **MISD (多指令流单数据流):** 不存在这样的计算机

- **MIMD (多指令流多数据流):** 同时执行多条指令分别处理多个不同的数据，MIMD分为多计算机系统和多处理器系统

- **多计算机系统**中的每个计算机节点都具有各自的**私有存储器**，并且具有**独立的主存**，必须通过消息传递访问不同节点的私有存储器
- **多处理器系统**共享单一地址空间，直接用存取指令访问系统中所有存储器

硬件多线程

- **细粒度多线程:** 多个线程之间轮流交叉执行指令，多个线程之间的指令是不相关的，可以乱序并行执行

在这种方式下，处理器能在每个时钟周期切换线程

- **粗粒度多线程:** 连续几个时钟周期都执行同一线程的指令序列，仅在当前线程出现了较大开销的阻塞时，才切换线程，如Cache 缺失

在这种方式下，当发生流水线阻塞时，必须清除被阻塞的流水线，新线程的指令开始执行前需要重载流水线，因此，线程切换的开销比细粒度多线程更大。

- **同时多线程**：上述两种多线程技术都实现了指令级并行，但线程级不并行。

同时多线程（SMT）是上述两种多线程技术的变体。它在实现指令级并行的同时，实现线程级并行

在同一个时钟周期中，发射多个不同线程中的多条指令执行

多核处理器

多核处理器是指将多个处理单元集成到单个CPU中，每个处理单元称为一个核（core），通常也称片上多处理器。

每个核既可以有自己的Cache，又可以共享同一个Cache，所有核通常共享主存储器

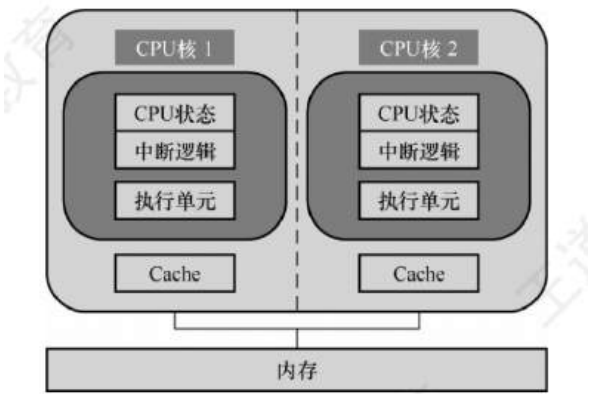


图 5.21 不共享 Cache 的双核 CPU 结构

核之间可以同时执行线程，从而实现物理意义上的并行

指令流水线

流水线概述

计算机中的流水线技术是将一个复杂的任务分解为若干个阶段，每个阶段与其它阶段并行处理

并行包括

- 时间上的并行：流水线处理技术
- 空间上的并行：多操作部件处理机或标量处理机

在先前的运算电路设计中，实际上已经采用了流水线的设计思想，这可以看作部件级的流水线

而将流水线技术应用于指令的解释执行过程则为指令级的流水线

MIPS指令流水线

MIPS指令的执行过程包括取指、译码、执行、访存、写回五个阶段

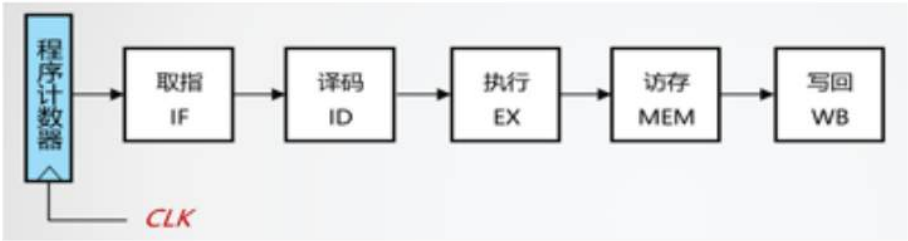


图7.1 单周期MIPS CPU逻辑架构

设 T_{total} 为5个阶段的延迟总和，现在在上图的基础上增加4个流水寄存器

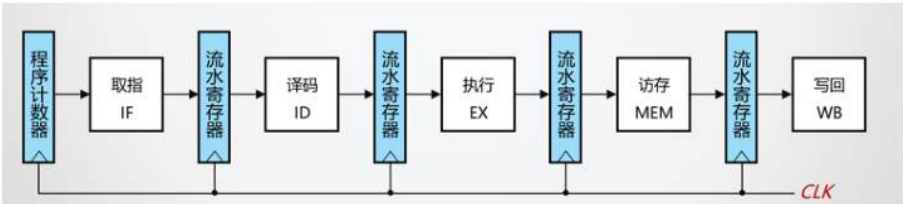


图7.2 MIPS指令流水线逻辑架构

当第1条指令进入WB段后，流水线各段都包含1条不同的指令，流水线中同时存在5条指令之后每隔1个时钟周期，都会有1条指令进入流水线，也会有1条指令流出流水线（执行完毕）

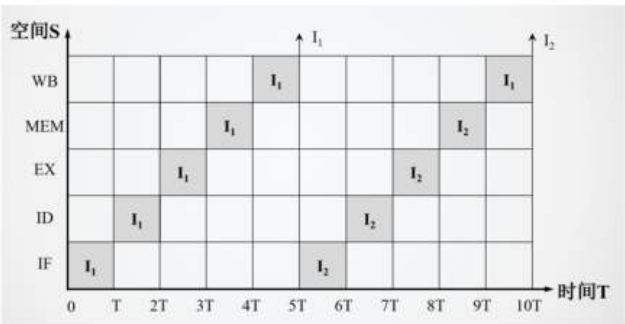


图7.3 单周期MIPS CPU时空图

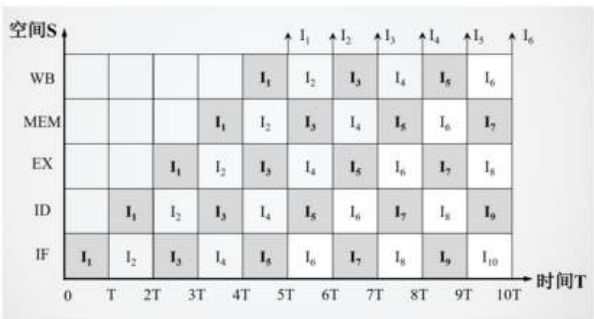


图7.4 5段MIPS流水线时空图

设 T_{max} 为5个功能段中的最大关键延迟，则执行 n 条指令需要 $(n + 4) * T_{max}$ ， $CPI = (n + 4)/n$

当 n 较大时，CPI约为1，指令流水线性能加速比为 T_{total}/T_{max}

i 流水线对指令集的要求

- 指令长度应尽量一致，有利于简化取指令和指令译码操作
- 指令格式应尽量规整，尽量保证源寄存器的位置相同，有利于在指令未知时就可取寄存器操作数
- 采用 LOAD/STORE 型指令，其他指令都不能访问存储器，这样可把 LOAD/STORE 指令的地址计算和运算指令的执行步骤规整在同一个周期中，有利于减少操作步骤
- 数据和指令在存储器中“按边界对齐”存放。这样，有利于减少访存次数，使所需数据在一个流水段内就能从存储器中得到。

流水线数据通路

单周期MIPS处理器数据通路为

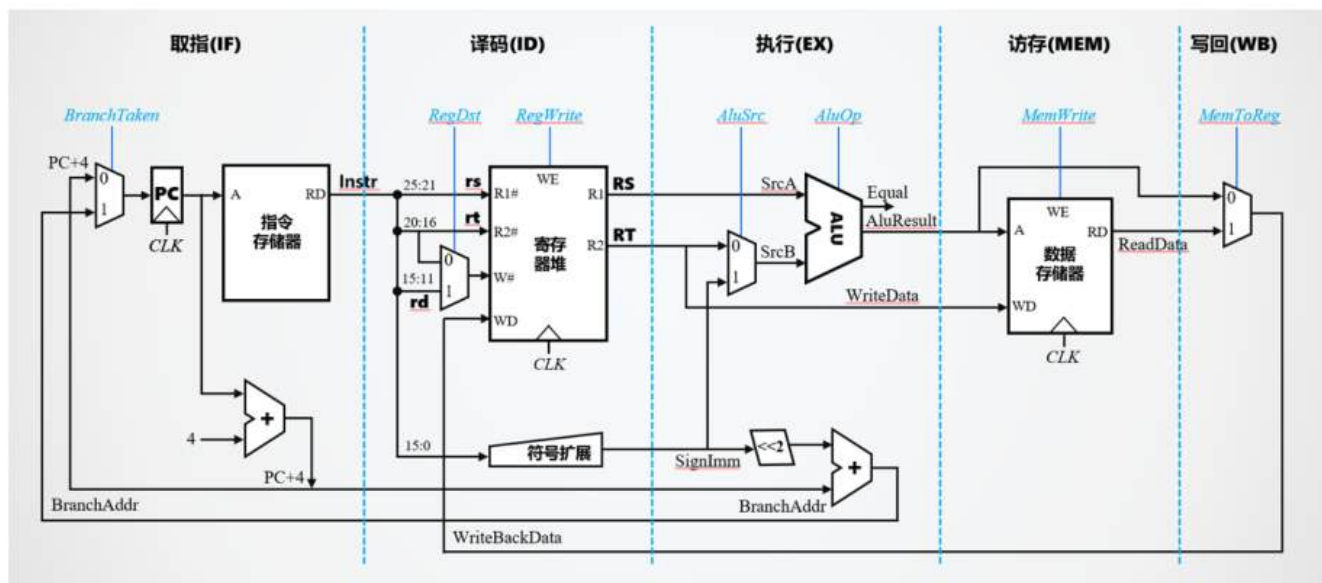


图7.5 单周期MIPS处理器数据通路

现在，在五部分间加入 4 个流水寄存器部件，流水寄存器锁存前一个段加工处理完毕后的数据和控制信号，为下一个段的功能部件提供数据输入

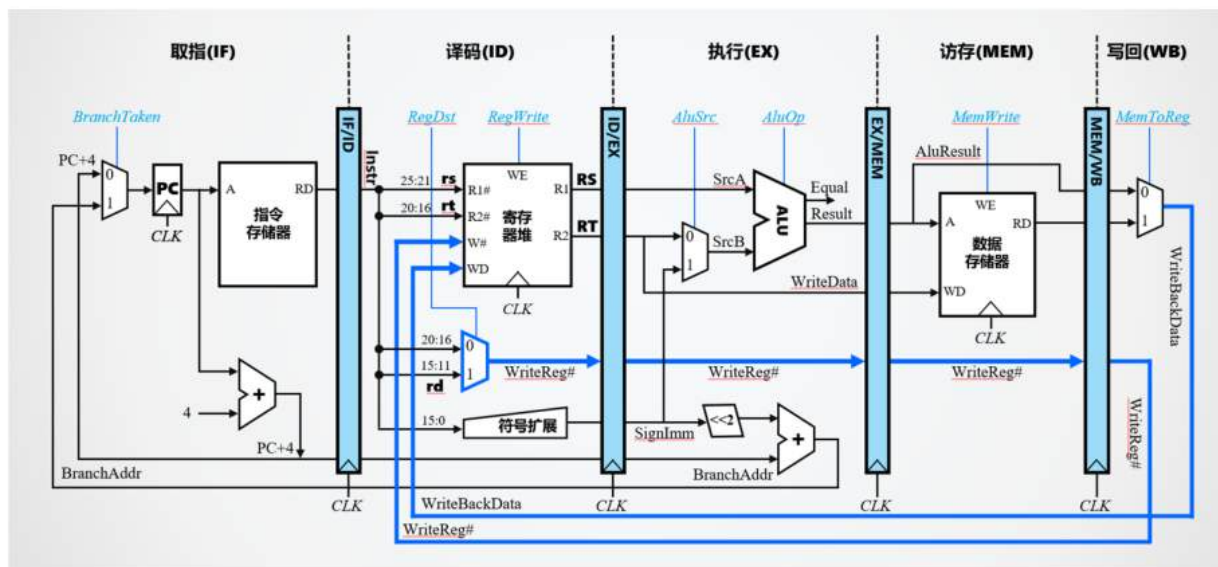


图7.7 流水线中写回数据通路改造

所有流水寄存器、程序计数器PC、寄存器堆、数据存储器均采用统一时钟CLK进行同步

- IF/ID流水寄存器锁存：指令字、PC+4。
- ID/EX流水寄存器锁存：RS、RT、WriteReg#、符号扩展的值、PC+4
- EX/MEM流水寄存器锁存：ALU的运算结果、数据存储器待写入数据WriteData、WriteReg#
- MEM/WB流水寄存器锁存：ALU的运算结果、数据存储器的读出数据、WriteReg#

⚠ 其中写回数据通路是比较特殊的，如果只添加了流水寄存器，则译码处的WD输入来自写回WB的输出，写入寄存器编号又来自ID

在流水线设计下，两者可能隶属于不同的指令，导致数据混乱

为此，将ID段的寄存器堆写入编号经过ID/EX、EX/MEM、MEM/WB流水寄存器后，再送到寄存器堆的W# 端口

这样的目的是控制时间差，保证写回数据写入正确的寄存器

流水线中控制信号及其传递

表7.1 控制信号分类

控制信号	位置	来源	功能说明
BranchTaken	IF	EX	分支跳转信号，为1表示跳转，由EX段的Branch信号和Equal标志进行逻辑与生成
RegDst	ID	ID	写入目的寄存器选择，为1时目的寄存器为rd寄存器，为0时目的寄存器为rt寄存器
RegWrite	ID	WB	控制寄存器堆写操作，为1时数据需要写回寄存器堆中的指定寄存器
AluSrc	EX	EX	ALU的第二输入选择控制，为0时输入寄存器rt，为1时输入扩展后的立即数
AluOp	EX	EX	控制ALU进行不同的运算，具体取值和位宽与ALU的设计有关
MemWrite	MEM	MEM	控制数据存储器写操作，为0时进行读操作，为1时进行写操作
MemToReg	WB	WB	为1时将数据存储器读出数据写回寄存器堆，否则将ALU运算结果写回

其中BranchTaken、RegWrite控制信号的位置和来源不一致，其他5个控制信号的位置与来源一致

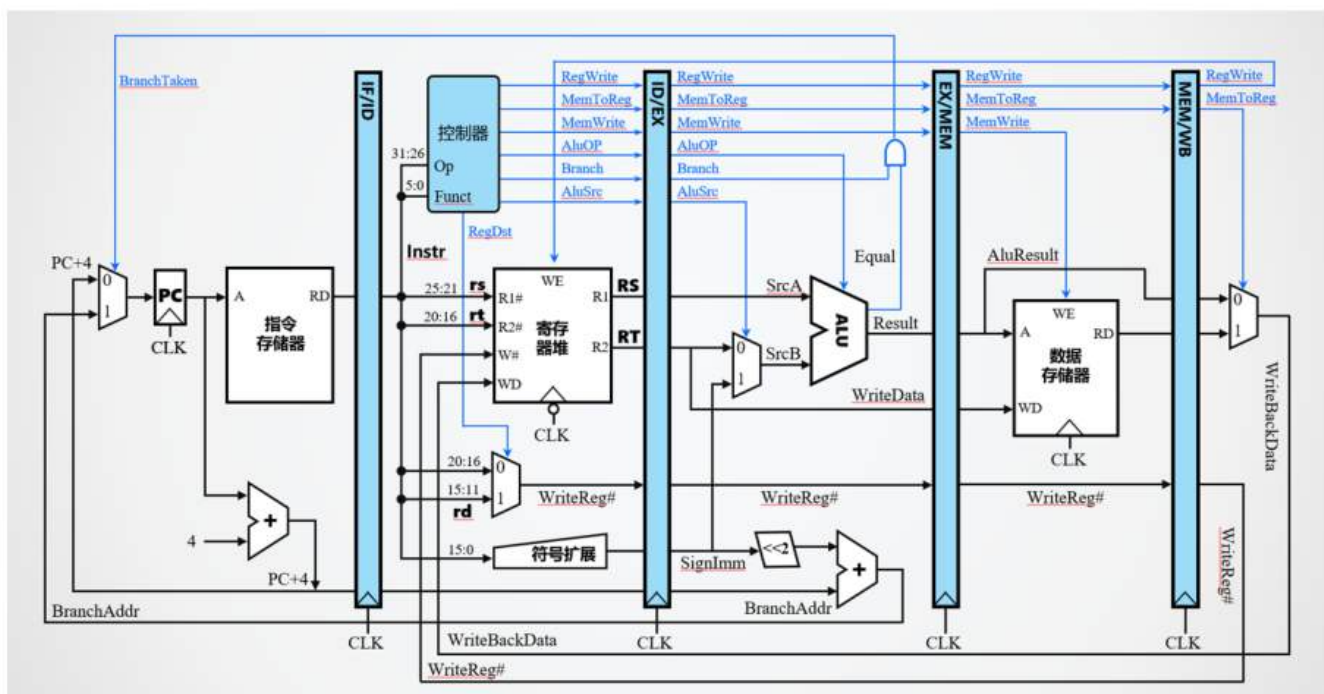


图7.8 流水线中写回数据通路改造

- 控制器输出的7个控制信号中的6个（除RegDst之外）要传递到ID/EX流水寄存器。
- ID/EX流水寄存器输出的6个控制信号中的3个（除AluSrc、AluOp、Branch之外）要传递到EX/MEM流水寄存器。

- EX/MEM流水寄存器输出的3个控制信号中的2个（除MemWrite之外）要传递到MEM/WB流水寄存器。
- MEM/WB流水寄存器输出2个控制信号（RegWrire、MemToReg）

指令在流水线中的执行过程

表7.2 指令在流水线中的执行情况

序号	指令	取指令IF	译码ID	执行EX	访存MEM	写回WB
1	lw	√	√	√	√	√
2	sw	√	√	√	√	
3	beq	√	√	√		
4	add	√	√	√		√
5	addi	√	√	√		√

下面以顺序执行五条指令为例

1. 取指令

lw指令进入IF段

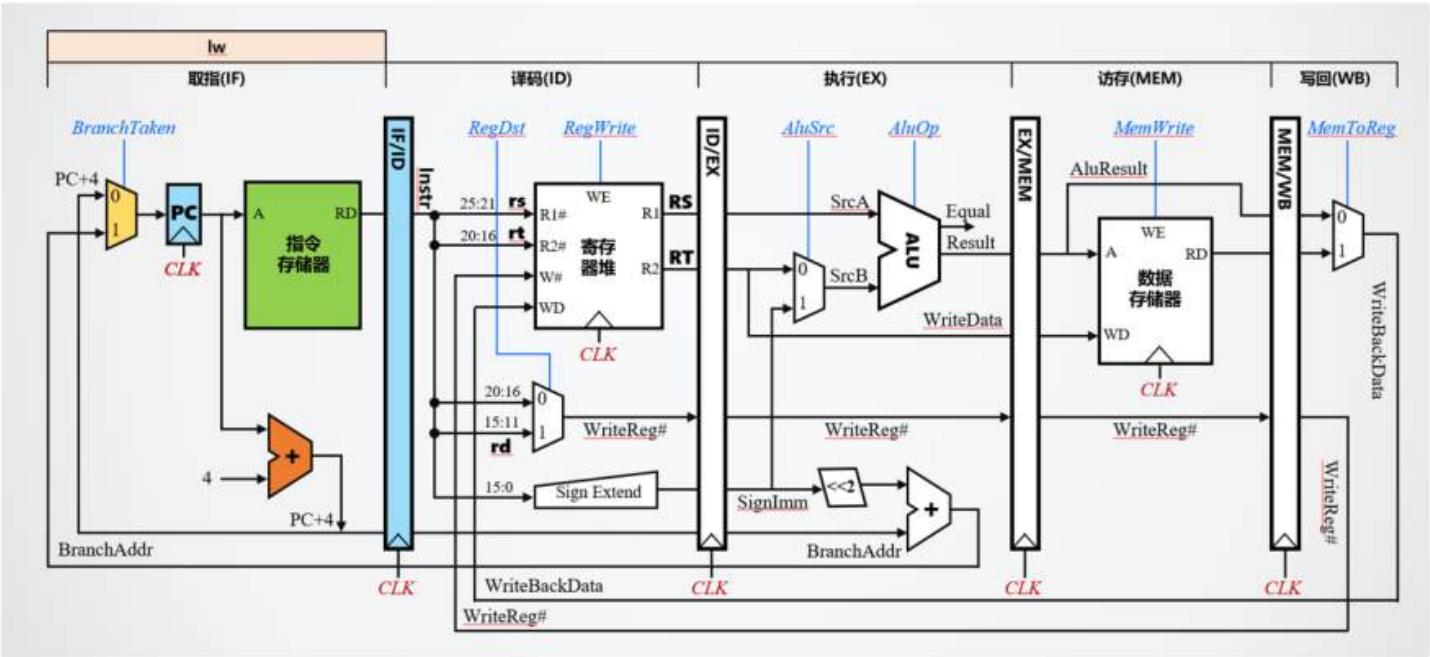


图7.9 lw指令进入IF段

1. 以程序计数器PC的值作为地址，从指令存储器中取出lw指令字，送IF/ID流水寄存器。
2. PC+4的值送PC寄存器的输入端和IF/ID流水寄存器。
3. 指令译码、取操作数

lw指令进入ID段

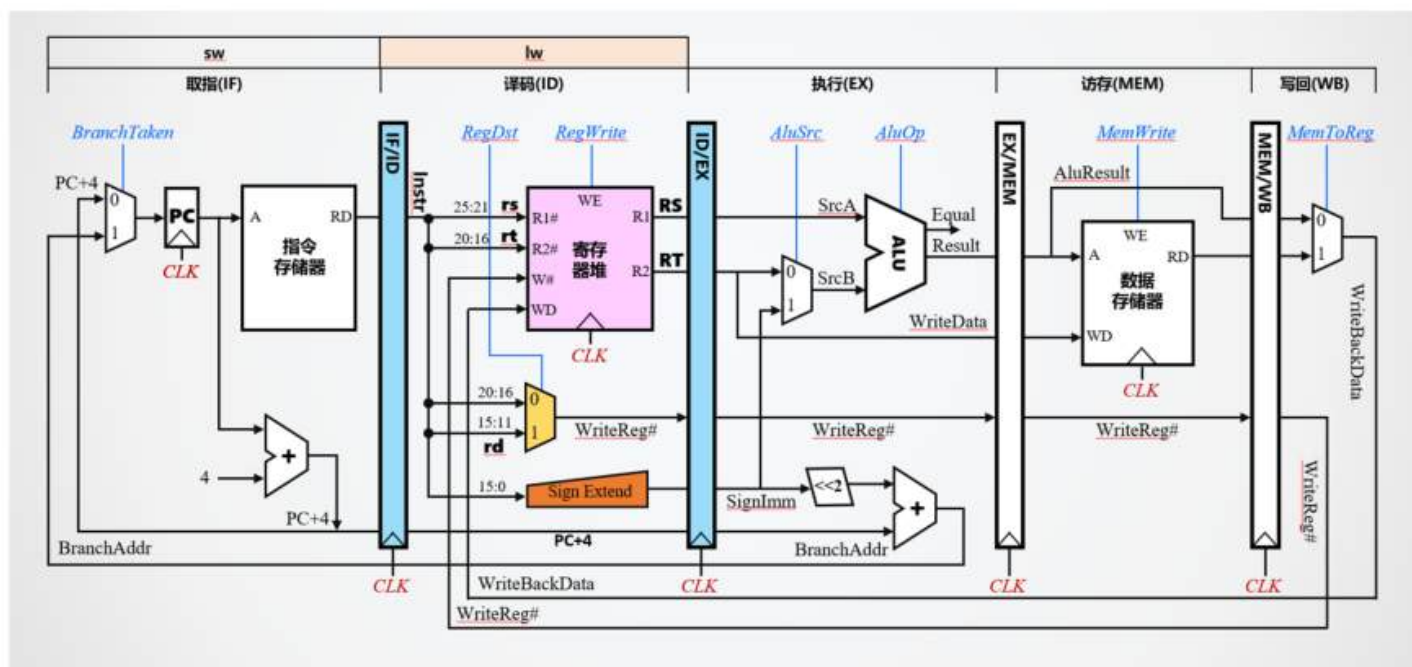


图7.10 lw指令进入ID段

1. 首先对指令字进行译码，得到7个控制信号，将其中的6个控制信号送ID/EX流水寄存器
2. 接下来取操作数，从寄存器堆中取出rs、rt寄存器的值（RS、RT），送ID/EX流水寄存器
3. WriteReg#、符号扩展器的输出，也送ID/EX流水寄存器；PC+4继续送ID/EX流水寄存器。
4. 同时，sw指令进入IF段。
5. 执行或访存地址运算

lw指令进入EX段

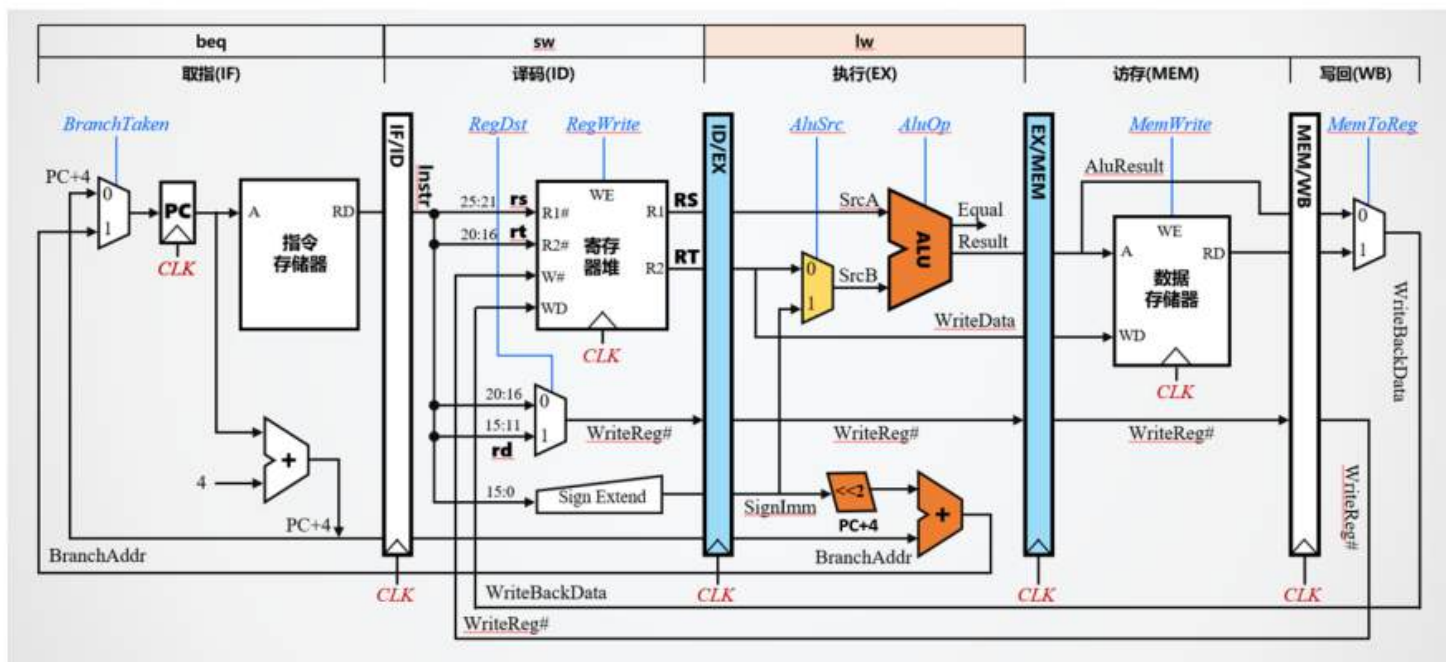


图7.11 lw指令进入EX段

1. 首先计算访存地址，将SrcA (RS) 与SrcB (符号扩展器的输出SignImm) 相加，得到访存地址，送EX/MEM流水寄存器。
2. 再计算分支目标地址，SignImm左移2位，与PC+4相加，得到分支目标地址，送PC的输入端。
3. rt寄存器的值 (RT) 作为数据存储器的写入数据，送EX/MEM流水寄存器；
4. WriteReg# 继续送ID/EX流水寄存器；3个控制信号送EX/MEM流水寄存器
5. 同时，sw指令进入ID段、beq指令进入IF段。
6. 存储器访问

lw指令进入MEM段

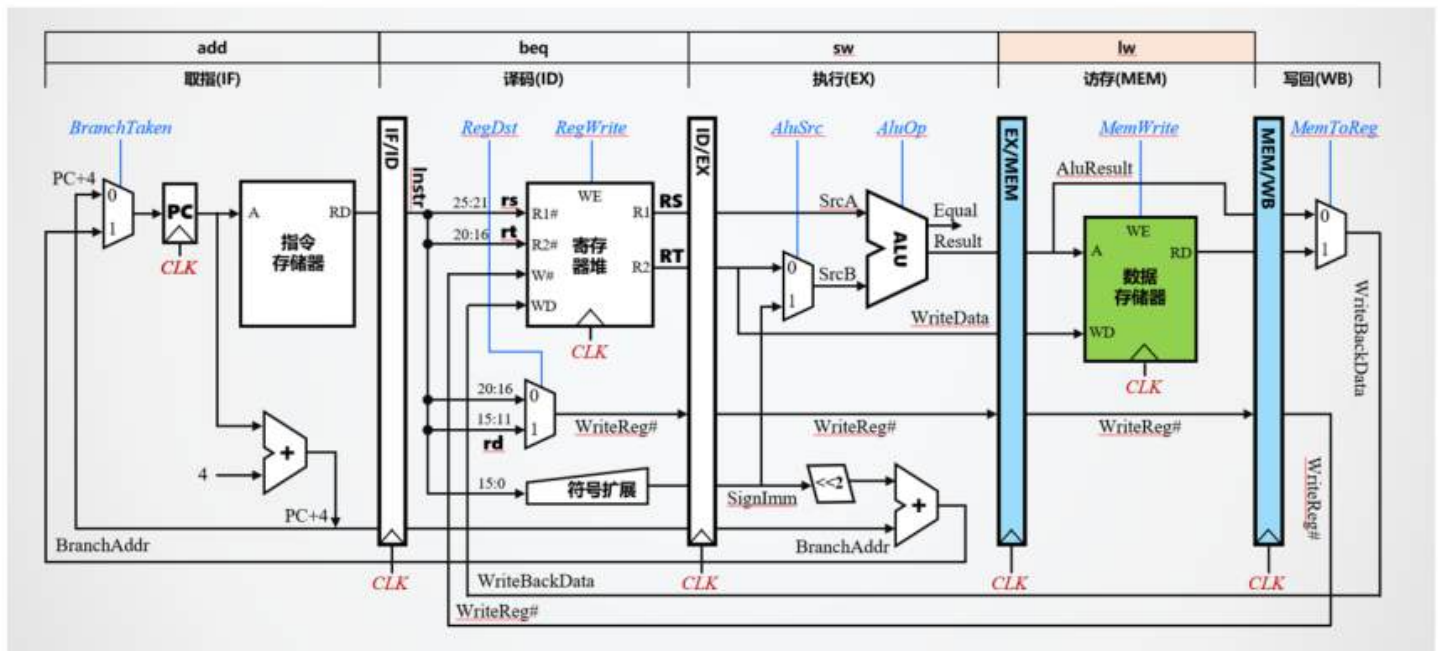


图7.12 lw指令进入MEM段

- 如为存储器读，以ALU的输出作为地址（A）读数据存储器，读出的数据（RD）送MEM/WB流水寄存器。
- 如为存储器写，以ALU的输出作为地址（A）、WriteData作为数据（WD），写数据存储器（写控制信号为MemWrite）。

ALU的输出送MEM/WB流水寄存器；WriteReg# 继续送MEM/WB流水寄存器；2个控制信号送MEM/WB流水寄存器

同时，sw指令进入EX段、beq指令进入ID段、add指令进入IF段。

5. 结果写回

lw指令进入WB段

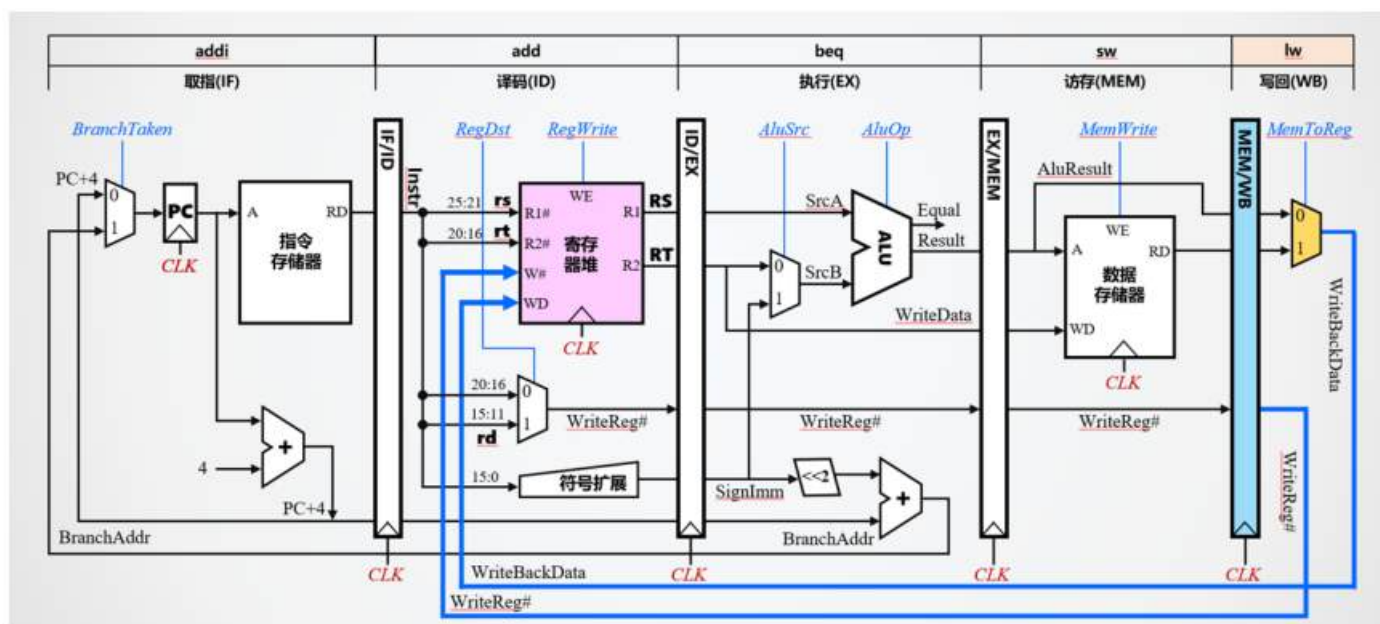


图7.13 lw指令进入WB段

将结果写入到寄存器堆中

寄存器堆的写入编号 (W) 为MEM/WB流水寄存器的输出WriteReg#。

- 如果是lw指令，寄存器堆的写入数据为数据存储器的输出
- 如果是add等指令，寄存器堆的写入数据为ALU的输出。

此时，lw指令执行完毕；同时，sw指令进入MEM段、beq指令进入EX段、add指令进入ID段、addi指令进入IF段。

流水线冲突与处理

流水线冲突是指由于指令相关的存在，指令流水线出现“阻塞”或“暂停”，下一条指令不能在预期的时钟周期内加载到流水线中

这也被称作**流水线冒险**

i 指令相关

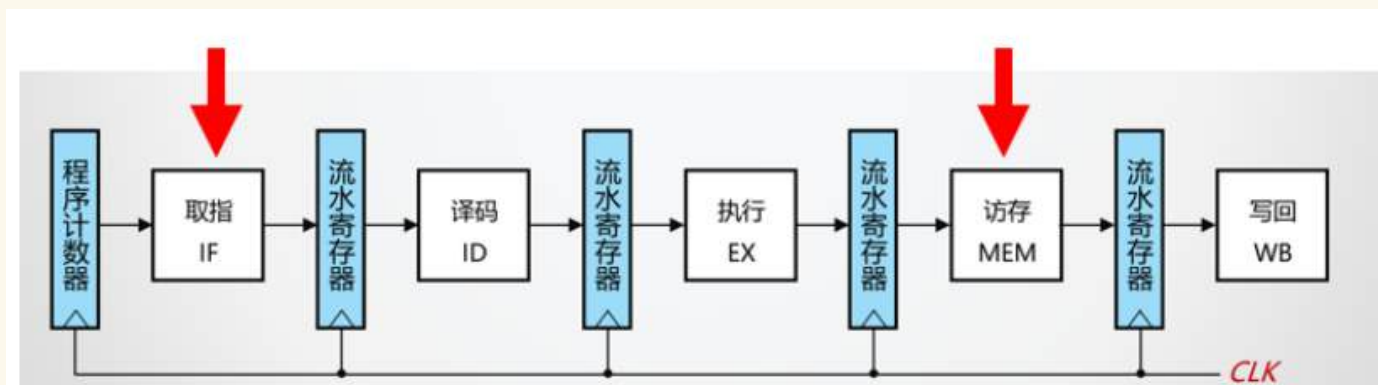
在指令流水线中，如果某指令的某个阶段必须等到它前面的某条指令的某个阶段完成后才能开始，则两条指令存在指令相关。

流水线冲突的种类

结构冲突

多条指令在同一时钟周期都需使用同一操作部件引起的冲突，称为**结构冲突**

! 假设流水线中只有一个存储器（例如多周期MIPS处理器就只有一个存储器），数据和指令都存放在同一个存储器中



当访存类指令（lw、sw）进入MEM段时，需要读、写存储器；此时，IF段也需要从存储器中读出指令字。

这时，就会产生**访存结构冲突**。

控制冲突

当流水线在分支指令、子程序调用指令、子程序返回指令时，之后载入流水线的相邻指令，可能因为分支跳转而不能进入执行阶段

这种冲突称为**控制冲突**，也称为**分支冲突**

💡 考虑下面的指令执行

```
S
beq $zero,$zero,8
add $s1,$s2,$s3
addi $s4,$s5,$s6
bne $s1,$s2,8
lw $s5,4($s1)
sw $s6,8($s1)
```


由于分支跳转是否进行要等到EX段才能确定，因此分支指令相邻的后续若干条指令（add、addi）已经被预取进入流水线

数据冲突

数据冲突是指当前指令要用到先前指令的操作结果，而这个结果尚未产生或尚未送达指定的位置

- 先写后读冲突

```
S
and $s1,$s2,$s3
sub $s2,$s1,$s3
```

第1条指令还没有完成对s1寄存器的写，第2条指令就读取s1寄存器的值
会导致第2条指令读取的是第1条指令之前的\$s1寄存器的值

- 先读后写冲突

```
S
sw $s3,0($zero)
sub $s3,$s1,$s2
```

第1条指令还没有完成对s3寄存器的读，第2条指令就完成对3寄存器的写
会导致第1条指令读取的是第2条指令改写后的\$3寄存器的值

- 写后写冲突

```
S
lw $s3,0($zero)
sub $s3,$s1,$s2
```

第1条指令还没有将存储器中的内容写到s3寄存器，第2条指令就完成对3寄存器的写
会导致第2条指令后的\$3的值，为第1条指令从存储器中读出的值

流水线冲突的处理

结构冲突处理

- 通过增设加法部件避免运算冲突
- 使用2个存储器（指令存储器、数据存储器）避免访存冲突
- 通过插入“气泡”延缓取指令的方式解决访存冲突
- 阻塞程序计数器PC，使IF段暂停一个时钟周期，等到访存操作结束后，再重新启动IF段；
该方法会使流水线暂停一个时钟周期，引起性能的损失

✓ MIPS中，不存在寄存器读写操作的结构冲突

控制冲突处理

- 将分支跳转信号作为流水线清空信号

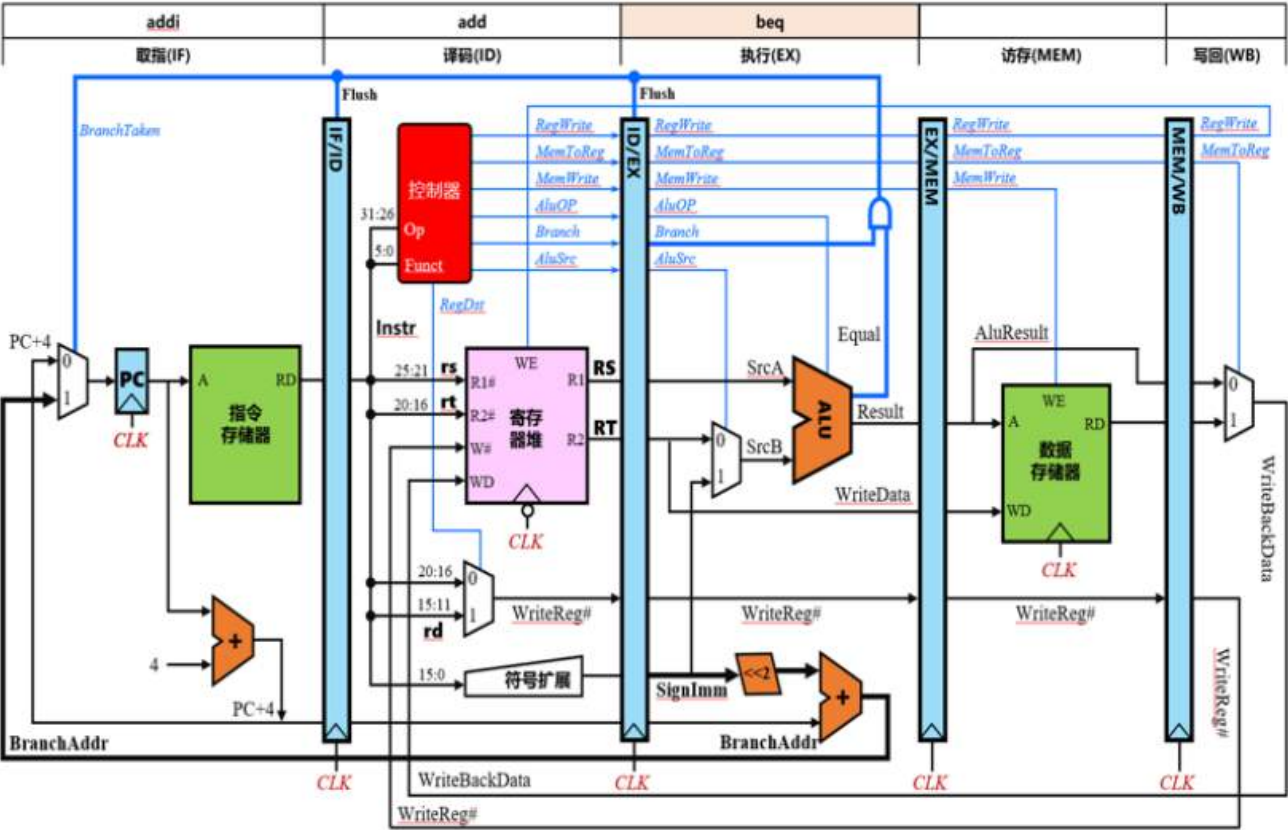


图7.14 MIPS流水线中的分支相关处理

- 在EX段执行分支指令



图7.16 在EX段执行分支指令（beq）的流水线时空图

- 在ID段执行分支指令



图7.17 在ID段执行分支指令（beq）的流水线时空图

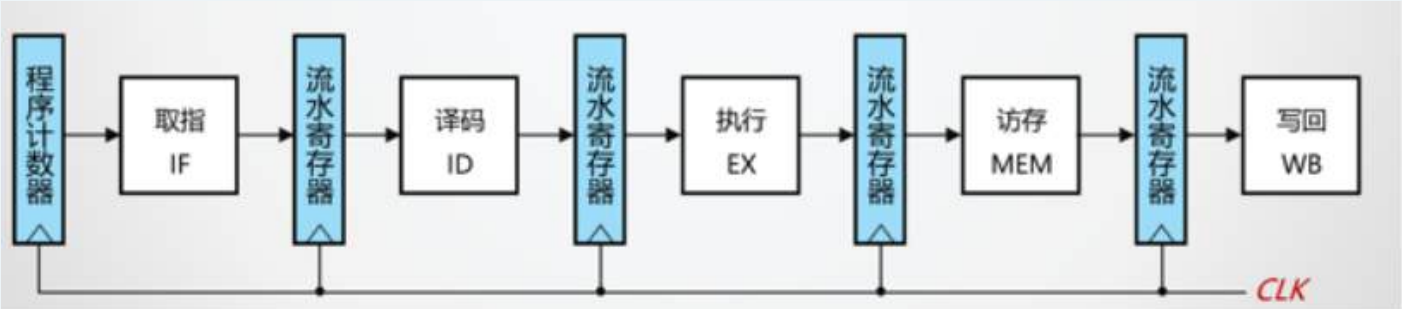
对于分支指令造成的控制相关，只需要在分支跳转时，将分支指令所在功能段左侧所有流水寄存器(如EX段位IF/ID和ID/EX)，同步清零即可

- 采用分支延迟槽技术，即分支指令后的一条或几条指令，无论分支跳转是否成功，都会进入执行阶段
- 基于软件或硬件的方法提前进行分支预测，预测当前指令的下一条指令的正确地址

插入气泡法解决冲突

i 程序中的数据相关

在5段流水线中，ID段从寄存器堆中取操作数时，才会发生数据相关



因此只需要处理ID段指令与EX、MEM、WB段的3条指令之间的数据相关性

而ID段与WB段的数据相关，可以采用对寄存器堆进行先写后读的方式解决(寄存器堆的写入控制采用下跳沿触发，寄存器堆的读出控制采用上跳沿触发)

只需要考虑ID段与EX、MEM段之间的数据相关性

下面的例子通过插入气泡(紫色框)解决了冲突

CLKs	取指令IF	译码ID	执行EX	访存MEM	写回WB
1	and \$s1,\$s1,\$s2				
2	sub \$s2,\$s1,\$zero	and \$s1,\$s1,\$s2			
3	add \$s3,\$s1,\$s1	sub \$s2,\$s1,\$zero	and \$s1,\$s1,\$s2		
4	add \$s3,\$s1,\$s1	sub \$s2,\$s1,\$zero		and \$s1,\$s1,\$s2	
5	add \$s3,\$s1,\$s1	sub \$s2,\$s1,\$zero			and \$s1,\$s1,\$s2
6	or \$s4,\$s5,\$s1	add \$s3,\$s1,\$s1	sub \$s2,\$s1,\$zero		

插入气泡需要增加硬件逻辑

- 检测ID段指令是否与EX段指令相关

$\text{DataHazard} = \text{RsUsed} \& (\text{rs} \neq 0) \& \text{EX.RegWrite} \& (\text{rsEX.WriteReg}) + \text{RtUsed} \& (\text{rt} \neq 0) \& \text{EX.RegWrite} \& (\text{rtEX.WriteReg})$

- 第一部分表示判断ID段的rs寄存器是不是0号寄存器（0号寄存器恒等于0，不存在数据相关），且与EX段的写入寄存器编号（EX.WriteReg#）相同，且EX段的寄存器写信号（EX.RegWrite）有效。
- 第二部分表示判断ID段的rt寄存器是不是0号寄存器（0号寄存器恒等于0，不存在数据相关），且与EX段的写入寄存器编号（EX.WriteReg#）相同，且EX段的寄存器写信号（EX.RegWrite）有效。
- 检测ID段指令是否与MEM段指令相关

$\text{DataHazard} = \text{RsUsed} \& (\text{rs} \neq 0) \& \text{MEM.RegWrite} \& (\text{rsMEM.WriteReg\#}) + \text{RtUsed} \& (\text{rt} \neq 0) \& \text{MEM.RegWrite} \& (\text{rtMEM.WriteReg\#})$

总的检测逻辑为

$$\text{DataHazzard} = \text{RsUsed} \& (\text{rs} \neq 0) \& \text{EX.RegWrite} \& (\text{rsEX.WriteReg}) + \text{RtUsed} \& (\text{rt} \neq 0) \& \text{EX.RegWrite} \& (\text{rtEX.WriteReg}) + \text{RsUsed} \& (\text{rs} \neq 0) \& \text{MEM.RegWrite} \& (\text{rsMEM.WriteReg}) + \text{RtUsed} \& (\text{rt} \neq 0) \& \text{MEM.RegWrite} \& (\text{rtMEM.WriteReg})$$

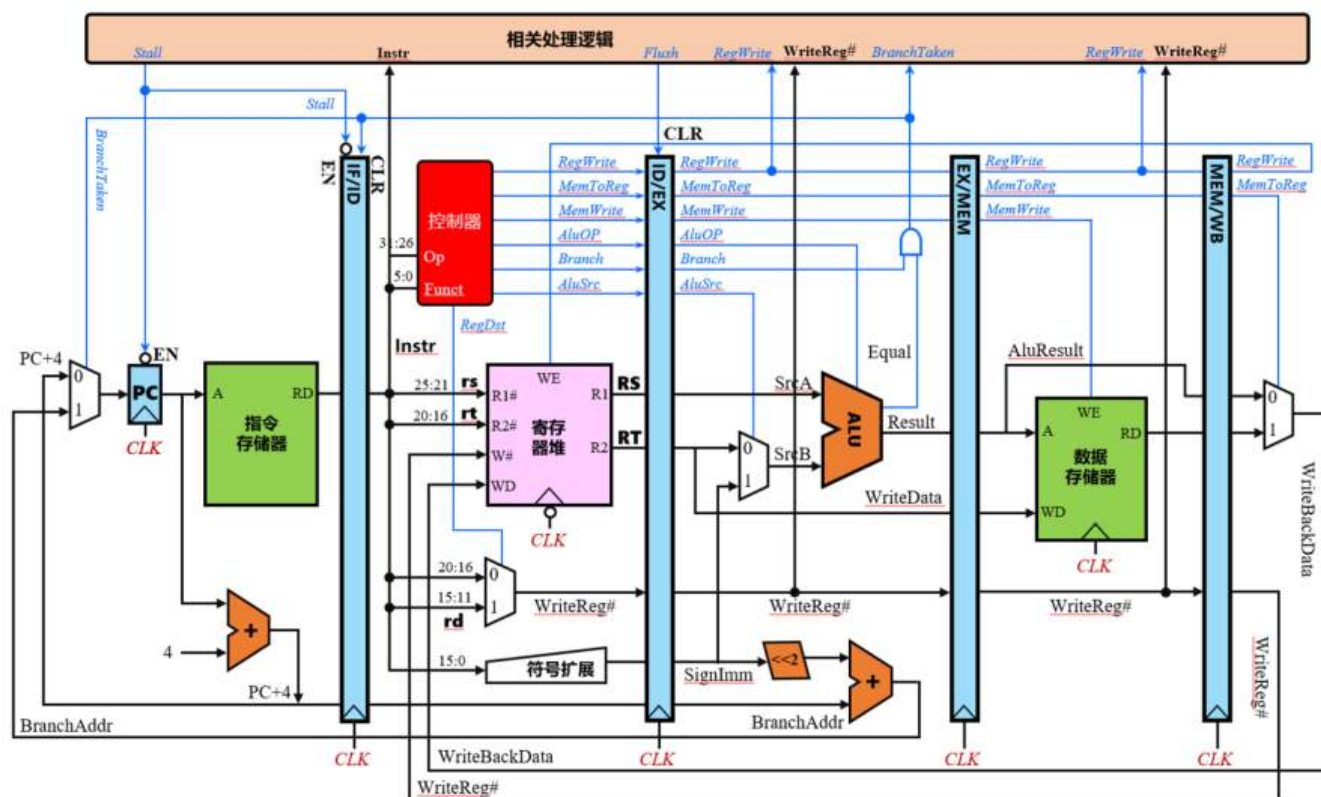


图7.19 气泡流水线顶层视图

结合控制冲突和数据冲突处理的流水线清空与阻塞信号逻辑公式为

- $\text{Stall} = \text{DataHazzard}$

数据相关时，要阻塞暂停IF、ID段指令的执行

- $\text{PC.EN} = \sim \text{Stall}$

阻塞IF段指令的执行（程序计数器PC使能端输入=0）

- $\text{IF/ID.EN} = \sim \text{Stall}$

阻塞ID段指令的执行（IF/ID流水寄存器使能端输入=0）

- $\text{IF/ID.CLR} = \text{BranchTaken}$

出现分支跳转时，清空IF/ID段流水寄存器

- $\text{ID/EX.CLR} = \text{Flush} = \text{BranchTaken} + \text{DataHazzard}$

出现分支跳转或数据相关时，清空ID/EX段流水寄存器

重定向解决数据冲突

❗ 插入气泡法中，大量气泡的插入会严重影响指令流水线的性能

重定向的思想是：先不考虑ID段所取的寄存器操作数是否正确，等到指令实际使用这些寄存器操作数时，再考虑其正确性问题

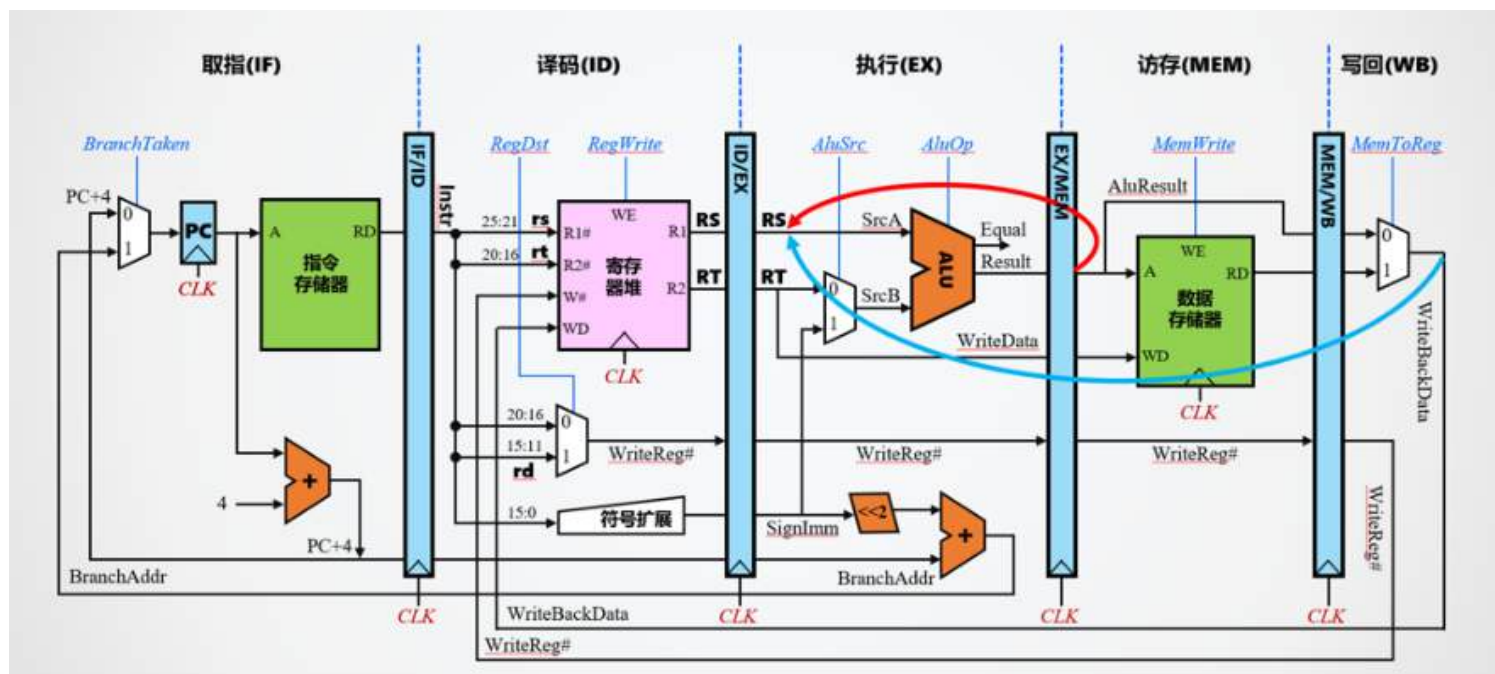


图7.22 数据重定向示意图

💡 假设EX段的指令与MEM、WB段的指令存在数据相关，EX段的寄存器操作数（RS、RT）就不能由ID/EX流水寄存器提供，而是由MEM段或WB段提供；

即可以通过重定向的方式，将MEM段的AluResult或WB段的WriteBackData，送到EX段

具体的实现上，可以考虑增加两个3路选择器

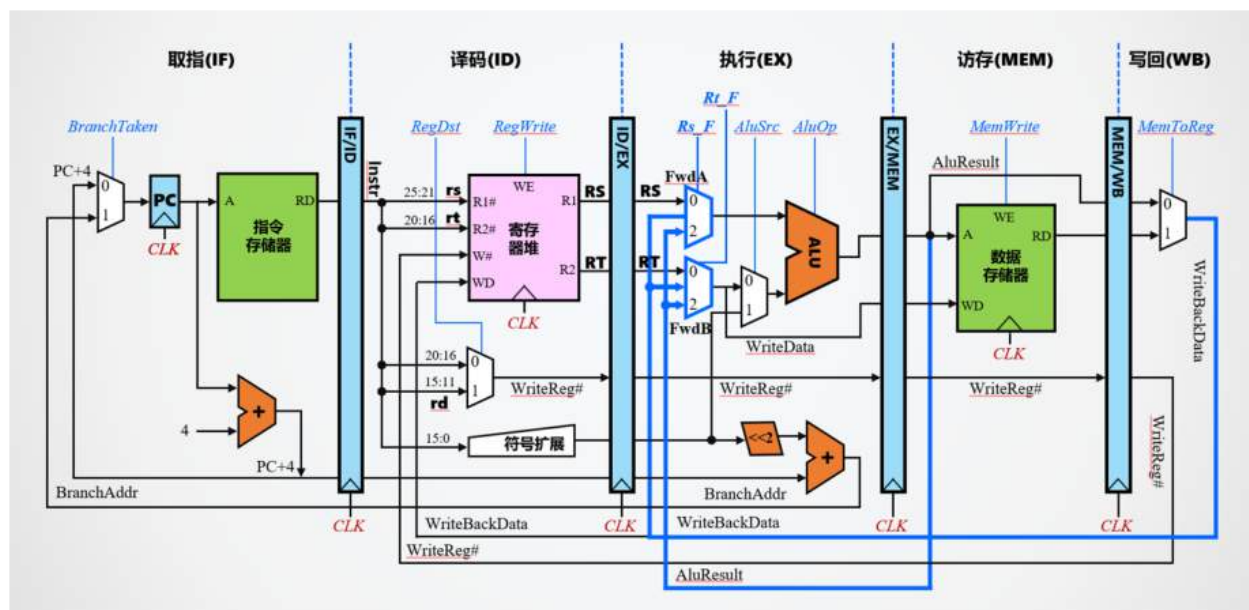


图7.23 数据重定向数据通路

不能用重定向解决的有

- EX段与MEM段的Load-Use相关

对于Load-Use相关的数据冲突不能采用重定向

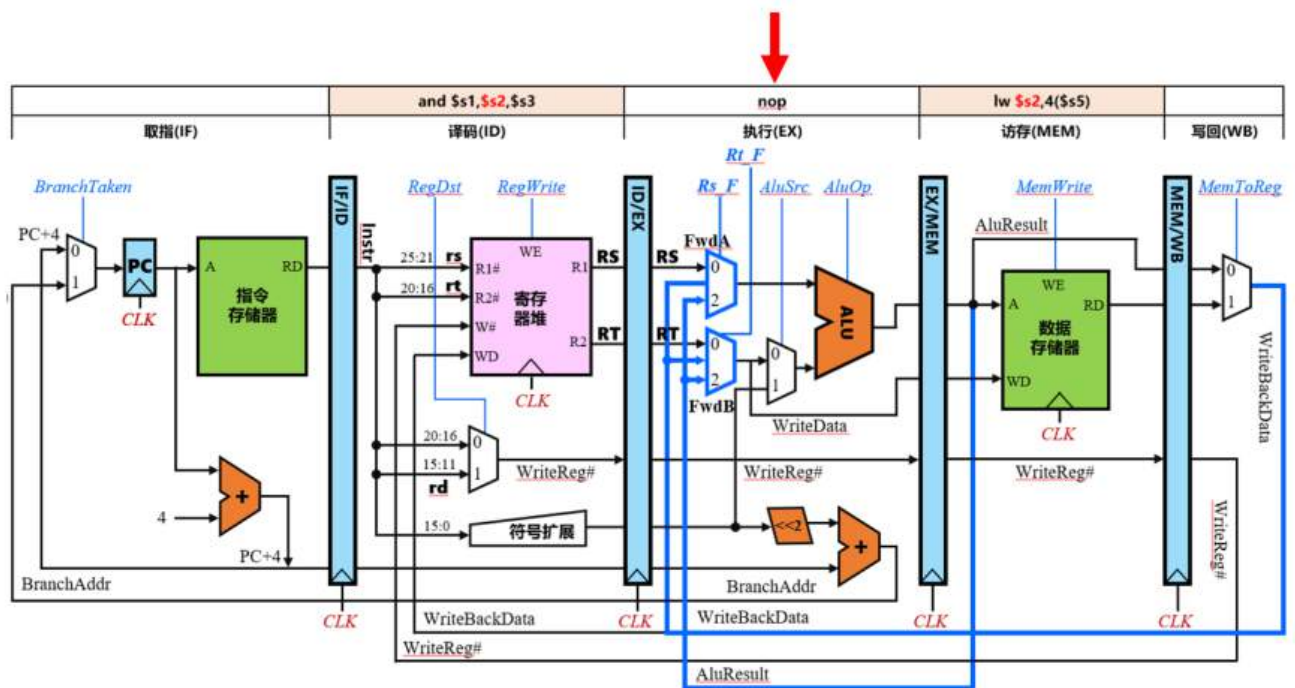
重定向只能在功能上解决相关问题，但重定向将导致EX段的关键路径延迟变为MEM段访存延迟加EX段运算器运算延迟，使得流水线频率降低

i Load-Use相关

第一条为取数指令，而第二条指令与第一条指令存在相关，例如

S

```
lw $s2, 4($5)
and $s1, $s2, $s3
```



- ID段分支指令与EX段的相关、MEM段的Load-Use相关

同理，也是功能上可行，但将ALU的运算结果和数据存储器的读出值重定向到ID段，会导致ID段关键路径延迟增加ALU的运算延迟、数据存储器的访存延迟

ARMASM

```
lw $s2,4($s5)
```

```
add $s3,$s4,$s5
```

```
beq $s2,$s3,next# 这里的beq在ID段时，和EX段的add，MEM段的lw有数据相关
```

此时唯一可以重定向的是MEM段的AluResult

在这些情形下，仍然要在相邻指令中加入一个气泡

数据相关检测和处理的相关公式有

- Load-Use检测

$$\text{LoadUse} = \text{RsUsed} \& (\text{rs} \neq 0) \& \text{EX.MemRead} \& (\text{rsEX.WriteReg\#}) + \text{RtUsed} \& (\text{rt} \neq 0) \& \text{EX.MemRead} \& (\text{rtEX.WriteReg\#}) ^$$

- 重定向选择信号RsForeware

$$\text{if}(\text{RsUsed} \& (\text{rs} \neq 0) \& \text{EX.RegWrite} \& (\text{rs} == \text{EX.WriteReg\#})) \text{RsForeware} = 2$$

```
elseif (RsUsed&(rs≠0)&MEM.RegWrite&(rs==MEM.WriteReg# )RsForware=1)
else RsForware=0
```

- 重定向选择信号RtForware

```
if(RtUsed&(rt≠0)&EX.RegWrite&(rt==EX.WriteReg# ))RtForware=2
else if(RtUsed&(rt≠0)&MEM.RegWrite&(rt==MEM.WriteReg# )RtForware=1)
else RtForware=0
```

当发生LoadUse相关时，需要暂停IF、ID段指令的执行，并在EX段插入气泡而EX段执行分支指令时，会清空ID段、EX段中的误取指令

因此各控制端口的逻辑如下

- Stall=LoadUse# 发生LoadUse相关时，要暂停IF、ID段指令的执行
- PC.EN=∼Stall# 暂停IF段指令的执行（程序计数器PC使能端输入=0）
- IF/ID.EN=∼Stall# 暂停ID段指令的执行（IF/ID流水寄存器使能端输入=0）
- IF/ID.CLR=BranchTaken# 出现分支跳转时，清空IF/ID段流水寄存器
- ID/EX.CLR=Flush=BranchTaken+LoadUse# 出现分支跳转或LoadUse相关时，清空ID/EX段流水寄存器

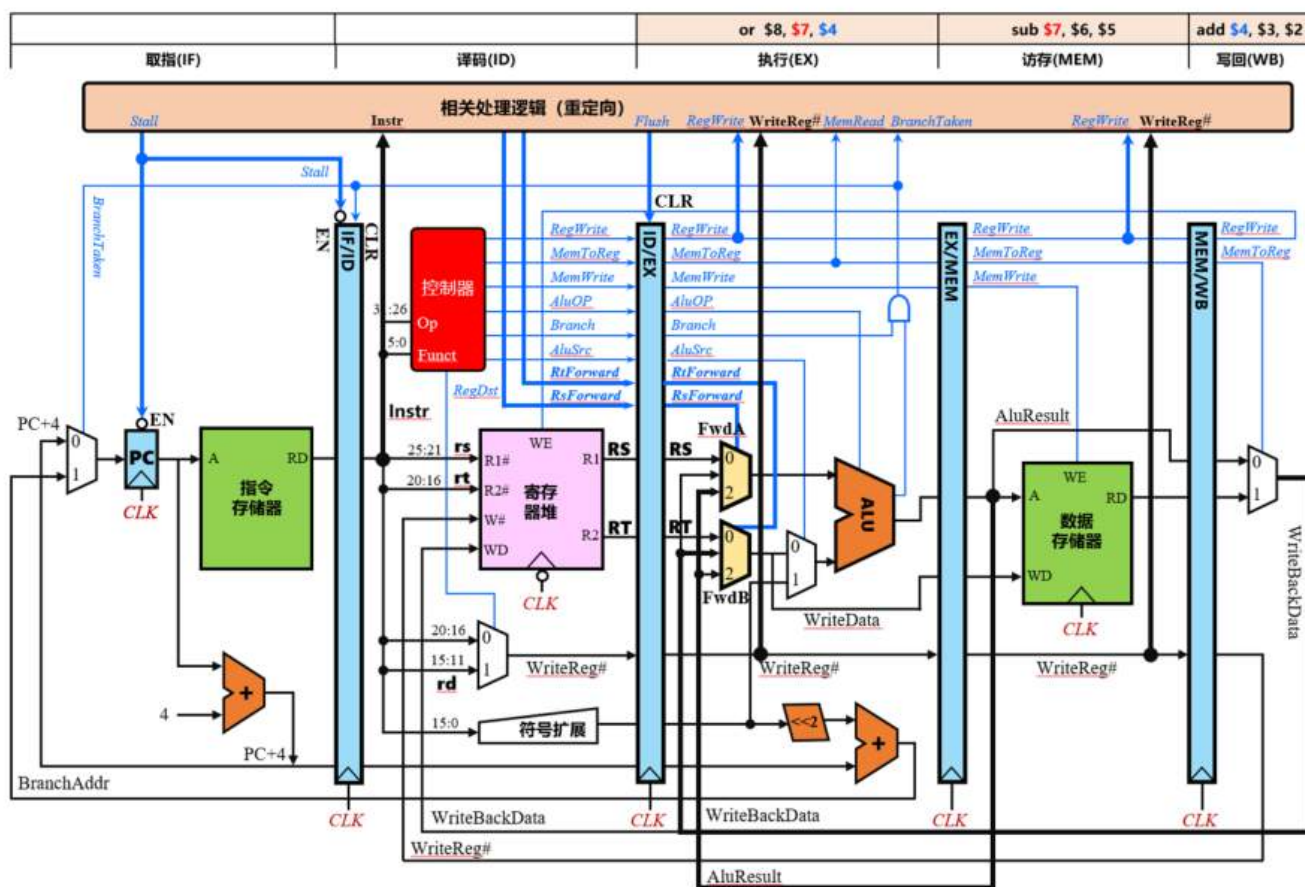


图7.27 重定向流水线顶层视图

分支预测

分支预测主要解决的是流水线中的控制冲突

静态分支预测

基于编译器的编译信息，对分支指令后续指令地址进行预测

静态分支预测给出的预测信息是静态的、不能改变的，与分支的实际执行情况无关，通常是采用一些简单的策略进行预测或处理

分支指令具有分支局部性，例如

- 循环指令执行循环时始终跳转，只在最后一次不执行回退的分支，这时候静态分支预测将预测它会回退
- 条件指令(回退)始终预测不跳转

动态分支预测

动态分支预测依据分支指令的分支跳转历史，不断地对预测策略进行动态调整，具有较高的预测准确率

✔ 动态分支预测正是利用了分支指令的分支局部性进行预测

动态分支预测实现依靠分支预测缓冲器(BTB)

i BTB

BTB用于存放分支指令的分支跳转历史统计信息

其本质为一个全相联cache，每一项包括valid位，分支指令地址，分支目标地址，历史跳转信息描述位，置换标志

表7.3 分支历史表BTB格式					
序号	valid	分支指令地址	分支目标地址	分支预测历史位	置换标记
0	1 (高电平有效)	XXXX	XXXX	11预测跳转	XXXX
1	1	XXXX	XXXX	10预测跳转	XXXX
2	1	XXXX	XXXX	01预测不跳转	XXXX
3	1	XXXX	XXXX	00预测不跳转	XXXX
4	0 (无效)	XXXX	XXXX	XX	XXXX
5	0	XXXX	XXXX	XX	XXXX
6	0	XXXX	XXXX	XX	XXXX
7	0	XXXX	XXXX	XX	XXXX

动态分支预测以分支指令地址为关键字，在BTB表(放在IF段中)中进行全相联并发比较，利用程序计数器PC值作为关键字。

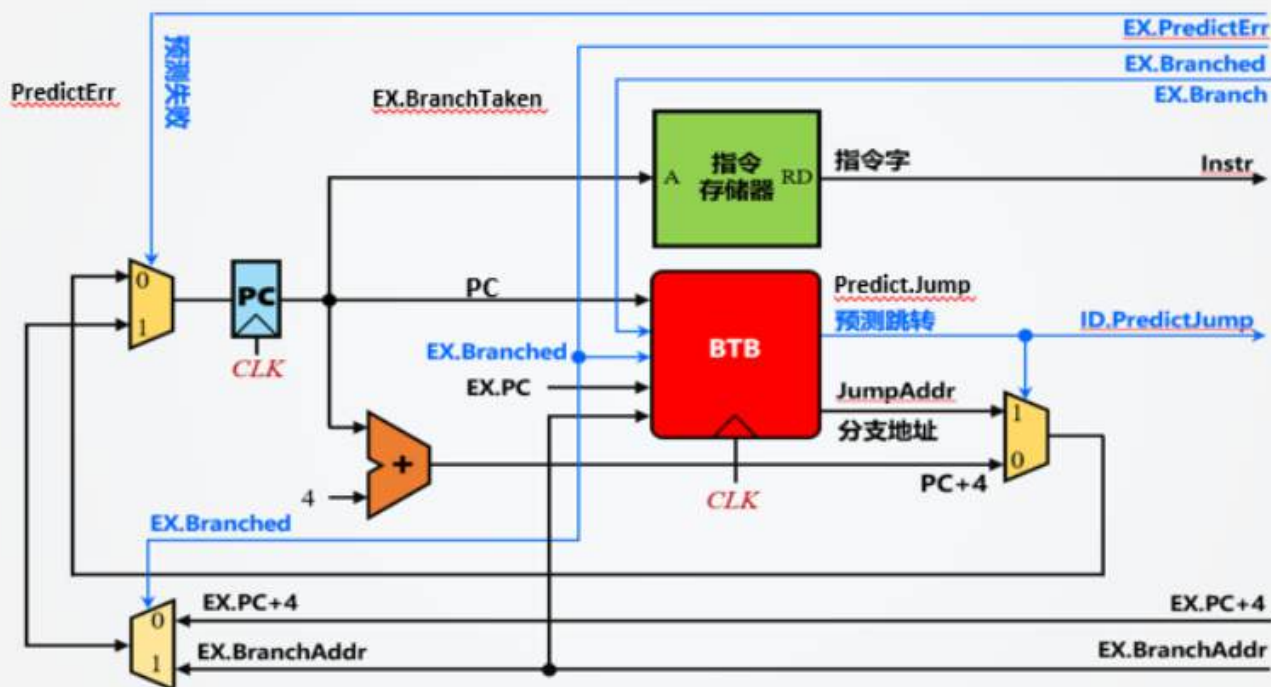


图7.30 动态分支预测BTB逻辑实现

此过程与取指令操作并发进行，不需要取出指令即可进行分支预测

- 数据缺失

表示当前分支指令不在BTB表中

需要将该分支指令的相关信息载入，并设置合适的分支预测历史位，清空误取指令，并重新修正程序计数器PC值，以取出正确的指令

- 数据命中

表明当前分支指令历史分支信息已存放在BTB表中

此时需要根据本次分支是否发生跳转的信息，调整对应表项中的分支预测历史位，并且处理与淘汰相关的置换标记信息，指令流水线不停顿

流水线性能分析

理想情况下，指令流水线的CPI=1，但由于流水阻塞或清空会损失一些周期，指令流水线的实际CPI>1

流水线的最小时钟周期取决于各功能段中最慢功能段的关键延迟

表7.6 流水线各功能段关键延迟

序号	功能段	标识	实际情况	预测情况
1	IF	T_{if_max}	$T_{clk_to_q} + T_{mem} + T_{setup}$	300ps=30+250+20
2	ID	T_{id_max}	$2(T_{clk_to_q} + T_{RF_read} + T_{setup})$	400ps=2*(30+150+20)
3	EX	T_{ex_max}	$T_{clk_to_q} + 2T_{mux} + T_{alu} + T_{setup}$	300ps=30+2*25+200+20
4	MEM	T_{mem_max}	$T_{clk_to_q} + T_{mem} + T_{setup}$	300ps=30+250+20
5	WB	T_{wb_max}	$T_{clk_to_q} + T_{mux} + T_{setup}$	75ps=30+25+20

可见，ID段是5个功能段的瓶颈，相关的性能指标还包括

• 流水线吞吐率

单位时间内流水线所完成的任务数量，或输出结果的数量，最基本的计算公式为

$$TP = \frac{n}{T_k}$$

n 为任务数， T_k 为处理完 n 个任务所用的总时间

进一步地，设 k 为流水段的段数， Δt 为时钟周期，吞吐率计算为

$$TP = \frac{n}{(k + n - 1)\Delta t}$$

当 $n \rightarrow \infty$ 时，最大吞吐率为 $1/\Delta t$

• 流水线加速比

完成同样一批任务，不使用流水线与使用流水线所用的时间之比，基本公式为

$$S = \frac{T_0}{T_k}$$

其中， T_0 为不使用流水线的总时间， T_k 为使用流水线的总时间，可进一步得到

$$S = \frac{kn}{(k+n-1)}$$

当 $n \rightarrow \infty$ 时，最大加速比为 k

□

总线、异常与中断

异常与中断处理

中断技术可以实现人机交互、故障处理、实时控制，并且提升计算机的并行性

异常与中断的基本概念

- **异常**：是CPU内部引起的异常事件，也称为**内部中断**或**软件中断**
 - **故障**：由指令执行引起的异常

如未定义指令、越权指令、段故障、缺页故障、存储保护违例、数据未对齐、除数为零、浮点溢出、整数溢出等

故障又可分为：可恢复故障（如数据缺页）和不可恢复故障（如未定义指令、越权指令等）
 - **自陷**：事先安排的“异常”事件；如系统调用指令、条件陷阱指令

例如x86的INT21H指令，MIPS的syscall指令
 - **终止**：随机出现的使得CPU无法继续执行的硬件故障，和具体指令无关

如机器校验错、总线错误、异常处理中再次异常的双错等
- **中断**：由外部设备向CPU发出的中断请求，要求CPU暂停当前正在执行的程序，转去执行为某个外部设备事件服务的中断服务程序
 - **可屏蔽中断**：通过可屏蔽中断请求线 INTR 向 CPU 发出的中断请求，但可以被屏蔽。屏蔽后 CPU 将不接受该中断请求
 - **不可屏蔽中断**：通过不可屏蔽中断请求线 NMI 向 CPU 发出的中断请求，无法被屏蔽，如电源掉电等

- 单级中断

执行中断服务程序时，不再响应其他中断请求

单级中断属于资源独占，有利于单个中断事务的完成，但不利于CPU对于紧急事务的处理

- 多重中断

也称为**嵌套中断**、**中断嵌套**。中断服务程序可以被更高优先级的中断请求中断

多重中断时，共享处理器资源，获得资源的 interrupt 事务，可以被其他 interrupt 事务打断

多重中断有利于CPU对于更紧急事务的处理，但当前处理的 interrupt 事务可能被多次中断而无法顺利完成

处理完毕后，再返回断点继续执行。它也被称为**外中断**

✎ MIPS处理器将异常和中断都称为“异常”

x86处理器将异常和中断都称为“中断”

- 来自CPU内部的中断称为“内部中断（软件中断）”
- 来自CPU外部的中断称为“外部中断（硬件中断）”

中断处理技术

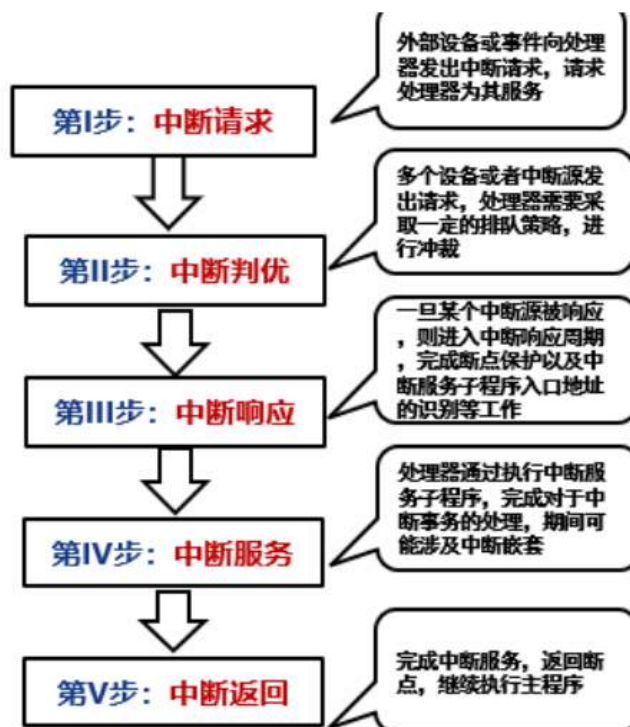
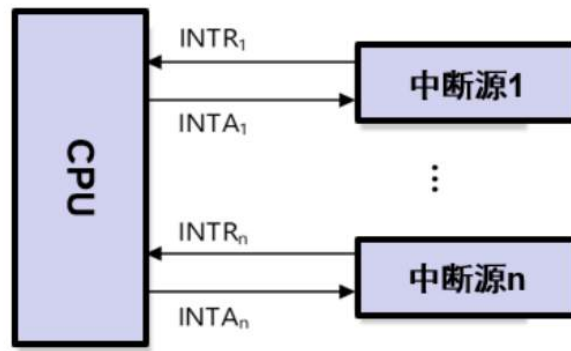


图 6.4 中断处理流程

中断请求与中断判优

- 中断请求方式
 - 独立请求方式

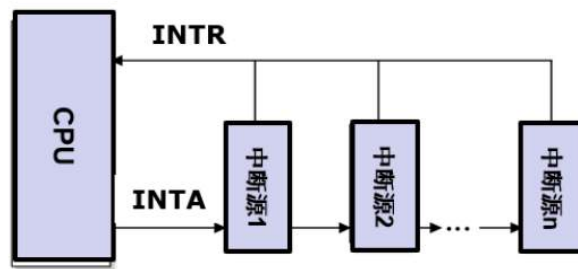


(a)独立请求方式

每个中断源有单独的中断请求线和中断应答线

速度快，成本高，线路多

- 链式请求方式



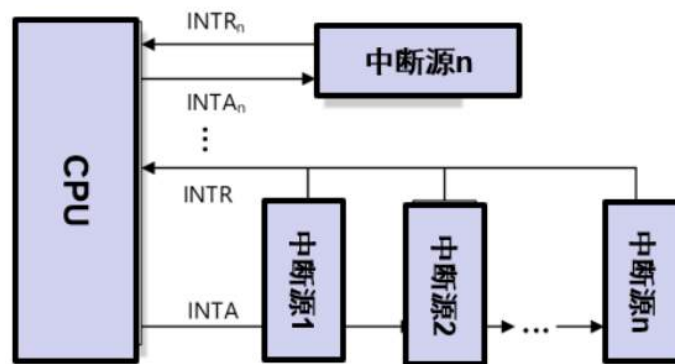
(b)链式请求方式

多个中断源共用一根公共的中断请求线INTR和中断应答线INTA

又分为硬件查询法和软件查询法两种方式。

线路少，成本低，但速度慢且中断优先级固定，不灵活，单点故障会导致整体瘫痪

- 分组链式请求方式



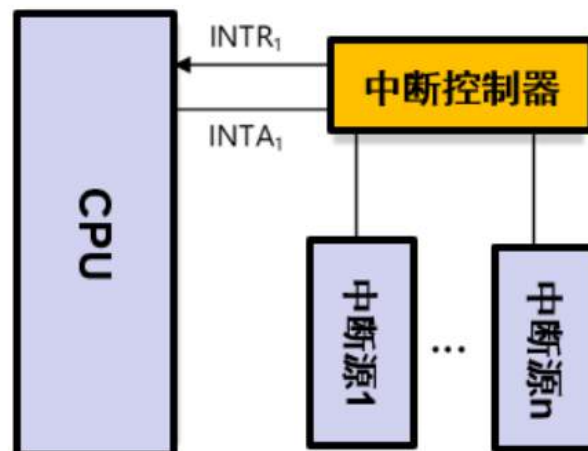
(d)分组链式请求方式

是独立请求方式与链式请求方式的折中

对于要求快速响应的中断请求采用独立请求方式，对于不要求快速响应的中断请求则采用链式请求方式。

其控制较为复杂，但兼具独立请求和链式请求的优势

- 中断控制器方式



(c)中断控制器

所有中断源通过独立请求方式与中断控制器连接，中断控制器与CPU连接

中断控制器分担了CPU的中断控制功能，负责实现外部中断的优先级仲裁逻辑与中断识别

这是目前普遍采用的形式，无需CPU直接干预，有独立的接口芯片

- 中断优先级

中断优先级是CPU响应并处理中断的先后次序

- **响应优先级**

指CPU对各设备中断请求进行响应的先后次序，其在硬件电路上一般固定，不便于变动
中断响应优先级的一般划分规律为

- 不可屏蔽中断>内部异常>可屏蔽中断
- 内部异常中：硬件终止（Abort）>指令异常或自陷等程序故障（Fault、Trap）
- DMA中断>I/O中断
- I/O中断中：高速设备>低速设备，输入设备>输出设备，实时控制>普通设备

- **处理优先级**

指中断嵌套的实际优先级处理次序

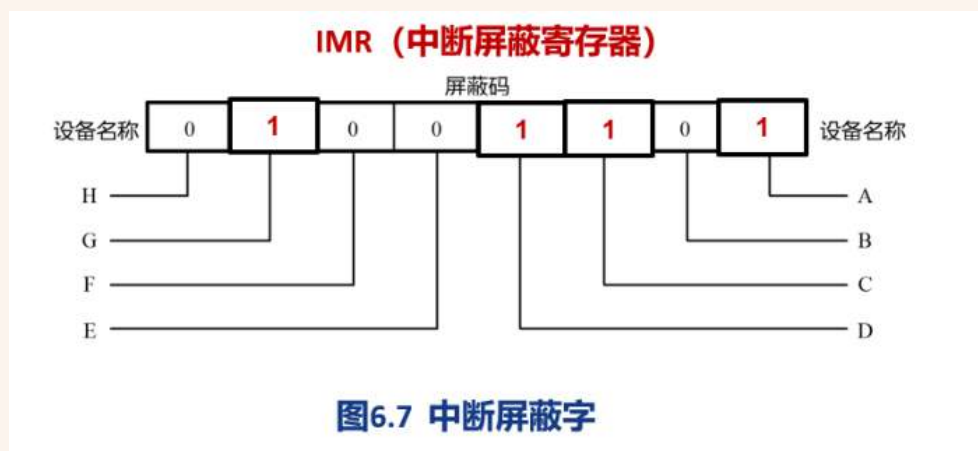
通常可以利用中断屏蔽技术动态调整，从而使低优先级的中断也可以中断高优先级的中断服务程序

💡 在不使用中断屏蔽技术时，处理优先级与响应优先级相同

- **中断屏蔽技术**

中断屏蔽技术是在中断控制器中设置中断屏蔽寄存器IMR，其值称为中断屏蔽字

💡 以下图为例，假设有8个设备，其中有4个(ACDG)被屏蔽



在下图中，假设4个设备的中断优先级为IR0最高、IR3最低

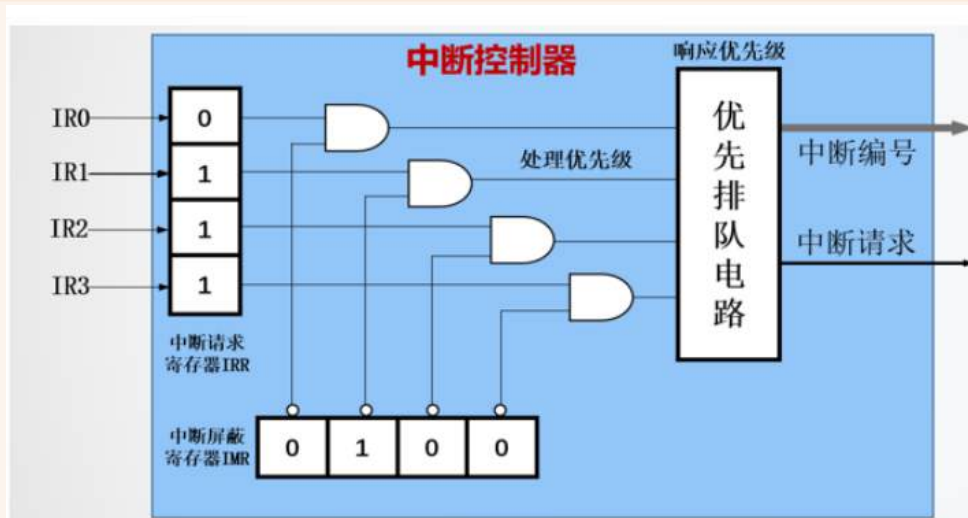


图6.8 中断屏蔽技术原理

- 如果没有中断屏蔽技术，3个设备（IR1、IR2、IR3）同时发出中断请求，将先处理IR1的中断请求
- 如果采用中断屏蔽技术，中断屏蔽寄存器IMR设置为0100，即屏蔽IR1
3个设备同时发出中断请求，将先处理IR2的中断请求（IR2>IR3）

中断响应

中断响应需要满足一定的条件

- 对应的中断请求未被屏蔽：IMR[i]=0。
- 当前没有更高优先级的其他中断请求：i>ISR。
- 如果CPU正在执行中断服务，则中断请求应符合嵌套条件。
- 中断使能位处于使能状态，也就是开中断状态

满足这些条件时，在CPU执行完一条指令的最后一个状态周期，CPU进入中断响应周期

中断响应包括下面的过程

1. **关中断**：关中断的目的是临时禁止新的中断请求，在中断响应周期以及中断服务程序中保护现场操作的完整性

只有这样才能保证中断服务程序执行完成后，能返回断点正确执行

2. **保存断点**：保存断点是保存将来返回被中断程序的位置

- 对于已经执行完毕的指令，断点是下一条指令的位置

- 对于缺页故障、段错等执行指令引起的故障异常，由于指令并没有执行，断点就是异常指令的PC值

3. 中断识别：中断识别的主要任务就是根据当前的中断请求识别中断来源，将中断服务程序入口地址送入程序计数器PC

由于中断响应过程中CPU不能执行其他任务，中断响应过程是由CPU执行中断隐指令完成的

中断识别

中断识别的任务是确定中断是由哪个中断源发出的，并获取中断服务程序入口地址

- **非向量中断**：属于共享中断请求信号的软件查询法

CPU响应中断后，跳转到固定地址去执行中断查询程序

由中断查询程序确定是哪一个设备申请中断，并获取中断服务程序入口地址

- **向量中断**：CPU根据中断号快速查找中断服务程序入口地址的方法

向量中断中每一个设备的中断源都有一个唯一的中断编号，称为中断号

中断号是由计算机系统统一分配，通常是固定不变的

💡 如x86计算机共有256个中断号

表6.2 8086中断号分配

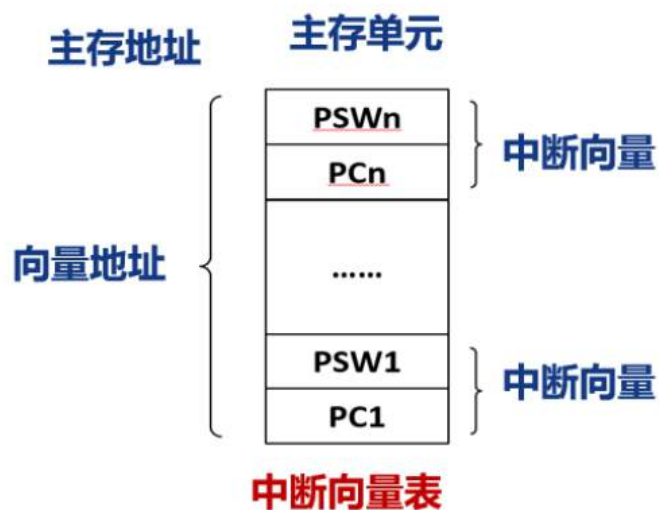
中断号	中断功能	中断号	中断功能	中断号	中断功能
00	除零异常	08	定时器	10 ~ 1F	BIOS中断调用
01	单步中断	09	键盘	10	视频显示I/O调用
02	不可屏蔽中断	0A	保留	11	设备配置检查调用
03	断点中断	0B	串口2	12	存储器容量检查
04	溢出异常	0C	串口1	13	磁盘I/O调用
05	打印屏幕	0D	硬盘	1B	Ctrl+Break控制
06 ~ 07	保留	0E	软盘	28 ~ 3F	DOS保留
08 ~ 0F	可屏蔽外部中断	0F	打印机	60 ~ 67	用户软中断保留

根据中断号，在中断向量表中，可以查找得到**中断服务程序入口地址**

中断向量表通常存放在内存中，中断向量表中存放的内容称为中断向量，即中断服务程序入口地址

🔧 有些计算机将中断服务程序入口地址和程序状态字合在一起称为**中断向量**

中断向量在中断向量表中的地址称为**向量地址**



💡 其中PC_i和PSW_i为第i个中断源的中断服务程序入口地址和程序状态字

在8086的中断向量表中，每个中断向量占4个字节，中断号为n的向量地址就为n×4

000H		除零异常（0号中断） 服务程序入口地址		IP
				CS
004H		单步中断（1号中断） 服务程序入口地址		IP
				CS
008H		NMI异常（2号中断） 服务程序入口地址		IP
				CS
3FFH				IP
				CS

图6.11 8086中断向量表

中断处理(软件实现)

- 单级中断处理流程

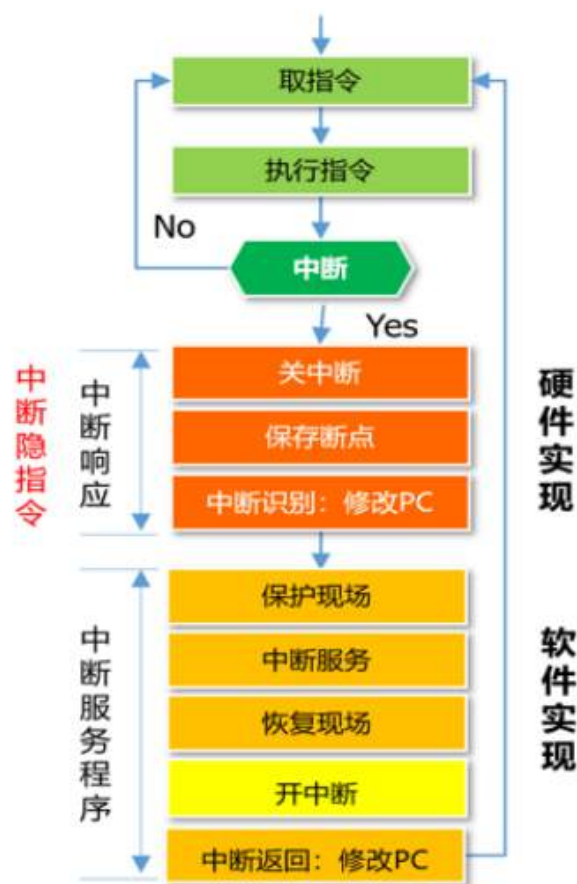


图6.12a 单级中断处理流程

1. 中断响应

由硬件自动完成，即所谓的中断隐指令；包括关中断、保存断点、中断识别

2. 保护现场

3. 保存中断服务程序中会被修改的寄存器，通常用压栈指令（push）实现

4. 中断服务

进行实际的中断事务处理，如数据传输、唤醒等待进程等操作

5. 恢复现场

将保护现场中保存的寄存器释放出来，通常用出栈指令（pop）实现出栈指令的顺序必须与压栈指令的顺序相反（先进后出）

6. 开中断

通过执行开中断指令（如x86的sti指令），将中断允许触发器IE/IF设置为“1”，使CPU可以接收新的中断请求

7. 中断返回

中断服务程序的最后一条指令是中断返回指令（x86的iret，MIPS的eret），将程序返回到主程序的断点处继续执行

- 多级中断处理流程

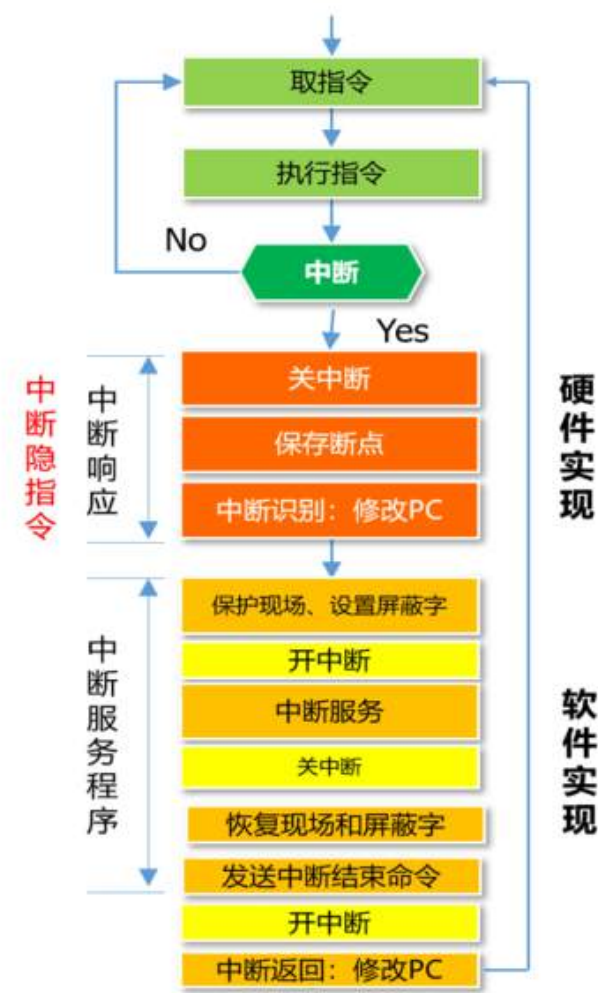


图6.12b 多重中断处理流程

1. 中断响应

与单级中断相同。

2. 保护现场、设置屏蔽字

这里除了要保护现场（寄存器）外，还要保护中断屏蔽字

还需要设置新的屏蔽字，从而改变中断处理的优先级，实现多重中断（中断嵌套）

3. 开中断

目的是使优先级更高的中断服务程序，可以中断当前的中断服务程序。

4. **中断服务**，与单级中断相同。

5. **关中断**

利用关中断指令（如x86的cli指令）关中断，保证恢复现场任务的完整性。

6. **恢复现场和屏蔽字**

这里除了恢复现场（寄存器）外，还要恢复中断屏蔽字。

7. **发送中断结束命令**

CPU向中断控制器发送中断结束命令EOI，通知中断控制器当前中断处理完毕

再请求中断控制器清除中断服务寄存器ISR中最高优先级的中断位。

8. **开中断**。与单级中断相同。

9. **中断返回**。与单级中断相同

X86ASM

```
;多重中断对应的服务程序汇编代码
PUSH AX;保护现场
.....
PUSH DX;保护现场
IN AL,8259_IMR_PORT;访问8259（中断控制器）的中断屏蔽寄存器IMR
PUSH AX;保护中断屏蔽寄存器的内容
MOV AL,Curr_Dev_IMR;设置新的中断屏蔽字
OUT 8259_IMR_PORT,AL;输出新的中断屏蔽字到8259的IMR
STI;开中断

.....;中断服务

CLI;关中断
POP AX;恢复中断屏蔽字
OUT 8259_IMR_PORT,AL;恢复中断屏蔽字
POP DX;恢复现场

.....
POP AX;恢复现场
MOV AL,20H;发送EOI命令
OUT 8259_CMD_PORT,AL;发送EOI命令到8259的命令口
```


STI;开中断

IRET;中断返回



中断服务程序与子程序的差异

- 调用方式不一样
 - 子程序（子函数）在程序中显式调用（x86为call指令，MIPS为jal指令）
 - 中断服务程序大多是随机调用的
- 保存并修改PC和子程序入口地址的方式不同
 - 中断服务程序保存并修改PC以及入口地址是由中断隐指令完成的
 - 子函数保存并修改PC是由调用指令（call指令，jal指令）实现的。
- 现场的内容不同
 - 因为中断服务程序没有调用者，所以需要保存ABI中约定的由调用者函数保存的寄存器
因此中断服务程序的现场包含的寄存器更多
 - 而子函数只需要保存可能被改写的被调用者保存的寄存器。
- 返回主程序的方式不同
 - x86中，子函数返回指令为ret，而中断返回指令为iret
 - MIPS中子函数返回指令为jr\$ra，而中断返回指令为eret。

总线概述

总线是一组能为多个部件分时和共享的公共信息传送线路。总线的特点包括

- **分时**：同一时刻只允许有一个部件向总线发送信息，若系统中有多多个部件，则它们只能分时地向总线发送信息
- **共享**：总线上可以挂接多个部件，各个部件之间互相交换的信息都可通过这组线路分时共享，多个部件可同时从总线上接收相同的信息。

总线的主要功能是在连接的功能部件之间进行数据传输

现代总线是指连接多个计算机内部功能部件或多个计算机的通信系统

总线既包括相关的硬件（总线控制器、总线接口）、软件，也包括相关的通信协议

总线分类

- 按照**数据传送方向**分为：单向传输总线、双向传输总线
- 按照**时序控制方式**分为：同步总线、异步总线
- 按照**信号线的功能**分为：数据总线、地址总线、控制总线
- 按照**信号传输的模式**分为：并行传输总线、串行传输总线。现代计算机更多的采用高速的串行传输总线
- 按照**总线在计算机系统中所处的位置**分为：片内总线、系统总线、I/O总线、外部总线

片内总线

片内总线是指芯片内部各组成部件之间的连接线，也称为**片上总线**

💡 如CPU芯片内部寄存器之间、寄存器与运算器之间的数据通路连接，控制器与执行部件之间的控制信号连接

系统总线

系统总线是CPU直接连接主存、I/O模块等主要功能部件的信息传输线，与CPU引脚直接相连，也称为主机总线、CPU总线、前端总线

现代计算机CPU主频为外频(主板的工作频率) $\times$ 倍频，系统总线频率为外频 $\times 2$ 或外频 $\times 4$

系统总线还可以被分为

- **数据总线**：在各部件之间传输数据、指令和中断类型号等，它是**双向传输总线**，数据总线的位数反映一次能传送的数据的位数
- **地址总线**：用来指出数据总线上源数据或目的数据所在的主存单元或I/O端口的地址，它是**单向传输总线**，地址总线的位数反映最大的寻址空间
- **控制总线**：用来传输各种命令、反馈和定时信号

I/O总线

I/O总线主要用于连接计算机内部的中低速I/O设备，通过桥接器与高速总线相连接

I/O总线将低速设备与高速总线相分离，提升总线的系统性能

外部总线

外部总线主要用于连接计算机与外部设备，使其在计算机系统或与其他系统之间进行数据通信，也称为**通信总线**

相对的，系统总线和I/O总线被称为内部总线

总线组成

总线系统包括一组连接各功能部件的传输线缆和总线控制器

总线控制器负责总线控制权的仲裁，以及总线资源的分配和管理

 如果是扩展总线控制器，还需要进行总线协议的转换

总线复用技术

复用总线是指一组传输线具有多种用途，用于分时传送不同类型的信息，可以减少引脚数目

总线设备分类

- 可以拥有总线控制权并主动进行总线传输控制的设备称为**主设备**
- 被主设备寻址访问的设备称为**从设备**

表8.1 主、从设备举例

主设备	从设备	功能说明
CPU	内存	取指令、取数据、写数据
CPU	I/O设备	初始化数据传输
DMA控制器	内存	第三方DMA数据传输
I/O设备	内存	总线主控I/O设备的第一方DMA数据传输
CPU	协处理器	CPU提交指令给协处理器
协处理器	CPU	协处理器从CPU取操作数

 总线主控技术

高速I/O设备内置了DMA控制器，无须第三方DMA控制器就可以直接获得总线控制权，不需要CPU直接与主存进行数据交互

采用总线控制技术的I/O设备可以作为主设备，MCA、EISA、PCI等总线均支持总线主控技术

总线标准

总线标准是关于总线及总线接口的物理特性、电气特性、功能特性与时间特性的详细规范和协议

总线标准包括：

- **机械规范**：规定总线的物理连接方式，包括总线的线数，总线的插头，插座的形状、尺寸，引脚线排列方式的规范等。
- **电气规范**：定义总线信号的传递方向及有效电平。
- **功能规范**：约定总线中每一根线的功能。
- **时序规范**：明确每根线的信息在什么时间有效，也就是明确总线上各信号有效的时序关系。

总线与三态门

三态门也称三态缓冲器，三台指的是低电平0，高电平1，高阻态

三态门可以用于总线的输出控制

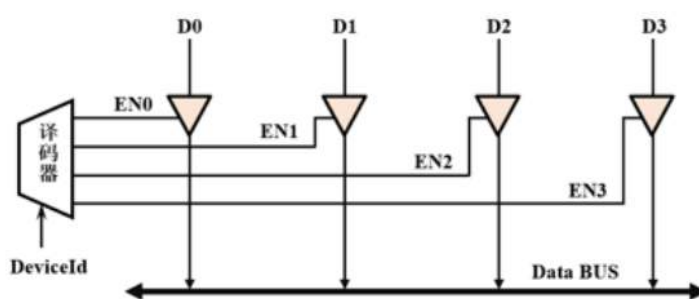


图8.4 基于三态门的输出缓冲

三态门和译码器结合，可以实现同一时刻只有一个部件向总线输出

三态门也可以用于构成有向总线

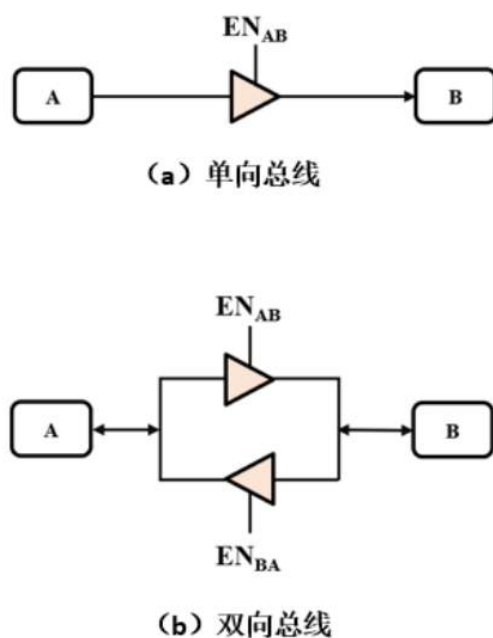


图8.5 基于三态门的单/双向总线

总线性能指标

- **总线宽度**

数据总线的根数，如8位、16位、32位等。

- **总线时钟频率**

时钟频率越快，传输速率越高

早期计算机的总线时钟频率与CPU的时钟频率相同，现代计算机CPU的频率要高于总线的时钟频率。

- **总线传输周期**

一次总线操作完成所需要的时间，包括总线申请阶段、寻址阶段、传输阶段、结束阶段等4个阶段的时间，简称总线周期。

- **单时钟传输次数**

指一个总线时钟周期内传输数据的次数，通常为1

- 采用DDR技术，在时钟的上、下跳沿分别传输一次数据，该值为2
- 采用QDR(QuadDataRate)技术，该值为4

- 采用AGPx8总线，该值为8

因此，总线实际工作频率=总线时钟频率x单时钟传输次数。

- 总线带宽

指在总线上的最大数据传输速率，单位为MB/s或GB/s

总线带宽计算公式为：同步总线带宽=总线宽度x总线时钟频率x单时钟传输次数。

表8.2 常见总线带宽

总线标准	总线宽度	总线时钟频率	单时钟传输次数	总线带宽
ISA-8	8	8.3 MHz	1	8 MB/s
ISA-16	16	8.3 MHz	1	16 MB/s
EISA	32	8.3 MHz	1	33 MB/s
MCA	32	10 MHz	1	40 MB/s
VESA	32	33 MHz	1	133 MB/s
PCI-32	32	33/66 MHz	1	133/266 MB/s
PCI-64	64	66 MHz	1	533 MB/s
AGP	32	66 MHz	1	266 MB/s
AGP*8	32	66 MHz	8	2133 MB/s
ATA133	16	66 MHz	1	133 MB/s
SCSI-3	16	80 MHz	4	640 MB/s
FSB	64	400 MHz	4	12.8 GB/s

- 总线负载能力

总线上能同时连接的设备数；如PCI总线插槽通常只能外接3个扩展设备。

总线传输机制

总线传输过程

1. 请求阶段

主设备发出总线请求信号，总线控制器将总线使用权分配给主设备

请求阶段可以细分为传输请求和总线仲裁两个阶段

2. 寻址阶段

主设备发出从设备的地址（存储器地址或I/O端口地址），以及有关控制命令，启动相应的从设备。

3. 传输阶段

也称数据阶段，主设备和从设备之间进行数据传输。

4. 结束阶段

传输结束后，主设备撤销总线请求，释放总线控制权

总线上一对主从设备之间的一次信息交换称为一个总线事务

典型的总线事务类型有：存储器读、存储器写、I/O读、I/O写、中断响应、DMA响应等

• 普通模式总线传输

由一个寻址阶段和一个数据阶段组成；一次只能传输一个计算机字长的数据信息，总线传输效率低

• 突发模式总线传输

由一个寻址阶段和多个数据阶段组成，一次可以传输多个计算机字长的数据信息，总线传输效率高

在寻址阶段首先得到首地址

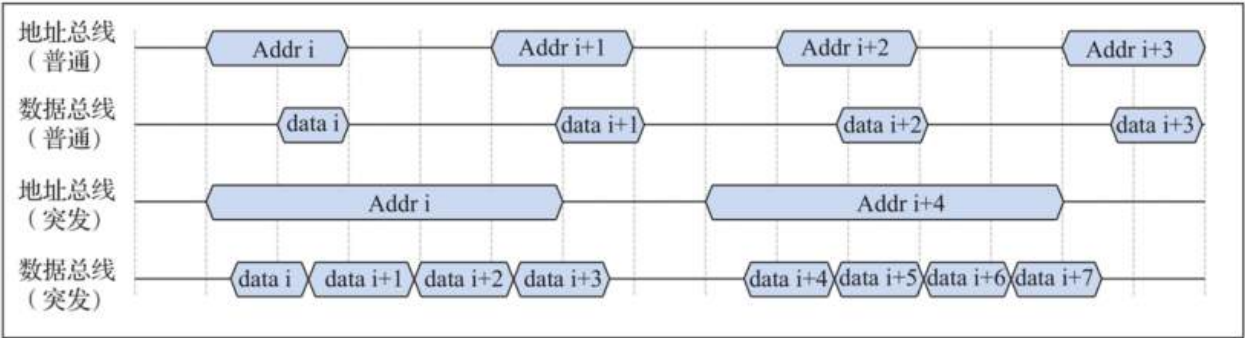


图8.6 普通模式和突发模式总线传输对比

总线的信息传送

信息的传送方式

• 并行传送

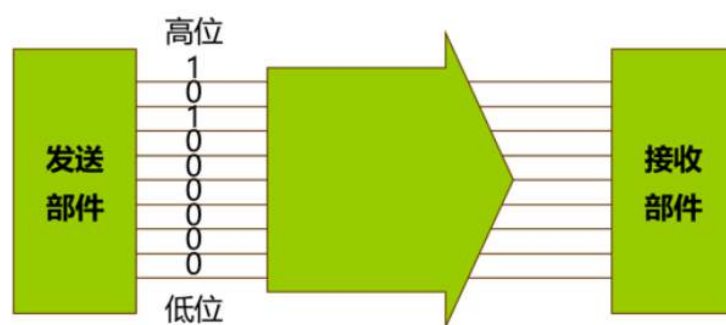


图8.7a 信息的传送方式（并行传送）

并行传送速度快，但线数多、需要导线等长、成本高，传输距离较长时会产生时钟偏移问题。计算机中的系统总线普遍采用并行传输方式；但是，当传输频率过高时，还会引起线间串扰问题，存在高频障碍。

因此，现代总线逐渐向高频串行总线发展，如PCIe、SATA等高速串行总线。

- 串行传送



图8.7b 信息的传送方式（串行传送）

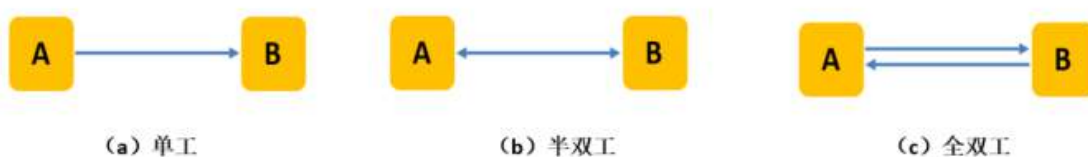


图8.8 3种数据传输方式

优点为传送成本低且传输距离远（最远可达几千米，但同等频率下比并行方式的传输速率低）。

根据传送方向的不同，串行传送方式可以进一步分为：单工、半双工、全双工等3种方式。

根据定时方式的不同，串行传送可分为同步串行通信和异步串行通信等2种方式。

- 同步串行通信

传输双方采用统一时钟，将多个字节数据组成一个信息帧进行传输。

每帧的开始为同步字符，结尾为结束字符，中间为若干个字节的数据（几十~几千个字节），以及CRC校验字节

- **异步串行通信**

传输双方各自都有独立的时钟，信息帧长度固定，按字符传输，一个信息帧传输一个字符
一个信息帧包括1个起始位（低电平）、5~7位有效数据、奇偶校验位（可以没有）、1~2个停止位（高电平）

常见的RS-232C和RS-485串行总线均采用异步串行通信方式

串行通信用波特率来描述传输速率，即每秒传送的二进制位数

数传率也称比特率，是指每秒传送的有效数据位；数传率小于波特率

串行通路也可以多组并发，此时串行总线带宽=总线时钟频率 $\times$ 编码效率 $\times$ 并发通路数

? 直流平衡问题

高速串行总线在传送多个连续的“1”或“0”时，可能会因为信号转换中的电压位阶关系，造成接收数据错误

为此，通常会将传输数据进行特殊编码后，再进行传输

例如在5个连续的“1”或“0”后，插入一位“0”或“1”，称为8bit/10bit编码方式，其编码效率为80%

- **并串行传送**

将信息分成若干组，组内采用并行传送，组间采用串行传送

💡 例如，Intel8088CPU，CPU内部总线为16位，系统总线（CPU外部总线）只有8位

CPU与主存或外设的通信就是采用并串行传送，即将一个16位字分成两个连续的8位字节进行串行传送

- **分时传送**

- **总线复用技术必须分时传送。**例如地址线与数据线复用，先传送地址，后传送数据。
- 共享总线的部件必须分时使用总线。挂在总线上的部件有多个时，同一时刻总线使用权只能由一个主设备控制。

总线仲裁

当总线上存在多个主设备时，需要有**总线控制器**来决定哪一个主设备获得总线使用权

获得总线控制权的主设备称为**活动主设备**

集中式仲裁

将总线裁决的逻辑集中在一起

- 链式查询

也称为菊花链查询方式

设置总线请求信号线BR、总线许可信号线BG、总线忙信号线BS三根控制线

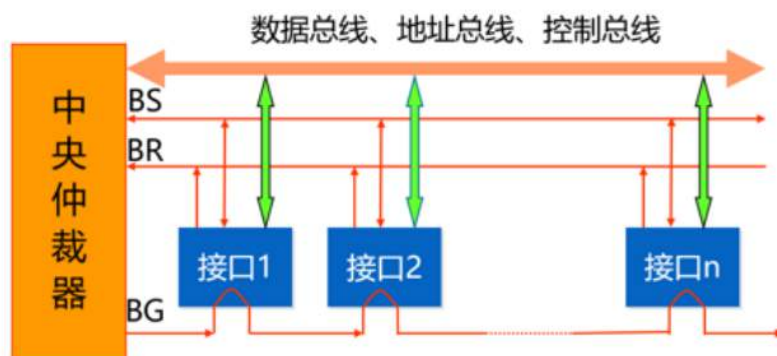


图8.11 链式查询方式

- 链式查询结构简单、扩充容易
- 链式查询中各设备优先级固定，优先级高的设备频繁使用总线时，会使优先级低的设备长时间不能使用总线(饥饿现象)
- 计数器定时查询方式

计数器定时查询方式采用一个计数器控制总线的使用权，用一组计数器地址线代替总线许可信号线BG

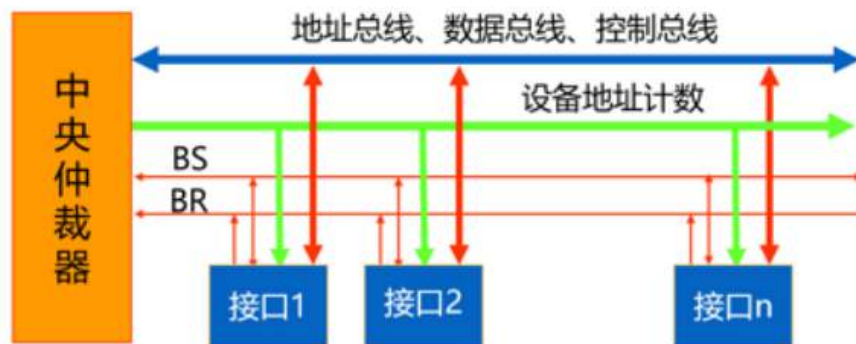


图8.12 计数器定时查询方式

当总线控制器收到总线请求信号BR且总线空闲时，设备地址计数器开始计数

当计数值与请求使用总线设备的地址（即接口号1~n）一致时，对应的主设备获得总线控制权

计数器的初值可以由程序设定，设置不同的初值就可以动态地改变设备的优先级

- 初值为5：接口为5的优先级最高
- 初值为0：则优先级与链式查询方式一致。
- 初值为上次获得总线使用权的设备地址，并且采用循环计数方式，则所有设备的优先级都是一样的。

计数器定时查询方式可以灵活调整设备优先级，避免单点故障；但控制复杂，控制线数量为 $2 + \log_2 n$

• 独立请求方式

每个主设备均有一组专属的总线请求信号线BR_i和总线许可信号线BG_i

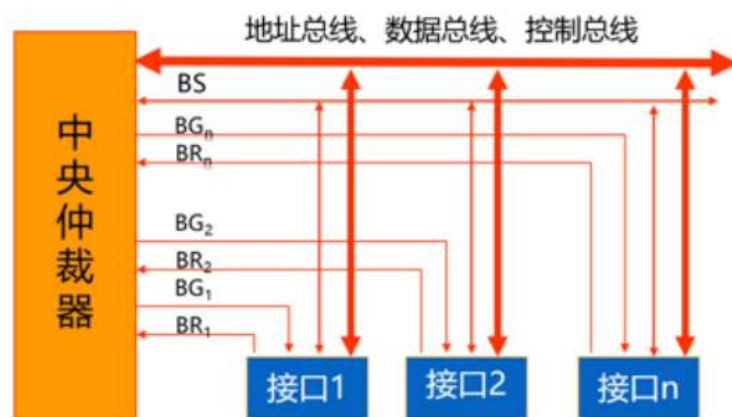


图8.13 独立请求方式

总线控制器按一定的优先次序决定响应哪个设备的总线请求

独立请求的优先级策略最为灵活，总线许可信号BG不再需要逐个地对设备进行串行查询，属于并行仲裁，响应时间快

但控制线也是最多的，需要 $2n+1$ 根

- **分组链式查询**

是链式查询方式和独立请求方式的结合，包括多个菊花链

每个菊花链具有一对独立的总线请求和总线许可信号

分布式仲裁

将总线裁决的逻辑分散在总线上连接的各个主设备中，不需要中央仲裁器

- **自举分散式仲裁**

每个主设备有专属的总线请求信号 BR_i ，总线忙信号BS则共享

每个主设备的仲裁器只连接并监测比自己优先级高的总线请求信号 BR_i

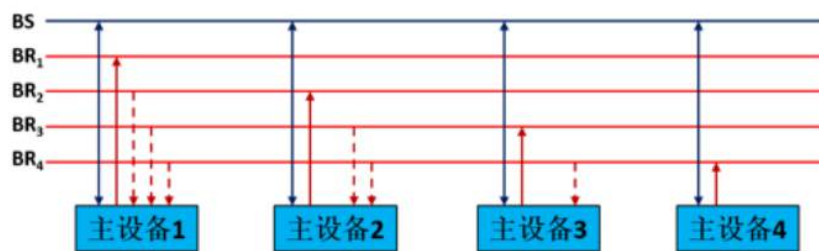


图8.14 自举分散式仲裁

每个需要请求使用总线的主设备，都将监测所有更高优先级设备的请求信号

如果发现其他高优先级的设备没有发出总线请求信号，则可获得总线控制权；否则不能使用总线控制权

自举分散式仲裁需要控制线 $n+1$ 根

• 并行竞争仲裁

不再使用单根的总线请求信号 BR_i ，而是采用编码的方式表示设备的总线请求信号，类似自举分散式仲裁

💡 如果有16个主设备自举分散式仲裁，需要16根请求信号线；但并行竞争仲裁就只要4根

• 冲突检测分散式仲裁

每个设备独立地请求使用总线

当多个同时申请使用总线的设备发生冲突时，发生冲突的设备停止总线申请，延迟一个时间段后，再重新发出总线请求

🔧 网络协议中的以太网协议采用冲突检测分散式仲裁

总线定时

同步定时

同步定时方式下通信双方均在统一总线时钟控制下进行信息传输

• 存储器读操作时序

分为寻址阶段、数据阶段、结束阶段

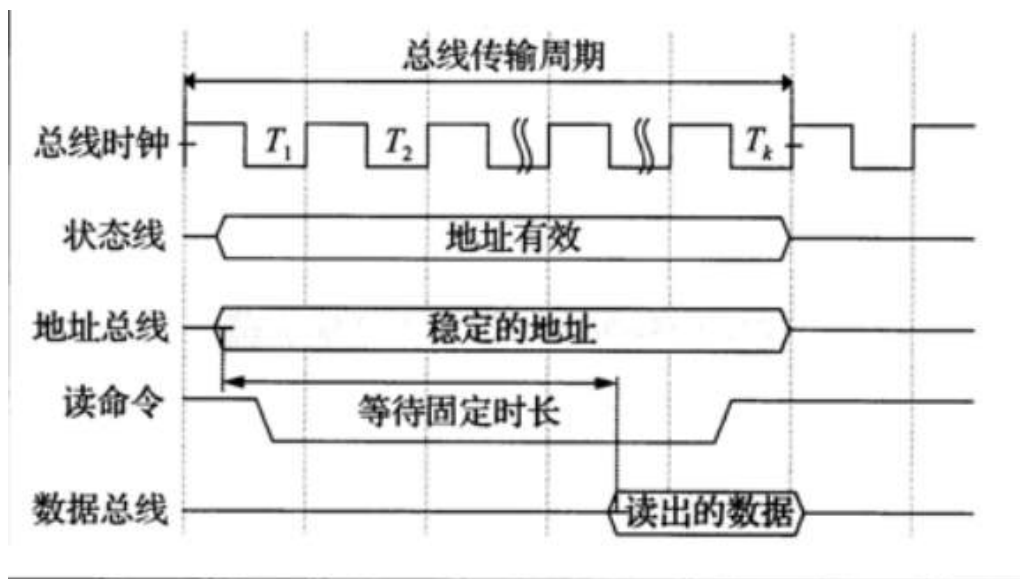


图8.15 同步方式下存储器读操作时序

1. 寻址阶段

第一个时钟周期将MEM送上AB，同时设置相关状态线，表明地址有效；读命令送上CB，等待数据出现在DB

2. 数据阶段

从设备根据状态线和地址与自己地址匹配，开始准备数据

在一个存储周期后，将data送上DB；主设备也在此时间段后拿走data

3. 结束阶段

主设备撤销所有信号，从设备撤销数据，BS=0，让出总线使用权

• 存储器写操作时序

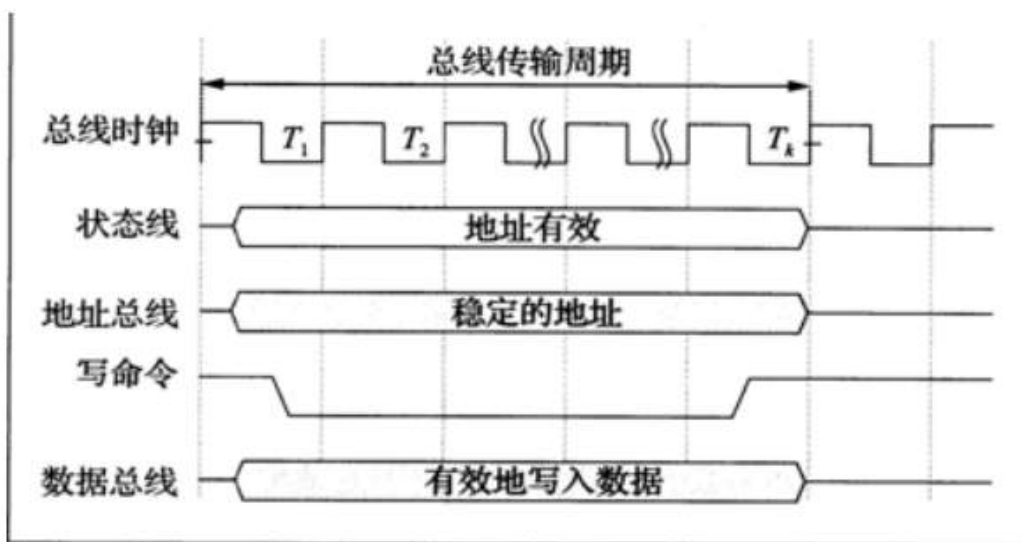


图8.16 同步方式下存储器写操作时序

异步定时

异步定时方式不需要统一时钟信号，主、从设备之间通过握手协议进行异步通信

异步实时方式使得高速设备可以和慢速设备通信，有效规避了同步定时中的时钟偏移问题，总线传输距离更长

• 存储器读操作时序

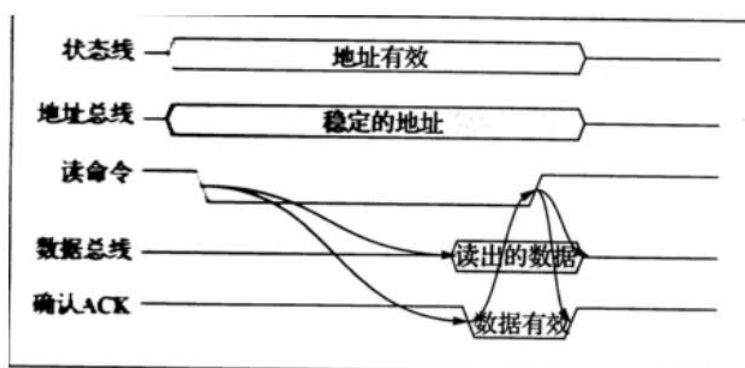


图8.17 异步方式下存储器读操作时序

1. 寻址阶段

CPU发送地址状态和读命令，等待从设备（MEM）的ACK信号

2. 数据阶段

MEM识别地址并匹配，准备好data，送上DB，同时向CPU发出ACK，CPU收到后从DB取走data

- Read生效后，mem准备数据，并将之送入DB，同时触发ACK生效；
- Read失效后，触发数据失效和ACK的撤销

3. 结束阶段

CPU撤销信号，MEM撤销data和ACK等信号

• 存储器写操作时序

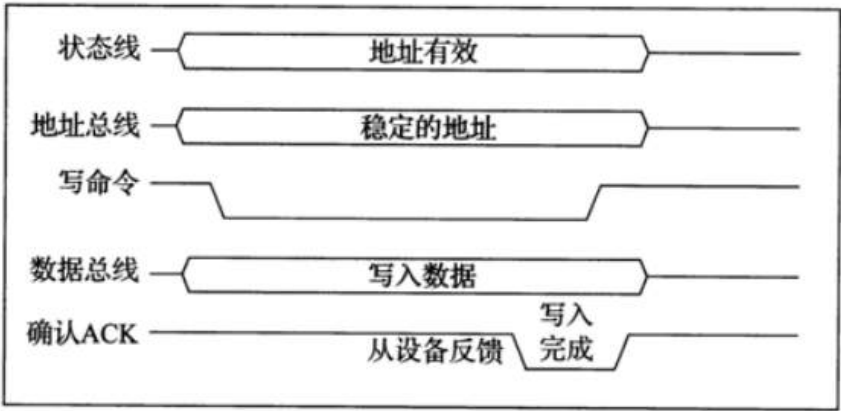


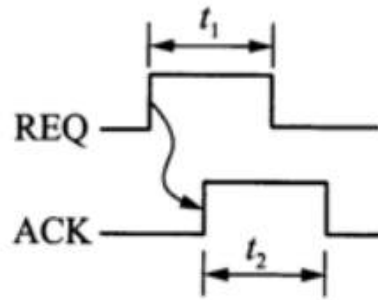
图8.18 异步方式下存储器写操作时序

在写入完成处，主设备撤销信号要等待从设备写操作完成，并反馈信号给CPU

异步定时可以根据请求信号REQ与应答信号ACK的建立和撤销是否相互依赖分为

• 非互锁方式

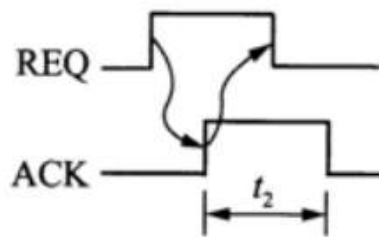
主设备发出请求信号REQ，从设备收到后会进行响应，完成请求后给出应答信号ACK
一段时间后，主、从设备分别撤销请求信号REQ和应答信号ACK



该方式只有一次握手

- **半互锁方式**

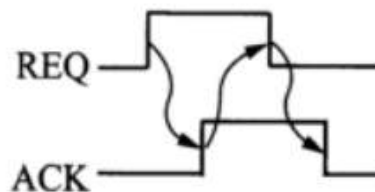
主设备请求信号REQ的撤销依赖于从设备应答信号ACK的建立



需要两次握手

- **全互锁方式**

从设备应答信号ACK的撤销也依赖于主设备请求信号ACK的撤销



需要三次握手

💡 全互锁操作实例

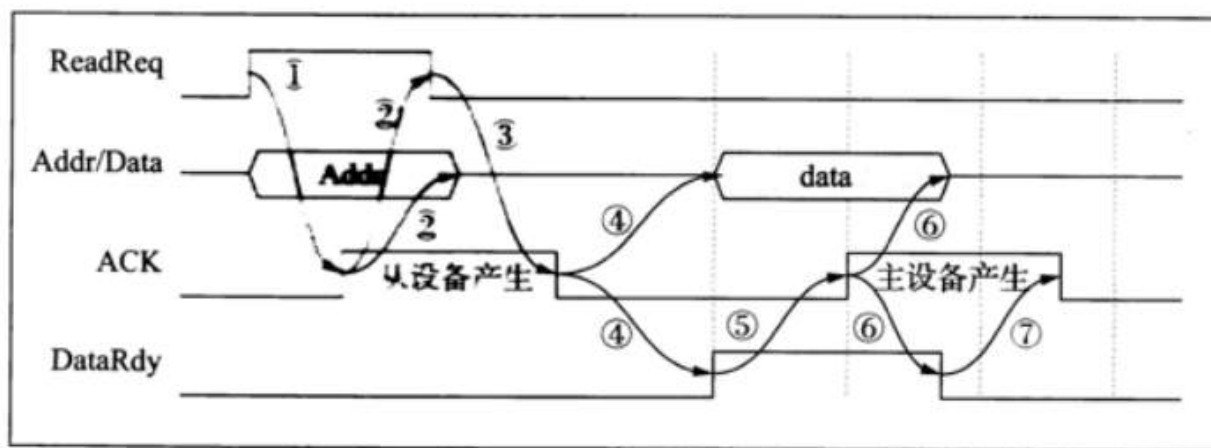


图8.20 全互锁异步方式下存储器读操作时序

1. 存储器收到CPU的ReadReq后，接收Addr，发挥应答信号ACK，表示收到地址和读请求信号；
2. 主设备收到确认信息后，撤销发出的信息；
3. 存储器发现主设备撤销后，也撤销ACK
4. 存储器将data送上DB，同时向CPU发出DataRdy
5. CPU从DB取走data，向存储器发出ACK，表明数据接收成功
6. 存储器收到CPU的应答信号后，撤销数据和DataRdy信号
7. 主设备发现上述信号撤销，也撤销ACK，一次异步传输完成。

半同步定时

是同步定时和异步定时的结合。

• 存储器读取数据时序

在同步存储器读时序的基础上引入一个等待信号Wait，Wait高电平表示存储器还没有准备好数据，CPU要一直等待

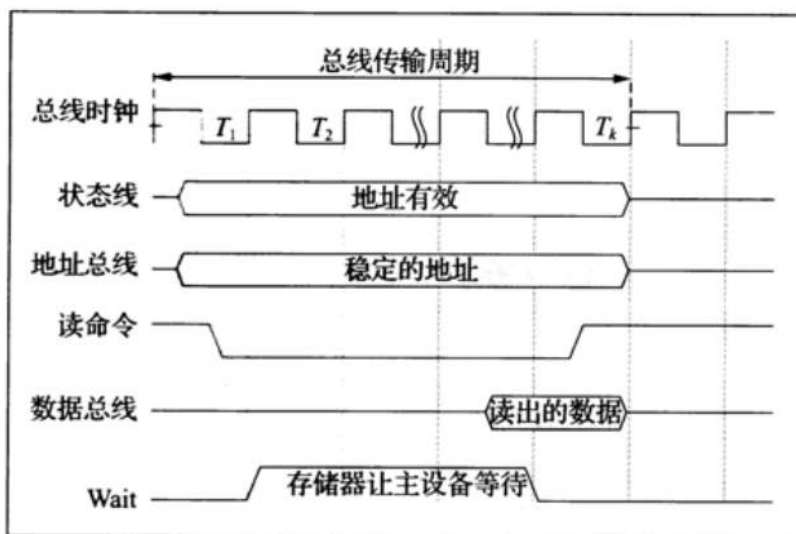


图8.21 半同步通信方式下存储器读操作时序

在同步方式下，等待固定时长后存储器就会自动取走数据总线上数据

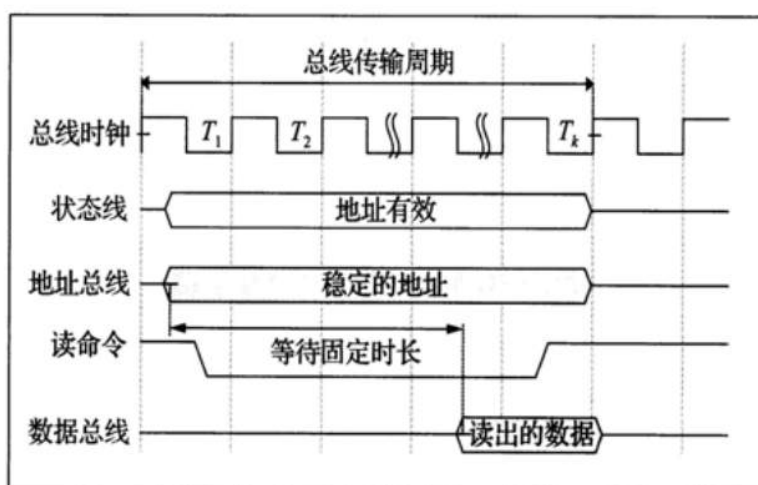


图8.15 同步方式下存储器读操作时序

分离事务通信方式

在半同步定时的基础上，当存储器准备数据时，释放总线让主设备处理其他总线事务

分离事务通信协议中，总线的交互过程为

1. 主设备向从设备发出读请求信号，给出地址和请求命令。
2. 当从设备进行应答后，主设备立即释放总线控制权。

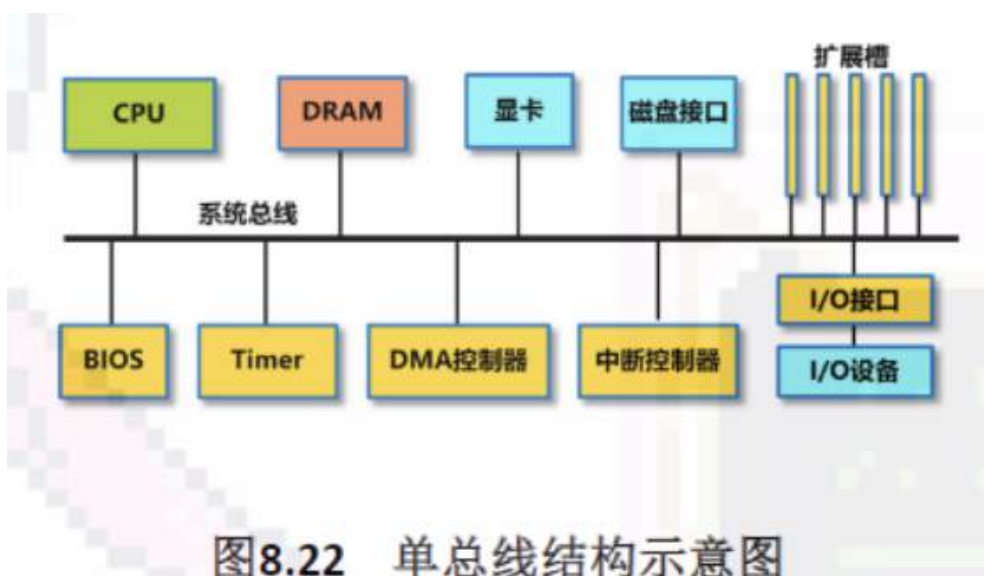
3. 从设备准备数据，此时总线用于处理其他总线事务。
4. 从设备准备好数据后，将作为主设备重新申请使用总线，并将数据放置在数据总线上。
5. 原主设备通过总线接收数据。

分离事务通信方式大大提高了总线利用率，但是其控制方式更加复杂

总线结构

单总线结构

单总线结构只有一个总线(系统总线/全局总线)

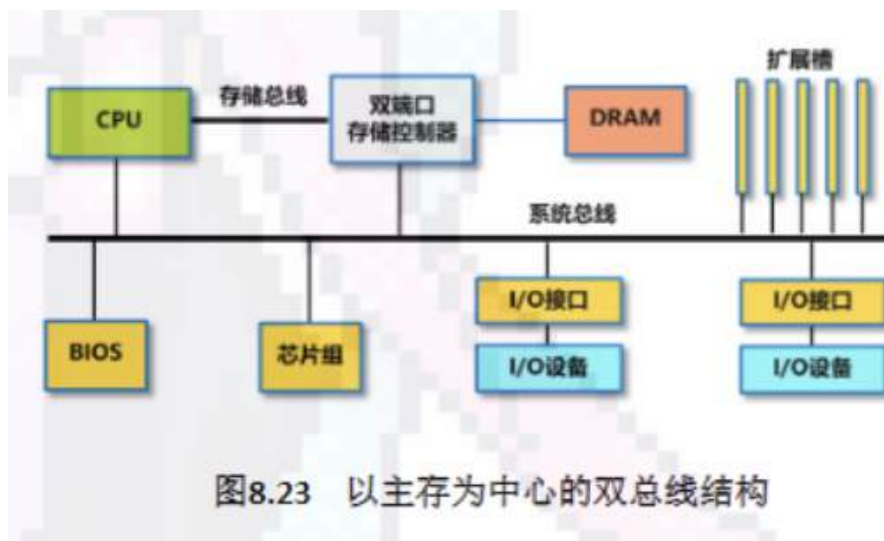


- I/O设备必须通过I/O接口与系统总线相连
- 显卡采用集成显卡的方式与系统总线直接相连；也可以采用独立显卡的方式插入扩展槽，与系统总线相连

单总线结构简单，使用灵活，扩展容易；但性能较差，高速设备的高速特性被限制

双总线结构

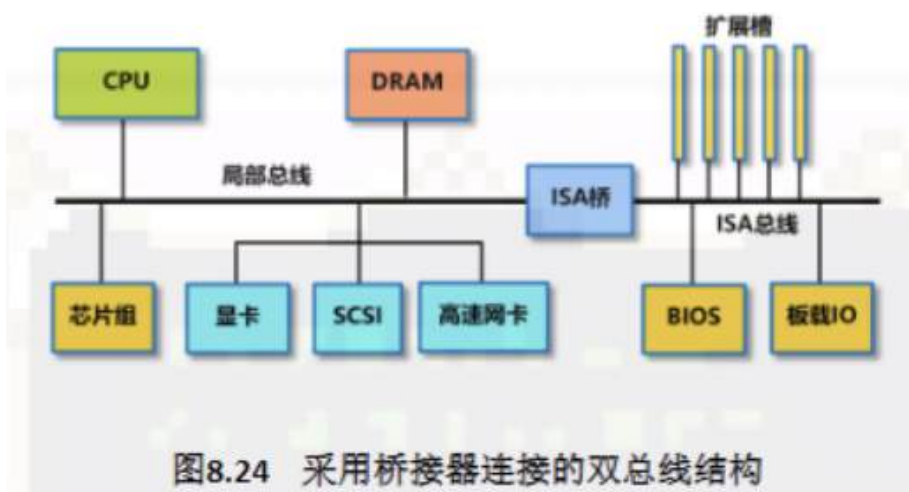
- 以主存为中心的双总线结构



双总线包括系统总线和存储总线(主存总线)，双端口存储控制器（也称为内存控制器）同时连接存储总线和系统总线

CPU通过存储总线访问主存，通过系统总线访问外部设备，外部设备与主存之间、CPU与主存之间的数据传送可并行进行

- 采用桥接器连接的双总线结构

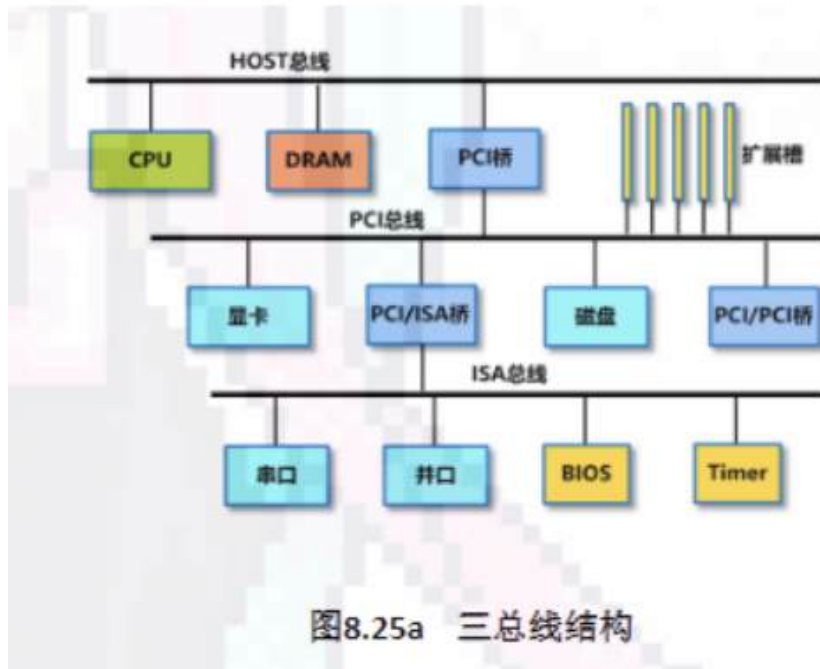


采用桥接器的双总线结构计算机包括系统总线（局部总线）和I/O总线

- CPU、主存以及一些高速设备（显卡、SCSI、高速网卡等）直接连接在系统总线（局部总线）上
- 慢速的I/O设备全部挂在I/O总线（ISA总线）上
- 系统总线和I/O总线之间通过桥接器相连

三总线结构

包括HOST总线(CPU总线、系统总线)、PCI总线、ISA总线(遗留总线)

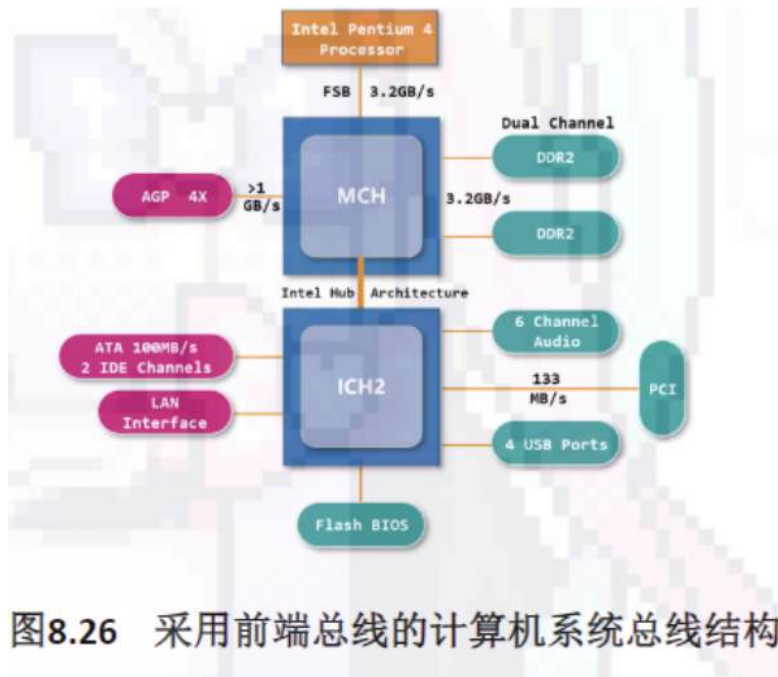


- CPU、DRAM连接在HOST总线上
- 显卡、磁盘等高速设备连接在PCI总线上
- 低速设备（串口、并口、BIOS、定时器Timer）连接在ISA总线上
- PCI桥连接HOST总线和PCI总线
- PCI/ISA桥连接PCI总线和ISA总线

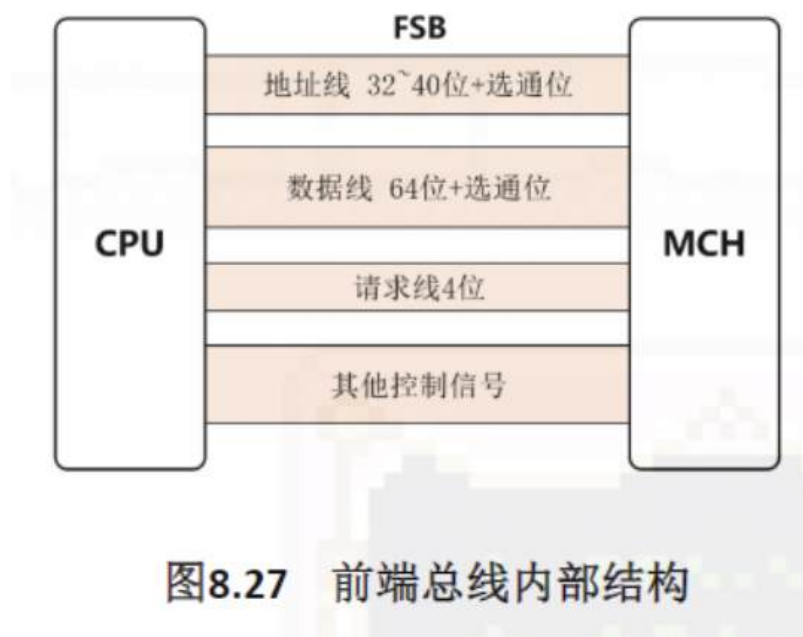
这大大提高了 I/O 设备的性能，但任意时刻只能使用一种总线，工作效率相对较低

高性能总线结构

采用前端总线的南北桥结构



- 前端总线(FSB)



用于连接CPU和北桥芯片MCH

- 北桥芯片(MCH)

用于连接高速的CPU、AGP显卡、内存（DDR2）和南桥芯片（ICH2）

- 南桥芯片(ICH2)

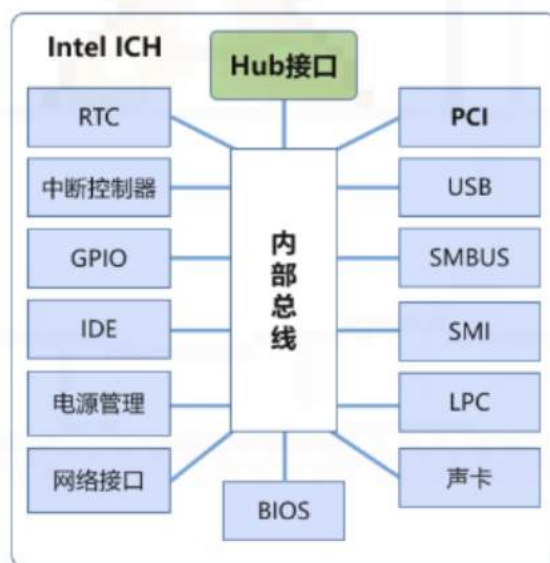


图8.28 南桥芯片内部结构

集成了IHA总线Hub接口、实时时钟控制器RTC、中断控制器、GPIO端口、IDE内存接口、电源管理模块、网络接口、音频接口、PCI总线控制器、USB总线控制器、SMBUS、SMI、LPC总线接口、BIOS等设备

采用QPI总线南北桥结构

用QPI总线代替前端总线FSB，DMI总线代替IHA总线，DDR3内存条直接与CPU连接

常用外部总线

- RJ-232-C：也称串口，其中的RS为推荐标准，是一种串行物理接口标准
- RS-485：对于远距离的串行通信，通常采用半双工的RS-485串行接口标准
- IEEE-488：也称并口，是一种8位并行接口标准
- USB：通用串行总线，采用分层的星型树状拓扑结构连接

通过USB集线器扩展可最多支持127个设备

- DVI：数字视频接口
- HDMI：高清晰度多媒体接口，是一种数字化视频/音频接口技术，适合影像传输的专用型数字化接口

可同时传送音频和影像信号

- **RJ45**: 是布线系统中信息插座连接器的一种

连接器由插头(接头、水晶头)和插座组成

输入与输出系统

输入输出设备与特性

输入输出系统主要用于实现CPU与外部设备、外部设备与主存之间的信息交换

输入输出系统是典型的软硬件协同系统, 包括

- I/O设备、I/O接口、总线、I/O管理部件等I/O硬件系统
- 驱动程序、软件访问接口、用户程序等I/O软件系统

输入输出设备的特性有

- 异步性

CPU与外设必须以异步方式交换数据, 因为两者速度差异过大

- 实时性

CPU必须及时处理外设准备好的数据

- 独立性

外设采用标准总线接口与CPU进行连接, 输入输出与具体设备类型无关

I/O接口

计算机中的所有 I/O 设备均使用 I/O接口 (总线接口) 与总线相连, CPU使用设备地址经总线与I/O接口通信来访问I/O设备

从不同角度, I/O 接口可以分为不同的类型

- 按**数据传送方式** (外设和接口一侧): 可分为**并行接口** (一字节或一个字的所有位同时传送) 和**串行接口** (一位一位地有序传送), 接口要完成数据格式的转换
- 按**主机访问I/O设备的控制方式**: 可分为**程序查询接口**、**中断接口**和**DMA接口**等
- 按**功能选择的灵活性**: 可分为**可编程接口** (通过编程改变接口功能) 和**不可编程接口**

I/O接口的功能

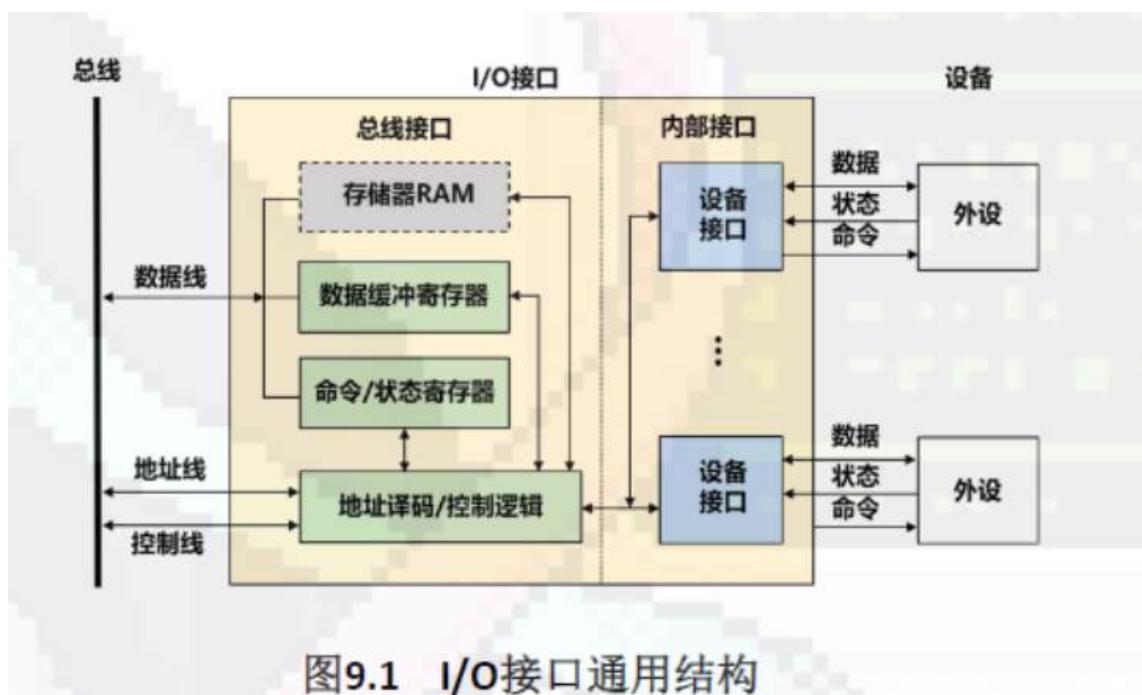
I/O接口应具有的功能有

- **设备寻址**：接收来自总线的地址信息，经译码电路，选择对应外部设备中的寄存器或存储器。
- **数据交互**：实现外部设备、主存与CPU之间的数据交换，这也是接口最基本的功能。
- **设备控制**：传送CPU命令。
- **状态检测**：反映外部设备的工作状态。
- **数据缓冲**：匹配CPU与外部设备的速度差距。
- **格式转换**：实现数据格式转换或逻辑电平信号转换。
- **中断、时序控制、数据检错和纠错**

I/O接口的结构

I/O接口主要包括

- **总线接口**：连接总线的总线接口必须按总线标准进行设计，这部分逻辑为接口的标准部分。
- **内部接口**：连接设备的内部接口逻辑因设备而异，是非标准的。



其基本功能部件有

- **数据缓冲寄存器（DBR）**

- 设备状态寄存器（DSR）
- 设备命令寄存器（DCR）
- 设备存储器：例如显卡中的显存。
- 地址译码器
- 数据格式转换逻辑：进行串并或并串传送的转换

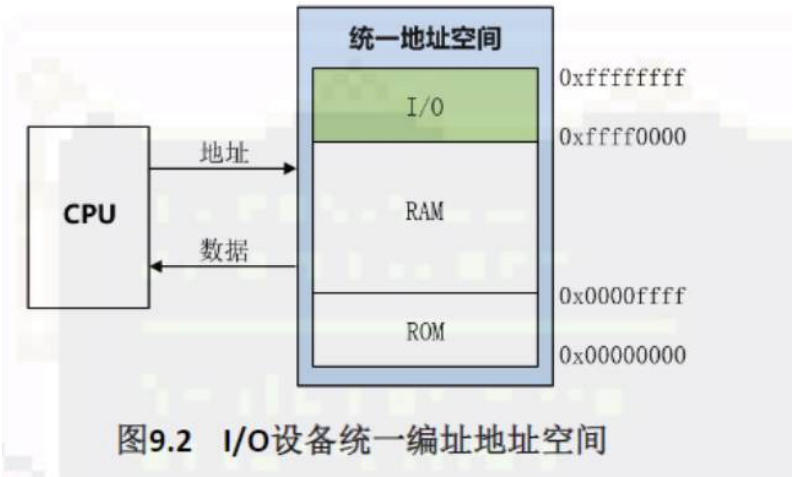
I/O接口的编址

I/O 接口电路中可被 CPU 直接访问的寄存器称为 **I/O 端口**，包括**数据端口**、**状态端口**、**控制端口**

I/O 端口要想能够被CPU访问，就必须要对各个端口进行编址，每个端口对应一个端口地址
对I/O 端口的编址方式有与存储器**独立编址**和**统一编址**两种

统一编址

又称为**内存映射编址**。外设与内存统一编址，逻辑上处于同一个地址空间



统一编址不需要设置专用I/O指令，只需要访存指令就可以访问外设

 由于I/O接口中的数据是动态变化的，因此统一编址中的I/O地址空间不能使用cache进行缓存，否则CPU无法了解设备状态的实时变化；
在C语言中接口变量应该声明为volatile型，表明该变量是会经常自动变化的

优点在于

- 不需要专门的 I/O 指令

- 使端口有较大编址空间
- 缺点在于
- 占用部分主存地址空间，主存容量变小
- 识别 I/O 端口时全部地址线都要参加译码

独立编址

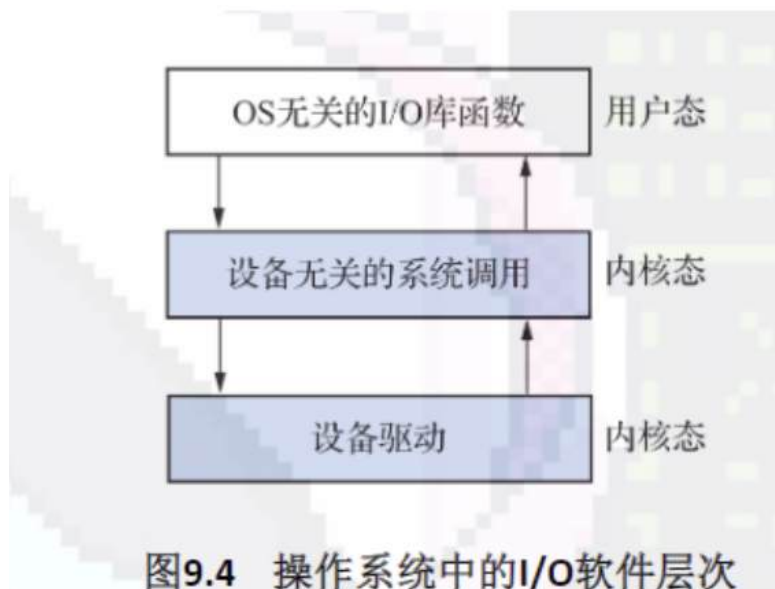
又称为**端口映射编址**，I/O地址空间与主存地址空间相互独立

表9.1 个人计算机中常见I/O端口		
I/O地址范围（十六进制）	IRQ中断号	设备
0000 ~ 000F	00000004	DMA控制器
0020 ~ 0021	无	中断控制器
0040 ~ 0043	00000004	计时器
0060 ~ 0064		PS/2鼠标、键盘
0070 ~ 0071	00000008	CMOS实时时钟
02FB ~ 02FF	00000004	串口
F060 ~ F07F	FFFFFFFE	SATA控制器
03D0 ~ 03DF	FFFFFFFB	显卡控制器

此时需要使用特殊的I/O指令访问外设，在80386处理器就需要使用IN/OUT指令

I/O接口的软件

现代计算机中，用户并不能直接访问设备，必须通过操作系统间接访问设备，主要包括三个层次



与操作系统无关的I/O库

通常工作在用户态下，与操作系统无关

如C语言的stdio.h的 `printf()`，用户程序通过调用I/O库中的函数来访问外设

与设备无关的操作系统调用库

通常工作在内核态，与设备无关，屏蔽了设备的具体访问细节，向用户提供了统一的I/O调用接口

如UNIX操作系统的 `open()`

独立的设备驱动程序

通常工作在内核态，通过具体的I/O指令或访存指令访问外设，与具体设备进行数据和命令交互

数据传输控制方式

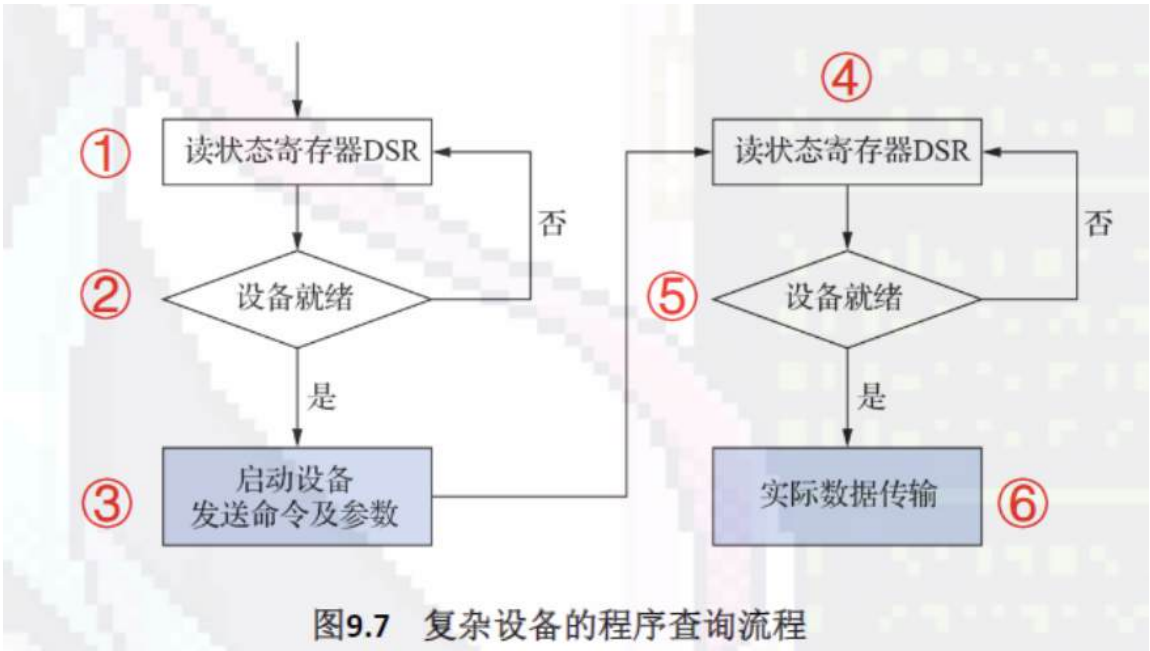
程序控制方式

是最原始的方式，可以分为程序查询方式(轮询方式)和直接传送方式

其中，程序查询方式是有条件传送方式，直接传送方式为无条件传送方式

程序控制方式流程为

- 1. CPU首先查询设备的状态，如果设备就绪，通过设置I/O接口的命令寄存器启动设备
- 2. 设备开始准备；设备准备的过程中，CPU通过读取I/O接口中的状态寄存器，查询设备是否已就绪
- 3. 接下来CPU不断查询设备状态，根据查询结果，决定下一步操作究竟是进行数据传送，还是等待设备就绪

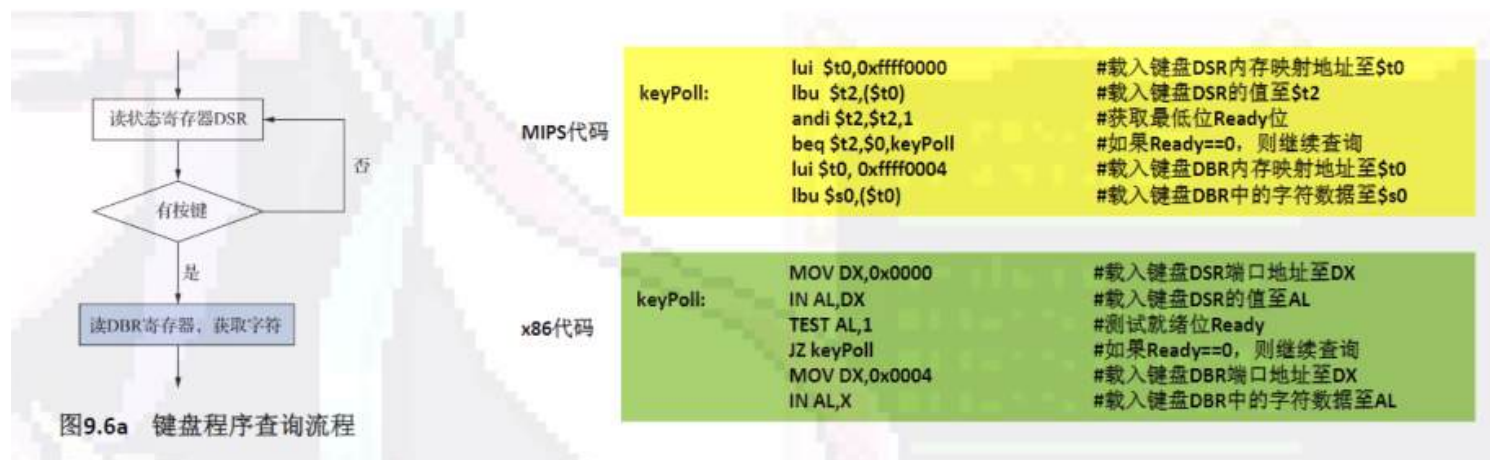


参考设备I/O地址如下

表9.2 设备I/O地址

设备	寄存器（8位）	内存映射地址	端口映射地址	备注
键盘设备	数据缓冲寄存器DBR	0xFFFF0004	0x0004	
键盘设备	设备状态寄存器DSR	0xFFFF0000	0x0000	最低位为Ready位
字符显示设备	数据缓冲寄存器DBR	0xFFFF000C	0x000C	
字符显示设备	设备状态寄存器DSR	0xFFFF0008	0x0008	最低位为Ready位

以键盘设备的程序查询流程为例

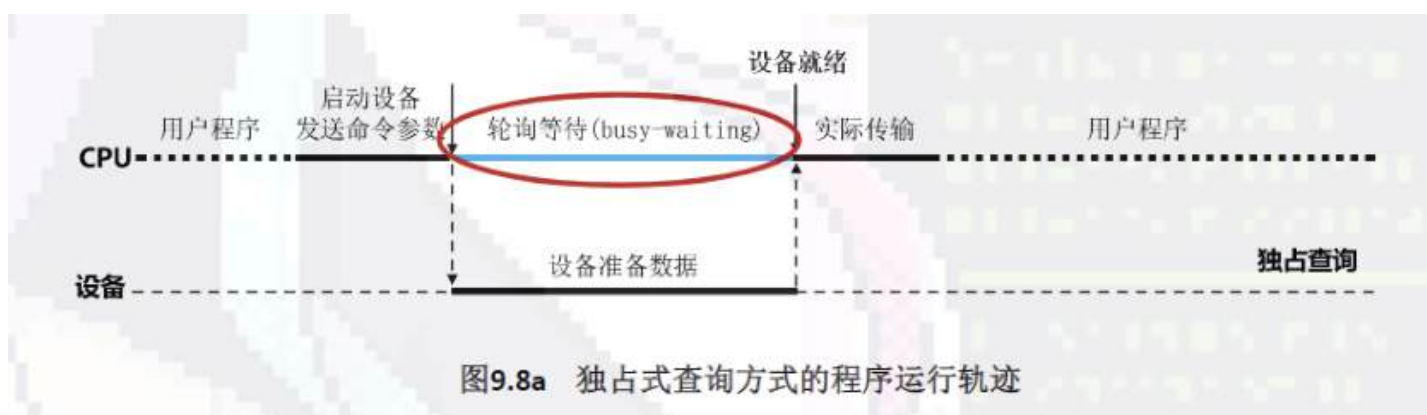


程序控制方式如采用轮询策略，CPU要不断查询I/O接口中的状态寄存器DSR

而轮询也有两种方案

- 忙等待

又称为**独占式查询**



在设备准备数据时，CPU不能执行任何任务，称为忙等待状态

这使得CPU浪费大量时间进行轮询操作

- 定时轮询

1. CPU启动设备后，会启动一个定时中断
2. 挂起当前用户进程P1，并放入I/O等待队列，调度用户进程P2运行
3. 定时时间到后，CPU会执行定时中断服务程序，查询设备状态
4. 如果设备准备好，则唤醒等待进程P1，否则将继续定时查询

定时轮询方式下，CPU可以执行其他任务，避免轮询等待CPU时间的浪费，有效节约了CPU时间

但定时轮询的时间间隔很关键

- 时间间隔过短，则中断处理过多，时间浪费较多
- 时间间隔过长，外部设备数据可能得不到及时处理



程序控制方式的I/O接口设计简单，但是CPU与外部设备只能串行工作，CPU浪费大量时间进行查询和等待，系统效率较低

程序中断控制方式

程序中断控制方式中，CPU与外部设备是并行工作，CPU利用率高

流程为

1. 用户系统调用：用户进程P1通过系统调用read()函数使操作系统进入内核态
2. 驱动启动设备：设备驱动程序发送命令参数，启动设备
3. 操作系统进程调度：操作系统将用户进程P1放入I/O等待队列，调度用户进程P2运行
4. 设备中断请求：当设备准备好数据后，立即主动向CPU发出中断请求

- 5. CPU中断服务: CPU暂停当前进程的执行, 转去执行中断服务程序 (完成数据交换), 唤醒用户进程P1, 进入就绪状态, 等待系统调度
- 6. CPU恢复运行: 中断服务程序执行完毕后, 返回到断点继续执行用户进程P2



表9.9 中断控制方式的程序运行轨迹

中断控制方式的额外时间开销在于进程切换和中断服务程序的时间开销

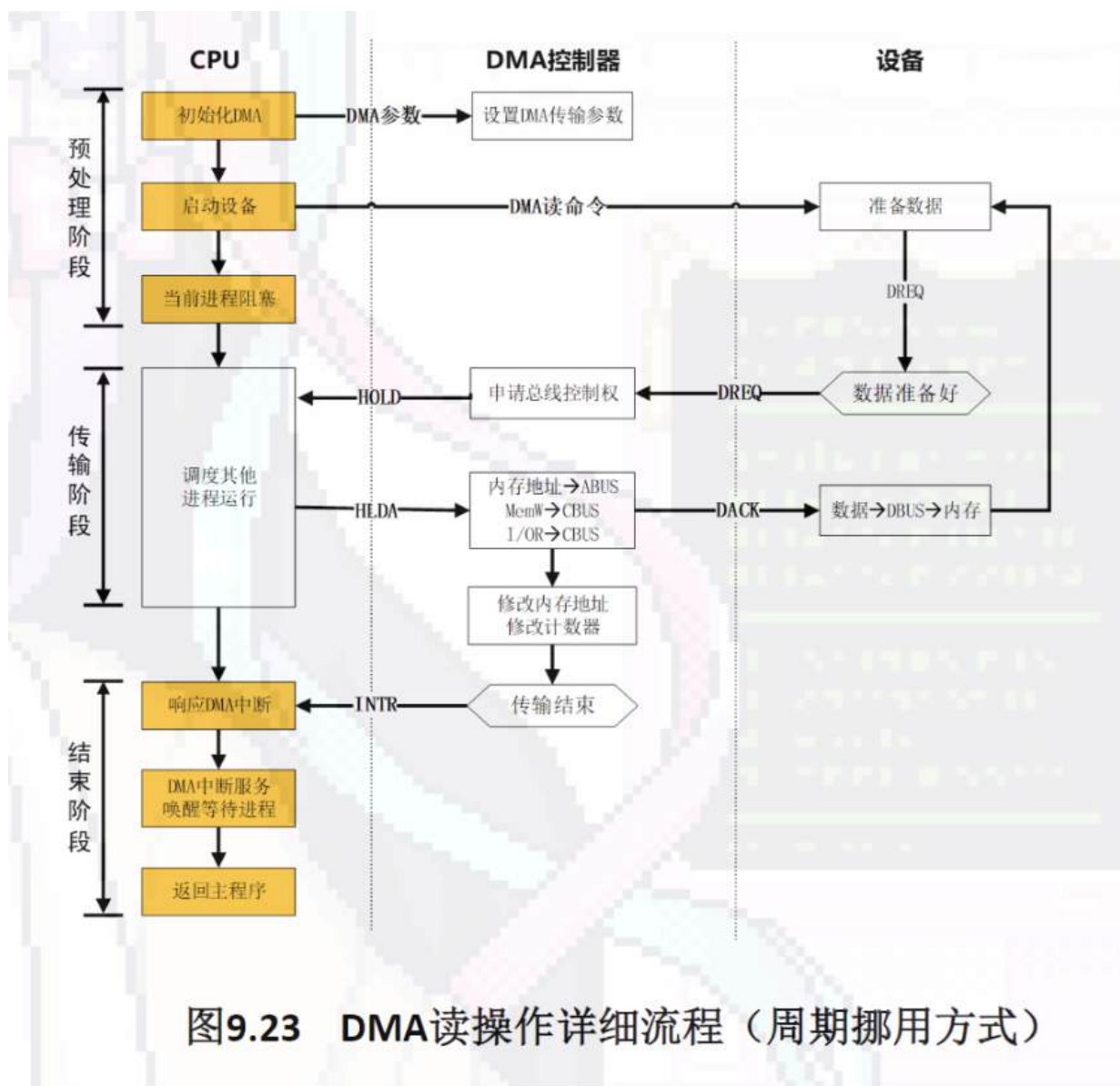
⚠ 程序中断控制方式并不一定优于程序控制方式, 如极高速设备准备数据的时间很短, 比两次上下文切换以及中断服务程序的开销还短, 反而使得效率更低

直接存储器访问方式(DMA方式)

由DMA控制器 (DMAC) 临时代替CPU控制总线, 控制外部设备与内存之间进行直接的数据交换, 信息传送不再经过CPU寄存器中转

传输流程为

1. 预处理阶段: CPU访问DMAC, 设置DMA传输参数 (包括主存起始地址、数据块长度、传输方向等), 并启动设备
2. 传输阶段: DMAC向CPU申请总线控制权用于数据传输, 此时CPU可以执行其他程序
3. 结束阶段: DMAC以中断方式请求CPU对数据缓冲区和DMAC进行后处理



💡 在1和3中，DMAC是从设备；在2中DMAC为主设备

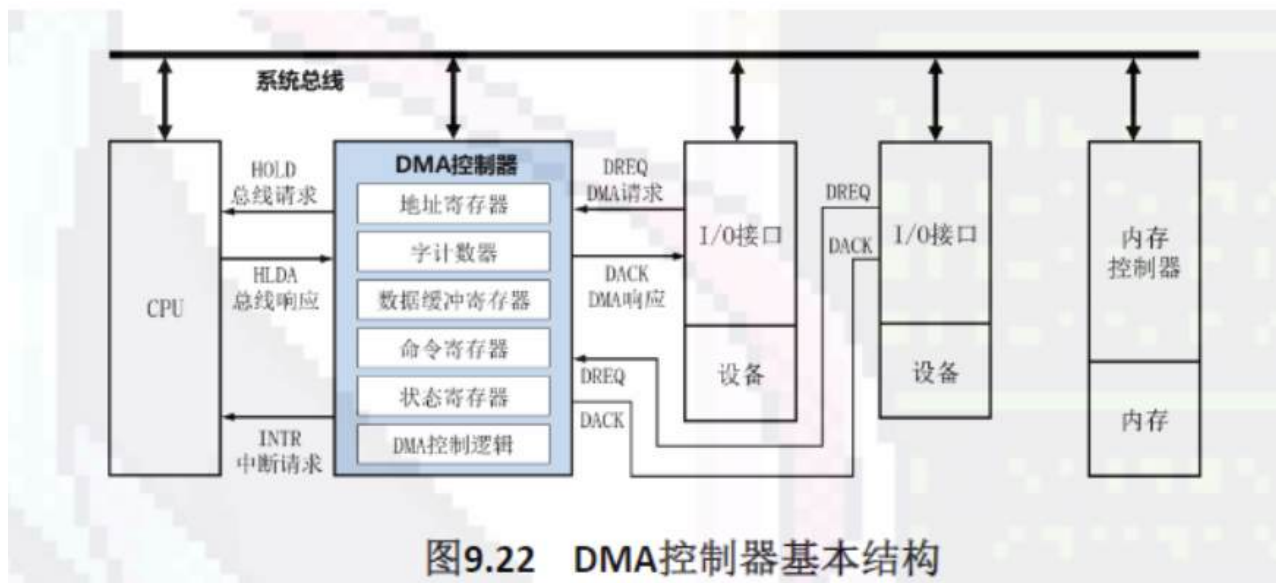
DMA方式主要用于存储器与外设之间的大量数据传送，也可以用于内存数据的内部搬迁，减少了I/O过程中CPU用于实际传输的开销

DMAC的结构

DMAC结构主要包括

- 地址计数器：初始值为数据块的起始地址，每传送1个字或字节，自动加1。
- 字计数器：初始值为数据块长度，每传送1个字或字节，自动减1。
- 数据缓冲寄存器（DBR）：DMAC作为从设备时，用来接收CPU传输的数据；DMAC作为主设备时，用来暂存数据。

- 命令寄存器（DCR）：用来接收CPU的控制命令。
- 状态寄存器（DSR）：负责向CPU反馈DMAC的状态。
- DMA控制逻辑：包括控制和时序电路，以及状态标志，支持中断机



DMAC传输控制信号有

- DREQ：DMA请求，由外部设备发给DMAC
- HOLD：总线请求，由DMAC发给CPU
- HLDA：总线响应，由CPU发给DMAC
- DACK：DMA响应，由DMAC发给外部设备

图中的DMAC为第三方DMA，现代总线技术普遍支持总线主控技术，将DMAC集成到I/O接口中，称为第一方DMA

DMA与中断方式的区别

- 虽然均采用“请求-应答”机制，但两者请求和响应时机有区别
 - 中断方式求的是CPU时间，响应时机是指令周期结束时刻
 - DMA方式请求的是总线控制权，响应时机是任何一个机器周期结束的时刻
- 中断方式通过CPU执行程序进行实际的数据传送，存在程序执行现场的保护和恢复问题
DMA方式依靠额外的硬件来实现数据传输，其不改变CPU现场，不影响系统性能
- 中断方式不仅可以实现数据传输，还可以用于处理各种随机事件，提高计算机的灵活性；

通道方式

为进一步减少CPU被I/O操作中中断的次数，提高CPU效率，由通道控制器(IOP)分担CPU的I/O管理，以提高系统效率

通道拥有独立的通道指令系统，可以通过执行通道程序来代替CPU独立执行I/O任务

通道具体功能包括

- 根据CPU要求，组织设备与系统连接
- 通过设备控制器向设备发出操作命令
- 指出数据在设备中的位置和在内存缓冲区内的位置
- 检查设备和设备控制器的工作状态
- 向CPU反映设备、设备控制器及通道本身的状态信息
- 进行必要的信息格式变换

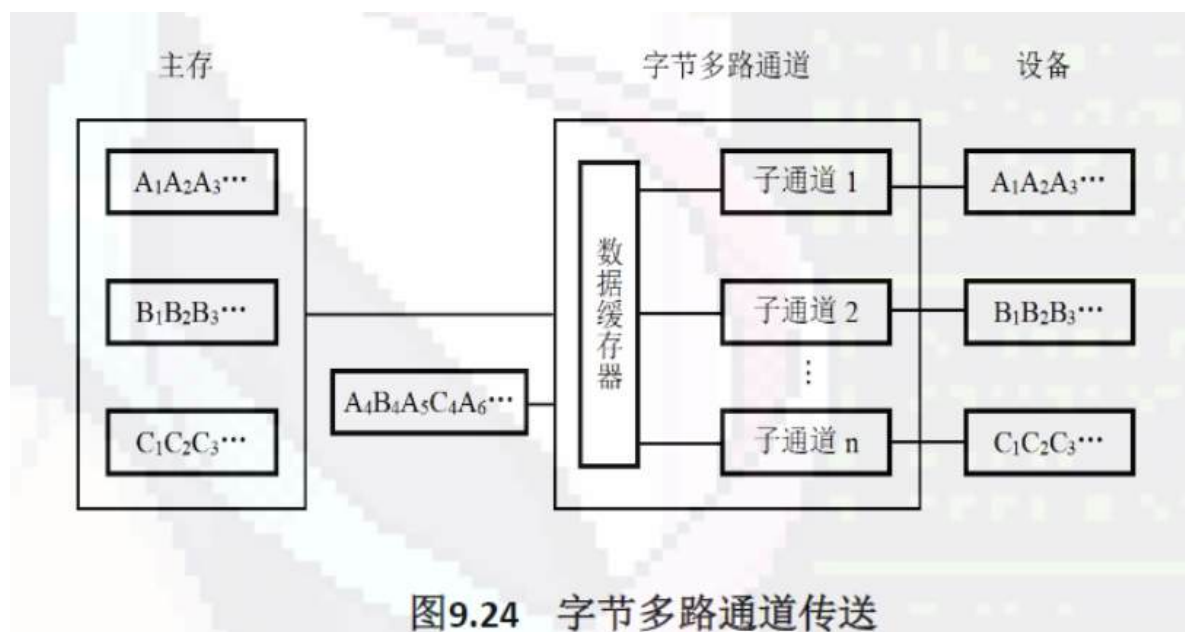
设备控制器

介于通道和设备之间，是通道对外部设备实行具体控制的部件

通道类型

- 字节多路通道

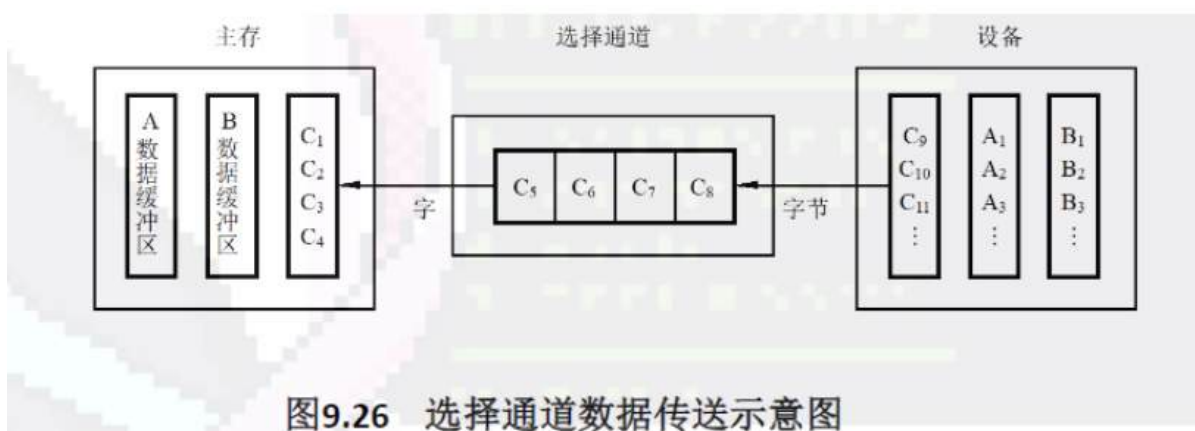
采用字节交叉方式，与通道连接的各个设备以字节为单位交叉使用通道



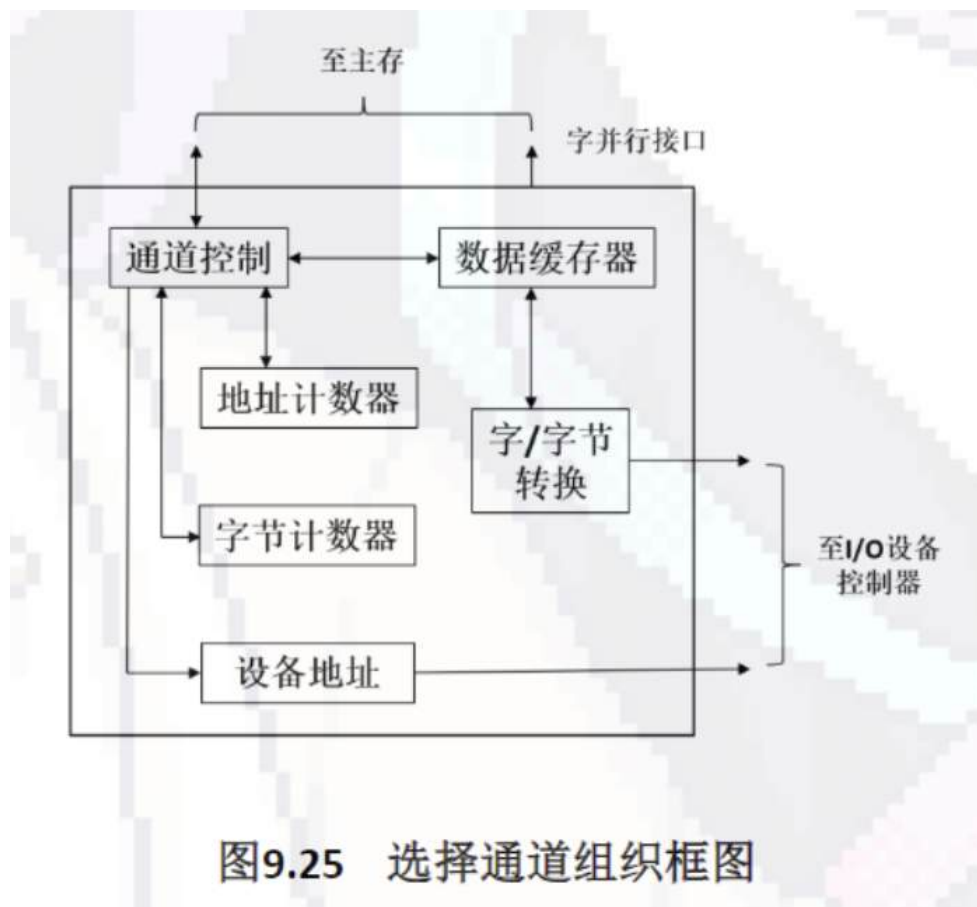
常用于连接低速设备，如打印机

- 选择通道

设备以成组数据连接传送方式占用通道，直到指定数量的数据全部传送完毕，通道才转为其他设备服务



选择通道在物理上可以连接多个设备，但这些设备不能同时工作



选择通道适用于大批量数据的高速传送

- 成组多路通道

成组多路通道是字节多路通道和选择通道的结合，多个设备以数据组（块）为单位交叉使用通道

当某设备占用通道时，连续传送一组数据（选择通道），然后将通道让给其他设备（字节多路通道）

在一批数据传送过程中，要多次与设备断开和连接，通道的硬件结构比较复杂

成组多路通道适用于中高速设备，如磁盘、磁带等

CPU对通道的控制

- 执行I/O指令

执行特权I/O指令STARTI/O来启动通道

CPU启动通道后，通道和外部设备将独立工作

- 处理来自通道的中断请求

- 当通道和外部设备发生异常或通道处理结束时，CPU接收来自通道的“中断”请求，对故障进行测试和处理并对通道传输进行后期处理
- 收集外部设备和通道自身的状态信息，并送入内存固定单元中存放，供CPU测试外部设备和通道的状态时使用

外围处理器方式(PPU)

是通道方式的进一步发展，通常用于大中型计算机系统中

PPU独立于CPU工作，有自己的指令系统

常见I/O设备

- **键盘**：最常用的输入设备，主要由按键开关、盘架、编码器和接口电路组成
- **鼠标**：也是最常用的输入设备，主要有机械式鼠标和光电式鼠标(最常用)两类
- **打印机**：最常用的输出设备，可分为击打式打印机（如针式打印机）和非击打式打印机（如喷墨打印机、激光打印机）两大类

计算机通过打印机语言控制打印机，主要有页描述语言和嵌入式语言

- **显示器**：也是最常用的输出设备，主要性能指标包括分辨率、长宽比、屏幕尺寸、灰度级(颜色的差别)、刷新率、显存

显存也称**刷新存储器**，用于暂存当前屏幕显示信息和不断刷新屏幕显示；

- 显存容量=分配率 $\times$ 灰度级位数
- 显存带宽=分辨率 $\times$ 灰度级位数 $\times$ 刷新频率

提高显存带宽，可以选择

- 使用高速度的DRAM芯片组成刷新存储器
- 刷新存储器采用多体交叉结构
- 提高刷新存储器内显示控制器的内部总线宽度
- 刷新存储器采用双端口存储器结构，将刷新端口与更新端口分开
- **硬盘存储器**：是磁表面存储器的一种，磁鼓、软盘、磁带都属于磁表面存储器

磁表面存储器利用磁性材料剩磁的两种磁化方向来记录信息，S $\rightarrow$ N表示1，N $\rightarrow$ S表示0

写入时电转磁，读出时磁转电

- **磁盘存储器**

- 内部结构

包括磁记录介质、磁盘控制器（软盘称为FDC，硬盘称为HDC）、磁盘驱动器（软盘称为FDD，硬盘称为HDD）

- 磁盘数据信息编址和记录格式

包括记录面、磁道、扇区，半径相等的磁道集合称为圆柱面

- 技术指标

- 存储密度：包括道密度和位密度
 - 存储容量：磁盘存储容量（双面盘）=盘片*2*磁道数*扇区数*扇区容量
 - 平均定位时间：包括寻道时间和等待时间
 - 数据传输速率：假设位密度为M（bit/英寸）、转速为V（英寸/s），则数据传输速率为 $M \times V$ （bit/s）。

- **磁盘阵列(RAID)**

磁盘阵列(RAID)将多个磁盘按照一定的方式进行组织和管理，构成一个大容量、高性能、高容错的存储系统

RAID具有以下特征

- 多个磁盘构成一个更大的逻辑存储空间，扩充存储系统容量。
 - 连续数据被分割成相同大小的数据块，相邻的数据块分布在不同的磁盘上，数据访问时多个磁盘并发工作，从而提升存储系统访问性能。
 - 采用校验编码提高多磁盘系统可靠性，在某个磁盘出现故障后，磁盘阵列仍可正常工作

- **光盘存储器**

- 按照访问模式可分为

- 只读型光盘（ROM，也称CD-ROM）
 - 一次写入型光盘（WORM）
 - 可擦重写型光盘（ReWrite）：又分为相变光盘和磁光盘（MO）两种。
 - 直接重写型光盘（OverWrite）

- 按照容量可分为
 - CD（也称CD-ROM）
 - 数字化视频光盘（DVD）
 - 蓝光光盘（BD）

还可以按照光盘盘片直径大小分类

□

□